
AD16N otp&flash SDK

发行版本 *v1.2*

liujie

2025 年 10 月 11 日

1	文档版本说明	3
1.1	v1.2.0 说明	3
1.2	v1.1.1 说明	4
1.3	v1.1.0 说明	4
1.4	v1.0.0 说明	5
2	环境介绍	7
2.1	硬件环境介绍	7
2.1.1	AD16N 硬件开发环境	7
2.2	快速上手	9
2.2.1	编译下载介绍	9
2.2.2	编译下载报错说明	13
2.3	SDK 芯片资源占用情况	14
2.3.1	I/O 口占用情况	14
2.3.2	软中断占用情况	15
2.3.3	定时器占用情况	15
2.3.4	串口占用情况	15
2.4	SDK 工程配置	15
2.4.1	应用配置文件《app_config.c》介绍	16
2.4.2	应用配置文件《app_config.h》介绍	17
2.4.3	APP 配置文件《app_modules.h》介绍	23
2.4.4	cpu 差异相关功能配置文件《cpu_config.c》介绍	25
2.5	AD16N 超频说明	26
2.6	AD16N 启动时间	26
3	下载目录 & 烧写说明	27
3.1	下载目录介绍	27
3.1.1	dowload_bat.c 文件介绍	28
3.1.2	isd_config.ini 文件介绍	29
3.2	派生密钥 3.0 介绍	31
3.3	二级密钥 V2.0 简单介绍	32
3.4	多算法授权简单介绍	32
3.5	烧写注意事项	33
3.6	FLASH 工程烧写补充说明	34
3.7	OTP 工程烧写补充说明	35
3.7.1	FLASH 调试模式与 OTP 模式区别	35
3.7.2	FLASH 调试模式与 OTP 模式的烧写	36

3.8	开发工具介绍	37
3.8.1	资源打包工具	37
3.8.2	音频转换工具	39
4	杰理工具文档介绍	45
5	中断系统	47
5.1	中断的注册与注销	47
5.1.1	函数 void HWI_Install(unsigned char index, unsigned int isr, unsigned char priority)	49
5.1.2	函数 void HWI_Uninstall(unsigned char index)	49
5.2	软中断说明	50
5.2.1	函数 void bit_set_swi(unsigned char index)	51
5.2.2	函数 void bit_clr_swi(unsigned char index)	51
5.3	软件不可屏蔽中断	51
5.3.1	函数 void _OS_ENTER_CRITICAL(u32 bit_list_l, u32 bit_list_h)	52
5.3.2	函数 void _OS_EXIT_CRITICAL(void)	52
5.3.3	函数 void vm_isr_response_index_register(u8 index)	52
5.3.4	函数 void vm_isr_response_index_unregister(u8 index)	52
5.3.5	函数 u32 get_vm_isr_response_index_l(void)	52
5.3.6	函数 u32 get_vm_isr_response_index_h(void)	53
5.4	硬件不可屏蔽中断	53
5.4.1	函数 void irq_unmask_set(u8 index)	53
5.4.2	函数 void irq_unmask_disable(u8 index)	53
6	系统 FLASH 写保护功能	55
6.1	A0 芯片如何添加写保护参数	55
6.1.1	flash 写保护工具说明	55
6.1.2	新增 flash 写保护参数与指令	56
6.1.3	Flash SR1、SR2 寄存器和写保护范围	57
6.1.4	写保护参数下载说明	62
6.2	内封 flash 芯片如何添加写保护参数 V1	62
6.2.1	配置烧写器获取内封 flash 写保护参数	63
6.2.2	配置强制下载工具获取内封 flash 写保护参数	63
6.3	系统 flash 写保护相关函数说明	63
6.3.1	函数 int norflash_set_write_protect(u8 enable_write_protect)	63
6.3.2	函数 u16 norflash_read_sr1_sr2(void)	64
6.3.3	函数 u32 flash_code_protect_callback(u32 offset, u32 len)	64
6.4	程序中开启与关闭写保护功能	64
6.4.1	常见问题说明	66
7	设备管理	67
7.1	设备注册表	68
7.2	设备函数	70
7.2.1	函数 int devices_init_api()	70
7.2.2	函数 void *dev_open(u32 index)	71
7.2.3	函数 u32 device_close(u32 index)	71
7.2.4	函数 void *device_obj(u32 index)	71
7.2.5	函数 u32 device_online(void)	71
7.2.6	函数 u32 device_status(u32 index, bool mode)	71
7.2.7	函数 int dev_bulk_read(void *_device, void *buf, u32 offset, u32 len)	72
7.2.8	函数 int dev_bulk_write(void *_device, void *buf, u32 offset, u32 len)	72
7.2.9	函数 int dev_ioctl(void *device, int cmd, u32 arg)	72
7.2.10	函数 int dev_close(void *device)	74
7.2.11	设备管理常见问题介绍	74

8	文件系统管理	75
8.1	文件系统介绍	76
8.2	文件系统使用一般流程	76
8.3	通用文件系统管理函数	77
8.3.1	函数 void fs_init(void)	77
8.3.2	函数 u32 fs_mount(void **ppvfs, void *device, void *type)	77
8.3.3	函数 u32 fs_openbypath(void *pvfs, void **ppvfile, const char *path)	78
8.3.4	函数 u32 fs_read(void *pvfile, void *buf, u32 len)	78
8.3.5	函数 u32 fs_seek(void *pvfile, u32 offset, u32 mode)	79
8.3.6	函数 u32 fs_file_close(void **ppvfile)	79
8.3.7	函数 u32 fs_fs_close(void **ppvfs)	80
8.3.8	函数 u32 fs_file_name(void *pvfile, void *name, u32 len)	80
8.3.9	函数 int fs_get_attrs(void *pvfile, void *pvfs_attr)	80
8.3.10	函数 int fs_ioctl(void *pvfile, int cmd, int arg)	80
9	sydfs 文件系统	81
10	norfs 文件系统	83
11	fat 文件系统	85
11.1	FAT 文件系统管理函数	86
11.1.1	函数 int fs_ftell(void *pvfile, void *parm)	86
11.1.2	函数 int vfs_get_fsize(void *pvfile, void *parm)	86
11.1.3	函数 struct vfscan *vfs_fscan_new(void *pvfs, const char *path, const char *arg, u8 max_depth, int (*callback)(void), struct vfscan *fsn, struct vfscan_reset_info *info)	86
11.1.4	函数 void fs_fscan_release(void *pvfs, struct vfscan *fs)	87
11.1.5	函数 int fs_get_folderinfo(void **pvfile, struct vfscan *fs, int *start_num, int *end_num)	87
11.1.6	函数 int fs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num)	87
11.1.7	函数 u32 vfs_file_name(void *pvfile, void *name, u32 len)	88
11.1.8	函数 int fs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg)	88
11.1.9	函数 int vfs_mk_dir(void *pvfs, char *folder, u8 mode)	88
11.1.10	函数 int vfs_delete_dir(void *pvfs, char *path)	89
11.1.11	函数 int vfs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num)	89
11.1.12	函数 u32 vfs_file_delete(void *pvfile)	89
11.1.13	函数 int vfs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg)	90
11.1.14	函数 int vfs_format(void **ppvfs, const char *dev_name, const char *type, u32 clust_size, u8 create_new)	90
11.1.15	函数 int vfs_ioctl(void *pvfile, int cmd, int arg)	91
12	解码接口	93
12.1	解码使用一般流程	94
12.1.1	在解码之前的设备以及文件的初始化	94
12.1.2	数据流获取	95
12.1.3	解码流程	96
12.1.4	DAC 数据处理说明	97
12.2	解码管理接口函数	99
12.2.1	函数 dec_obj *decoder_io(void *pfile, u32 dec_ctl, dp_buff *dbuff, u8 loop)	99
12.2.2	dec_obj *decoder_list(dec_data_stream *p_strm, u32 dec_ctl, dp_buff *dbuff, u8 loop, u32 output_sr)	100
12.2.3	函数 void irq_decoder_ret(dec_obj *obj, u32 ret)	102
12.3	解码功能控制函数	103
12.3.1	函数 void decoder_pause(dec_obj *obj)	103
12.3.2	函数 void decoder_stop(dec_obj *obj, DEC_STOP_WAIT wait)	103

12.3.3	函数 bool decoder_ff(dec_obj *obj, u8 step)	103
12.3.4	函数 bool decoder_fr(dec_obj *obj, u8 step)	103
12.3.5	函数 int decoder_time(dec_obj *p_dec)	104
12.4	各个格式的解码函数	104
12.4.1	统一介绍 decode_api	104
12.5	解码常见问题	104
12.5.1	播歌有卡音或者杂音如何查找问题	104
12.5.2	解码开始后声音变调问题	106
13	各解码格式配置说明	107
13.1	fla 解码格式	108
13.2	ump3 解码格式	108
13.3	midi 解码和 midi 琴解码格式	109
13.3.1	midi 驱动版本选择说明	109
13.3.2	midi 相关资源文件说明	112
13.3.3	midi 相关功能配置说明	112
13.4	a 解码格式	113
13.5	标准 MP3 解码格式	113
13.6	WAV 解码格式	113
13.7	OPUS 解码格式	114
13.8	ADPCM-IMA 解码格式	114
13.9	SBC/MSBC/USBC 解码格式	115
13.10	speex 解码格式	117
13.11	jla_lw 解码格式	117
14	编码接口	119
14.1	编码功能控制函数	119
14.1.1	编码使用一般流程	120
14.1.2	文件编码功能控制接口函数	120
14.2	函数 void encoder_file(u32(*fun)(void *, void *, void *), void *pfile)	120
14.3	函数 void stop_encode_file(enc_obj *p_enc_obj, u32 dlen)	120
14.3.1	音频传输编码功能控制接口函数	120
14.4	函数 enc_obj *audio2rf_encoder_io(u32(*enc_fun)(void *, void *, void *), void *p_queue_ops, void *ops)	120
14.5	函数 u32 audio2rf_encoder_start(enc_obj *obj)	121
14.6	函数 u32 rf_enc_output(void *priv, u8 *data, u16 len)	121
14.7	函数 void audio2rf_encoder_stop(queue_obj *p_queue, ENC_STOP_WAIT wait)	121
15	各编码格式配置说明	123
15.1	标准 MP2 编码	124
15.2	ump2 编码	124
15.3	A 格式编码	125
15.4	OPUS 格式编码	125
15.5	ADPCM_IMA 格式编码	125
15.6	sbc/msbc/usbc 编码格式	126
15.7	speex 编码格式	128
15.8	JLA_LW 编码格式	129
16	MIDI	131
16.1	midi 驱动版本选择说明	131
16.1.1	midi 驱动版本选择步骤	132
16.2	MIDI 基本功能配置	136
17	JL_MIO 功能	139
17.1	JLmio 一般流程	139

17.1.1	使用 JLMio.exe 创建 mio 文件	139
17.1.2	使用打包工具将音频文件与所制成的 mio 文件放于同一目录下	140
17.1.3	在 download.bat 中添加相应 mio 文件夹	140
17.1.4	在 app_music.c 中添加相应 mio 文件夹	141
17.1.5	在 mio_api.c 中将相应宏开关打开	142
17.2	JLMio 接口介绍	143
17.2.1	void mio_a_pwm_init(u32 chl)	143
17.2.2	void mio_a_pwm_run(u32 chl, u32 pwm_var)	144
17.2.3	void mio_a_io_init(u32 mask)	144
17.2.4	void mio_a_io_run(u32 mask, u32 io_ver)	144
18	OTP 小音箱应用流程	145
18.1	音乐解码模式	145
18.1.1	函数 bool music_play_control(MBOX_MUSIC_CMD cmd, u32 index, DEC_STOP_WAIT mode)	145
18.1.2	函数 bool mbox_simple_play_file(u8 dindex, u32 findex)	146
18.1.3	函数 u32 device_mount(play_control *ppctl)	146
18.1.4	函数 void fs_dev_close(void)	146
18.2	2 USB 从机模式	147
18.3	LINEIN 模式	147
19	FLASH 工程应用流程	149
19.1	音乐解码模式	149
19.2	USB 从机模式	149
19.3	LINEIN 模式	149
19.4	MIDI 解码 / MIDI 琴模式	150
19.5	玩具多路解码模式	150
19.5.1	函数 u32 simple_dev_fs_mount(play_control *ppctl)	150
19.5.2	函数 u32 simple_dev_fs_close(play_control *ppctl)	151
19.5.3	函数 u32 simple_play_file_byindex(play_control *ppctl)	151
19.5.4	5.4 函数 u32 get_decoder_status(dec_obj *obj)	151
19.5.5	函数 static bool simple_switch_device(play_control *ppctl)	151
19.5.6	函数 static bool simple_next_dir(play_control *ppctl)	151
19.6	录音模式	152
19.6.1	函数 int encode_start(Encode_Control *obj)	152
19.6.2	函数 void encode_stop(Encode_Control *obj)	152
19.6.3	函数 int fatfs_enc_file_create(Encode_Control *obj)	153
19.6.4	函数 dec_obj *fatfs_enc_file_decode(Encode_Control *obj, u16 dec_type)	153
19.6.5	函数 int norfs_enc_file_create(Encode_Control *obj)	153
19.6.6	函数 dec_obj *norfs_enc_file_decode(Encode_Control *obj, u16 dec_type)	153
19.7	LoudSpeaker 扩音模式	154
19.7.1	函数 void audio_adc_speaker_start(u32 sr)	154
19.7.2	函数 void audio_adc_speaker_reless(void)	155
20	AUDIO 公共	157
20.1	audio 采样率列表	157
20.2	audio 工作模式列表	158
20.2.1	AUDIO DAC 工作模式	158
20.2.2	AUDIO ADC - MIC 工作模式	158
20.2.3	AUDIO ADC - LINEIN 工作模式	158
20.3	《app_config.c》中的开机默认配置	158
20.4	Audio 公共配置	159
21	AUDIO DAC	161
21.1	《app_config.c》中的开机默认配置	162

21.1.1	Audio 公共部分	162
21.1.2	AUDIO DAC 部分配置	162
21.2	《audio_dac.h》中的开机默认配置	163
21.2.1	工作模式	164
21.2.2	位宽	164
21.3	AUDIO DAC 示例	164
21.3.1	开打 AUDIO DAC 硬件	165
21.3.2	关闭 AUDIO DAC 硬件	166
21.3.3	关闭 AUDIO DAC 与 AUDIO 硬件	166
21.4	AUDIO 全关闭示例	167
21.5	AUDIO DAC 管理架构	167
21.5.1	AUDIO DAC 驱动模式	168
21.5.2	AUDIO DAC 共 VCM0 模式	169
21.6	AUDIO DAC 相关函数	170
21.6.1	函数 void dac_mode_init(void)	170
21.6.2	函数 void dac_init_api(u32 sr)	170
21.6.3	函数 dac_sr_api(u32 sr)	170
21.6.4	函数 void dac_off_api(void)	171
21.6.5	函数 u8 stereo_dac_vol(char set, u8 vol_l, u8 vol_r)	171
21.6.6	函数 u8 dac_vol(char set, u8 vol)	171
21.6.7	函数 bool dac_mute(bool mute)	171
21.6.8	函数函数 bool regist_dac_channel(void *psound, void *kick)	171
21.6.9	函数函数 bool unregist_dac_channel(void *psound)	172
21.6.10	函数 void fifo_dac_fill(u8 *buf, u32 len, AUDIO_TYPE type)	172
21.6.11	函数 void audio_dac_analog_vol(AUDIO_DAC_ANA_VOL vol_l, AUDIO_DAC_ANA_VOL vol_r)	172
21.6.12	函数 void audio_amux_analog_vol(AUDIO_DAC_AUMX_VOL vol_l, AUDIO_DAC_AUMX_VOL vol_r)	172
22	AUDIO ADC	173
22.1	《app_config.c》中的开机默认配置	174
22.1.1	Audio 公共部分	174
22.1.2	AUDIO ADC 部分配置	174
22.2	AUDIO ADC 示例	176
22.2.1	打开 AUDIO ADC 硬件	177
22.2.2	停止 AUDIO ADC 运行	177
22.2.3	关闭 AUDIO ADC 硬件	177
22.2.4	关闭 AUDIO 以及 AUDIO ADC 硬件	177
22.3	AUDIO 全关闭示例	178
22.4	AUDIO ADC 管理架构	179
22.4.1	AUDIO ADC 驱动模式	180
22.5	AUDIO ADC 相关函数	181
22.5.1	函数 void audio_adc_mode_init(void)	181
22.5.2	函数 u32 audio_adc_init_api(u32 sr, ADC_MODE mode, u32 throw_sp_num)	182
22.5.3	函数 void audio_adc_off_api(void)	182
22.5.4	函数 bool regist_audio_adc_channel(void *psound, void *kick)	182
22.5.5	函数 bool unregist_audio_adc_channel(void *psound)	182
22.5.6	函数 void fill_audio_adc_fill(u8 *buf, u32 len)	183
23	软硬件淡入淡出	185
23.1	淡入淡出效果配置	185
23.2	淡入淡出相关函数	186
23.2.1	void dac_fade(void)	186
23.2.2	dac_fade_in_api()	186

23.2.3	dac_fade_out_api()	186
24	软件 EQ (PCM_EQ)	189
24.1	1 配置参数说明	189
24.2	2 相关函数	190
24.2.1	2.1 函数 void *link_pcm_eq_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u32 channel)	190
24.2.2	2.2 函数 void *pcm_eq_api(void *obuf, u32 sr, u32 channel, void **ppsound)	190
24.3	3 EQ 工具使用	190
24.3.1	3.1 EQ 工具使用介绍	190
24.3.2	3.2 SDK 配置说明	192
25	完整的浮点 EQ(audio_eq)	195
25.1	浮点 EQ 配置 (Flash 工程)	195
25.1.1	旧版本 EQ 模式管理	197
25.1.2	新版本 EQ 模式管理	197
25.2	浮点 EQ 相关接口 (Flash 工程)	198
25.2.1	函数 void *link_eq_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u8 ch)	198
25.2.2	函数 void *link_eq_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u8 ch)	198
25.2.3	函数 int eq_mode_sw(void *parm)	198
25.2.4	函数 u8 get_eq_mode_total_num(void *parm)	198
25.2.5	函数 u8 get_eq_mode_total_num(void *parm)	198
25.3	在线 EQ 调试助手使用一般流程	199
25.3.1	SDK 里开启 PC 模式和使用在线 EQ 调试工具宏定义	199
25.3.2	小机进入 PC 模式后打开使用 AD16N 在线 EQ 调试助手	199
25.3.3	在音效调节界面双击点击 musicEQ 进入在线调节 EQ 曲线界面	201
25.3.4	把生成的.bin 文件下载到小机上	202
25.3.5	SDK 具体使用 EQ 样例	203
26	简单浮点 EQ (audio_eq)	207
26.1	简单 EQ 配置 (OTP 工程)	207
26.2	简单 EQ 相关接口 (OTP 工程)	208
26.2.1	函数 void audio_eq_init_api(void)	208
26.2.2	函数 void audio_eq_open_api(u8 ch, u16 sr)	208
26.2.3	函数 void audio_eq_switch_tab(void)	208
26.2.4	函数 void audio_eq_close_api(void)	208
27	混响音效 (echo)	209
27.1	配置参数说明	209
27.2	混响音效相关函数	210
27.2.1	函数 void *link_echo_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)	210
27.2.2	函数 void *echo_api(void *obuf, u32 sr, void **ppsound)	211
28	移频啸叫抑制 (pitchshifter_howling)	213
28.1	配置参数说明	213
28.2	相关函数	214
28.2.1	函数 void *link_pitchshift_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)	214
28.2.2	函数 void *pitchshift_howling_api(void *obuf, u32 sr, void **ppsound)	214
29	自适应陷波啸叫抑制 (notch_howling)	215
29.1	配置参数说明	215
29.2	相关函数	216

29.2.1	函数 void *link_notch_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)	216
29.2.2	函数 void *notch_howling_api(void *obuf, u32 sr, void **ppsound)	216
30	变音音效 (vo_pitch)	217
30.1	变音音效参数配置	217
30.2	变音音效算法链路链接函数	218
30.2.1	函数 sound_out_obj *link_voice_pitch_sound(sound_out_obj *p_curr_sound, void *p_dac_cbuf, void **pp_effect, u32 cmd)	218
30.2.2	函数 void *voice_pitch_api(void *obuf, u32 cmd, void **ppsound)	219
30.2.3	函数 void rap_callback(void *priv, int pos)	219
30.2.4	函数 void rap_reopen(void)	219
30.3	变音音效算法参数函数	219
30.4	两种 RAP 音效的区别	220
30.5	RAP 和 RAP_REALTIME 模式文件替换说明	220
31	实时变速变调 (SPEED, 用于扩音)	223
31.1	配置参数	223
31.2	相关函数	224
31.2.1	函数 void *link_speed_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 in_sr)	224
32	歌曲变速变调 (SONG_SPEED, 用于解码)	225
32.1	配置参数	225
32.2	相关函数	226
32.2.1	函数 void *link_song_speed_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 in_sr, u8 ch)	226
32.2.2	函数 void *song_speed_phy(void *obuf, PS69_CONTEXT_CONF *psp_parm, void **ppsound)	226
33	混合变音音效 (voicechanger)	227
33.1	可选音效	227
33.2	配置参数说明	228
33.3	相关函数接口	228
33.3.1	函数 void *link_voice_changer_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 in_sr)	228
33.3.2	函数 void *voice_changer_api(void *obuf, u32 sr, void **ppsound)	229
33.3.3	函数 void update_voice_changer_parm(VOICECHANGER_AV_PARM *new_vc_parm, VOICESYN_AV_PARM *new_vsyn_ctrl)	229
34	音频降噪算法 (ANS)	231
34.1	涉及文件	232
34.2	相关参数	233
34.3	相关函数	234
34.3.1	void *link_ans_sound(void *p_sound_out, void *p_ans_obuf)	234
34.3.2	int ans_run(void *hld, short *inbuf, int len)	235
35	频谱分析算法	237
35.1	涉及文件	237
35.2	使用流程	238
35.2.1	在解码链路中使用频谱分析算法	238
35.2.2	在扩音链路中使用频谱分析算法	238
35.2.3	硬件 EQ 复用说明	239
35.2.4	获取频点幅度	239
35.3	相关函数	239

35.3.1	void *link_spectral_anal_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u8 ch)	239
35.3.2	void getFollowdB_api(void)	240
36	系统记忆存储	241
36.1	系统记忆存储接口	242
36.1.1	函数 int sysmem_init_api(u32 mem_addr, u32 mem_size)	242
36.1.2	函数 int sysmem_read_api(u32 id, u8 *data_buf, u16 len)	242
36.1.3	函数 int sysmem_write_api(u32 id, u8 *data_buf, u16 len)	243
36.1.4	函数 void sysmem_pre_erase_api(void)	243
37	设备记忆 (OTP 工程)	245
37.1	设备记忆存储接口	246
37.1.1	函数 u32 device_memory_control(u8 devmm_index, u8 *buf, u32 offset, u32 len, char set)	246
38	VM 掉电存储详细说明	247
38.1	VM 记忆的基本原理	248
38.2	VM 记忆的系统局限性	248
38.3	NEW VM 的预擦除	248
38.4	NEW VM 常见 BUG	248
38.5	VM 掉电存储使用说明	249
38.6	VM 掉电存储版本简单说明	249
38.7	新旧 VM 文件组成及版本选择	250
38.8	VM 相关参数介绍	250
38.8.1	ID 号 (vm_api.h)	251
38.8.2	读/写入数据长度	251
38.8.3	vm 存储区域大小	252
38.8.4	新版 VM 格式整理时缓存空间设置 new_vm_buff (new_vm.h)	252
38.8.5	新版 vm 格式整理后是否立即擦除旧半区区域功能	253
38.8.6	新版 vm 多段读功能开关	253
38.9	VM 掉电记忆存储接口	254
38.9.1	函数 u32 syscfg_vm_init(u32 mem_addr, u32 mem_size)	254
38.9.2	函数 u32 sysmem_write_api(u32 id, u8 *data_buf, u32 len)	254
38.9.3	函数 u32 sysmem_read_api(u32 id, u8 *data_buf, u32 len)	254
38.9.4	函数 u32 sysmem_pre_erase_api(void)	255
38.9.5	函数 void nvm_pre_erasure_next(NEW_VM_OBJ *p_nvm, u16 using_next, u16 idle_next)	255
38.10	VM 常见问题介绍	255
38.10.1	VM 简单使用 demo:	255
38.10.2	调整 VM 区域大小设置:	256
38.10.3	新版 VM 在关机或者掉电后可能存在数据丢失 BUG:	257
38.10.4	VM 格式整理原理:	257
39	消息与事件	259
39.1	消息与事件使用一般流程	260
39.2	消息 (msg) 函数	261
39.2.1	函数 void message_init()	261
39.2.2	函数 int post_msg(int argc, ...)	261
39.2.3	函数 int get_msg(int len, int *msg)	262
39.2.4	函数 void clear_all_message(void)	262
39.3	事件 (EVENT) 函数	262
39.3.1	函数 void event_init(void)	264
39.3.2	函数 u32 event2msg_api(u32 event)	264
39.3.3	函数 int post_event(int event)	265
39.3.4	函数 bool get_event_status(u32 event)	265

40 按键系统 Key	267
40.1 按键类型介绍	268
40.1.1 IO 按键介绍	268
40.1.2 AD 按键介绍	269
40.1.3 红外按键介绍	269
40.1.4 触摸按键介绍	270
40.2 按键接口介绍	270
40.2.1 函数 void key_init(void)	270
40.2.2 函数 void key_table_sel(void *msg_filter)	270
40.2.3 函数 void key_scan(void)	270
40.3 按键消息配置	271
41 低功耗与唤醒	273
41.1 低功耗相关参数配置	273
41.2 唤醒相关参数配置	274
41.3 低功耗相关函数	275
41.3.1 函数 void music_idle_deal(void) (OTP 工程)	275
41.3.2 函数 void app_powerdown_deal(u8 is_busy) (FLASH 工程)	275
41.3.3 函数 void sys_softoff(void)	275
41.3.4 函数 void board_set_soft_poweroff(void)	276
41.3.5 函数 void sys_power_down(u32 usec)	276
41.4 唤醒相关函数	276
41.4.1 函数 u8 is_wakeup_source(enum WAKEUP_REASON index)	276
41.4.2 函数 void wakeup_callback(u8 index)	276
42 LCD 段码屏驱动说明	277
42.1 简介	277
42.2 使用流程	277
42.3 与低功耗的配合使用和注意事项	278
43 RTC 功能	279
43.1 RTC 功能介绍	279
43.2 RTC 配置参数	280
43.3 RTC 接口函数	281
43.3.1 void rtc_dev_init(const struct rtc_config_init *arg);	281
43.3.2 void read_current_time();	281
43.3.3 void write_clock_time(u16 year, u8 month, u8 day, u8 hour, u8 min, u8 sec);	281
43.3.4 void rea_alarm_time();	281
43.3.5 void write_alarm_time(u16 year, u8 month, u8 day, u8 hour, u8 min, u8 sec);	281
43.3.6 void time_wakeup_set(u8 index, u8 en);	282
43.3.7 void rtc_alarm_en(u32 en);	282
43.4 RTC 简单使用示例	282
44 Reserved_Area 区域说明	283
44.1 认证码相关信息	283
44.2 认证码功能 isd_config_ini.c 配置	283
44.3 认证码信息接口	285
44.3.1 void rsv_auth_analysis(void)	285
44.4 认证码文件说明	285
45 异常	287
45.1 异常分析	288
45.2 常见异常	290
46 双备份升级	291

46.1	支持双备份升级的芯片以及对应 SDK 版本	291
46.2	SDK 相关配置	291
46.3	相关接口说明	292
46.3.1	函数 u32 jifs_get_idle_bank_info(u32 *bank_addr, u32 *bank_size)	292
46.3.2	函数 u32 jifs_updata_dual_bank_info(u32 bank_addr, u16 data_crc)	293
46.3.3	函数 u32 jifs_check_dual_bank_info(u32 update_bank_addr)	293
46.4	常见客户问题	293
46.5	双备份升级接口使用示例	294
47	Indices and tables	295

开源社区仓库链接:

<https://gitee.com/Jieli-Tech/fw-AD16N>

<https://github.com/Jieli-Tech/fw-AD16N>

开源社区交流群:

AD14/AD15/AD...

528人

服务



扫一扫群二维码，立刻加入该群。

开源社区钉钉群二维码

文档作者: 刘杰

本页介绍历史版本

1.1 v1.2.0 说明

日期：2025 年 9 月 16 日

1、添加 **midi** 驱动版本选择说明，**midi** 开源并支持自动切换 4 字节和 2 字节地址的库：

- “[midi 驱动版本选择说明](#)”

2、添加外挂 **flash** 下载资源文件流程：

- “[通过二次打包将文件下载至外挂 flash 流程](#)”

3、新增双备份升级章节：

- “[双备份升级](#)”

4、新增中断系统章节：

- “[中断系统](#)”

5、新增 **ANS** 音频降噪算法章节：

- “[音频降噪算法 \(ANS\)](#)”

6、新增频谱分析算法章节：

- “[频谱分析算法](#)”

7、更新编码和解码部分章节：

- 更新接口说明” [解码接口](#)”
- 更新接口说明” [编码接口](#)”
- 添加各芯片对应 SDK 对各个解码格式支持情况的说明” [各解码格式配置说明](#)”

- 添加各芯片对应 SDK 对各个编码格式支持情况的说明” 各编码格式配置说明”

8、更新 VM 章节说明：

- “VM 掉电存储详细说明”

9、更新环境介绍章节：

- 添加 cpu_config.c 章节说明” cpu 差异相关功能配置文件《cpu_config.c》介绍”
- 添加 SDK 芯片资源占用情况说明” SDK 芯片资源占用情况”

10、更新 RTC 章节：

- 更新 RTC 章节备注和接口说明” RTC 功能”

11、更新系统 flash 写保护章节：

- 系统 FLASH 写保护功能说明” 系统 FLASH 写保护功能”

1.2 v1.1.1 说明

日期：2023 年 7 月 13 日

- 订正解码数据输出接口 mp_output 代码-> “数据流获取”；
- 订正低功耗 flash_pg_keep 配置说明->” 低功耗相关参数配置 “；

1.3 v1.1.0 说明

日期：2023 年 4 月 26 日

1、系统频率与系统 flash 波特率修改：

- 修改章节：《环境介绍》-> “SDK 工程配置”-> “应用配置文件《app_config.h》介绍”-> “时钟配置”；
- 添加章节：《环境介绍》->” AD16N 超频说明 “；

2、完整浮点 EQ 版本更新修改：

- 修改章节：《完整的浮点 EQ》-> “完整的浮点 EQ(audio_eq)”；
- 修改章节：《完整的浮点 EQ》-> “浮点 EQ 配置 (Flash 工程)”；
- 修改章节：《完整的浮点 EQ》-> “浮点 EQ 相关接口 (Flash 工程)”；
- 添加章节：《完整的浮点 EQ》-> “浮点 EQ 配置 (FLASH 工程)”-> “旧版本 EQ 模式管理”；
- 添加章节：《完整的浮点 EQ》-> “浮点 EQ 配置 (FLASH 工程)”-> “新版本 EQ 模式管理”；
- 添加章节：《完整的浮点 EQ》-> “在线 EQ 调试助手使用一般流程”；

3、LCD 章节更新修改：

- 添加说明：《LCD 段码屏驱动说明》-> “与低功耗的配合使用和注意事项”；

4、外挂 flash 章节更新修改：

- 添加外挂 flash 单线说明-> “外挂 FLASH 配置 (FLASH Configuration)”；

5、新增 sys_momery 系统记忆章节：

- 新增 sys_memory 说明-> “系统记忆存储”;
 - vm 读写接口由 sys_memory 管理-> “VM 掉电记忆存储接口”;
- 6、新增二级密钥 V2.0 系统:
- 二级密钥 V2.0 简单介绍-> “二级密钥 V2.0 简单介绍”;

1.4 v1.0.0 说明

日期: 2023 年 3 月 30 日

建立文档, 文档内容包括:

环境介绍
下载目录&工具使用环境介绍
设备管理
文件系统管理
解码接口
编码接口
JL_MIO 功能
OTP 小音箱应用流程
FLASH 工程应用流程
AUDIO 公共
AUDIO DAC
AUDIO ADC
软硬件淡入淡出

混响音效 (echo)
移频啸叫抑制 (pitchshifter_howling)
自适应陷波啸叫抑制 (notch_howling)
简单EQ
完整的浮点EQ
变音音效 (vo_pitch)
实时变速变调 (SONG_SPEED, 用于扩音)
歌曲变速变调 (SONG_SPEED, 用于解码)
FLASH 硬件写保护功能
设备记忆 (OTP 工程)
VM 掉电存储 (FLASH 工程)
消息与事件
按键系统key
低功耗与唤醒
LCD 段码屏驱动说明
RTC 功能

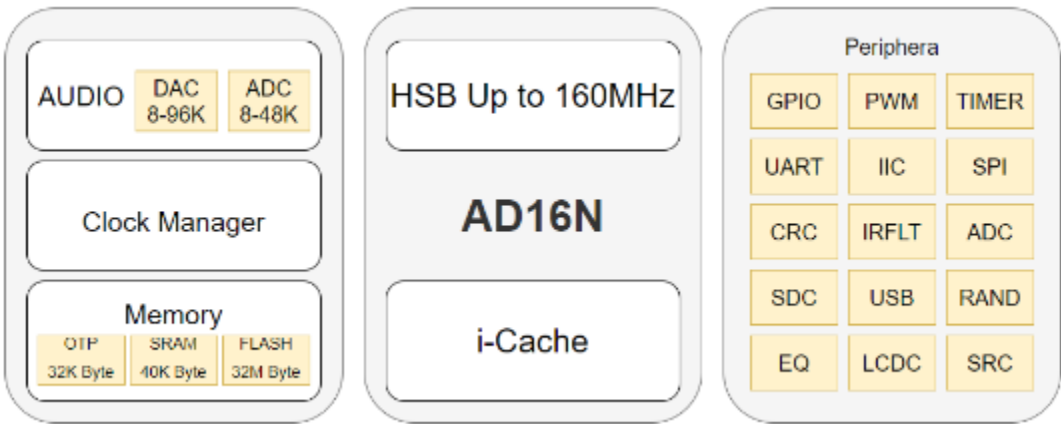
本章节主要介绍软硬件开发环境。

2.1 硬件环境介绍

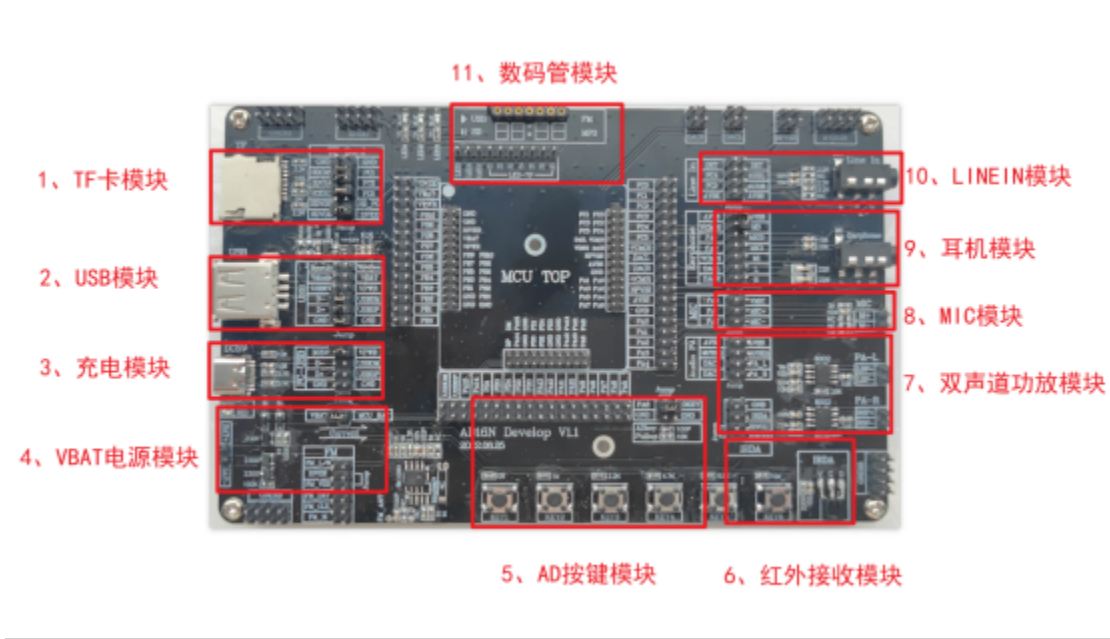
AD16N FLASH 方案硬件开发环境分为开发顶板和开发底板。

2.1.1 AD16N 硬件开发环境

AD16N 芯片框图如下图所示：



AD16N 使用的硬件环境开发板由底板和顶板两部分组成。
底板包含的模块如下图所示：



其中模块：

模块	模块详情
TF 卡模块	可驱动一线模式的 TF 卡，支持 IO 供电以及内部 SD_PG 供电；
USB 模块	可连接 U 盘或者 PC 从机，或通过 VPWR 给开发板供电或充电；U 盘由 VBAT 经过升压后供电，具体请参考开发板原理图；
TYPE C 模块	可连接 PC 从机，或通过 VPWR 给开发板供电或充电；
VBAT 电源模块	可正极接 VBAT+，负极接 VBAT-，并短接 VBAT 到 MCU_VBAT；注：VBAT 供电电压不可超过 4.3V
AD 按键模块	可支持 6 个 AD 按键，外接 10k 上拉电阻；
红外接收模块	可用于接收红外信号，需将 IRVDD 与 IOVDD 短接；
双声道功放模块	可支持双声道功放输出以及功放 mute；
MIC 模块	可连接麦克风，OTP 方案暂不支持 MIC 输入；
耳机模块	可连接 3.5mm 耳机；
Linein 模块	可支持双声道 Linein 以及检测；
数码管模块	可连接 7 脚 MP3 数码管；

顶板包含的模块如下图所示：

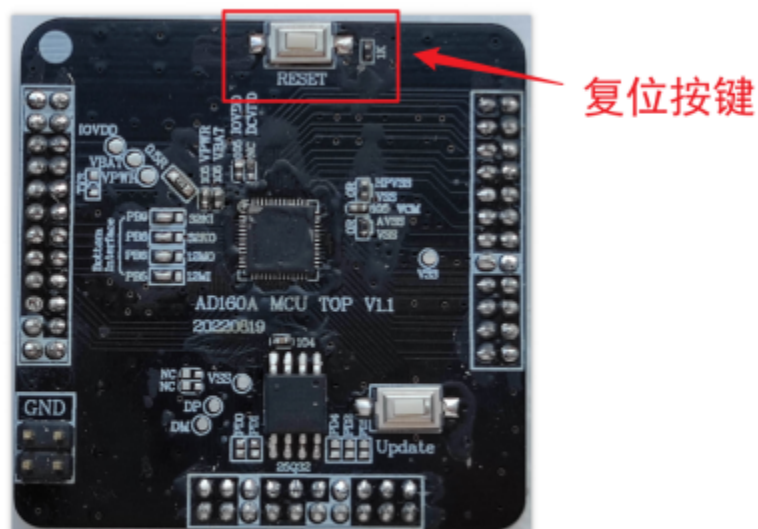


图 1 DAD16N 核心板

按下顶板的“复位按键”后，系统会进行复位。

在线调试时，芯片 OTP 需要烧写 usb bootloader (usbboot.bin) 后才能使用 USB 升级。

可以通过强制升级工具升级。

2.2 快速上手

2.2.1 编译下载介绍

本节将介绍如何打开工程，编译下载后进入音乐模式：

使用 CodeBlocks 打开工程（两种办法）

1. 直接在目录下找到 CBP 文件，用 codeblocks 打开；

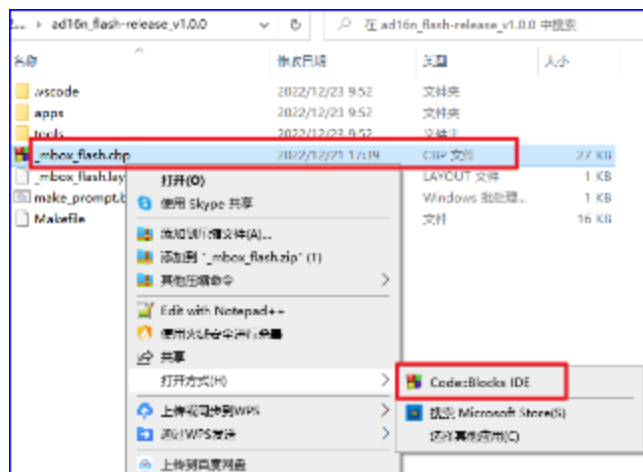


图 2.1 打开 CodeBlock 工程方式一

2. 打开 codeblocks 后, File->Open-> 找到选中 CBP 文件-> 打开;

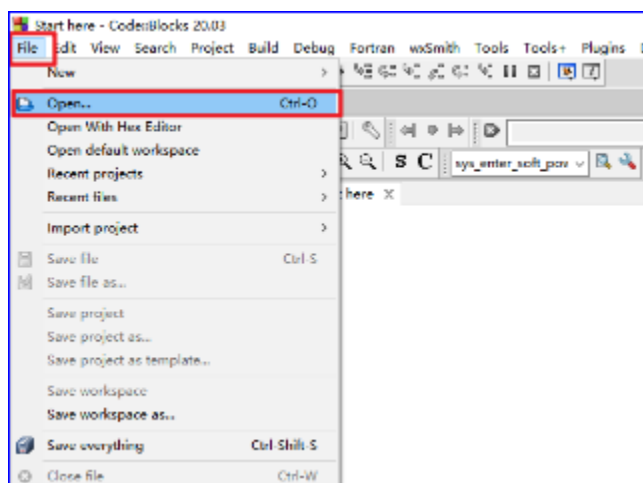


图 2.2 打开 CodeBlock 工程方式二

打开工程后找到 main.c 文件

打开工程后会显示在左侧的 Management, 点击打开 Sources -> apps -> app -> bsp -> start -> main.c, 里面的 c_main 函数即是我们的工程的主函数。

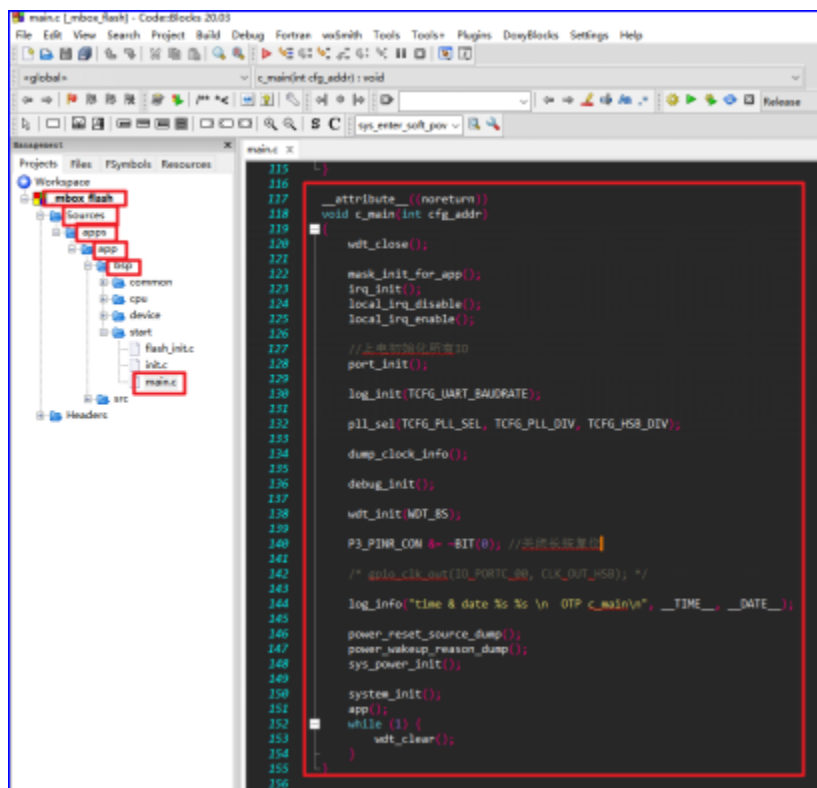


图 2.3 打开 main.c 文件

USB 强制下载工具连接开发板

- 第一步：将 USB 强制下载工具的 7 号拨片拨至 ON，其余拨片拨至 OFF；
第二步：短按下载器的按键，让闪烁的绿灯灭灯，即进入下载模式；

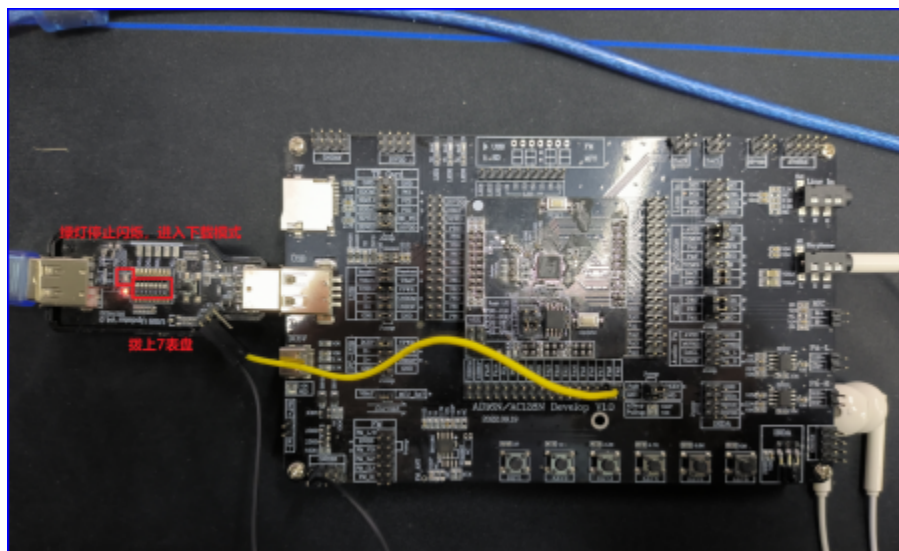


图 2.4 USB 强制下载工具使用

编译下载程序并运行

点击 codeblocks 的编译 (Build), 如果有改动 build_option 或者库文件, 需要点击全编译 (Rebuild);

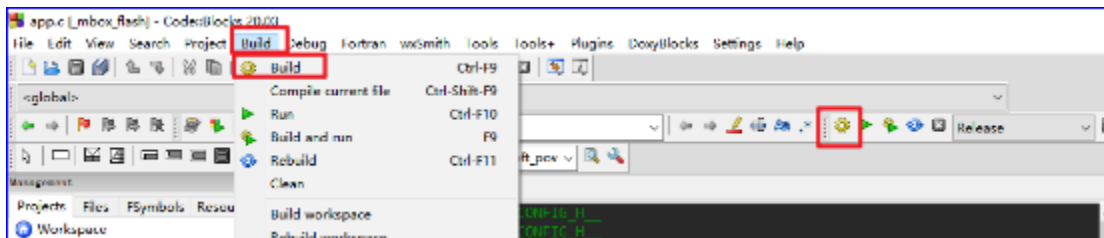


图 2.5 CodeBlocks 编译

下方 logs & others (快捷键 F2) 会显示下载结果, 有下载完毕即下载成功;

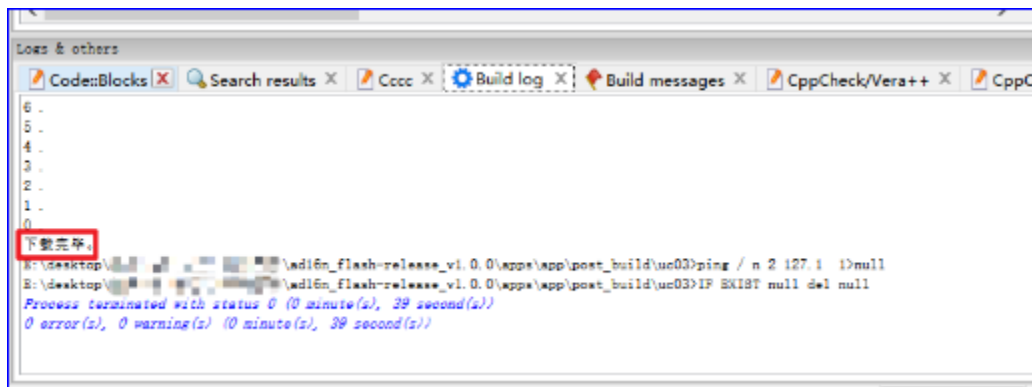


图 2.6 CodeBlocks 编译下载成功

使用 Type-c 供电, 接上 SDK 默认打印口 PA6, 观察打印信息, 开机打印信息如下, 即小机运行成功;

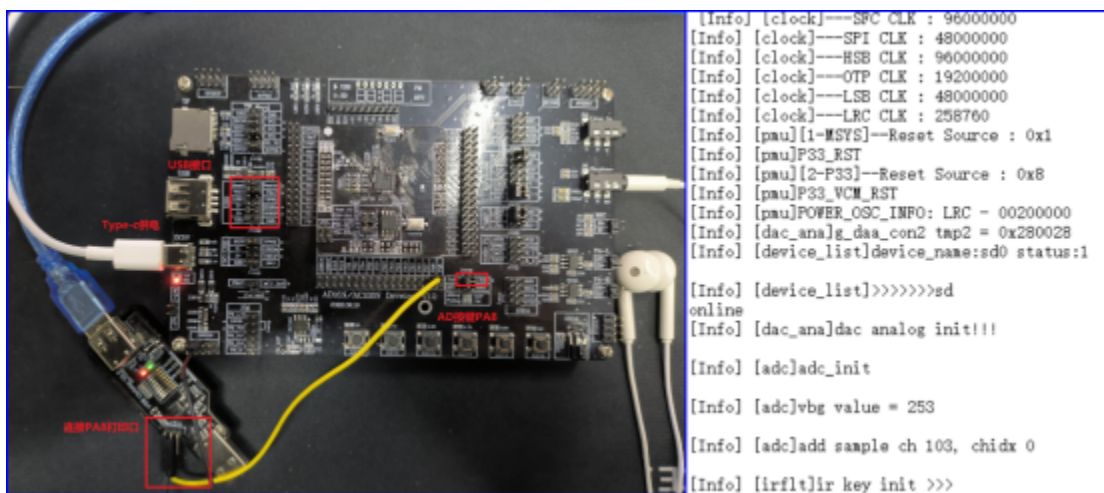


图 2.7 小机运行打印

2.2.2 编译下载报错说明

如下图所示，AD16N SDK 需使用杰理 PI32V2 r1 large lto Compiler 编译器进行编译：

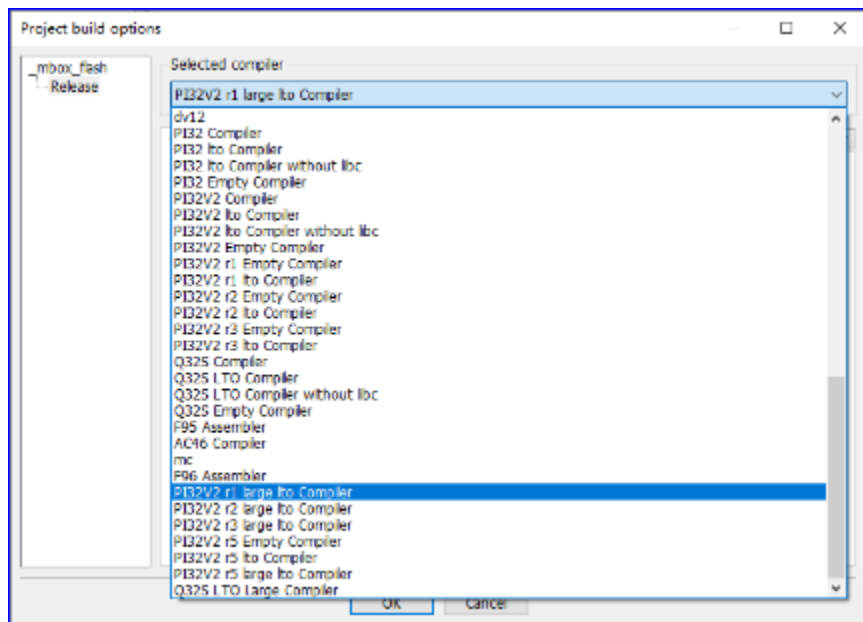


图 2.8 AD16N CodeBlocks 编译器选择

首次编译时若出现如下图所示的报错信息，需保证将 CodeBlocks 软件安装在 C 盘目录下，并将 JL 工具链和编译器更新至最新版本即可；

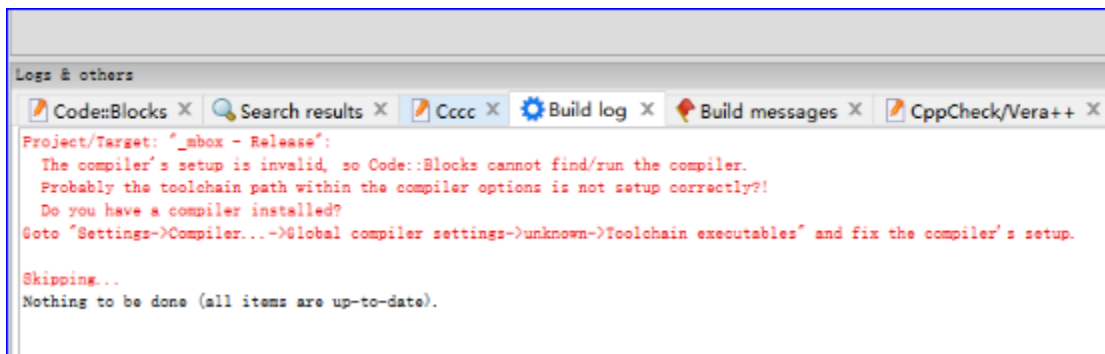


图 2.9 AD16N CodeBlocks 编译器报错信息

2.3 SDK 芯片资源占用情况

2.3.1 I/O 口占用情况

PA 口 I/O	默认占用情况
PA0	MIC 偏置电压 (MICBIAS) 输出引脚
PA1	单端 MIC(MIC-IN) 输入引脚 / 差分 MIC P 端输入引脚 / IOKEY0 按键检测引脚
PA2	差分 MIC N 端输入引脚 / IOKEY1 按键检测引脚
PA3	MATRIX 矩阵按键检测引脚
PA4	MATRIX 矩阵按键检测引脚
PA5	默认打印脚 / MATRIX 矩阵按键检测引脚
PA6	IRKEY 红外按键检测引脚
PA7	TOUCH 触摸按键检测引脚
PA8	ADKEY 按键检测引脚 / 默认唤醒引脚 / TOUCH 触摸按键检测引脚
PA9	UI_PORT6
PA10	UI_PORT5
PA11	IOKEY2 按键检测引脚 / UI_PORT4
PA12	MATRIX 矩阵按键检测引脚 / UI_PORT3
PA13	MATRIX 矩阵按键检测引脚 / UI_PORT2
PA14	MATRIX 矩阵按键检测引脚 / UI_PORT1
PA15	IOKEY3 按键检测引脚 / UI_PORT0

PB 口 I/O	占用情况
PB0	外挂资源 flash_cs 片选脚
PB1	外挂资源 flash_clk 引脚
PB2	外挂资源 flash_do 引脚
PB3	外挂资源 flash_di 引脚
PB4	暂无使用
PB5	暂无使用
PB6	暂无使用
PB7	暂无使用
PB8	外挂 32K 晶振作为 RTC 时钟源
PB9	外挂 32K 晶振作为 RTC 时钟源
PB10	暂无使用

PC 口 I/O	占用情况
PC0	sd 卡 dat 引脚
PC1	sd 卡 cmd 引脚
PC2	sd 卡 clk 引脚
PC3	暂无使用
PC4	模拟 Linein 输入 L 端
PC5	模拟 Linein 输入 R 端

2.3.2 软中断占用情况

软中断	默认占用情况
SOFT0_IRQ	解码中断使用
SOFT1_IRQ	编码中断使用
SOFT2_IRQ	录音写数据
SOFT3_IRQ	stream_frame 数据流使用

2.3.3 定时器占用情况

定时器	默认占用情况
TIMER0	IRKEY 红外按键
TIMER1	powerdown 延时使用
TIMER2	暂无使用
TIMER3	暂无使用

2.3.4 串口占用情况

串口	默认占用情况
UART0	SDK 打印
UART1	暂无使用
UART2	暂无使用

2.4 SDK 工程配置

目前 AD16N 发布的 SDK 共有两个工程，分别是 FLASH 工程和 OTP 工程：

- flash 应用工程：

该工程有小音箱以及玩具功能，能实现音箱解码、USB 从机、Linein、midi、录音以及玩具内置 flash 解码功能；同时拥有 AD、红外、矩阵按键、低功耗、EQ、VM 等功能。

.vscode	2022/12/20 8:53	文件夹	
apps	2022/12/20 8:53	文件夹	
obj	2022/12/20 8:54	文件夹	
tools	2022/12/20 8:53	文件夹	
_mbox_flash.cbp	2022/12/19 17:09	project file	27 KB
_mbox_flash.depend	2022/12/20 10:34	DEPEND 文件	47 KB
make_prompt.bat	2022/12/19 17:09	Windows 批处理...	1 KB
Makefile	2022/12/19 17:09	文件	16 KB
tmp_app2022.txt	2022/12/20 10:34	TXT 文件	2 KB

图 3.1 AD16N flash 工程目录

- mbox_otp 应用工程：
该工程有小音箱功能，能实现音箱解码、USB 从机和 Linein 功能；同时拥有 AD、红外按键、低功耗、EQ、设备记忆等功能。

 .vscode	2022/9/6 22:06	文件夹
 apps	2022/9/6 10:02	文件夹
 tools	2022/9/6 10:02	文件夹
 _mbox.cbp	2022/9/6 10:02	project file
 make_prompt.bat	2022/9/6 10:02	Windows 批处理...
 Makefile	2022/9/6 10:02	文件

图 3.2 AD16N otp 工程目录

2.4.1 应用配置文件《app_config.c》介绍

应用配置文件《app_config.c》能配置与系统相关的配置：

- 1、中断优先级；
- 2、config_asser：内核异常打印。

① 0：关闭内核异常打印；

② 1：开启内核异常打印；

注：生产的工程要关闭内核异常打印，内核异常打印会关闭看门狗并且while(1)停留！

备注：打开内核异常打印后，系统出现异常会关闭看门狗并且在异常中断里面 while(1) 停留不再退出！
所以，生产的工程要关闭内核异常打印！避免出现异常卡死！！
所以，生产的工程要关闭内核异常打印！避免出现异常卡死！！
所以，生产的工程要关闭内核异常打印！避免出现异常卡死！！

- 3、config_audio_adc_enable：audio adc 模块使能。

① 0：不使用audio adc模块；

② 1：使用audio adc模块；

- 4、config_usbslave_ctl_mic：usb 从机模式控制是否开启 AUDIO_ADC。

① 0：不开启AUDIO_ADC；

② 1：开启AUDIO_ADC；

- 5、libs_debug：打印总开关。

① FALSE：关闭打印；

② TRUE：开启打印；

- 6、log_tag_const_x_yyyy：分支打印开关，仅仅在打印总开关打开的时候有效。

- ① x=i: 控制打印函数 (log_info / log_info_hexdump) 的打印开关;
- ② x=d: 控制打印函数 (log_debug / log_debug_hexdump) 的打印开关;
- ③ x=e: 控制打印函数 (log_error / log_error_hexdump) 的打印开关;
- ④ x=c: 控制打印函数 (log_char) 的打印开关;
- ⑤ yyyy: 控制相应的打印块;

注: 若要打开相应打印, 将CONFIG_DEBUG_LIBS(0) 改为CONFIG_DEBUG_LIBS(1) 即可!

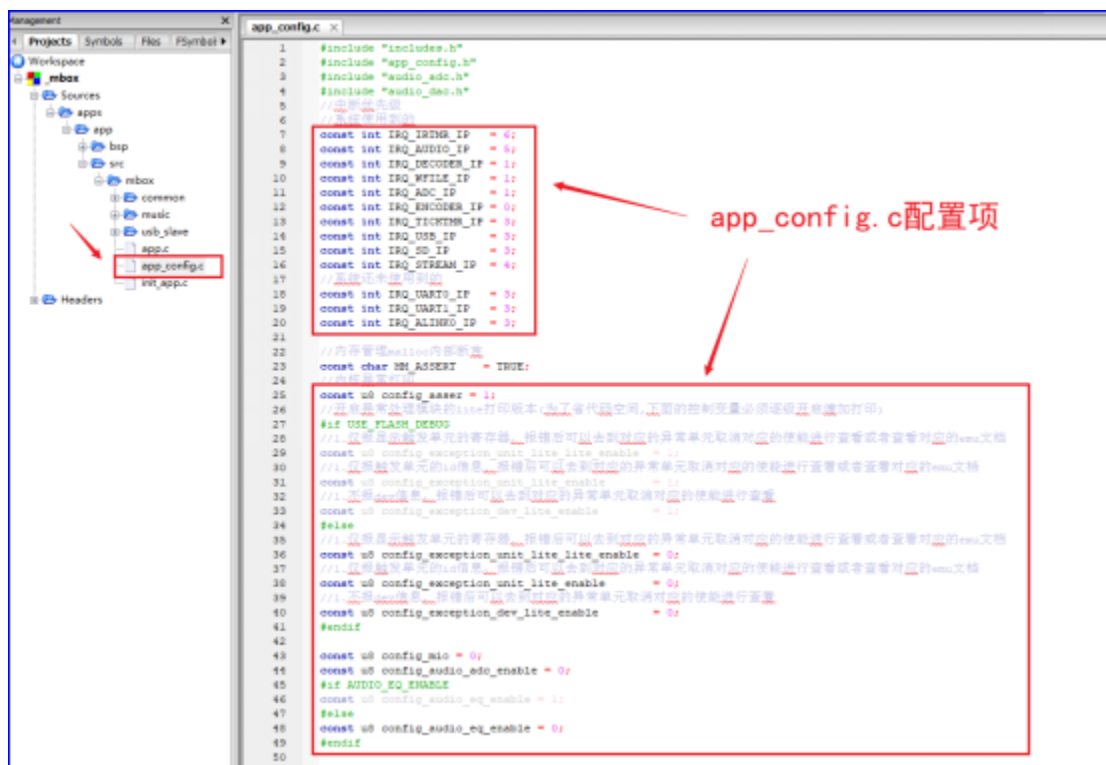


图 3.3 app_config.c 配置

2.4.2 应用配置文件《app_config.h》介绍

应用配置文件《app_config.h》能配置与系统相关的配置:

OTP / FLASH 烧写配置 (仅 OTP 方案 SDK 拥有该配置)

1、USE_FLASH_DEBUG: FLASH 调试模式

- ① 0: 使用OTP烧写配置;
- ② 1: 使用FLASH调试模式;

注: 芯片OTP空间只能烧写一次, 此宏置1用于USB调试; 需要生成OTP烧写文件时, 需要将此宏置0!

时钟配置

配置定义	含义
TCFG_PLL_SEL	系统时钟源
TCFG_PLL_DIV	系统时钟源一级分频
TCFG_HSB_DIV	系统时钟源二级分频

- 注 1：选择系统时钟时，优先调整二级分频，无法满足所需要的时钟下再调整一级分频！
- V1.3.0 版本及往后的 SDK 支持最高超频到 192MHz 系统时钟，存放代码的系统 flash spi 波特率默认不超过 50MHz，最高可超频到 96MHz；
 - 用户可通过修改 clock.c 文件的 SPI_MAX_CLK 变量提高 spi 时钟，但需要确保电源环境稳定且 A0 芯片自选 Flash 支持该速度！
 - 如下图，21 行的静态变量 “static u32 SPI_MAX_CLK = 50 * MHz_UNIT;” 是系统 flash 波特率的限制，修改其值可以改变系统 flash spi 波特率的上限！

注：系统时钟超过 160Mhz 使用，系统 flash 波特率的 50Mhz 限制修改一定要阅读本章的 “AD16N 超频说明”!!!!

```
19 #define SPI_TSHSL 80
20 static u32 LSB_MAX_CLK = 96 * MHz_UNIT;
21 static u32 SPI_MAX_CLK = 50 * MHz_UNIT;
22 static u32 OTP_MAX_CLK = 21 * MHz_UNIT;
23
24 static u32 sys_clock = 16000000;
25 #define MAX_RANGE 3
26 const u32 pll_clock_tab0[4][6] = {
27     //0 d3p5 d2p5 d2p0 d1p5 d1p0
28     {0, 27400000L, 38400000L, 48000000L, 64000000L, 96000000L},
29     {0, 41100000L, 57600000L, 72000000L, 96000000L, 144000000L},
30     {0, 54800000L, 76800000L, 96000000L, 128000000L, 192000000L},
31     {0, 68600000L, 96000000L, 120000000L, 160000000L, 240000000L}
32 };
33 const u32 pll_max_clock_tab[MAX_RANGE][4] = {
34     //dvdd sys max lsb max otp max
35     {SYSVDD_VOL_SEL_120V, 128000000L, 64000000L, 21333334L}, //1.20v
36     {SYSVDD_VOL_SEL_132V, 160000000L, 80000000L, 20000000L}, //1.32v
37     {SYSVDD_VOL_SEL_138V, 192000000L, 96000000L, 19200000L} //1.38v
38 };
39
40 const u8 div_tab0[] = {
41     1, 3, 5, 7
42 };
43
44 const u8 div_tab1[] = {
45     1, 2, 4, 8
46 };
47
48 apps/app/bsp/cpu/uc03/clock.c
```

默认spi时钟不超过50MHz

系统 flash spi 时钟配置

Cache 配置

配置定义	含义
CPU_USE_CACHE_WAY_NUMBER	配置 cache_way, 范围 2~4

串口配置

配置定义	含义
TCFG_UART_TX_PORT	串口打印输出引脚
TCFG_UART_BAUDRATE	串口打印波特率

按键配置

配置定义	含义
KEY_DOUBLE_CLICK	使能按键多击
KEY_IO_EN	IO 按键使能
KEY_AD_EN	AD 按键使能
KEY_IR_EN	红外按键使能
KEY_MATRIX_EN	矩阵按键使能
KEY_TOUCH_EN	触摸按键使能
D_HAS_KEY_VOICE	按键音使能
AD_KEY_CH_SEL	AD 按键采集通道选择, 具体查看 saradc.h 文件
AD_KEY_IO_SEL	AD 按键采集 I/O 选择, 需要与采集通道匹配
IR_KEY_IO	红外按键采集 I/O 口选择;
IR_KEY_TIMER	红外按键采集定时器选择;
IR_KEY_IRQ_IDX	红外按键采集定时器中断号选择, 需要与选择的定时器匹配; 注: 按键部分功能需要在 key.h 中配置!
TOUCH_KEY_IO_SEL	触摸按键 I/O 口选择;

低功耗配置

配置定义	含义
POWER_WAKEUP_IO	低功耗唤醒 I/O 选择, 具体请查看 power_api.c 中的配置
TCFG_VDDIOM_LEVEL	VDDIO 电压选择

AUDIO 配置

1、SR_DEFAULT：默认 DAC 采样率，支持以下采样率

①

8000;

②

11025;

③

12000;

④

16000;

⑤

22050;

⑥

24000;

⑦

32000;

⑧

44100;

⑨

48000;

⑩

64000;

11

88200;

12

96000;

充电模块配置

配置定义	含义
TCFG_CHARGE_ENABLE	充电模块使能
TCFG_CHARGE_FULL_V	充满电压选择
TCFG_CHARGE_FULL_MA	充满电流选择
TCFG_CHARGE_MA	最大充电电流选择
TCFG_CHARGE_TRICKLE_MA	涓流电流选择

1、TCFG_CHARGE_FULL_V：充满电压选择

①

CHARGE_FULL_V_4041：最小充满电压；；

②

.....

③

CHARGE_FULL_V_4534：最大充满电压；

2、TCFG_CHARGE_FULL_MA：充满电流选择

①

CHARGE_FULL_mA_DIV5：充电电流达到恒流充电电流的1/5时，判断已充满电；；

②

.....

③

CHARGE_FULL_mA_DIVD20：充电电流达到恒流充电电流的1/20时，判断已充满电；

3、TCFG_CHARGE_MA：最大充电电流选择

①

CHARGE_mA_20：最小恒流充电电流20mA；；

②

.....

③

CHARGE_mA_180：最大恒流充电电流180mA；

4、TCFG_CHARGE_TRICKLE_MA：涓流电流选择

注：充电功能部分配置请查看charge.h文件！

SD 模块配置

配置定义	含义
TFG_SD_EN	SD 卡模块使能
TFG_SDPG_ENABLE	SD 卡模块使用 SD_PG (PC3) 供电使能

EQ 模块配置

配置定义	含义
AUDIO_EQ_ENABLE	EQ 模块使能总开关
TCFG_CFG_TOOL_ENABLE	EQ 配置工具使能, 需要在线调试 EQ 时打开, 默认关闭

设备升级配置

配置定义	含义
TFG_DEV_UPGRADE_SUPPORT	设备升级使能
TFG_UPGRADE_FILE_NAME	设备升级 ufw 文件默认路径” /update.ufw”

USB 模块配置

配置定义	含义
TCFG_PC_ENABLE	PC 模块使能
TCFG_USB_EXFLASH_UDISK_ENABLE	外置 FLASH 模拟 U 盘使能
TCFG_UDISK_ENABLE	U 盘模块使能
USB_DEVICE_CLASS_CONFIG	USB 从机功能选择

1、USB_DEVICE_CLASS_CONFIG: USB 从机功能选择

- ① MASSSTORAGE_CLASS: 大容量存储 (读卡器) 功能;
- ② SPEAKER_CLASS: AUDIO 功能;
- ③ HID_CLASS: HID 控制 功能

外挂 FLASH 配置 (FLASH Configuration)

配置定义	含义
TFG_EXT_FLASH_EN	外挂 flash 使能
TFG_SPI_HW_NUM	外挂 flash 使用的硬件 spi 组
TFG_SPI_UNIDIR_MODE_EN	外挂 flash 单线模式使能,flash do 与 di 短接并连接到 do 的 i/o 引脚
TFG_SPI_WORK_MODE	spi 模式选择
TFG_SPI_READ_DATA_WIDTH	spi 位宽选择
TFG_SPI_CS_PORT_SEL	spi cs 引脚选择
TFG_SPI_CS_PORT	spi cs 引脚选择, 用于配置 ini 文件下载数据到外挂 flash
TFG_SPI_CLK_PORT_SEL	spi clk 引脚选择
TFG_SPI_CLK_PORT	spi clk 引脚选择, 用于配置 ini 文件下载数据到外挂 flash
TFG_SPI_DO_PORT_SEL	spi do 引脚选择
TFG_SPI_DO_PORT	spi do 引脚选择, 用于配置 ini 文件下载数据到外挂 flash
TFG_SPI_DI_PORT_SEL	spi di 引脚选择
TFG_SPI_DI_PORT	spi di 引脚选择, 用于配置 ini 文件下载数据到外挂 flash

UI 显示配置

配置定义	含义
LED_5X7	LED 数码管使能, 会占用部分 I/O, 请查看 app.h 中的引脚分布
LCD_4X8_EN	LCD 断码屏使能, 会占用部分 I/O, 请查看驱动文件中的引脚分配
KEY_MATRIX_LCD_REUSE_E	矩阵按键与 LCD 复用宏
POWERDOWN_KEEP_LCD	LCD 断码屏 powerdown 低功耗保持

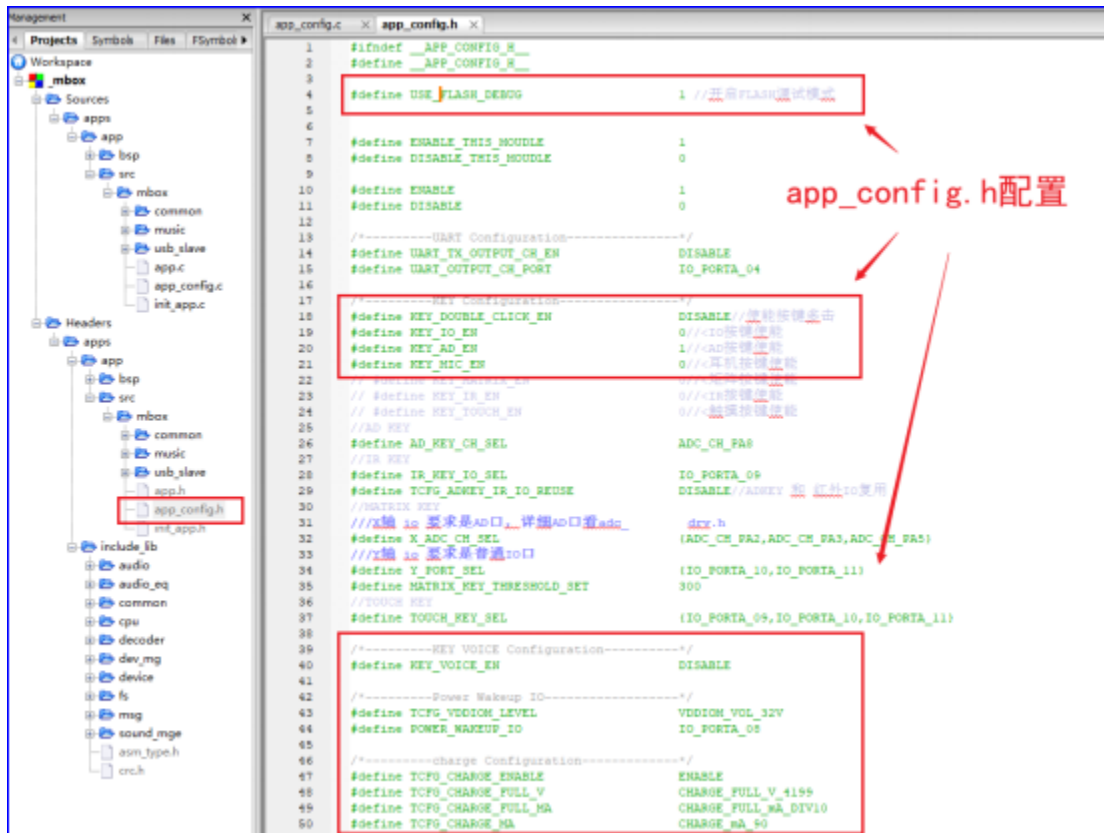


图 3.4 app_config.h 配置

2.4.3 APP 配置文件《app_modules.h》介绍

应用配置文件《app_modules.h》中的宏定义能配置支持的编解码格式、以及 APP 模式相关的配置；每一个配置项都可独立开关，关闭没有用到的宏定义，可以减少生成的代码量，节省程序空间和运行空间*。

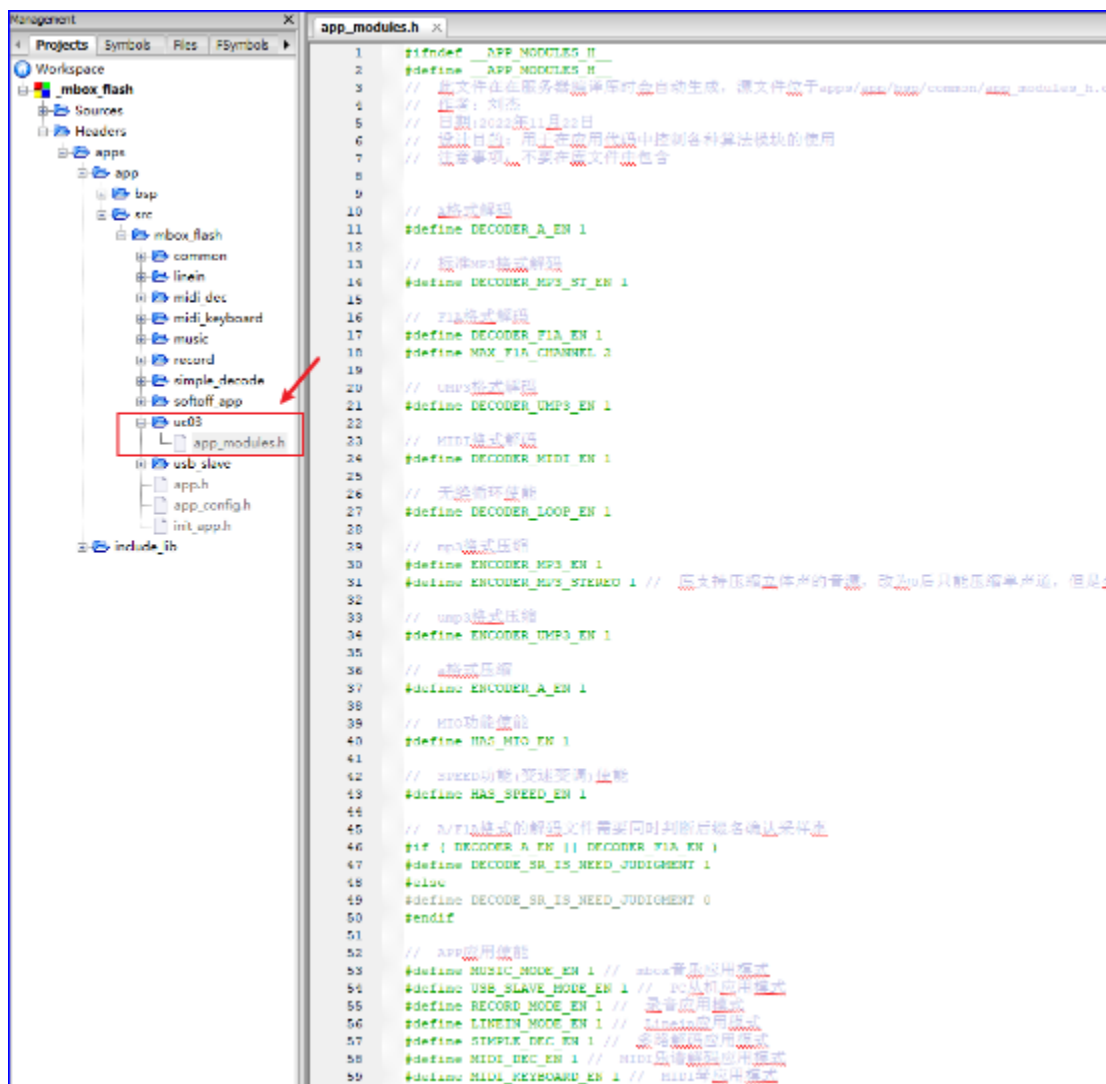


图 3.5 app_modules.h 配置

注：AD16N_V1.4.0 的 app_modules.h 新增了较多的宏，但是由于功能不支持，因此以下列举的宏不可修改。

```

RUN_APP_RC
RUN_APP_DONGLE
DECODER_IMA_EN
DECODER_OPUS_EN
DECODER_SPEEX_EN
DECODER_SBC_EN
DECODER_JLA_LW_EN
ENCODER_OPUS_EN
ENCODER_SPEEX_EN
ENCODER_IMA_EN
ENCODER_SBC_EN
ENCODER_JLA_LW_EN
RF_REMOTECONTROL_MODE_EN

```

SYS_TIMER_EN

2.4.4 cpu 差异相关功能配置文件《cpu_config.c》介绍

该文件主要用于配置和芯片差异相关的功能模块，主要包含：::

1、异常模块 2、cache_way 配置 3、设备升级配置 4、SD 卡宫殿口配置 5、音频模块相关配置 6、低功耗保持 LCD 显示配置

1、config_exception_enable：异常模块使能。

- ① 0：不使能；
- ② 1：使能；

2、config_exception_unit_lite_enable：显示内核异常触发单元的寄存器。

- ① 0：不显示；
- ② 1：显示；

3、config_exception_unit_lite_enable：显示内核异常触发单元 id 信息。

- ① 0：不显示；
- ② 1：显示；

4、config_exception_dev_lite_enable：显示内核异常触发单元 dev 信息。

- ① 0：不显示；
- ② 1：显示；

5、config_vcm_cap_addon：vcom 外挂电容配置，需根据封装以及用料进行配置。

- ① 0：vcom 不挂电容；
- ② 1：vcom 外挂电容，可改善音质；

6、audio_dac_vcmo0_enable / audio_dac_vcmo1_enable：vcm0 / vcm1 使能配置。

- ① 0：vcm0 / vcm1 不输出电压；
- ② 1：vcm0 / vcm1 输出 audio 中间电位，dac 或 adc 可省去隔直电容；

7、config_dac_points_combine：双声道解码数据合成输出配置（DAC 差分输出时配置）。

- /* 样点合成公式： $L \cdot L_{\text{gain}} / \text{Totalgain} + R \cdot R_{\text{gain}} / \text{Totalgain}$ */
- ① config_dac_points_Lgain / config_dac_points_Rgain：左 / 右声道增益；
- ② config_dac_points_Totalgain：增益计算因子；

8、audio_adc_mic_pga_6db：audio_adc 前级增益配置。

- ① 0：0dB；
- ② 1：6dB；

9、audio_adc_mic_pga_g：audio_adc 后级增益配置。

- ① 详情请查看《audio_adc.h》文件中的 AUDIO_MICPGA_G 枚举成员；

10、audio_adc_aux_pga_g：数字 linein 增益配置。

① 详情请查看《audio_adc.h》文件中的AUDIO_MICPGA_G枚举成员；

2.5 AD16N 超频说明

在 release_v1.3.0 之前的 SDK AD16N 的系统时钟最高运行 160Mhz，内部的系统 Flash 的波特率限制不超过 50Mhz；

在 release_v1.3.0 的 SDK 系统时钟最高放宽 192Mhz，内置的系统 Flash 的波特率限制可在一定条件下放宽到 96Mhz！

系统时钟不超过	内核电压挡位	内置系统 Flash	A0 芯片外挂系统 Flash
128Mhz	1.20v	支持	支持
160mhz	1.32v	支持	支持
192Mhz	1.38v	支持	支持

表系统时钟支持情况

系统 Flash 波特率	内置系统 Flash	A0 芯片外挂系统 Flash
“<= 50Mhz”	支持	支持
“>=50mhz” & “<=96Mhz”	支持	不能直接支持
“>96MHZ”	不支持	不支持

表系统 Flash 波特率支持情况

注：当使用 A0 芯片时，系统 Flash 需要外挂，由于 Flash 的型号以及性能，加上 PCB 布线等情况的影响，不能预测系统 Flash 波特率运行在超过 50Mhz、甚至是到 96MHZ 的具体情况；在此种情况下，客户如果需要系统 Flash 波特率运行在超过 50Mhz，需要自行评估。

本章“SDK 工程配置”->“应用配置文件《app_config.h》介绍”->“时钟配置”有时钟配置的章节介绍。

2.6 AD16N 启动时间

AD16 的启动时间如下 (测试芯片为 uboot 启动芯片), 第一行是公版 SDK 默认配置

冷启动 (跑到 main 函数)	softoff 启动时间 (从触发唤醒跑到 main)	mask_io_cfg
31.24412ms	8.89977ms	sfc_fas_boot:1; do_flash_reset:0
32.41109ms	10.98155ms	sfc_fas_boot:1; do_flash_reset:1
31.34513ms	24.72249ms	sfc_fas_boot:0; do_flash_reset:0
31.15557ms	26.71502ms	sfc_fas_boot:0; do_flash_reset:1

下载目录 & 烧写说明

AD16N 编译完成后，会在下载目录生成代码文件，并与资源文件整合形成烧录文件，通过 CodeBlocks 调用强制下载工具烧写到芯片内置 FLASH 中；同时我司有许多工具支持压缩转换和打包音频，将资源文件下载到内置或外挂 FLASH 中。

3.1 下载目录介绍

AD16N 下载目录位于 `app\post_build\uc03`，一共分为三种文件：

- 生产相关文件：
 - `app.bin`：编译生成的代码文件
 - `jl_isd.bin`：FLASH 烧录文件，包含代码文件和资源文件
 - `output_flash.fw`：用于我司烧写器批量生产的 FLASH 烧录文件
 - `output_otp.fw`：用于我司烧写器批量生产的 OTP 烧录文件（仅 OTP SDK 使用）
- 下载工具：
 - `download_bat.c`：下载批处理文件，编译时会生成 `download.bat` 文件
 - `isd_config_ini.c`：工程配置文件，编译时会生成 `isd_config.ini` 文件
 - 下载目录工具更新 `jlproj`：该工具可在线更新下载目录的 `exe` 工具

本小节着重介绍下载工具的 `download_bat.c` 以及 `isd_config.ini` 文件

AD14N / AD15N / AC104N 下载目录位于 `app\post_build\sh5x\voice_toy`(工程名称)，一共分为三种文件：

3.1.1 download_bat.c 文件介绍

download.bat 脚本由 download_bat.c 文件编译生成，该脚本支持把代码和资源文件以及部分配置烧写到内置 FLASH，详细的功能说明图 2.1.1 所示，具体参数如下：

-app app.bin：

该指令会将代码文件 app.bin 转换成用于 FLASH 烧写的文件 jl_isd.bin。

-otp app.bin：（仅 OTP SDK 使用）

该指令会将代码文件 app.bin 转换成用于 OTP 烧写的文件，需要与-output-fw 以及-package-only 指令一起使用，生成用于我司烧写器批量生产的 OTP 烧录文件。

-format all：

该指令会擦除整颗内置 FLASH。

-output-fw：

该指令会生成用于我司烧写器批量生产的烧录文件。

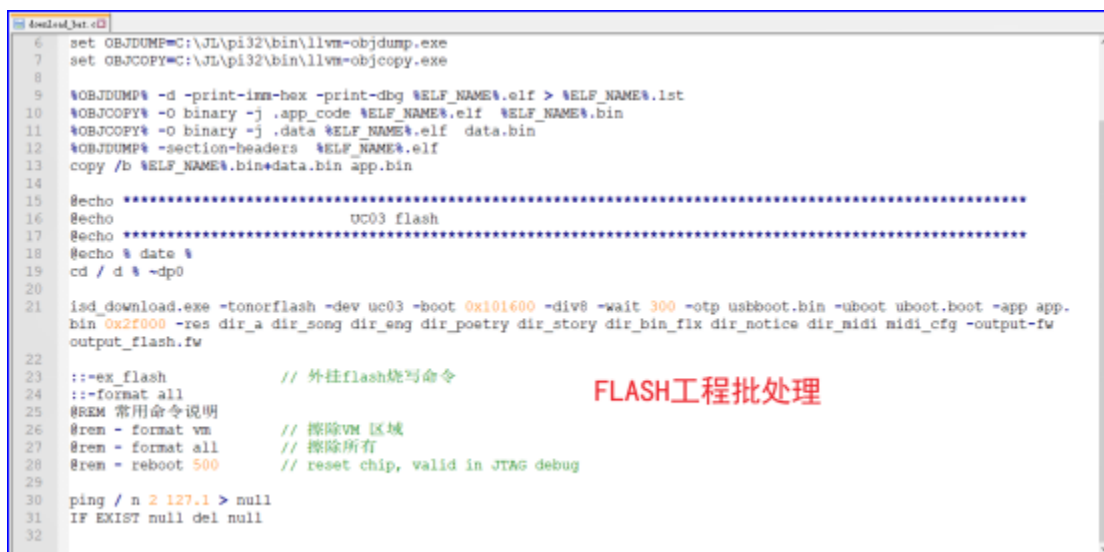
- key *.key：

该指令会加密烧写文件。参数 *.key 为申请的 key 文件，不同系列不可混用 key 文件。

- ex_flash dir_xxx： 外挂 flash 烧写命令

该指令会根据 isd_config.ini 中定义的外挂 flash 引脚，将指定的资源文件下载到外挂 flash 中，外挂 flash 引脚配置将于后文 isd_config.ini 文件介绍中说明。

注意：准备下载到外挂 flash 的 dir 文件，需要经过 packres.exe 指定下载地址再次打包生成新的 dir 文件，才可使用-ex_flash 命令。例如将 dir_song 文件下载到外挂 flash，需要通过 packres 脚本添加 packres.exe -n res -o dir_song_ext dir_song 0 -normal 命令指定 0 地址将 dir_song 文件打包成 dir_song_ext 文件，在 download_bat.c 中添加-ex_flash dir_song_ext 指令即可将 dir_song 文件完整烧写至外挂 flash 中。



```

1  set OBJDUMP=C:\JL\api32\bin\llvm-objdump.exe
2  set OBJCOPY=C:\JL\api32\bin\llvm-objcopy.exe
3
4  %OBJDUMP% -d -print-imm-hex -print-dbg %SELF_NAME%.elf > %SELF_NAME%.lst
5  %OBJCOPY% -O binary -j .app_code %SELF_NAME%.elf %SELF_NAME%.bin
6  %OBJCOPY% -O binary -j .data %SELF_NAME%.elf data.bin
7  %OBJDUMP% -section-headers %SELF_NAME%.elf
8  copy /b %SELF_NAME%.bin+data.bin app.bin
9
10 @echo *****
11 @echo                               UC03 flash
12 @echo *****
13 @echo % date %
14 cd / d % ~dp0
15
16 isd_download.exe -tonerflash -dev uc03 -boot 0x101600 -div8 -wait 300 -otp usboot.bin -uboot uboot.boot -app app.
17 bin 0x2f000 -res dir_a dir_song dir_eng dir_poetry dir_story dir_bin_fix dir_notice dir_midi midi_cfg -output-fw
18 output_flash.fw
19
20
21
22
23 ::-ex_flash                // 外挂flash烧写命令
24 ::-format all
25 @REM 常用命令说明
26 @rem - format vm          // 擦除VM 区域
27 @rem - format all         // 擦除所有
28 @rem - reboot 500         // reset chip, valid in JTAG debug
29
30 ping / n 2 127.1 > null
31 IF EXIST null del null
32

```

图 1.1 FLASH 工程 download_bat.c 文件说明

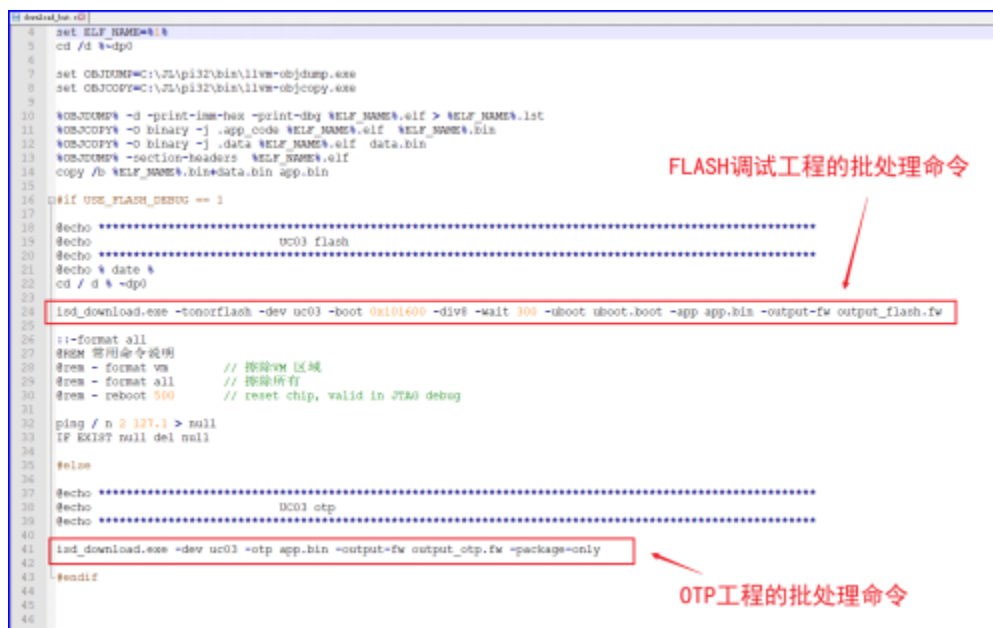


图 1.2 OTP 工程 download_bat.c 文件说明

3.1.2 isd_config.ini 文件介绍

isd_config.ini 配置文件由 isd_config_ini.c 文件编译生成，该配置文件可以配置程序中与烧写、启动的相关参数。

EXTRA_CFG_PARAM

- PID: 工程标识，不可超过 16 字节；
- VID: 工程版本号，用于升级判断；
- NEED_RESERVED_4K: 4K 保留区配置，详情请看《第十九章 RESERVED_AREA 区域说明》；

- ① NEED_RESERVED_4K = YES: 保留 4K 配置区；
- ② NEED_RESERVED_4K = NO: 不保留 4K 配置区；

- SPECIAL_OPT: 选择是否实时根据 FLASH 支持的对齐方式组织文件

- ① SPECIAL_OPT = 0
 - 0: 根据下载时所连样机 FLASH 的对齐方式组织文件，只生成 1 份 4K 或者 256 对齐的烧录文件；
- ② SPECIAL_OPT 不存在: 生成多份不同对齐方式的烧录文件；

*注: 烧录文件在组织过程中支持按 4K 或者 256 字节对齐；若用 256 字节对齐的烧录文件，去生产最小擦除单位

- FORCE_4K_ALIGN: 选择是否强制只生成 1 份 4K 字节对齐的 FLASH 烧录文件

注: 如果方案需要在线升级，建议使用强制 4K 对齐，SDK 默认只生成 1 份 4K 对齐的烧录文件！

- FLASH_SIZE: 指定 FLASH 大小，用于生成指定大小的烧录文件

- ① 0x800000: 按照 8M 字节的 FLASH 的空间生成烧录文件，如果在线下载时发现实际的 FLASH 与指定的大小不匹配
- *注: 在没有直接指定 FLASH 大小的情况下，最后生成的下载文件会以市面 FLASH 能支持的最小文件的形式组织；

(续下页)

(接上页)

- *a. 当所有文件小于256K字节时，系统会以总大小为256K为标准为录音分配剩余空间；*
- *b. 当所有文件大小等于300K时，系统会以512K为标准为录音分配剩余空间；*

SYS_CFG_PARAM

- SPI: 芯片内置 FLASH 的 SPI 配置，2_3_0 表示使用双线模式，0_3_0 表示使用单线模式，根据内置 FLASH 接线方式进行选择；

- ① 2_3_0: 内置FLASH使用双线数据模式；
- ② 0_3_0: 内置FLASH使用单线数据模式；

- CACHE_WAY: 芯片使用 cache 的 WAY 数，AD16N 可选 1~4，多出来的 cache 可以作为 sram 使用，该功能会影响芯片运行效率，需要修改 ld 文件；
- UTTX: UBOOT 串口打印脚选择；

- ① PA00~PA15、PB00~PB10、PC00~PC05: 使用普通I/O口作为串口打印脚；
- ② USBDM、USBDP: 使用USB引脚作为串口打印脚；

- UTBD: UBOOT 串口打印波特率，默认为 1000000；
- EX_FLASH: 外挂 flash 烧写配置；

- ① PB00_1A_NULL: 配置顺序为“CS-SPI端口-电源口”；
- ② 用VDDIO供电电源口参数为NULL，使用时配置CS脚I/O即可；

- EX_FLASH_IO: 外挂 flash 烧写引脚配置：

- ① 1_PB01_PB02_PB03_NULL_NULL: 配置顺序为“数据宽度-CLK-DO-DI-D2-D3”；
- ② 双线模式时数据宽度为1；单线模式时数据宽度为0；D2、D3口为NULL；
- ③ 四线模式时数据宽度为4，需要配置D2、D3口；
- ④ 该配置在isd_config_ini.c中使用了宏拼接，修改数据宽度时需要修改拼接的第一个参数

BURNER_PASSTHROUGH_CFG

- OTP_BOOT: 芯片启动模式选择，为烧写器配置项，会影响芯片开机，该位在 OTP 工程 SDK 中由 app_config.h 文件中的 USE_FLASH_DEBUG 宏定义自动选择控制，在 FLASH 工程 SDK 中默认配置为 0；

- ① OTP_BOOT = 0: 芯片从FLASH启动，用于FLASH工程以及OTP工程的FLASH调试模式；
- ② OTP_BOOT = 1: 芯片从OTP启动；

BURNER_CFG

- LVD: 芯片低电复位电压默认档位选择，为烧写器配置项；该档位仅影响烧写器默认选择的档位，用户仍可自行修改烧写器上的 LVD 档位进行烧写；如无该配置项，则烧写器默认选择最高档位 2.840v；该配置项为一次性烧写，请慎重选择！

- ① 可选档位：2.078v、2.188v、2.297v、2.406v、2.515v、2.624v、2.733v、2.840v；

RESERVED_CONFIG

- XXXX_ADR: 区域起始地址

- ① 0: VM区域起始位置;
- ② AUTO: 由工具自动分配起始地址;

- XXXX_LEN: 区域大小

- XXXX_OPT: 区域操作属性

- ① 0: 下载代码时擦除指定区域;
- ② 1: 下载代码时不操作指定区域;
- ③ 2: 下载代码时给指定区域加上保护;

注: VM 区域的起始地址为 0, 且实际大小为系统剩余的空间, 扩展录音空间可查看 (一) 中 FLASH_SIZE 配置的说明; EEPROM 区域一般用于代码中的 VM 断电记忆存储, 区域大小不少于 8K, 否则 VM 初始化会失败; 用户可自行定义其他空间区域;

3.2 派生密钥 3.0 介绍

AD16N FLASH 方案使用派生密钥 3.0 (r3key/l3key) 的 KEY 方案。该密钥系统是为了代理商为多家客户供货而设计的。目的是代理商手握一个 r3key, 并以此为母本, 生成出多个不同的 l3key; 代理商为其下游不同客户分配不同的 l3key, 再以 l3key 加密 flash, 从而确保芯片程序加密的安全性。

与 AD14N / AD15N 系列上的派生密钥 2.0 相比, 用户在使用派生密钥 3.0 的变化在于:

- 1) r3key 不是完整的密钥, 不能直接用于加密, 其派生出的 l3key 才是可用于加密的完整密钥。
- 2) 空片可以烧录成 r3key; 已经烧录过 r3key 的芯片可以进一步烧录该 r3key 派生出来的 l3key; 空片也可以直接烧录成 l3key, 以上行为均不可逆!
- 3) AD14N / AD15N 系列使用的是派生密钥 2.0, 与派生密钥 3.0 不能混用!

派生密钥 3.0 的使用可参考 download_bat.c 中 isd_download.exe 命令”-key”说明, 用户加密程序时, 在 download_bat.c 批处理中, 添加申请到的 l3key 文件名, 并将 l3key 文件置于 isd_download.exe 同一目录下, 重新编译程序即可加密程序。



图 2.1 L3key 添加到工程进行加密

3.3 二级密钥 V2.0 简单介绍

AD16N FLASH 方案 V1.3.0 以及之后版本的 SDK 支持二级密钥 V2.0 系统，该系统不兼容旧版本二级密钥；使用该功能前请咨询杰理助理；

(一) 使用场景

1. 公司 A 出售芯片给公司 B。
2. 公司 A 在杰理 SDK 基础开发一个 SDK 提供给公司 B 做二次开发

(二) 双向保护机制

1. 公司 A 在自己的核心库里面配置好密钥后，可保护公司 A 的 SDK 不外泄。在没有获得授权的情况下 SDK 无法正常运行，可避免公司 B 从其他渠道购买芯片运行公司 A 所开发的 SDK。
2. 公司 A 没有公司 B 的 key1 文件，公司 A 拿到 fw 文件之后也无法生产，从而保护了公司 B 二次开发之后新 SDK 的权益。

3.4 多算法授权简单介绍

使用该功能前请咨询杰理助理；

多算法授权，指的是：在一个 SDK 中，使用了算法提供商的库函数，而算法提供商可以控制芯片的烧录次数。

(一) 使用场景

1. 算法提供商想要控制生产商使用算法库的次数（即：算法提供商按生产的样机收取费用）。

具体详细说明可以点击下方链接：
[多算法授权使用说明](#)

3.5 烧写注意事项

FLASH 工程批量烧写时需注意一下事项：

- 1) 开发使用的样片，必须使用我司一拖二烧写器进行校准（烧写器需要对芯片内部时钟 LRC 和 DAC 电压做校准，这两项在芯片出厂阶段不做校准）；
- 2) SOP 和 QSOP 使用一拖二烧写器时，需要使用转接小板。左板与右板的连接线最好使用焊接固定，避免因机台动作导致接触不良；
 - 使用加粗的连接线，建议至少焊接 2 根以上地线，保证共地良好。
 - 使用机台转接小板烧写 SOP 和 QSOP 封装时，请注意必须关闭 1t2 烧写器配置的 REF TRIM 选项，手工烧写时需要勾选 REF TRIM；
 - 其他机械手烧写 QFN 或 LQFP 封装时，需要勾选 REF TRIM；

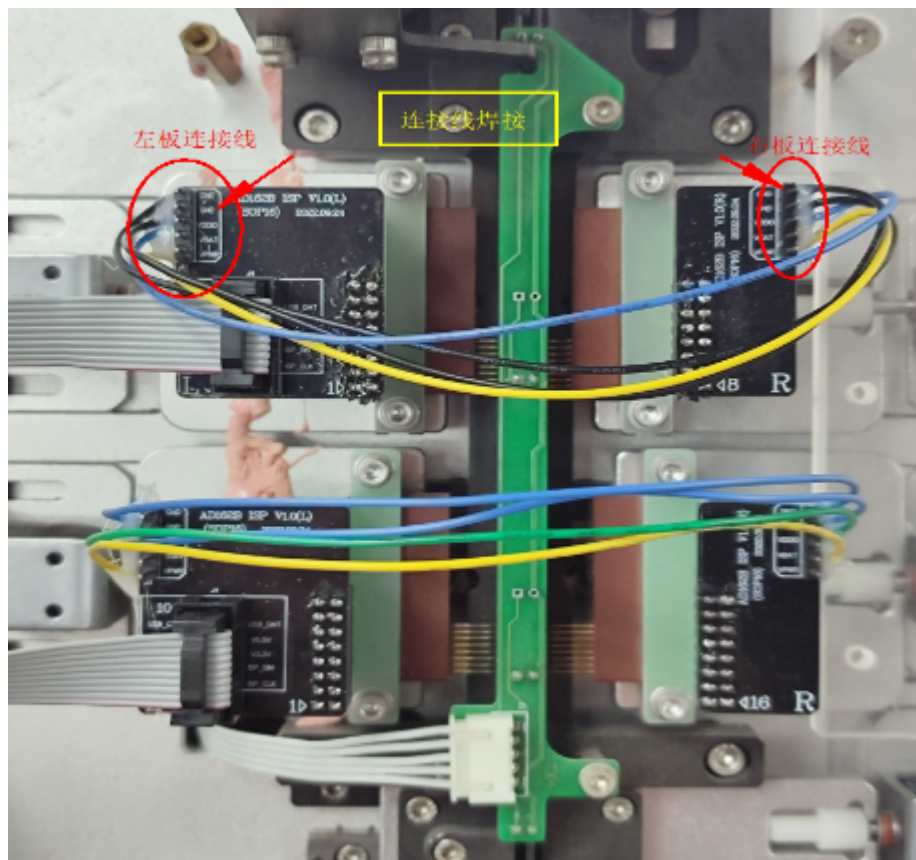


图 3.1 机台转接小板连接示意图

- 3) LVD 电压配置必须大于 FLASH 最低工作电压，OTP 方案 LVD 电压配置必须大于 2.3V；

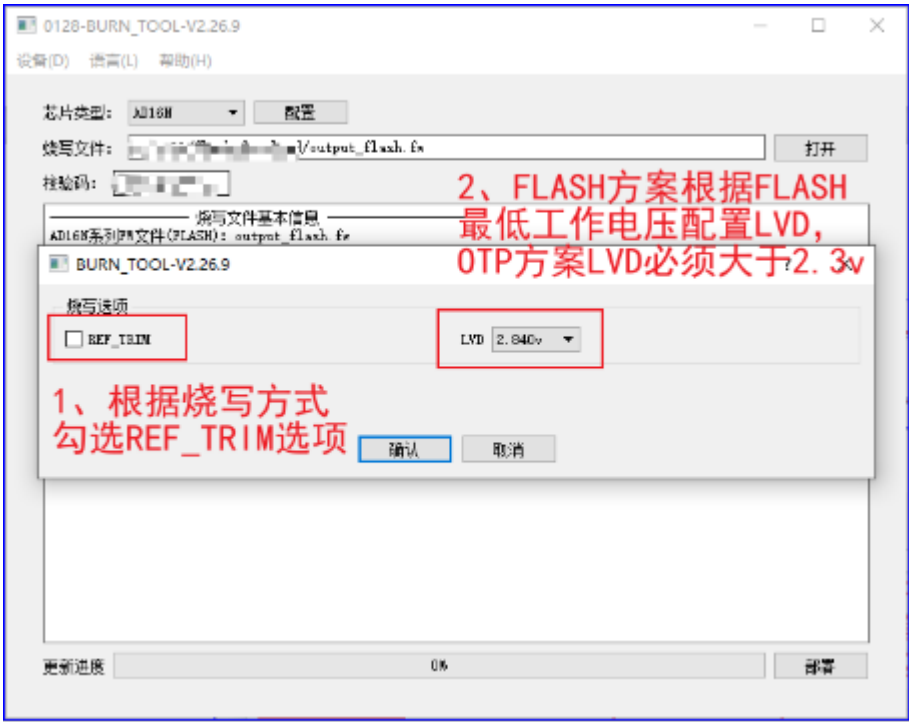


图 3.2 烧写器 REF_TRIM 选项和 LVD 选项

3.6 FLASH 工程烧写补充说明

AD16N 空片无法使用 USB 升级! FLASH 方案的封装片在二次开发前, 必须使用我司一拖二工具烧写底层驱动文件, 才能正常使用 USB 强制升级工具进行程序升级。

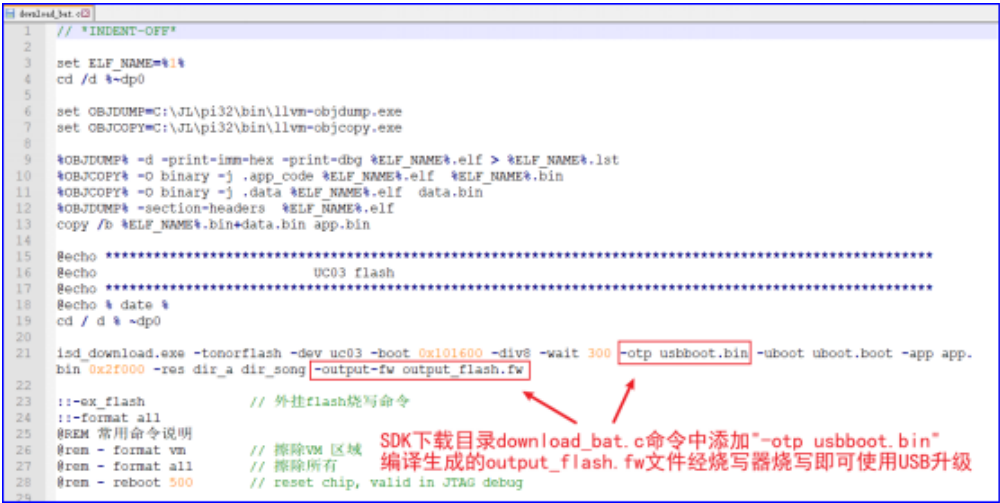
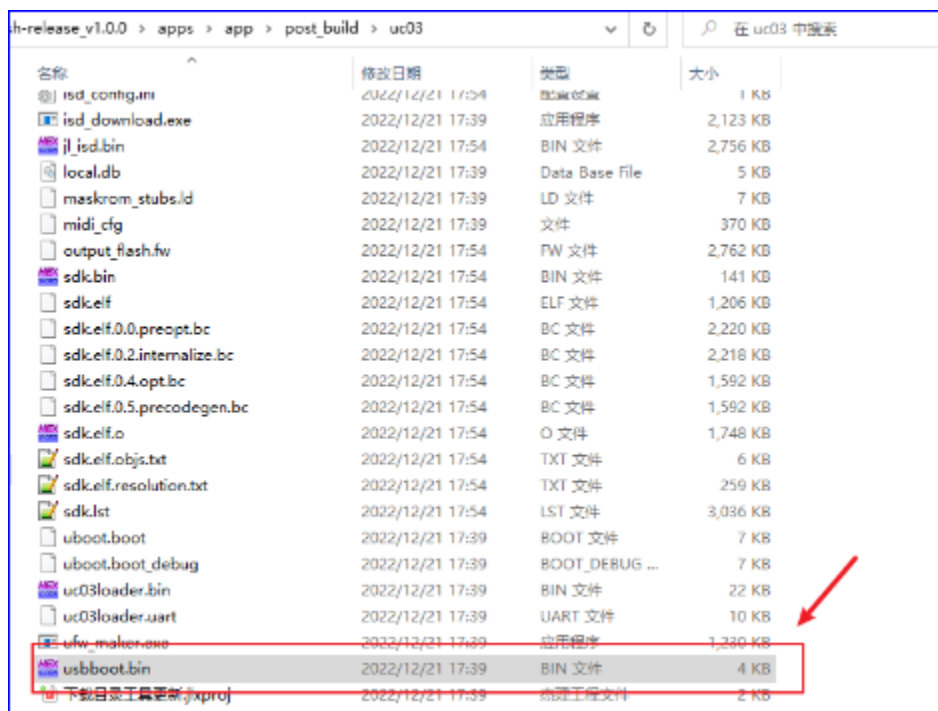


图 4.1 FLASH 工程使用 USB 强制升级工具 download_bat.c 说明



名称	修改日期	类型	大小
isid_config.ini	2022/12/21 17:54	配置文件	1 KB
isid_download.exe	2022/12/21 17:39	应用程序	2,123 KB
il_isid.bin	2022/12/21 17:54	BIN 文件	2,756 KB
local.db	2022/12/21 17:39	Data Base File	5 KB
maskrom_stubs.ld	2022/12/21 17:39	LD 文件	7 KB
midl_cfg	2022/12/21 17:39	文件	370 KB
output_flash.fw	2022/12/21 17:54	FW 文件	2,762 KB
sdic.bin	2022/12/21 17:54	BIN 文件	141 KB
sdic.elf	2022/12/21 17:54	ELF 文件	1,206 KB
sdic.elf.0.0.preopt.bc	2022/12/21 17:54	BC 文件	2,220 KB
sdic.elf.0.2.internalize.bc	2022/12/21 17:54	BC 文件	2,218 KB
sdic.elf.0.4.opt.bc	2022/12/21 17:54	BC 文件	1,592 KB
sdic.elf.0.5.precodegen.bc	2022/12/21 17:54	BC 文件	1,592 KB
sdic.elf.o	2022/12/21 17:54	O 文件	1,748 KB
sdic.elf.objs.txt	2022/12/21 17:54	TXT 文件	6 KB
sdic.elf.resolution.txt	2022/12/21 17:54	TXT 文件	259 KB
sdic.lst	2022/12/21 17:54	LST 文件	3,036 KB
uboot.boot	2022/12/21 17:39	BOOT 文件	7 KB
uboot.boot_debug	2022/12/21 17:39	BOOT_DEBUG ...	7 KB
uc03loader.bin	2022/12/21 17:39	BIN 文件	22 KB
uc03loader.uart	2022/12/21 17:39	UART 文件	10 KB
ufw_maker.exe	2022/12/21 17:39	应用程序	1,330 KB
usbboot.bin	2022/12/21 17:39	BIN 文件	4 KB
下载目录工具更新工程proj	2022/12/21 17:39	工程文件	2 KB

图 4.2 FLASH 工程使用 USB 强制升级工具驱动文件

3.7 OTP 工程烧写补充说明

OTP 方案适合最追求极致性价比的产品，属于一次性烧写存储方式，烧写后不可再次升级，样机测试必须严谨细致。量产时，建议选择使用内置 FLASH 型号试产，待试产稳定后再切换为 OTP 方案，降低生产风险。

OTP 方案在工程开发阶段，需要使用 FLASH 多次下载代码进行开发和调试；在最终生产时，将相同的工程代码烧写到 OTP 中。OTP 只能通过烧写器烧写。

3.7.1 FLASH 调试模式与 OTP 模式区别

OTP 工程 SDK 中通过 app_config.h 文件中的 USE_FLASH_DEBUG 宏定义，控制代码下载方式。

由于 OTP 空间只有 32K 字节，相较于 FLASH 调试模式，OTP 模式的区别在于：

- 关掉了所有 log 打印
- 没有使能异常模块，没有异常打印
- 不会记录唤醒源和复位源（可自行开启）

3.7.2 FLASH 调试模式与 OTP 模式的烧写

FLASH 调试模式

- 将 app_config.h 文件中的 USE_FLASH_DEBUG 宏定义置为 1
- 开发板连接 USB 强制下载工具，进入下载模式
- 编译工程，生成 jl_isd.bin 直接下载到开发板的 FLASH 中
- 编译工程还会生成 output_flash.fw 用于烧写器下载

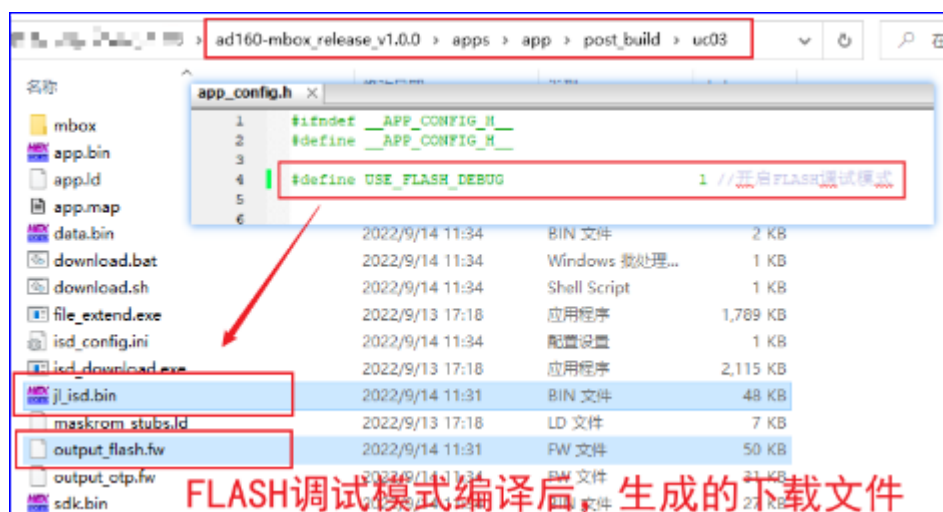


图 5.1 FLASH 调试模式下载目录文件说明

OTP 模式

- 将 app_config.h 文件中的 USE_FLASH_DEBUG 宏定义置为 0
- 编译工程，生成 output_otp.fw 文件用于烧写器烧写

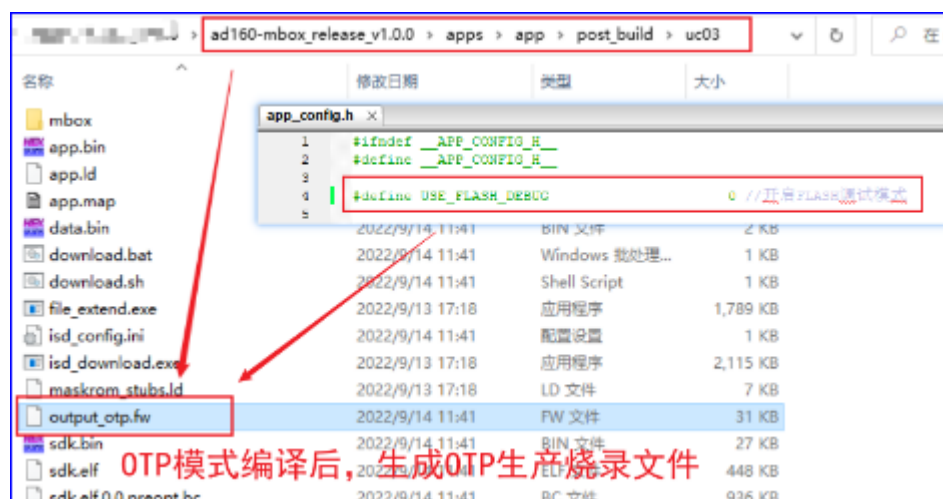


图 5.2 OTP 模式下载目录文件说明

3.8 开发工具介绍

杰理 AD 系列芯片的 SDK 支持的开发工具如下：

- 资源打包工具：能将多个文件打包成一个文件；
- 音频文件转换工具：能将音频文件转换为系统能支持的格式；
- MIDI 转换工具：能将 midi 文件转换为系统能支持的格式以及制作 midi 音色库；
- MIO 工具：能制作波形随音频律动的 MIO 文件；
- SYD 系统制作工具：能格式化 SD 卡/U 盘并挂载我司 SYD 文件系统的工具；

以上工具可在 AD16N_gitee 开源网站上或询问我司深圳办人员获取，部分工具内含说明文档，本节将介绍文件打包工具和音频转换工具；

3.8.1 资源打包工具

下载到 FLASH 的资源文件需要经过资源打包工具处理之后才能使用，打包工具包括：new_packres.bat、packres.exe 和 pRFiles.exe；其中 new_packres.bat 支持的命令详情请看同目录下的 readme.md 文件。

少量文件直接打包

1、将需要打包的文件放在工具根目录下，打开 new_packres.bat 文件，指定需要打包的文件以及输出的文件夹名；

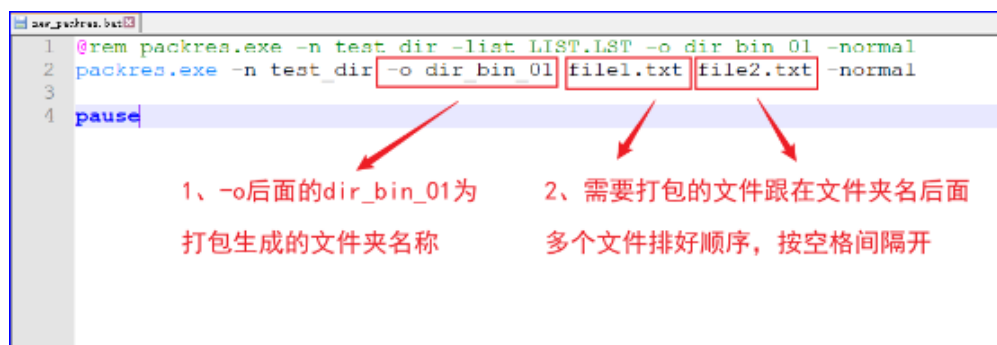


图 6.1 new_packres.bat 资源打包批处理说明

2、保存 new_packres.bat 文件，并双击该批处理，工具根目录下生成打包文件夹表示打包成功；

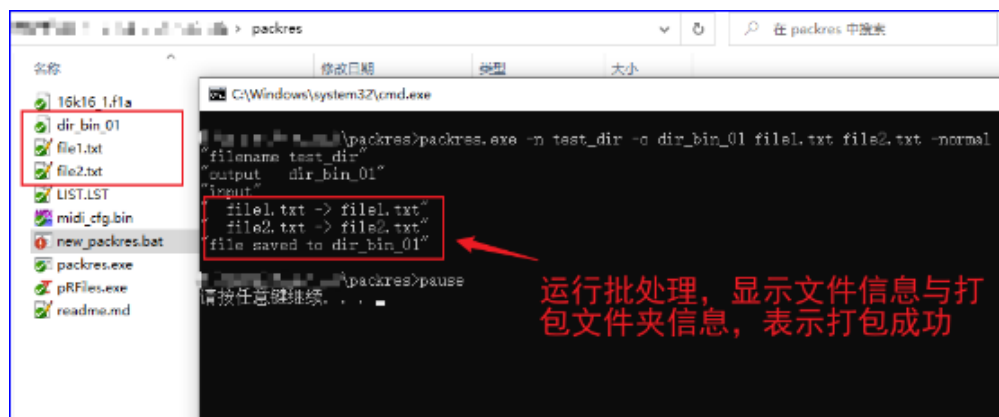


图 6.2 new_packres.bat 运行结果图

大量文件通过 pRFiles.exe 批量打包

1、将大量文件放在文件夹中，打开 pRFiles.exe，如下图所示进行操作，保存为 LST 文件，以保存为 TEST.LST 为例；

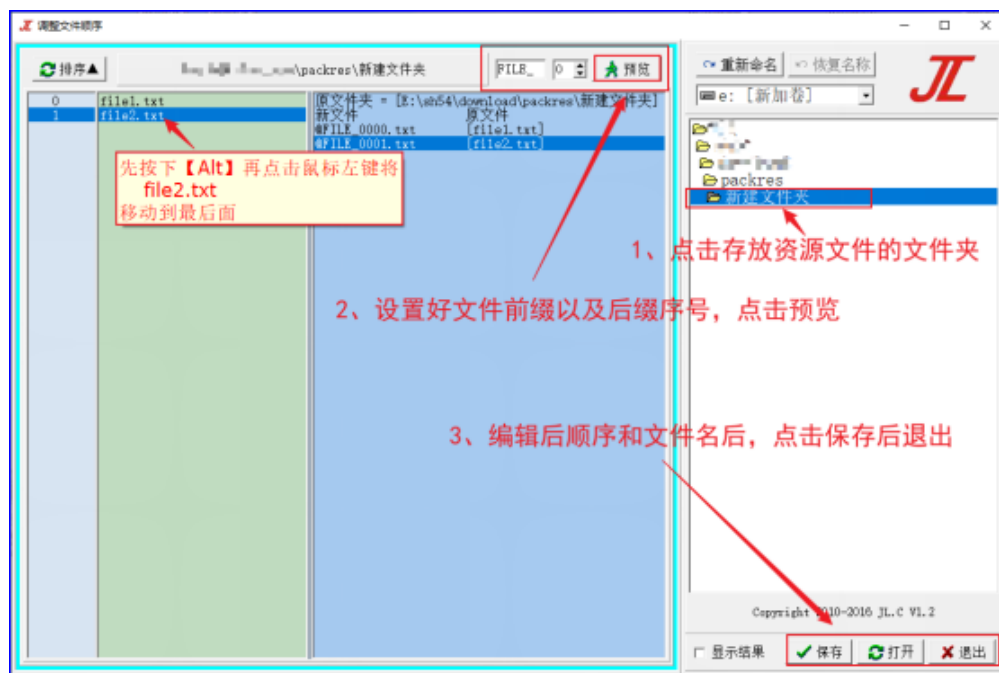


图 6.3 pRFiles.exe 打包工具操作图

2、将大量文件放在文件夹中，打开 pRFiles.exe，如下图所示进行操作，保存为 LST 文件，以保存为 TEST.LST 为例；

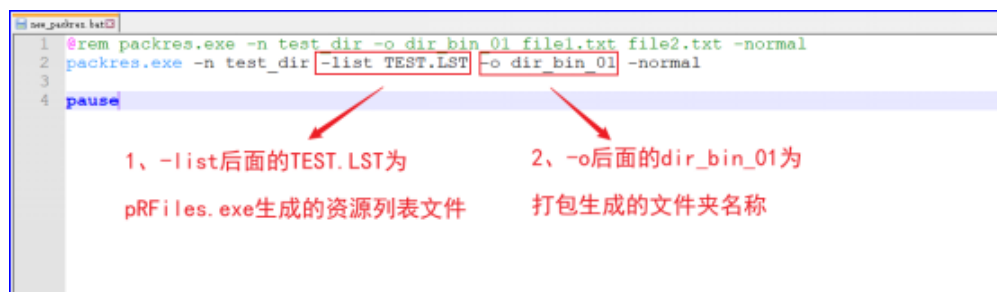


图 6.4 new_packres.bat 资源打包批处理说明

3、保存 new_packres.bat 文件，把资源文件全部放到工具根目录下，并双击该批处理，工具根目录下生成打包文件夹表示打包成功；

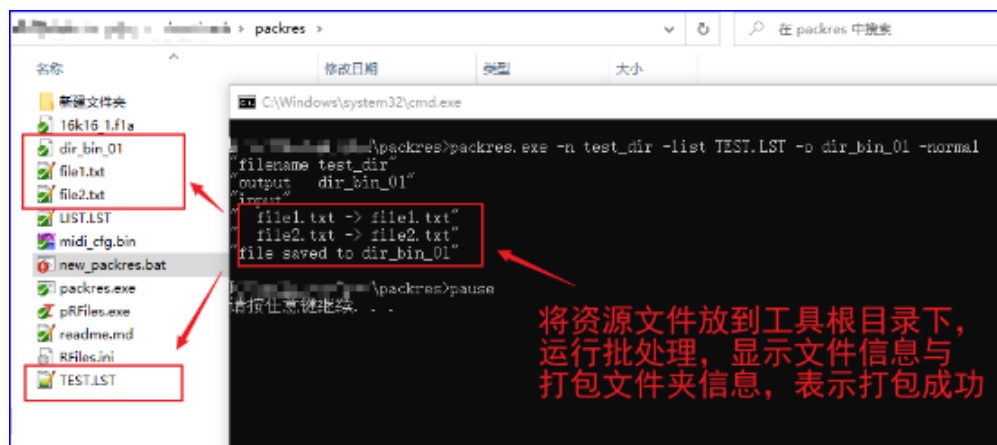


图 6.5 new_packres.bat 运行结果图

3.8.2 音频转换工具

音频文件转换工具 _1.2.17.exe 是我司用于压缩转换 WAV、MP3 和 WMA 音频的工具，可将音频转换成我司的 A (B、E) 格式、F1A (F1B、F1C) 格式、F1X 格式以及 UMP3 格式；

音频转换步骤

- 1、添加需要转换的音频文件；
- 2、选择输出音频的保存目录；
- 3、选择需要转换的格式、采样率、码率以及其他选项；
- 4、点击开始转换；

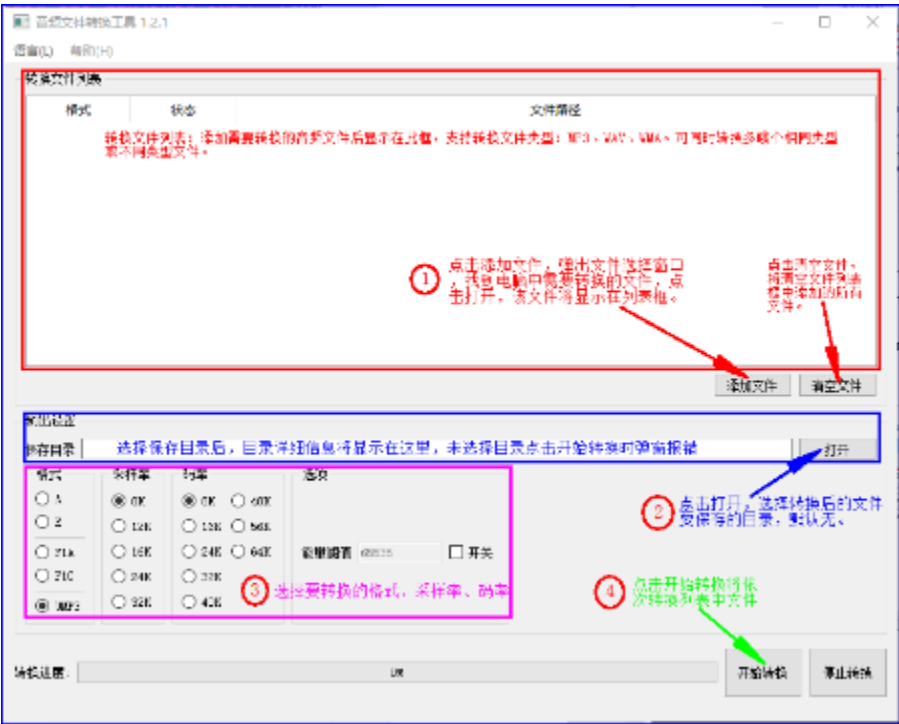


图 6.6 音频文件转换步骤

目标格式选择

转换工具可生成三种类型文件, 第一类 *.a、.b、.e; 第二类 *.fla、.flb、*.flc; 第三类 .ump3。每种类型都有采样率或码率供自由选择, 一般采样率高了对应选择的码率也会高些, 采样率跟码率都影响音质。

类型格式选择: 选哪个格式, 一般考虑音质、设备资源以及系统运行速度。例如需要音质高而 flash 大小够用, 系统资源够跑, 就选用高压缩比的类型。一般要低码率的话, 可以先选 fla/flb/flc, 然后系统需要速度快, 就选 a。如果 fla/flc 没法满足要求, 如压音乐之类的, 可以选 ump3。如需要重复播报的音频段或者音频间无缝衔接的, 可以选择 flx。

1、a、b、e 格式:

该格式转换压缩比较低, 速度快, 资源小。可用于人声或者背景音乐。该格式特点是音质较差, 但占用内存小。



图 6.7 a、b、e 格式配置图

2、fla、flb、flc、flx 格式:

该格式转换压缩比较高, 比较适用人声和音乐, 音质一般, 其中 fla、flb、flc 可用于无缝衔接, flx 格式可用于音频拼接。

格式	采样率	码率
☐ A	☒ 8K	☐ 8K ☐ 48K
☐ E	☐ 12K	☐ 16K ☒ 56K
☐ F1A	☐ 16K	☐ 24K ☐ 64K
☐ F1C	☐ 24K	☐ 32K
☐ F1X	☐ 32K	☐ 40K
☒ UMP3		

图 6.8 f1a、f1b、f1c、f1x 格式格式配置图

3、ump3 格式：

该格式有较多的采样率和码率可以选择，可以按情况来压缩音乐或者人声，该格式特点是占用内存大，音质较好。

格式	采样率	码率	选项
☐ A	☒ 8K	☒ 8K ☐ 48K	能量阈值 65535 ☐ 开关
☐ E	☐ 12K	☐ 16K ☐ 56K	
☐ F1A	☐ 16K	☐ 24K ☐ 64K	
☐ F1C	☐ 24K	☐ 32K	
☒ UMP3	☐ 32K	☐ 40K	

图 6.9 ump3 格式格式配置图

其他参数说明

1. f1a / f1b / f1c 无缝衔接转换和提高小文件压缩比

如果勾选是否无缝循环，则可以首尾无缝对接循环重复播放同一个音频。其能量转换阈值范围 0 到 65536，代表 0 到 1，默认参数配成 10，即砍掉接近完全静音的部分。运算过程中的音频也是归一化到 0 到 1 来计算的。所以实际应用中，调整的阈值范围一般是选取 0 到 0.1，约 0 到 6000。可以设成 2048。试下循环效果。如果勾选循环附加预处理，会对 f1a/f1b/f1c 文件再多做一次处理，分析计算文件前后的帧叠位置，计算前后的砍帧个数，然后对文件前后做一次前后自适应砍帧的动作。如果勾选提高小文件压缩比，会使压缩后的目标文件码率较为接近目标码率。f1a 是 cbr 的编码。需要长时间来平均来达到接近目标码率。如果文件都很小的时候，压出来的目标文件码率可能会距离目标码率差距较大。勾选提高小文件压缩比，会接近些。注：建议无缝循环的时候，只勾选是否无缝循环选项，听得出卡顿的文件，再勾选循环附加预处理，因为这个步骤的砍帧可能会出现有些文件前后会被砍掉较多的情况！

2. f1x 制作说明

制作 f1x 音频时，需要选择音频头字节数和头尾去除帧数。音频头字节数用于文件寻址，与原始转换文件大小有关。头字节数为 3 时，生成的 f1x 文件超过 16M 字节会报错，文件较多时可节约空间；头字节数为 4 时，则无限制。头尾去除帧数用于选择去除的 f1x 内各个音频文件头尾的音频数据包数，可以调整衔接效果。但不可完全依赖该功能，对于衔接效果不理想的音频，需要从音源文件去改善。

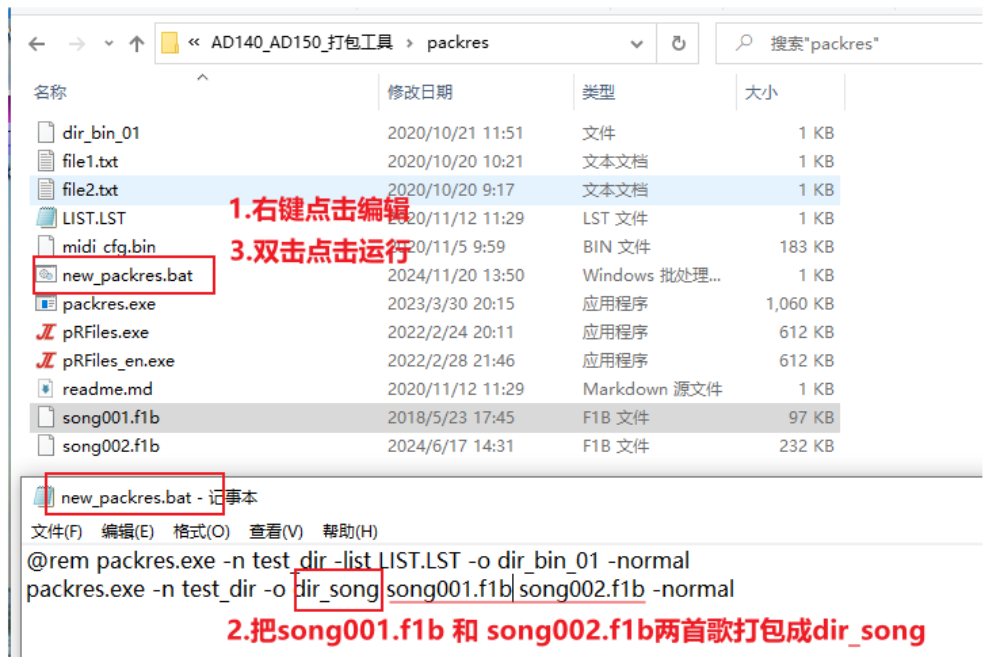
3. ump3 的能量阈值

ump3 格式有能量阈值选项，勾选开关便可以设置阈值，音频前后端会根据这个值砍掉一些低能量的音频，使播放开始和结束时更柔和。阈值范围：0-2147483648，默认参数配成 65535。即砍掉接近完全静音的部分，值越大能量剪切越多。鼠标悬停在能量阈值输入框无任何操作两秒以上会显示阈值范围。

通过二次打包将文件下载至外挂 flash 流程

1、将音频文件转换之后初次打包（具体打包参考：少量文件直接打包）；

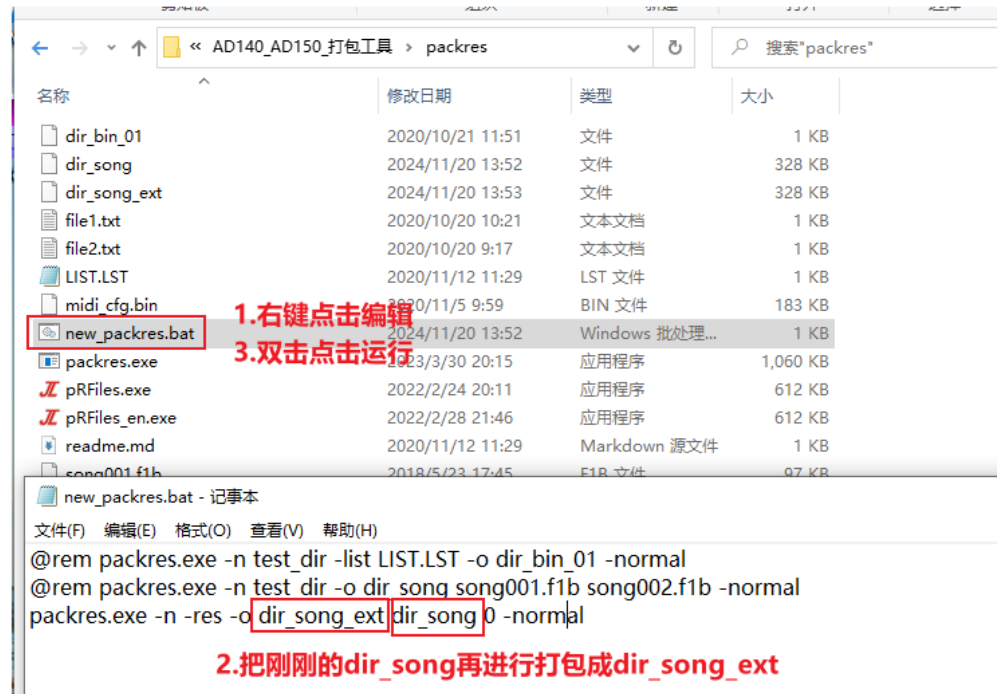
命令：packres.exe -n test_dir -o dir_song dx.f1b -normal



初次打包流程图

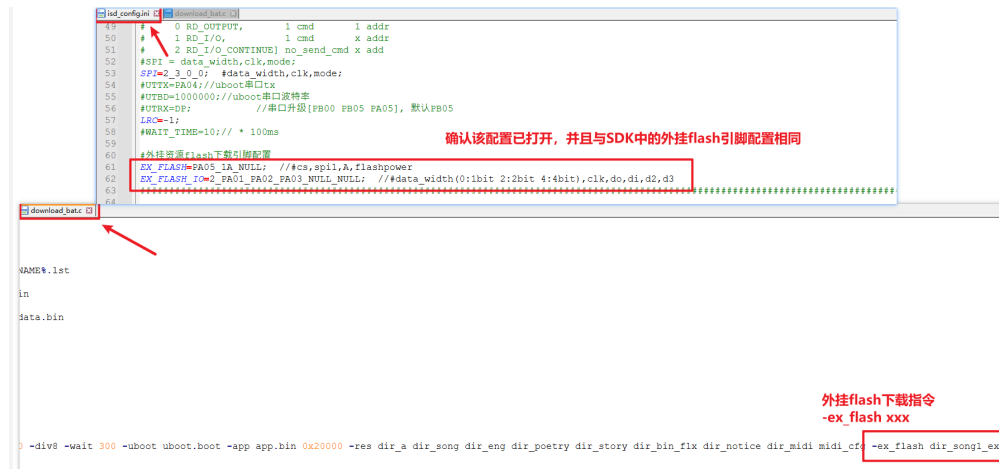
2、二次打包；

命令：packres.exe -n res -o dir_song2_ext dir_song 0 -normal



二次打包流程图

3、外挂 flash 的部分配置；



外挂 flash 配置图

4、正常编译下载后，可正常播放

杰理工具文档介绍

杰理工具文档都能在杰理科技的官网上找到, 杰理工具在线文档链接:

- **杰理工具文档根目录:**
<https://doc.zh-jieli.com/Tools/zh-cn/index.html>
 - **isd_config.ini 配置文件说明:**
https://doc.zh-jieli.com/Tools/zh-cn/dev_tools/toolchains/ini_cfg.html#isd-config-ini
 - **杰理科技强制升级工具 4.0 使用说明:**
https://doc.zh-jieli.com/Tools/zh-cn/dev_tools/forced_upgrade/index.html
-

AD16N 支持众多硬件中断，拥有 8 级中断优先级，支持中断嵌套；除了硬件中断，还支持软件中断；中断数量如下表所示：

CPU	中断数量	中断优先级
AD14N / AC104N	32 个（包含 4 个软中断）	0~7，共 8 级
AD15N	32 个（包含 4 个软中断）	0~7，共 8 级
AD17N	32 个（包含 4 个软中断）	0~7，共 8 级
AD18N	64 个（包含 4 个软中断）	0~7，共 8 级
AD16N	64 个（包含 4 个软中断）	0~7，共 8 级
AW30N	64 个（包含 8 个软中断）	0~7，共 8 级

注：中断优先级 0~7，共 8 级，数值越大，优先级越高

5.1 中断的注册与注销

玩具系列 SDK 注册一个中断，需要中断号、中断响应函数以及中断优先级；

- 中断号：每一个中断会分配一个唯一的中断号，AW30N 的中断号在 irq_vec.h 查看，玩具 AD 系列其余芯片可在 hwi.h 文件中查看。

注：中断号不可更改

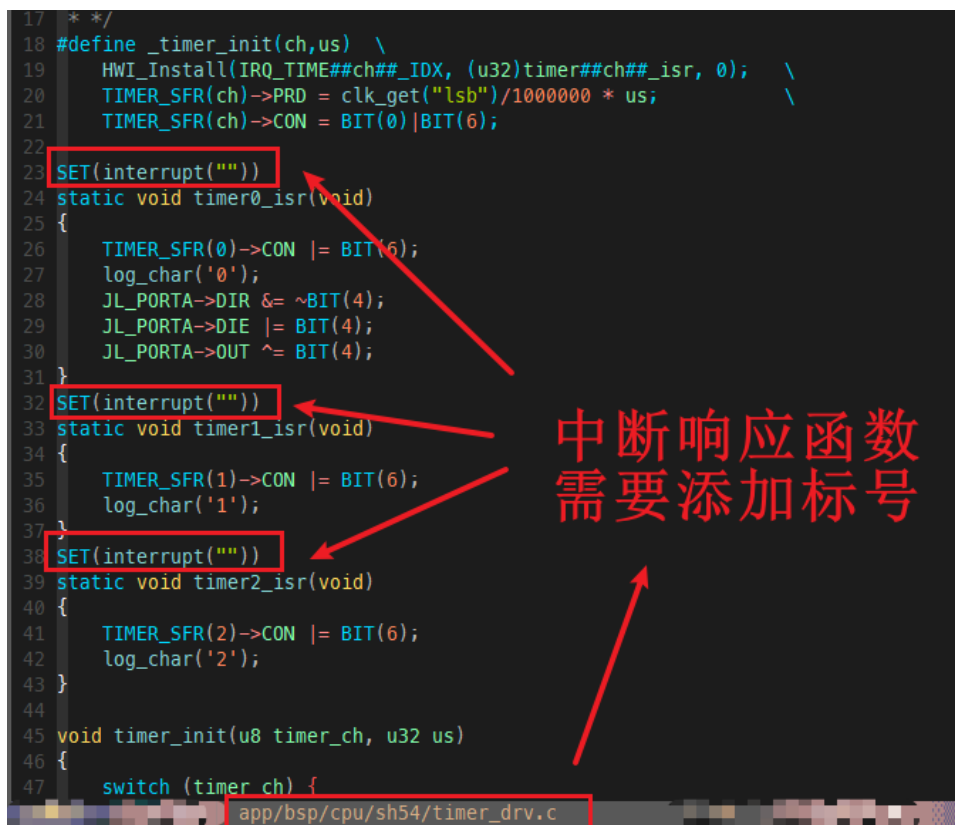
```

4
5 #define IRQ_BPCTL_IDX      0
6 #define IRQ_EXCEPTION_IDX  1
7 #define IRQ_SYSCALL_IDX    2
8 #define IRQ_TICKTMR_IDX    3
9 #define IRQ_TIME0_IDX      4
10 #define IRQ_TIME1_IDX      5
11 #define IRQ_TIME2_IDX      6
12 #define IRQ_UART0_IDX      10
13 #define IRQ_UART1_IDX      11
14
15 #define IRQ_SPI0_IDX        13
16 #define IRQ_SPI1_IDX        14
17
18 #define IRQ_SD0_IDX          16
19 #define IRQ_IIC_IDX          17
20 #define IRQ_USB_SOF_IDX      18
21 #define IRQ_USB_CTRL_IDX     19
22 #define IRQ_PMU_SOFT_IDX     20
23
24
25 #define IRQ_PORT_IDX         25
26 #define IRQ_ADC_IDX          26
27 #define IRQ_LED_IDX          27
28 #define IRQ_OSA_IDX          28
29 #define IRQ_LRCT_IDX         29
30 #define IRQ_GPCNT_IDX        30
31 #define IRQ_LEDC_IDX         31
32
33 #define IRQ_RDEC0_IDX        33
34 #define IRQ_SLCD_IDX         34
35 #define IRQ_PPS_IDX          35
36 #define IRQ_PPM_IDX          36
37
38 #define IRQ_MCPWM_TMR_IDX    38
39 #define IRQ_MCPWM_CHX_IDX    39
40
41 #define IRQ_AUDIO_IDX        46
42
43 #define IRQ_SRC_HW_IDX        49
44 #define IRQ_EQ_IDX           50
45
46 #define IRQ_SOFT0_IDX        60
47 #define IRQ_SOFT1_IDX        61
48 #define IRQ_SOFT2_IDX        62
49 #define IRQ_SOFT3_IDX        63
50
51 #define MAX_IRQ_ENTRY_NUM 64
52
53
54 //系统使用到的
55 extern const int IRQ_IRTMR_IP;
56 extern const int IRQ_AUDIO_IP;
57 extern const int IRQ_DECODER_IP;
58 //apps/include_lib/cpu/uc03/hwi.h

```

AW30N 中断号示意图

- 中断服务函数：中断服务函数类型必须为无返回值的 void 类型，且中断函数定义时需添加中断标志；

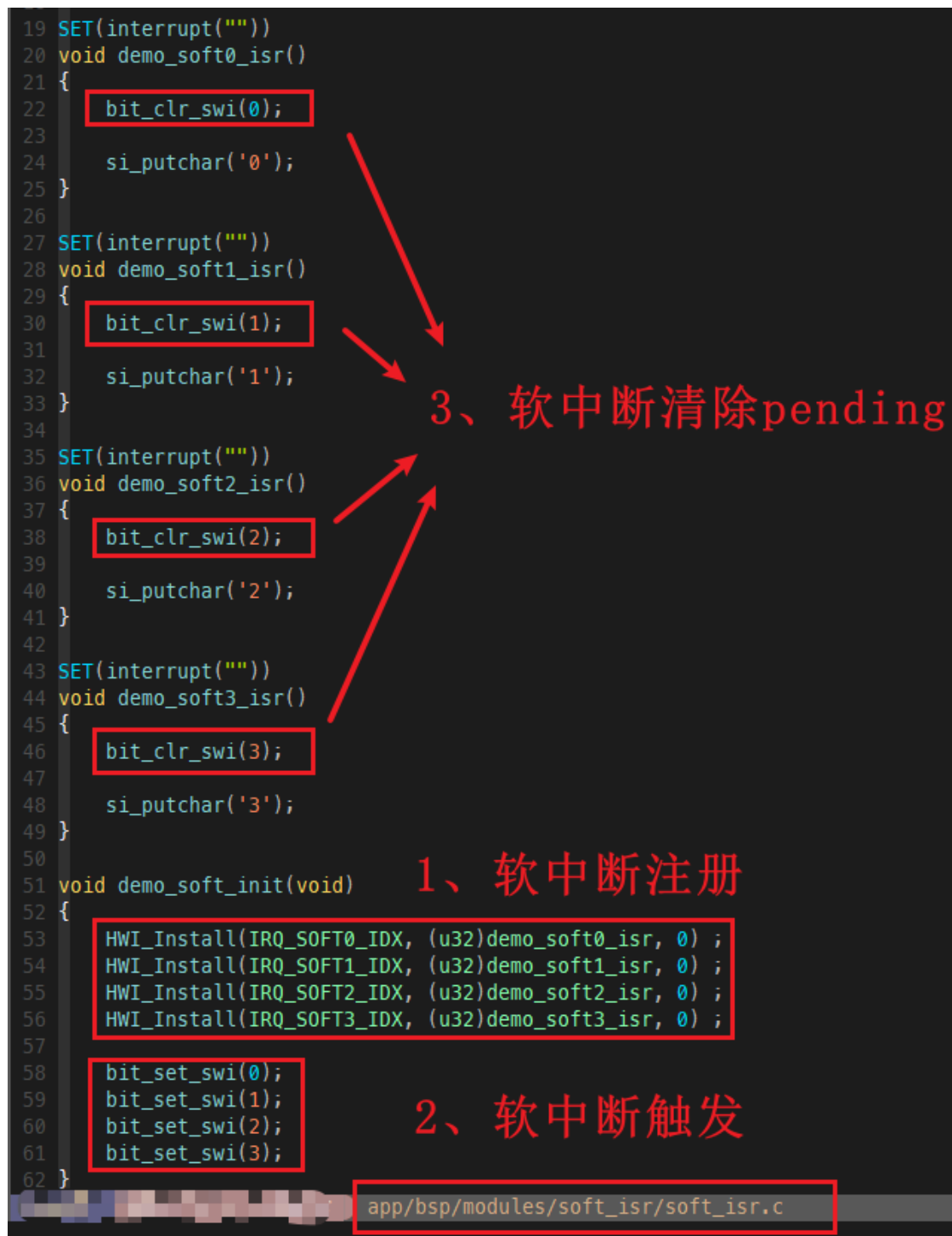


```

17  */
18  #define _timer_init(ch,us) \
19      HWI_Install(IRQ_TIME##ch##_IDX, (u32)timer##ch##_isr, 0); \
20      TIMER_SFR(ch)->PRD = clk_get("lsb")/1000000 * us; \
21      TIMER_SFR(ch)->CON = BIT(0)|BIT(6);
22
23  SET(interrupt(" "))
24  static void timer0_isr(void)
25  {
26      TIMER_SFR(0)->CON |= BIT(6);
27      log_char('0');
28      JL_PORTA->DIR &= ~BIT(4);
29      JL_PORTA->DIE |= BIT(4);
30      JL_PORTA->OUT ^= BIT(4);
31  }
32  SET(interrupt(" "))
33  static void timer1_isr(void)
34  {
35      TIMER_SFR(1)->CON |= BIT(6);
36      log_char('1');
37  }
38  SET(interrupt(" "))
39  static void timer2_isr(void)
40  {
41      TIMER_SFR(2)->CON |= BIT(6);
42      log_char('2');
43  }
44
45  void timer_init(u8 timer_ch, u32 us)
46  {
47      switch (timer_ch) {
48          case 0: timer0_isr(); break;
49          case 1: timer1_isr(); break;
50          case 2: timer2_isr(); break;
51      }
52  }
53  #endif
54
55  #endif
56
57  #endif
58
59  #endif
60
61  #endif
62
63  #endif
64
65  #endif
66
67  #endif
68
69  #endif
70
71  #endif
72
73  #endif
74
75  #endif
76
77  #endif
78
79  #endif
80
81  #endif
82
83  #endif
84
85  #endif
86
87  #endif
88
89  #endif
90
91  #endif
92
93  #endif
94
95  #endif
96
97  #endif
98
99  #endif
100
101  #endif
102
103  #endif
104
105  #endif
106
107  #endif
108
109  #endif
110
111  #endif
112
113  #endif
114
115  #endif
116
117  #endif
118
119  #endif
120
121  #endif
122
123  #endif
124
125  #endif
126
127  #endif
128
129  #endif
130
131  #endif
132
133  #endif
134
135  #endif
136
137  #endif
138
139  #endif
140
141  #endif
142
143  #endif
144
145  #endif
146
147  #endif
148
149  #endif
150
151  #endif
152
153  #endif
154
155  #endif
156
157  #endif
158
159  #endif
160
161  #endif
162
163  #endif
164
165  #endif
166
167  #endif
168
169  #endif
170
171  #endif
172
173  #endif
174
175  #endif
176
177  #endif
178
179  #endif
180
181  #endif
182
183  #endif
184
185  #endif
186
187  #endif
188
189  #endif
190
191  #endif
192
193  #endif
194
195  #endif
196
197  #endif
198
199  #endif
200
201  #endif
202
203  #endif
204
205  #endif
206
207  #endif
208
209  #endif
210
211  #endif
212
213  #endif
214
215  #endif
216
217  #endif
218
219  #endif
220
221  #endif
222
223  #endif
224
225  #endif
226
227  #endif
228
229  #endif
230
231  #endif
232
233  #endif
234
235  #endif
236
237  #endif
238
239  #endif
240
241  #endif
242
243  #endif
244
245  #endif
246
247  #endif
248
249  #endif
250
251  #endif
252
253  #endif
254
255  #endif
256
257  #endif
258
259  #endif
260
261  #endif
262
263  #endif
264
265  #endif
266
267  #endif
268
269  #endif
270
271  #endif
272
273  #endif
274
275  #endif
276
277  #endif
278
279  #endif
280
281  #endif
282
283  #endif
284
285  #endif
286
287  #endif
288
289  #endif
290
291  #endif
292
293  #endif
294
295  #endif
296
297  #endif
298
299  #endif
300
301  #endif
302
303  #endif
304
305  #endif
306
307  #endif
308
309  #endif
310
311  #endif
312
313  #endif
314
315  #endif
316
317  #endif
318
319  #endif
320
321  #endif
322
323  #endif
324
325  #endif
326
327  #endif
328
329  #endif
330
331  #endif
332
333  #endif
334
335  #endif
336
337  #endif
338
339  #endif
340
341  #endif
342
343  #endif
344
345  #endif
346
347  #endif
348
349  #endif
350
351  #endif
352
353  #endif
354
355  #endif
356
357  #endif
358
359  #endif
360
361  #endif
362
363  #endif
364
365  #endif
366
367  #endif
368
369  #endif
370
371  #endif
372
373  #endif
374
375  #endif
376
377  #endif
378
379  #endif
380
381  #endif
382
383  #endif
384
385  #endif
386
387  #endif
388
389  #endif
390
391  #endif
392
393  #endif
394
395  #endif
396
397  #endif
398
399  #endif
400
401  #endif
402
403  #endif
404
405  #endif
406
407  #endif
408
409  #endif
410
411  #endif
412
413  #endif
414
415  #endif
416
417  #endif
418
419  #endif
420
421  #endif
422
423  #endif
424
425  #endif
426
427  #endif
428
429  #endif
430
431  #endif
432
433  #endif
434
435  #endif
436
437  #endif
438
439  #endif
440
441  #endif
442
443  #endif
444
445  #endif
446
447  #endif
448
449  #endif
450
451  #endif
452
453  #endif
454
455  #endif
456
457  #endif
458
459  #endif
460
461  #endif
462
463  #endif
464
465  #endif
466
467  #endif
468
469  #endif
470
471  #endif
472
473  #endif
474
475  #endif
476
477  #endif
478
479  #endif
480
481  #endif
482
483  #endif
484
485  #endif
486
487  #endif
488
489  #endif
490
491  #endif
492
493  #endif
494
495  #endif
496
497  #endif
498
499  #endif
500
501  #endif
502
503  #endif
504
505  #endif
506
507  #endif
508
509  #endif
510
511  #endif
512
513  #endif
514
515  #endif
516
517  #endif
518
519  #endif
520
521  #endif
522
523  #endif
524
525  #endif
526
527  #endif
528
529  #endif
530
531  #endif
532
533  #endif
534
535  #endif
536
537  #endif
538
539  #endif
540
541  #endif
542
543  #endif
544
545  #endif
546
547  #endif
548
549  #endif
550
551  #endif
552
553  #endif
554
555  #endif
556
557  #endif
558
559  #endif
560
561  #endif
562
563  #endif
564
565  #endif
566
567  #endif
568
569  #endif
570
571  #endif
572
573  #endif
574
575  #endif
576
577  #endif
578
579  #endif
580
581  #endif
582
583  #endif
584
585  #endif
586
587  #endif
588
589  #endif
590
591  #endif
592
593  #endif
594
595  #endif
596
597  #endif
598
599  #endif
600
601  #endif
602
603  #endif
604
605  #endif
606
607  #endif
608
609  #endif
610
611  #endif
612
613  #endif
614
615  #endif
616
617  #endif
618
619  #endif
620
621  #endif
622
623  #endif
624
625  #endif
626
627  #endif
628
629  #endif
630
631  #endif
632
633  #endif
634
635  #endif
636
637  #endif
638
639  #endif
640
641  #endif
642
643  #endif
644
645  #endif
646
647  #endif
648
649  #endif
650
651  #endif
652
653  #endif
654
655  #endif
656
657  #endif
658
659  #endif
660
661  #endif
662
663  #endif
664
665  #endif
666
667  #endif
668
669  #endif
670
671  #endif
672
673  #endif
674
675  #endif
676
677  #endif
678
679  #endif
680
681  #endif
682
683  #endif
684
685  #endif
686
687  #endif
688
689  #endif
690
691  #endif
692
693  #endif
694
695  #endif
696
697  #endif
698
699  #endif
700
701  #endif
702
703  #endif
704
705  #endif
706
707  #endif
708
709  #endif
710
711  #endif
712
713  #endif
714
715  #endif
716
717  #endif
718
719  #endif
720
721  #endif
722
723  #endif
724
725  #endif
726
727  #endif
728
729  #endif
730
731  #endif
732
733  #endif
734
735  #endif
736
737  #endif
738
739  #endif
740
741  #endif
742
743  #endif
744
745  #endif
746
747  #endif
748
749  #endif
750
751  #endif
752
753  #endif
754
755  #endif
756
757  #endif
758
759  #endif
760
761  #endif
762
763  #endif
764
765  #endif
766
767  #endif
768
769  #endif
770
771  #endif
772
773  #endif
774
775  #endif
776
777  #endif
778
779  #endif
780
781  #endif
782
783  #endif
784
785  #endif
786
787  #endif
788
789  #endif
790
791  #endif
792
793  #endif
794
795  #endif
796
797  #endif
798
799  #endif
800
801  #endif
802
803  #endif
804
805  #endif
806
807  #endif
808
809  #endif
810
811  #endif
812
813  #endif
814
815  #endif
816
817  #endif
818
819  #endif
820
821  #endif
822
823  #endif
824
825  #endif
826
827  #endif
828
829  #endif
830
831  #endif
832
833  #endif
834
835  #endif
836
837  #endif
838
839  #endif
840
841  #endif
842
843  #endif
844
845  #endif
846
847  #endif
848
849  #endif
850
851  #endif
852
853  #endif
854
855  #endif
856
857  #endif
858
859  #endif
860
861  #endif
862
863  #endif
864
865  #endif
866
867  #endif
868
869  #endif
870
871  #endif
872
873  #endif
874
875  #endif
876
877  #endif
878
879  #endif
880
881  #endif
882
883  #endif
884
885  #endif
886
887  #endif
888
889  #endif
890
891  #endif
892
893  #endif
894
895  #endif
896
897  #endif
898
899  #endif
900
901  #endif
902
903  #endif
904
905  #endif
906
907  #endif
908
909  #endif
910
911  #endif
912
913  #endif
914
915  #endif
916
917  #endif
918
919  #endif
920
921  #endif
922
923  #endif
924
925  #endif
926
927  #endif
928
929  #endif
930
931  #endif
932
933  #endif
934
935  #endif
936
937  #endif
938
939  #endif
940
941  #endif
942
943  #endif
944
945  #endif
946
947  #endif
948
949  #endif
950
951  #endif
952
953  #endif
954
955  #endif
956
957  #endif
958
959  #endif
960
961  #endif
962
963  #endif
964
965  #endif
966
967  #endif
968
969  #endif
970
971  #endif
972
973  #endif
974
975  #endif
976
977  #endif
978
979  #endif
980
981  #endif
982
983  #endif
984
985  #endif
986
987  #endif
988
989  #endif
990
991  #endif
992
993  #endif
994
995  #endif
996
997  #endif
998
999  #endif
1000
1001  #endif
1002
1003  #endif
1004
1005  #endif
1006
1007  #endif
1008
1009  #endif
1010
1011  #endif
1012
1013  #endif
1014
1015  #endif
1016
1017  #endif
1018
1019  #endif
1020
1021  #endif
1022
1023  #endif
1024
1025  #endif
1026
1027  #endif
1028
1029  #endif
1030
1031  #endif
1032
1033  #endif
1034
1035  #endif
1036
1037  #endif
1038
1039  #endif
1040
1041  #endif
1042
1043  #endif
1044
1045  #endif
1046
1047  #endif
1048
1049  #endif
1050
1051  #endif
1052
1053  #endif
1054
1055  #endif
1056
1057  #endif
1058
1059  #endif
1060
1061  #endif
1062
1063  #endif
1064
1065  #endif
1066
1067  #endif
1068
1069  #endif
1070
1071  #endif
1072
1073  #endif
1074
1075  #endif
1076
1077  #endif
1078
1079  #endif
1080
1081  #endif
1082
1083  #endif
1084
1085  #endif
1086
1087  #endif
1088
1089  #endif
1090
1091  #endif
1092
1093  #endif
1094
1095  #endif
1096
1097  #endif
1098
1099  #endif
1100
1101  #endif
1102
1103  #endif
1104
1105  #endif
1106
1107  #endif
1108
1109  #endif
1110
1111  #endif
1112
1113  #endif
1114
1115  #endif
1116
1117  #endif
1118
1119  #endif
1120
1121  #endif
1122
1123  #endif
1124
1125  #endif
1126
1127  #endif
1128
1129  #endif
1130
1131  #endif
1132
1133  #endif
1134
1135  #endif
1136
1137  #endif
1138
1139  #endif
1140
1141  #endif
1142
1143  #endif
1144
1145  #endif
1146
1147  #endif
1148
1149  #endif
1150
1151  #endif
1152
1153  #endif
1154
1155  #endif
1156
1157  #endif
1158
1159  #endif
1160
1161  #endif
1162
1163  #endif
1164
1165  #endif
1166
1167  #endif
1168
1169  #endif
1170
1171  #endif
1172
1173  #endif
1174
1175  #endif
1176
1177  #endif
1178
1179  #endif
1180
1181  #endif
1182
1183  #endif
1184
1185  #endif
1186
1187  #endif
1188
1189  #endif
1190
1191  #endif
1192
1193  #endif
1194
1195  #endif
1196
1197  #endif
1198
1199  #endif
1200
1201  #endif
1202
1203  #endif
1204
1205  #endif
1206
1207  #endif
1208
1209  #endif
1210
1211  #endif
1212
1213  #endif
1214
1215  #endif
1216
1217  #endif
1218
1219  #endif
1220
1221  #endif
1222
1223  #endif
1224
1225  #endif
1226
1227  #endif
1228
1229  #endif
1230
1231  #endif
1232
1233  #endif
1234
1235  #endif
1236
1237  #endif
1238
1239  #endif
1240
1241  #endif
1242
1243  #endif
1244
1245  #endif
1246
1247  #endif
1248
1249  #endif
1250
1251  #endif
1252
1253  #endif
1254
1255  #endif
1256
1257  #endif
1258
1259  #endif
1260
1261  #endif
1262
1263  #endif
1264
1265  #endif
1266
1267  #endif
1268
1269  #endif
1270
1271  #endif
1272
1273  #endif
1274
1275  #endif
1276
1277  #endif
1278
1279  #endif
1280
1281  #endif
1282
1283  #endif
1284
1285  #endif
1286
1287  #endif
1288
1289  #endif
1290
1291  #endif
1292
1293  #endif
1294
1295  #endif
1296
1297  #endif
1298
1299  #endif
1300
1301  #endif
1302
1303  #endif
1304
1305  #endif
1306
1307  #endif
1308
1309  #endif
1310
1311  #endif
1312
1313  #endif
1314
1315  #endif
1316
1317  #endif
1318
1319  #endif
1320
1321  #endif
1322
1323  #endif
1324
1325  #endif
1326
1327  #endif
1328
1329  #endif
1330
1331  #endif
1332
1333  #endif
1334
1335  #endif
1336
1337  #endif
1338
1339  #endif
1340
1341  #endif
1342
1343  #endif
1344
1345  #endif
1346
1347  #endif
1348
1349  #endif
1350
1351  #endif
1352
1353  #endif
1354
1355  #endif
1356
1357  #endif
1358
1359  #endif
1360
1361  #endif
1362
1363  #endif
1364
1365  #endif
1366
1367  #endif
1368
1369  #endif
1370
1371  #endif
1372
1373  #endif
1374
1375  #endif
1376
1377  #endif
1378
1379  #endif
1380
1381  #endif
1382
1383  #endif
1384
1385  #endif
1386
1387  #endif
1388
1389  #endif
1390
1391  #endif
1392
1393  #endif
1394
1395  #endif
1396
1397  #endif
1398
1399  #endif
1400
1401  #endif
1402
1403  #endif
1404
1405  #endif
1406
1407  #endif
1408
1409  #endif
1410
1411  #endif
1412
1413  #endif
1414
1415  #endif
1416
1417  #endif
1418
1419  #endif
1420
1421  #endif
1422
1423  #endif
1424
1425  #endif
1426
1427  #endif
1428
1429  #endif
1430
1431  #endif
1432
1433  #endif
1434
1435  #endif
1436
1437  #endif
1438
1439  #endif
1440
1441  #endif
1442
1443  #endif
1444
1445  #endif
1446
1447  #endif
1448
1449  #endif
1450
1451  #endif
1452
1453  #endif
1454
1455  #endif
1456
1457  #endif
1458
1459  #endif
1460
1461  #endif
1462
1463  #endif
1464
1465  #endif
1466
1467  #endif
1468
1469  #endif
1470
1471  #endif
1472
1473  #endif
1474
1475  #endif
1476
1477  #endif
1478
1479  #endif
1480
1481  #endif
1482
1483  #endif
1484
1485  #endif
1486
1487  #endif
1488
1489  #endif
1490
1491  #endif
1492
1493  #endif
1494
1495  #endif
1496
1497  #endif
1498
1499  #endif
1500
1501  #endif
1502
1503  #endif
1504
1505  #endif
1506
1507  #endif
1508
1509  #endif
1510
1511  #endif
1512
1513  #endif
1514
1515  #endif
1516
1517  #endif
1518
1519  #endif
1520
1521  #endif
1522
1523  #endif
1524
1525  #endif
1526
1527  #endif
1528
1529  #endif
1530
1531  #endif
1532
1533  #endif
1534
1535  #endif
1536
1537  #endif
1538
1539  #endif
1540
1541  #endif
1542
1543  #endif
1544
1545  #endif
1546
1547  #endif
1548
1549  #endif
1550
1551  #endif
1552
1553  #endif
1554
1555  #endif
1556
1557  #endif
1558
1559  #endif
1560
1561  #endif
1562
1563  #endif
1564
1565  #endif
1566
1567  #endif
1568
1569  #endif
1570
1571  #endif
1572
1573  #endif
1574
1575  #endif
1576
1577  #endif
1578
1579  #endif
1580
1581  #endif
1582
1583  #endif
1584
1585  #endif
1586
1587  #endif
1588
1589  #endif
1590
1591  #endif
1592
1593  #endif
1594
1595  #endif
1596
1597  #endif
1598
1599  #endif
1600
1601  #endif
1602
1603  #endif
1604
1605  #endif
1606
1607  #endif
1608
1609  #endif
1610
1611  #endif
1612
1613  #endif
1614
1615  #endif
1616
1617  #endif
1618
1619  #endif
1620
1621  #endif
1622
1623  #endif
1624
1625  #endif
1626
1627  #endif
1628
1629  #endif
1630
1631  #endif
1632
1633  #endif
1634
1635  #endif
1636
1637  #endif
1638
1639  #endif
1640
1641  #endif
1642
1643  #endif
1644
1645  #endif
1646
1647  #endif
1648
1649  #endif
1650
1651  #endif
1652
1653  #endif
1654
1655  #endif
1656
1657  #endif
1658
1659  #endif
1660
1661  #endif
1662
1663  #endif
1664
1665  #endif
1666
1667  #endif
1668
1669  #endif
1670
1671  #endif
1672
1673  #endif
1674
1675  #endif
1676
1677  #endif
1678
1679  #endif
1680
1681  #endif
1682
1683  #endif
1684
1685  #endif
1686
1687  #endif
1688
1689  #endif
1690
1691  #endif
1692
1693  #endif
1694
1695  #endif
1696
1697  #endif
1698
1699  #endif
1700
1701  #endif
1702
1703  #endif
1704
1705  #endif
1706
1707  #endif
1708
1709  #endif
1710
1711  #endif
1712
1713  #endif
1714
1715  #endif
1716
1717  #endif
1718
1719  #endif
1720
1721  #endif
1722
1723  #endif
1724
1725  #endif
1726
1727  #endif
1728
1729  #endif
1730
1731  #endif
1732
1733  #endif
1734
1735  #endif
1736
1737  #endif
1738
1739  #endif
1740
1741  #endif
1742
1743  #endif
1744
1745  #endif
1746
1747  #endif
1748
1749  #endif
1750
1751  #endif
1752
1753  #endif
1754
1755  #endif
1756
1757  #endif
1758
1759  #endif
1760
1761  #endif
1762
1763  #endif
1764
1765  #endif
1766
1767  #endif
1768
1769  #endif
1770
1771  #endif
1772
1773  #endif
1774
1775  #endif
1776
1777  #endif
1778
1779  #endif
1780
1781  #endif
1782
1783  #endif
1784
1785  #endif
1786
1787  #endif
1788
1789  #endif
1790
1791  #endif
1792
1793  #endif
1794
1795  #endif
1796
1797  #endif
1798
1799  #endif
1800
1801  #endif
1802
1803  #endif
1804
1805  #endif
1806
1807  #endif
1808
1809  #endif
1810
1811  #endif
1812
1813  #endif
1814
1815  #endif
1816
1817  #endif
1818
1819  #endif
1820
1821  #endif
1822
1823  #endif
1824
1825  #endif
1826
1827  #endif
1828
1829  #endif
1830
1831  #endif
1832
1833  #endif
1834
1835  #endif
1836
1837  #endif
1838
1839  #endif
1840
1841  #endif
1842
1843  #endif
1844
1845  #endif
1846
1847  #endif
1848
1849  #endif
1850
1851  #endif
1852
1853  #endif
1854
1855  #endif
1856
1857  #endif
1858
1859  #endif
1860
1861  #endif
1862
1863  #endif
1864
1865  #endif
1866
1867  #endif
1868
1869  #endif
1870
1871  #endif
1872
1873  #endif
1874
1875  #endif
1876
1877  #endif
1878
1879  #endif
1880
1881  #endif
1882
1883  #endif
1884
1885  #endif
1886
1887  #endif
1888
1889  #endif
1890
1891  #endif
1892
1893  #endif
1894
1895  #endif
1896
1897  #endif
1898
1899  #endif
1900
1901  #endif
1902
1903  #endif
1904
1905  #endif
1906
1907  #endif
1908
1909  #endif
1910
1911  #endif
1912
1913  #endif
1914
1915  #endif
1916
1917  #endif
1918
1919  #endif
1920
1921  #endif
1922
1923  #endif
1924
1925  #endif
1926
1927  #endif
1928
1929  #endif
1930
1931  #endif
1932
1933  #endif
1934
1935  #endif
1936
1937  #endif
1938
1939  #endif
1940
1941  #endif
1942
1943  #endif
1944
1945  #endif
1946
1947  #endif
1948
1949  #endif
1950
1951  #endif
1952
1953  #endif
1954
1955  #endif
1956
1957  #endif
1958
1959  #endif
1960
1961  #endif
1962
1963  #endif
1964
1965  #endif
1966
1967  #endif
1968
1969  #endif
1970
1971  #endif
1972
1973  #endif
1974
1975  #endif
1976
1977  #endif
1978
1979  #endif
1980
1981  #endif
1982
1983  #endif
1984
1985  #endif
1986
1987  #endif
1988
1989  #endif
1990
1991  #endif
1992
1993  #endif
1994
1995  #endif
1996
1997  #endif
1998
1999  #endif
2000
2001  #endif
2002
2003  #endif
2004
2005  #endif
2006
2007  #endif
2008
2009  #endif
20
```

5.2 软中断说明

玩具系列 SDK 支持软件中断; 软件中断的注册、注销方式与硬件中断一致; 用户可在 AD14N_SDK 的 soft_isr.c 文件中查看示例:



软件中断使用示例

5.2.1 函数 void bit_set_swi(unsigned char index)

该函数实现触发一个软件中断，其中参数：

1、index：需要触发的软件中断序号（软中断0~软中断3分别对应0~3）；

注：需要注意此处 *index* 与中断号不同；例如软中断2的中断号在 *ADI4N* 上是30，需要触发该中断时，该接口传参的 *index* 传2即可！

5.2.2 函数 void bit_clr_swi(unsigned char index)

该函数实现清除一个软中断的 Pending，其中参数：

1、index：需要清除pending的软件中断序号（软中断0~软中断3分别对应0~3）；

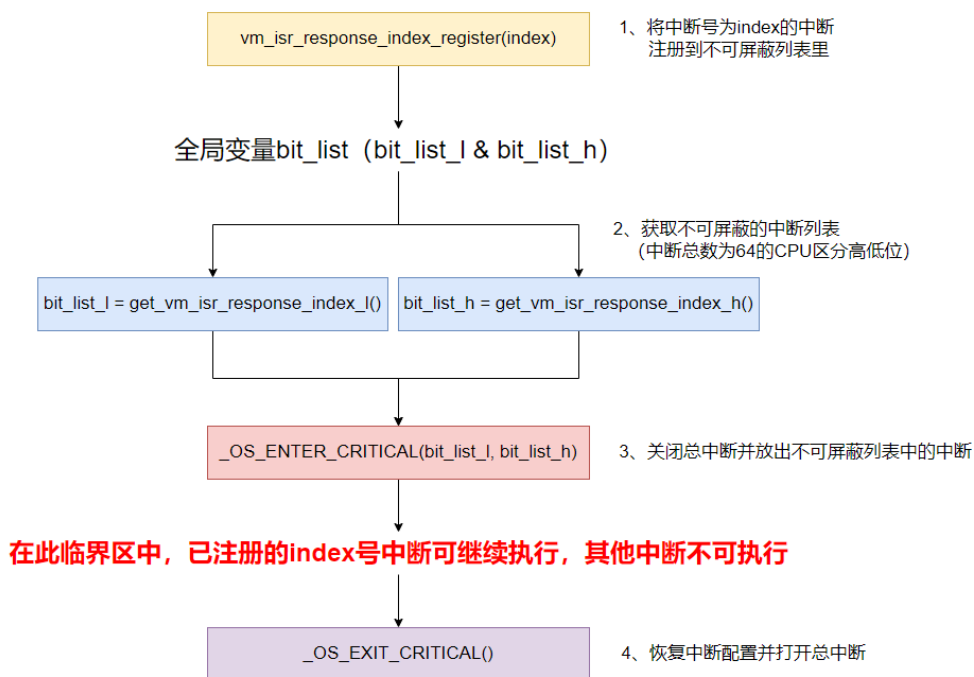
注：需要注意此处 *index* 与中断号不同；软中断2的中断号在 *ADI4N* 上是30，需要清除该中断 *pending* 时，该接口传参的 *index* 传2即可！

5.3 软件不可屏蔽中断

在进行 VM 擦写时系统会关闭总中断并操作系统 flash，该过程耗时较长，可能会导致播歌卡顿或其他外设中断无法及时响应；

SDK 提供了软件不可屏蔽中断，在关闭总中断的同时允许放出部分中断继续执行（例如放出 audio 中断优化操作 vm 卡音问题），该部分中断函数以及依赖的变量都需要放在 ram 里；

本小节介绍软件不可屏蔽中断相关的接口与使用，流程如下：



软件不可屏蔽中断运行流程图

5.3.1 函数 void _OS_ENTER_CRITICAL(u32 bit_list_l, u32 bit_list_h)

该函数适用于中断总数为 64 的芯片 (AW30N / AD16N / AD18N 等), 实现关闭总中断的同时放出 bit_list_l 和 bit_list_h 参数中置 1 的中断, 其中参数:

- 1、bit_list_l: 需要单独放出的中断list的低32位, bit0~bit31分别对应0号~
→31号中断;
- 2、bit_list_h: 需要单独放出的中断list的高32位, bit0~bit31分别对应32号~
→63号中断;

例如 AD18N 需要同时放出音频 APA 中断 (中断号为 37) 和 ticktimer(中断号为 3), 那么 bit_list_h 传入 0x20, bit_list_l 传入 0x8 即可;

5.3.2 函数 void _OS_EXIT_CRITICAL(void)

该函数适用于中断总数为 32 和 64 的芯片, 实现恢复中断配置并退出关中断状态;

5.3.3 函数 void vm_isr_response_index_register(u8 index)

SDK 中存在一个全局变量用于存储需要放出来的中断 list, 该函数实现往该变量中添加中断 bit, 其中参数:

- 1、index: 需要添加的不屏蔽中断号; 例如AD14N的audio中断号为9, 则传入9即可;

5.3.4 函数 void vm_isr_response_index_unregister(u8 index)

该函数适用于中断总数为 32 和 64 的芯片, 实现恢复中断配置并退出关中断状态, 其中参数:

- 1、index: 删除已添加的不屏蔽中断号;

5.3.5 函数 u32 get_vm_isr_response_index_l(void)

该函数实现获取全局的不可屏蔽中断 list 的低 32 位, 其中:

- 1、对于中断总数为32的芯片, 获取到完整的bit_list;
- 2、对于中断总数为64的芯片, 获取到低32位中断的bit_list;

5.3.6 函数 u32 get_vm_isr_response_index_h(void)

该函数实现获取全局的不可屏蔽中断 list 的高 32 位，仅中断总数为 64 的芯片有该接口；

5.4 硬件不可屏蔽中断

硬件不可屏蔽中断允许中断在关闭总中断 (即调用了函数 local_irq_disable()) 的情况下继续运行；目前支持硬件不可屏蔽中断的芯片有：AD16N、AD17N、AD18N 以及 AW30N；

注：若需要中断完全不被打断，需要软硬件不可屏蔽中断功能搭配使用；

5.4.1 函数 void irq_unmask_set(u8 index)

该函数实现将传入的 index 中断的优先级设置为最高，并加入硬件不可屏蔽中断列表；其中参数：

1、index：需要添加的不屏蔽中断号；

5.4.2 函数 void irq_unmask_disable(u8 index)

该函数实现关闭硬件不可屏蔽中断功能；

系统 FLASH 写保护功能

本章主要介绍如何将系统 flash 的代码与资源区域设置为写保护状态，该功能适用于 ad16n_flash_release_v1.1.0 以及之后的 SDK 版本，本章介绍内容包括以下几点：

- 介绍 SDK 中与写保护相关的工具，以及写保护参数文件；
- 对于 A0 芯片，用户如何根据 flash 手册添加写保护参数；
- 介绍程序中如何打开与关闭内置 flash 写保护；

本章节分为以下几个部分进行介绍说明：

1. “A0 芯片如何添加写保护参数”
2. “内封 flash 芯片如何添加写保护参数 VI”
3. “系统 flash 写保护相关函数说明”
4. “常见问题说明”

6.1 A0 芯片如何添加写保护参数

6.1.1 flash 写保护工具说明

SDK 的下载目录（路径：apps\app\post_build\uc03）中存放有“flash_write_protect”文件夹，文件夹中包含以下文件：

flash_wp_info.csv：该文件以文本形式记录各种flash的写保护参数
csv2dir.bat：该脚本实现将上述csv文件转成小机可识别的dir_sys_info文件

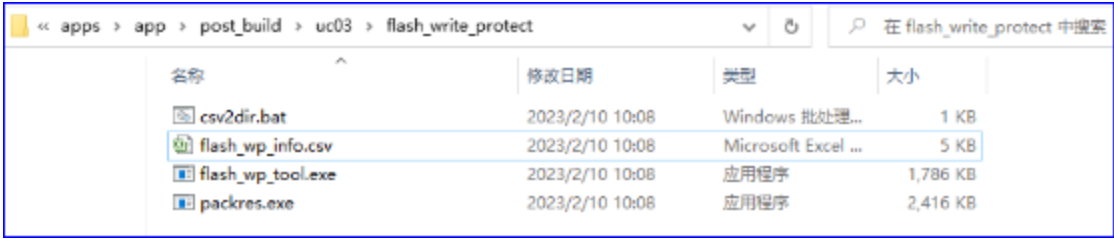


图 1.12 flash 写保护工具

其中，flash_wp_info.csv 文件中涵盖了内封 flash 的绝大部分型号。对于 A0 芯片，用户可自行根据使用的 flash 型号 datasheet 添加相应的写保护参数以及指令。

添加完成后，点击 csv2dir.bat 脚本，即可生成 dir_sys_info 文件，将该文件放在下载目录中，添加到 download.bat 的资源文件中下载到小机中，小机即可识别并添加写保护。

运行 csv2dir.bat 脚本时，需要注意是否有执行成功。

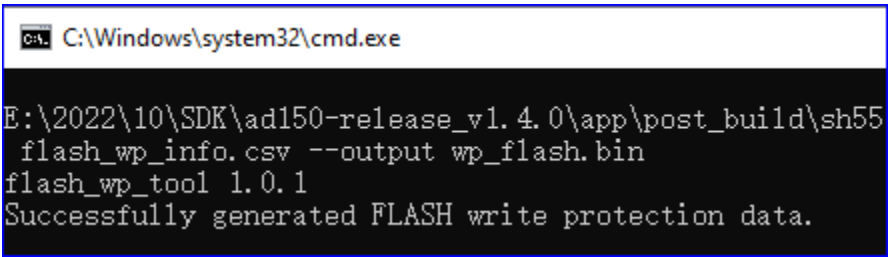


图 1.2 csv2dir.bat 脚本执行成功显示信息

1-3-fwp-csv2dir-succ.png 图 2 csv2dir.bat 脚本执行成功显示信息

没有执行成功时，需要关注错误信息，重新修改 flash_wp_info.csv 文件；下图的错误信息显示同时存在两组相同的 flash 写保护参数。

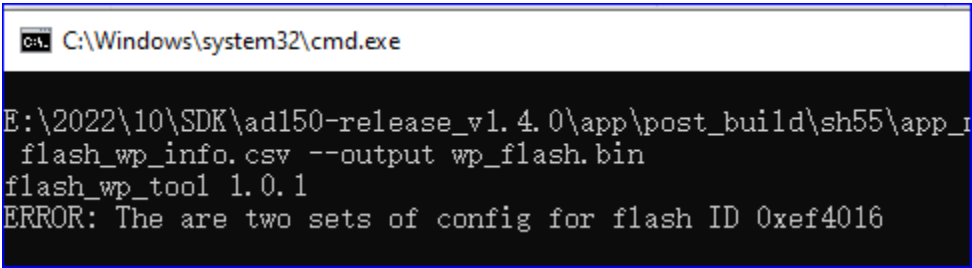


图 1.3 csv2dir.bat 脚本执行失败显示信息

6.1.2 新增 flash 写保护参数与指令

如下对于 A0 的芯片，用户可自行根据需要使用的 flash 的 datasheet，在 flash_wp_info.csv 文件中添加写保护参数以及相关指令，下文以华邦 W25Q32 FLASH 为例，介绍根据手册添加参数。

6.1.3 Flash SR1、SR2 寄存器和写保护范围

在 Flash datasheet 中，可通过查找关键字“BP1”快速找到 SR1 寄存器，查找关键字“LB1”快速找到 SR2 寄存器；检查 SR1 寄存器中是否有 SEC、TB、BP2、BP1、BP0 位，检查 SR2 寄存器是否有 CMP 位；

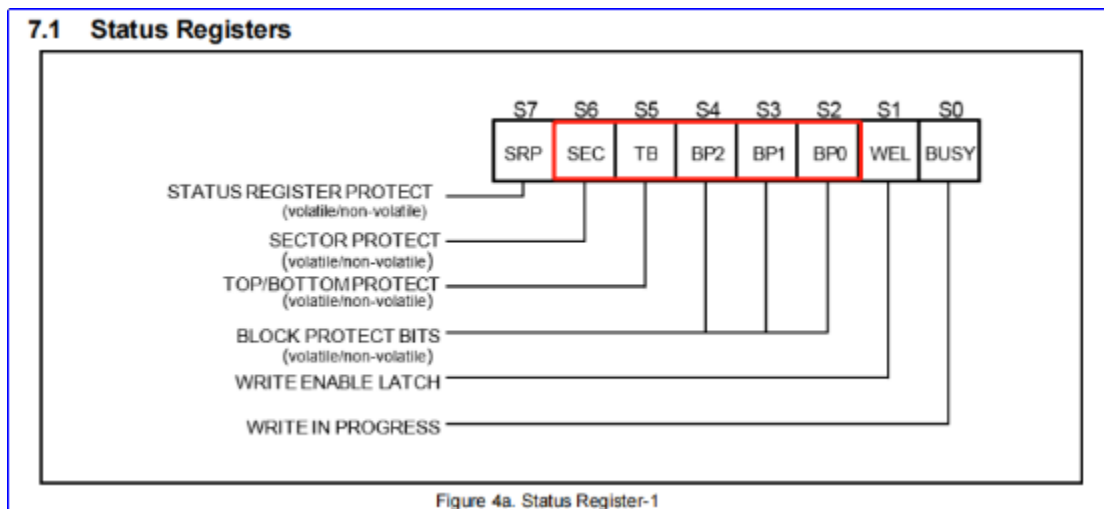


图 1.4 flash SR1 寄存器

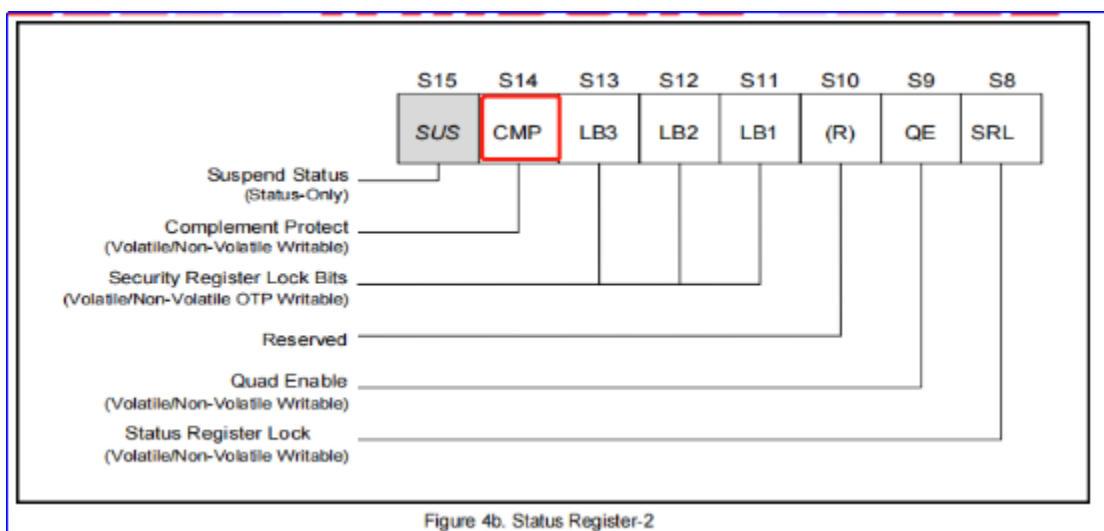


图 1.5 flash SR2 寄存器

通过查找关键字“Density”快速搜索写保护列表，并查看 flash 支持的写保护范围

7.1.8 Status Register Memory Protection (WPS = 0, CMP = 0)									
STATUS REGISTER ⁽¹⁾					W25Q32JV (32M-BIT) MEMORY PROTECTION ⁽²⁾				
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION ⁽²⁾	
X	X	0	0	0	NONE	NONE	NONE	NONE	
0	0	0	0	1	63	3F0000h – 3FFFFFFh	64KB	Upper 1/64	
0	0	0	1	0	62 and 63	3E0000h – 3FFFFFFh	128KB	Upper 1/32	
0	0	0	1	1	60 thru 63	3C0000h – 3FFFFFFh	256KB	Upper 1/16	
0	0	1	0	0	56 thru 63	380000h – 3FFFFFFh	512KB	Upper 1/8	
0	0	1	0	1	48 thru 63	300000h – 3FFFFFFh	1MB	Upper 1/4	
0	0	1	1	0	32 thru 63	200000h – 3FFFFFFh	2MB	Upper 1/2	

图 1.6 flash 写保护范围-1

7.1.9 Status Register Memory Protection (WPS = 0, CMP = 1)									
STATUS REGISTER ⁽¹⁾					W25Q32JV (32M-BIT) MEMORY PROTECTION ⁽²⁾				
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION ⁽²⁾	
X	X	0	0	0	0 thru 63	000000h – 3FFFFFFh	4MB	ALL	
0	0	0	0	1	0 thru 62	000000h – 3EFFFFh	4,032KB	Lower 63/64	
0	0	0	1	0	0 and 61	000000h – 3DFFFFh	3,968KB	Lower 31/32	
0	0	0	1	1	0 thru 59	000000h – 3BFFFFh	3,840KB	Lower 15/16	
0	0	1	0	0	0 thru 55	000000h – 37FFFFh	3,584KB	Lower 7/8	
0	0	1	0	1	0 thru 47	000000h – 2FFFFFFh	3MB	Lower 3/4	
0	0	1	1	0	0 thru 31	000000h – 1FFFFFFh	2MB	Lower 1/2	

图 1.7 flash 写保护范围-2

如何添加 flash 写保护参数

打开 flash_wp_info.csv 文件并根据 1.2 中的信息填写参数，我司配置 flash 写保护范围从 0 地址开始，向后覆盖。

表格找到上一组 flash 数据空两至三行后，在第一行填写 flash 标号、flash id 以及写使能（Write Enable）指令。

	A	B	C	D	E	F	G	H
1	addr	cmp	BP4	BP3	BP2	BP1	BP0	
222	flash标号	id	写使能命令					
223	W25Q32	0xef4016	06H					
224	128	0	0					
225	256	0	0					
226	512	0	0					
227	1024	0	0					
228	2048	0	0					
229	3072	1	0	0	1	0	1	
230	3584	1	0	0	1	0	0	
231	3968	1	0	0	0	1	0	
232	4032	1	0	0	0	0	1	
233	RDSR1	05H						
234	RDSR2	35H						
235	WRSR	01H	sr1	sr2				
236								
237								

8.2 Instruction Descriptions

8.2.1 Write Enable (06h)

8.2.2 Write Enable for Volatile Status Register (50h)

8.2.3 Write Disable (04h)

8.2.4 Read Status Register-1 (05h), Status Register-2 (35h)

8.2.5 Write Status Register-1 (01h), Status Register-2 (31h)

8.2.6 Read Data (03h)

图 1.8 flash 标号、ID 以及写使能指令信息

第二行开始填写 flash 写保护参数，最大支持 9 组 flash 写保护参数。

通过保护范围的大小，设置 SEC、TB、BP0、BP1、BP2（不同的 flash SEC 和 TB 标识可能是其它，如 BP4、BP3）和 CMP 的值。

例如：当 CMP = 0 时，SEC-TB-BP2-BP1-BP0 为 0b01101，可保护 000000h - 0ffffh 范围内 1MB 的数据。

	A	B	C	D	E	F	G
1	addr	cmp	BP4	BP3	BP2	BP1	BP0
222							
223	W25Q32	0xef4016	06H				
224	128	0	0	1	0	1	0
225	256	0	0	1	0	1	1
226	512	0	0	1	1	0	0
227	1024	0	0	1	1	0	1
228	2048	0	0	1	1	1	0
229	3072	1	0	0	1	0	1
230	3584	1	0	0	1	0	0
231	3968	1	0	0	1	0	0
232	4032	1	0	0	0	0	1
233	RDSR1						
234	RDSR2						
235	WRSR						
236							
237							
238							
239							
240							
241							
242							
243							

7.1.8 Status Register Memory Protection (WPS = 0, CMP = 0)

STATUS REGISTER ⁽¹⁾					W25Q32JV (32M-BIT) MEMORY PROTECTION ⁽¹⁾			
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION ⁽²⁾
0	1	0	0	1	0	000000h - 00FFFFh	64KB	Lower 1/64
0	1	0	1	0	0 and 1	000000h - 01FFFFh	128KB	Lower 1/32
0	1	0	1	1	0 thru 3	000000h - 03FFFFh	256KB	Lower 1/16
0	1	1	0	0	0 thru 7	000000h - 07FFFFh	512KB	Lower 1/8
0	1	1	0	1	0 thru 15	000000h - 0FFFFFFh	1MB	Lower 1/4
0	1	1	1	0	0 thru 31	000000h - 1FFFFFFh	2MB	Lower 1/2
X	X	1	1	1	0 thru 63	000000h - 3FFFFFFh	4MB	ALL

图 1.9 flash 写保护范围填写说明（1）

例如：当 CMP = 1 时，SEC TB BP2 BP1 BP0 为 0b00001，可保护 000000h - 3effffh 范围内 4032KB 的数据。

	A	B	C	D	E	F	G	H
1	addr	cmp	BP4	BP3	BP2	BP1	BP0	
222								
223	W25Q32	0xef4016	06H					
224	128	0	0	1	0	1	0	
225	256	0	0	1	0	1	1	
226	512	0	0	1	1	0	0	
227	1024	0	0	1	1	0	1	
228	2048	0	0	1	1	1	0	
229	3072	1	0	0	1	0	1	
230	3584	1	0	0	1	0	0	
231	3968	1	0	0	0	1	0	
232	4032	1	0	0	0	0	1	
233	RDSR1	05H						
234	RDSR2	35H						
235	WRSR	01H						
236								
237								
238								
239								
240								
241								
242								
243								
244								

7.1.9 Status Register Memory Protection (WPS = 0, CMP = 1)

STATUS REGISTER ⁽¹⁾					W25Q32JV (32M-BIT) MEMORY PROTECTION ⁽²⁾			
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION ⁽³⁾
X	X	0	0	0	0 thru 63	000000h - 3FFFFFFh	4MB	ALL
0	0	0	0	1	0 thru 62	000000h - 3FFFFFFh	4,032KB	Lower 63/64
0	0	0	1	0	0 and 61	000000h - 3DFFFFFFh	3,668KB	Lower 31/32
0	0	0	1	1	0 thru 59	000000h - 3BFFFFFFh	3,840KB	Lower 15/16
0	0	1	0	0	0 thru 55	000000h - 37FFFFFFh	3,584KB	Lower 7/8
0	0	1	0	1	0 thru 47	000000h - 2FFFFFFh	3MB	Lower 3/4
0	0	1	1	0	0 thru 31	000000h - 1FFFFFFh	2MB	Lower 1/2

图 1.10 flash 写保护范围填写说明 2

Flash 读写 SR1、SR2 指令

Flash 配置写保护时，需要读写 SR1、SR2 寄存器。查找手册找到读写 SR1、SR2 的指令，并填入表格中。如果 datasheet 上有 31H 命令，优先使用分开写 SR1、SR2 模式；

addr	cmp	BP4	BP3	BP2	BP1	BP0		
W25Q32	0xef4016	06H						
128	0	0	1	0	1	0		
256	0	0	1	0	1	1		
512	0	0	1	1	0	0		
1024	0	0						
2048	0	0						
3072	1	0						
3584	1	0						
3968	1	0						
4032	1	0						
RDSR1	05H							
RDSR2	35H							
WRSR	01H	sr1						

8.2.4 Read Status Register-1 (05h) Status Register-2 (35h) & Status Register-3 (15h)

The Read Status Register instructions allow the 8-bit Status Registers to be read. The instruction by driving /CS low and shifting the instruction code "05h" for Status Register-1, "35h" for Status Register-2 or "15h" for Status Register-3 into the DI pin on the rising edge of CLK. The status register bit is shifted out on the DO pin at the falling edge of CLK with most significant bit (MSB) first as shown in Figure 8. Refer to section 7.1 for Status Register descriptions.

8.2.5 Write Status Register-1 (01h) Status Register-2 (31h) & Status Register-3 (11h)

The Write Status Register instruction allows the Status Registers to be written. The writable Status Register bits include: SEC, TB, BP[2:0] in Status Register-1; CMP, LB[3:1], QE, SRL in Status Register-2; DRV0, WPS in Status Register-3. All other Status Register bit locations are read-only and are not affected by the Write Status Register instruction. LB[3:1] are non-volatile OTP bits, once it is programmed, it cannot be cleared to 0.

SR1 SR2寄存器分开写模式

图 1.11 flash 分开写 SR1 和 SR2 寄存器指令

若上述分开写 SR1、SR2 模式在芯片运行中添加写保护不成功，则使用 01H 指令连续写 SR1、SR2 模式；

	A	B	C	D	E	F	G	H
1	addr	cmp	BP4	BP3	BP2	BP1	BP0	
222								
223	W25Q32	0xef4016	06H					
224	128	0	0	1	0	1	0	
225	256	0	0	1	0	1	1	
226	512	0	0	1	1	0	0	
227	1024	0	0	1	1	0	1	
228	2048	0						
229	3072	1						
230	3584	1						
231	3968	1						
232	4032	1						
233	RDSR1	05H						
234	RDSR2	35H						
235	WRSR	01H	sr1	sr2				
236								
237								
238								
239								
240								
241								
242								

8.2.4 Read Status Register-1 (05h), Status Register-2 (35h).
The Read Status Register instructions allow the 8-bit Status Register by driving /CS low and shifting the instruction code "05h" for Status Register-1 or "35h" for Status Register-2 into the DI pin on the rising edge of CLK and shifted out on the DO pin at the falling edge of CLK with most significant bit first. Refer to section 7.1 for Status Register descriptions.

8.2.5 Write Status Register-1 (01h), Status Register-2 (31h).
The Write Status Register instruction allows the Status Registers to be written. The bits include: SEC, TB, BP[2:0] in Status Register-1; CMP, LB[3:1] in Status Register-2; DRV0, WPS in Status Register-3. All other Status Register bits are not affected by the Write Status Register instruction. LB[3:1] are non-volatile and cannot be cleared to 0.

图 1.12 flash 连续写 SR1 和 SR2 寄存器指令

部分 Flash 不存在 SR2 寄存器，则无需写 SR2 相关指令。

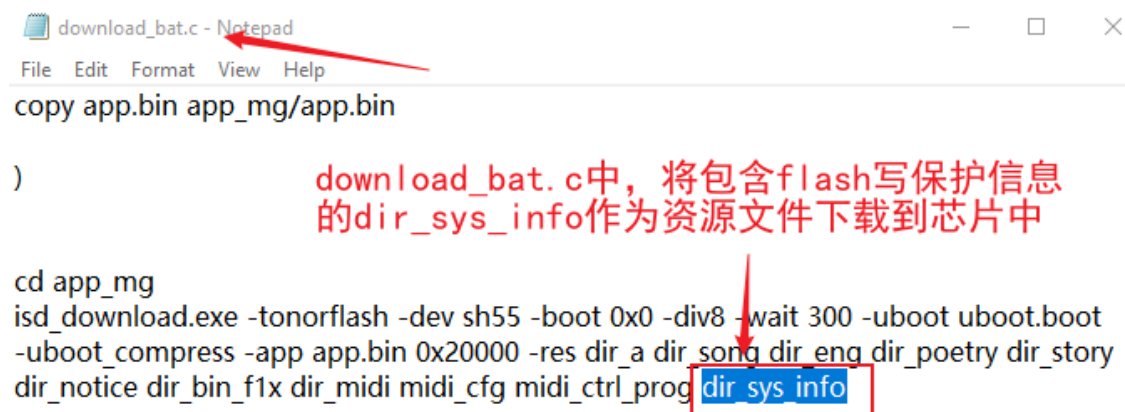
	A	B	C	D	E	F	G	
1	addr	cmp	BP4	BP3	BP2	BP1	BP0	
106								
107	4_ZE ZB25	0x5E3213	06H					
108	256	0	0	0	0	1	1	0
109	384	0	0	0	0	1	0	1
110	448	0	0	0	0	1	0	0
111	480	0	0	0	0	0	1	1
112	496	0	0	0	0	0	1	0
113	504	0	0	0	0	0	0	1
114	RDSR1	05H						
115	WRSR	01H	sr1					
116								

没有SR2寄存器的flash, 删除SR2相关字符

图 1.13 flash 只读写 SR1 寄存器指令

6.1.4 写保护参数下载说明

添加好写保护参数后，根据 2.1 的说明执行脚本生成 dir_sys_info 文件，并将其添加到 SDK 资源列表中，小机通过路径搜索找到该文件并进行解析。

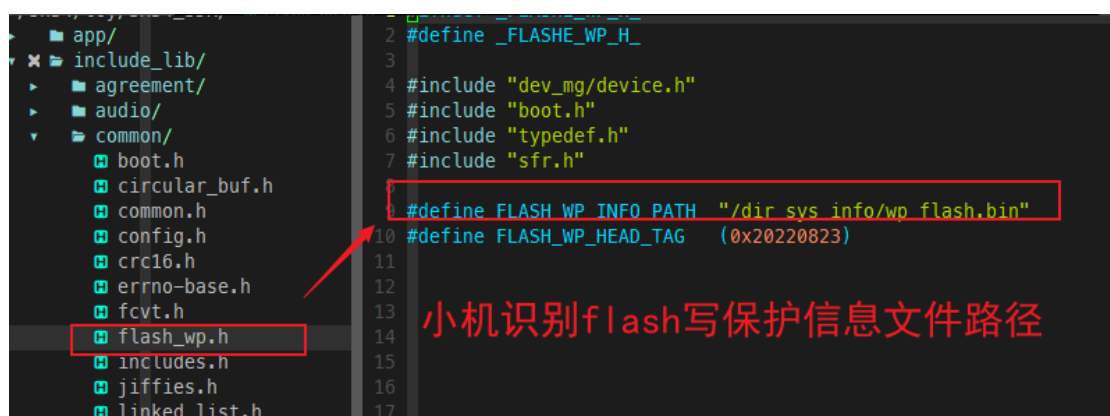


```
download_bat.c - Notepad
File Edit Format View Help
copy app.bin app_mg/app.bin

)

cd app_mg
isd_download.exe -tonorflash -dev sh55 -boot 0x0 -div8 -wait 300 -uboot uboot.boot
-uboot_compress -app app.bin 0x20000 -res dir_a dir_song dir_eng dir_poetry dir_story
dir_notice dir_bin_f1x dir_midi midi_cfg midi_ctrl_prog dir_sys_info
```

download_bat.c 中将 dir_sys_info 添加到资源列表中



```
#define _FLASH_WP_H_
#include "dev_mg/device.h"
#include "boot.h"
#include "typedef.h"
#include "sfr.h"

#define FLASH_WP_INFO_PATH "/dir_sys_info/wp_flash.bin"
#define FLASH_WP_HEAD_TAG (0x20220823)
```

小机识别 flash 写保护信息文件路径

6.2 内封 flash 芯片如何添加写保护参数 V1

上文通过表格的方式保存 flash 写保护参数，存在更新不及时的问题。对于杰理非 A0 的芯片，内封的 flash 型号可能会发生变动，不适合用表格形式记录。

我司烧写器可联网更新 flash 写保护参数，烧写器可在烧写阶段将内封 flash 对应的写保护参数记录在 flash 最尾部，由程序去查找该文件并配置写保护。

本文将介绍如何配置 ini 文件实现该功能：

支持 V1 版本的内封 flash 写保护策略的芯片：

支持写保护策略的芯片	AD14N / AC104N、AD15N、AD16N、AD17N
------------	----------------------------------

6.2.1 配置烧写器获取内封 flash 写保护参数

在 SDK 的下载目录（路径：apps\app\post_build\uc03\mbox_flash（或其他工程）\）中打开 isd_config_ini.c 文件，并在其中添加烧写器 flash 写保护配置项。

```
/* [BURNER_PASSTHROUGH_CFG] FLASH_WRITE_PROTECT=YES; */
```

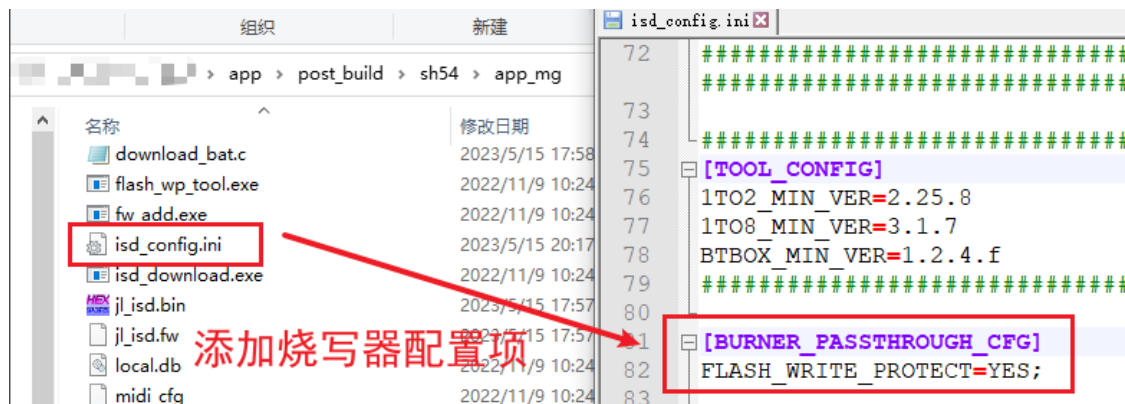


图 3.1 isd_config.ini 文件添加烧写器写保护配置项

6.2.2 配置强制下载工具获取内封 flash 写保护参数

isd_config.ini 文件添加了 FLASH_WRITE_PROTECT=YES 配置后，使用强制升级工具更新程序会报如下错误：

```
错误：在中没有发现FLASH cd6013的写保护参数。
请添加FLASH cd6013 写保护参数到文件。
```

图 3.2 强制升级工具报错

解决办法是将 FLASH_WRITE_PROTECT 注释掉或者配置为 NO。

6.3 系统 flash 写保护相关函数说明

6.3.1 函数 int norflash_set_write_protect(u8 enable_write_protect)

该函数实现开启或关闭内置 flash 写保护功能，其中参数：

1. Enable_write_protect

- 1：开启写保护
- 0：关闭写保护

2. 返回值

- 开启写保护成功：写保护的地址（即 flash 的 0~ 返回值这块区域保护）；
- 关闭写保护成功/开启写保护失败：

失败问题分析步骤：

1. 开写保护打印（在 app_config.c 里 EEPROM 区的打印打开）；

2. 检查打印, 如果没有任何打印直接返回 0, 则用户直接跳到步骤 3; 如果只有写保护失败打印, 则跳到步骤 4; 如果打印相关寄存器值出来和失败打印, 则跳到步骤 4
3. 写保护参数没有放进 flash 当中, 用户可以按照文档逐步操作:

“A0 芯片如何添加写保护参数”

“内封 flash 芯片如何添加写保护参数 VI”

4. 确认自己芯片的 flash 是否在 csv 烧写文件里; 确认 app.bin + 资源文件少于对应 flash 的最小保护区域, 用户可以在下载目录里加一些无用的资源文件。
5. 把 flash 手册和打印信息保存联系深圳办或者钉钉开源群联系我们。

详细可以看“写保护失败解决办法”

6.3.2 函数 u16 norflash_read_sr1_sr2(void)

该函数实现获取内置 flash 的 Status Register 1 和 Status Register 2 寄存器, 其返回值高 8 位为 SR2, 低 8 位为 SR1;

6.3.3 函数 u32 flash_code_protect_callback(u32 offset, u32 len)

该函数实现软件上限制驱动操作内置 flash (存放代码的 flash), 内置 flash 进行写或擦除操作前会回调该函数, 判断会操作到代码与资源区域, 则不进行相应操作, 可减低程序跑飞导致程序以外擦写 flash 的概率。其中参数:

- 1.offset: 设备进行写或擦除操作的地址
- 2.len: 设备进行写或擦除操作的长度
- 3.返回值:
 - 0: 可正常进行写或擦除操作
 - 1: 不进行写或擦除操作;

6.4 程序中开启与关闭写保护功能

SDK 中默认会在内置 flash 初始化的最后, 配置 flash 写保护状态。

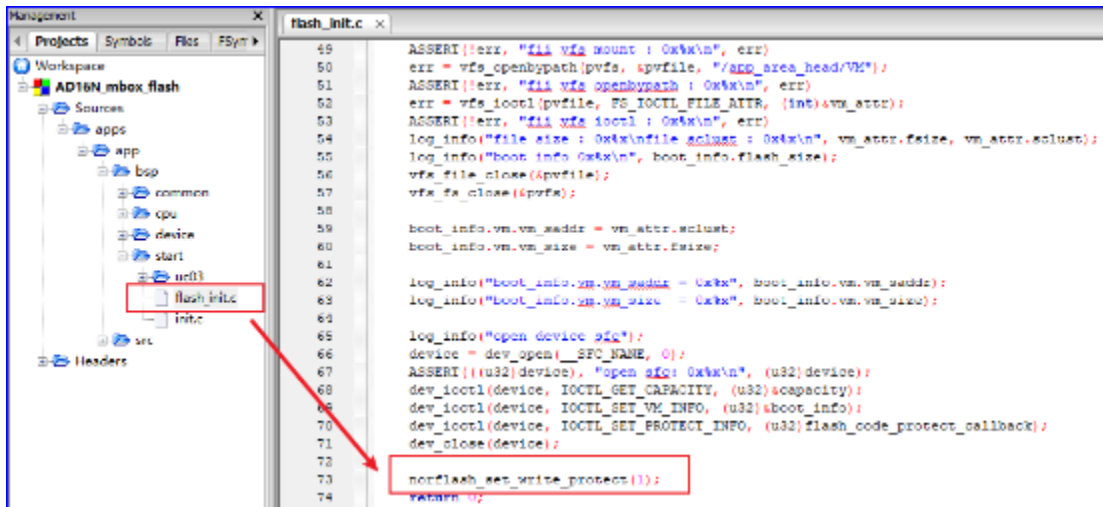


图 2.1 SDK 中添加写保护位置

配置 flash 写保护时, SDK 会从 res 资源文件中查找 dir_sys_info, 并解析其中的文件, 获取与正在运行的内置 flash 相匹配的写保护信息, 再操作内置 flash 添加写保护功能;

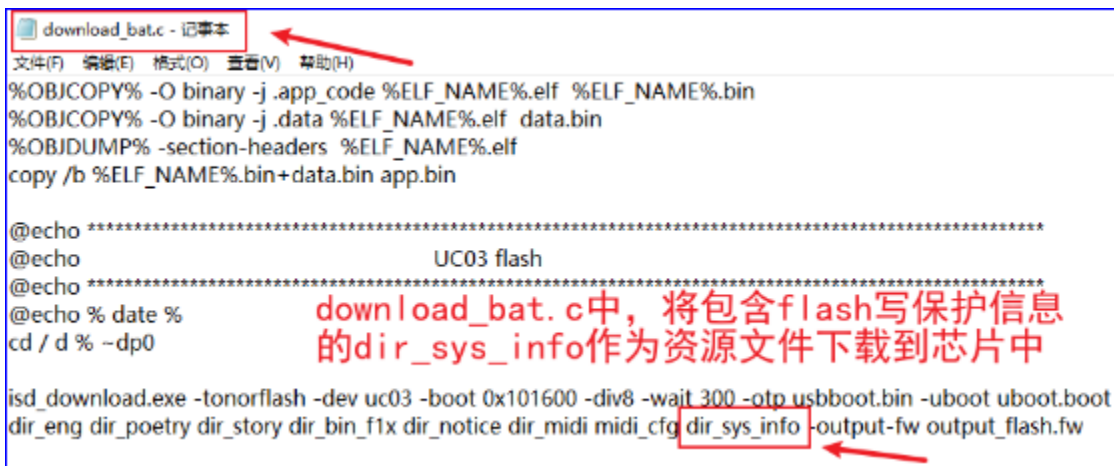


图 2.2 download.bat.c 中将 dir_sys_info 添加到资源列表中

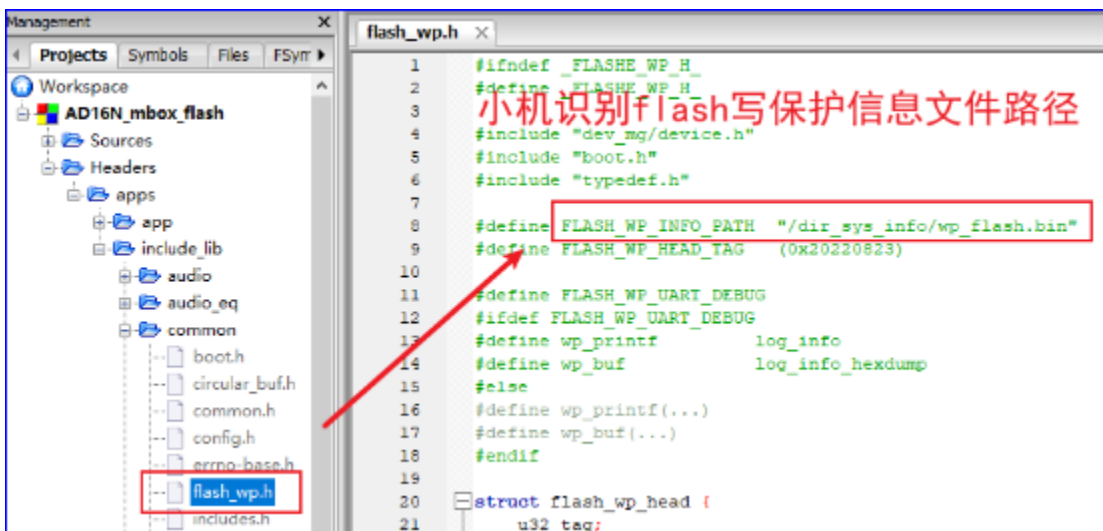


图 2.3 小机识别 flash 写保护信息文件路径

6.4.1 常见问题说明

flash 保护 csv 文件说明

用户需要了解写保护参数文件的一些相关意思：

- 1. 芯片 flash 型号
- 2. 芯片 flash_id
- 3. flash 的最小保护区域

14	WRSR	01H	sr1	sr2					
15									
16	flash型号	flash id							举例
17	8_TL TH25Q-80UA	0xeb6014	06H						
18		128	0	0	1	0	1	0	
19		256	flash最小保护区域	0	1	0	1	1	
20		512	0	0	1	1	0	0	
21		768	1	0	0	0	1	1	
22		896	1	0	0	0	1	0	
23		960	1	0	0	0	0	1	
24		992	1	1	0	1	0	0	
25		1008	1	1	0	0	1	1	
26		1016	1	1	0	0	1	0	
27	RDSR1	05H							
28	RDSR2	35H							
29	WRSR	01H	sr1	sr2					
30									

csv 文件简单了解

写保护失败解决办法

写保护失败现象有三：

- 1. 打印直接返回 0，没有写保护失败打印；
- 2. 打印返回 0，有写保护失败（set write protect fail）；
- 3. 打印返回 0，有相关寄存器配置和写保护失败打印（set write protect fail）；

写保护失败原因有三（一一对应上面失败现象）：

- 1. 写保护参数没有烧写到 flash 当中；
 “A0 芯片如何添加写保护参数”
 “内封 flash 芯片如何添加写保护参数 VI”
- 2. flash 的最小写保护区域大于 app.bin + 资源文件，涉及到了 VM 区域就会写保护失败；“flash 保护 csv 文件说明”
- 3. 某些 flash 配置特殊，需要找开发人员协助；

AD16N、AW30N 支持的设备有：系统 flash、数据 flash、SDMMC、u 盘。

设备	otp 方案	flash 方案
系统 flash	无	支持
数据 flash	不支持	支持
SDMMC	支持	支持
u 盘	支持	支持

设备管理模块依赖的头文件有：

```
apps\include_lib\dev_mg\device.h;
apps\app\bsp\device\device_app.h;
```

设备模块涉及的文件有：

```
apps\app\bsp\device\device_list.c;
apps\app\bsp\device\device_app.c;
```

其中应用层主要的函数有：

```
void *device_open(u32 index);
u32 device_close(u32 index);
void *device_obj(u32 index);
u32 device_online(void);
u32 device_status(u32 index, bool mode);
```

其中驱动层主要的函数有：

```
int devices_init_api();
bool dev_online(const char *name);
void *dev_open(const char *name, void *arg);
```

(续下页)

(接上页)

```

int dev_ioctl(void *device, int cmd, u32 arg);
int dev_close(void *device);
int dev_byte_read(void *_device, void *buf, u32 offset, u32 len);
int dev_byte_write(void *_device, void *buf, u32 offset, u32 len);
int dev_bulk_read(void *_device, void *buf, u32 sector, u32 sector_num);
int dev_bulk_write(void *_device, void *buf, u32 sector, u32 sector_num);

```

一般设备使用流程:

- a. 定义并赋值设备表;
- b. 开机调用函数 `devices_init_api()` 完成对所有设备的初始化;
- c. 调用函数 `device_open()` 并传入指定设备号, 得到对应设备句柄;
- d. 调用函数 `dev_bulk_read()` / `dev_bulk_write()` 对设备进行读写等操作;
- e. 操作完成后, 调用函数 `device_close()` 并传入指定设备号, 关闭对应设备;

7.1 设备注册表

设备注册表 `device_tab` 中注册了全部需要使用到的设备, 其中成员如下:

1. `name`: 设备名称的字符串 (自定义名称)
2. `ops`: 设备内部接口函数集合 (不可更改)
3. `priv_data`: 设备使用的底层硬件资源及相关模式配置 (可根据需要自行配置)

如果要添加设备, 直接在下图注册表中接着添加。开机时初始化程序会自动执行初始化。

如果要添加设备, 直接在下图注册表中接着添加。开机时初始化程序会自动执行初始化, 设备注册表定义及初始化如图 31.1 所示。

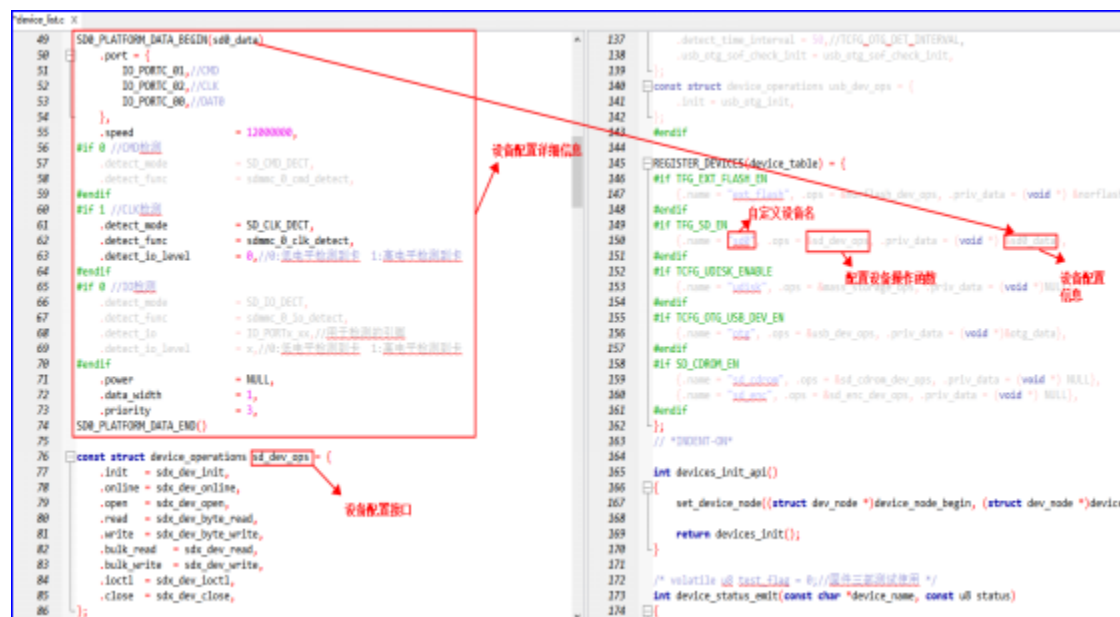


图 1.1 设备注册表

上图对 SD 接口的配置包括了所使用的 IO 口、波特率、检测模式（包含检测方式和检测函数等）、数据宽度（可支持 1 线宽和 4 线宽）、中断优先级（0-7，数字越大，优先级越高）。

注册 norflash_dev_ops 接口时，起始地址及长度以 512 字节为单位，比如读取 (dev_bulk_read) 时带入地址 1 实际是从 1*512=512 开始读取，且写函数内部自带擦除操作，可以直接写入。

```

1164
1165
1166 /*
1167  * 1. 外部调用时以512字节为单位的地址和长度，且需要驱动write自己处理擦除，
1168  * 请使用norflash_dev_ops接口，否则使用本文件内的其他ops
1169  *
1170  * 2. 如果不需要驱动自己处理擦除，可以把宏FLASH_CACHE_ENABLE清零，或者把
1171  * norflash_dev_read()里面调用的_norflash_read()的宏cache置0，
1172  * norflash_dev_write()同理
1173  *
1174  * 3. norflash_dev_ops可以被多个设备名注册，每个设备名被认为是不同分区，所以
1175  * 需要填不同的分区起始地址和大小，若分区地址有重叠或者最大分区结束地址大于
1176  * flash容量，会触发ASSERT()
1177  *
1178  * 4. 关于IOCTL_GET_CAPACITY，有多个分区注册时返回分区的大小，如果只注册了1
1179  * 个分区，分区起始地址 == 0时返回flash容量，起始地址 != 0时返回分区大小，
1180  * norflash_dev_ops返回的长度以512字节为单位
1181  *
1182  * 5. 本文件内的各个ops可以同时使用
1183  */
1184 □ const struct device_operations norflash_dev_ops = {
1185     .init = norflash_dev_init,
1186     .online = norflash_dev_online,
1187     .open = norflash_dev_open,
1188     .read = NULL,
1189     .write = NULL,
1190     .bulk_read = norflash_bulk_read, //按块(bulk)读取
1191     .bulk_write = norflash_bulk_write, //按块写入，写前会检查非空则进行擦除（支持擦除单位为4k的flash）
1192     .ioctl = norflash_bulk_ioctl,
1193     .close = norflash_dev_close,
1194 };
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753
2754
2755
2756
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
2779
2780
2781
2782
2783
2784
2785
2786
2787
2788
2789
2790
2791
2792
2793
2794
2795
2796
2797
2798
2799
2800
2801
2802
2803
2804
2805
2806
2807
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820
2821
2822
2823
2824
2825
2826
2827
2828
2829
2830
2831
2832
2833
2834
2835
2836
2837
2838
2839
2840
2841
2842
2843
2844
2845
2846
2847
2848
2849
2850
2851
2852
2853
2854
2855
2856
2857
2858
2859
2860
2861
2862
2863
2864
2865
2866
2867
2868
2869
2870
2871
2872
2873
2874
2875
2876
2877
2878
2879
2880
2881
2882
2883
2884
2885
2886
2887
2888
2889
2890
2891
2892
2893
2894
2895
2896
2897
2898
2899
2900
2901
2902
2903
2904
2905
2906
2907
2908
2909
2910
2911
2912
2913
2914
2915
2916
2917
2918
2919
2920
2921
2922
2923
2924
2925
2926
2927
2928
2929
2930
2931
2932
2933
2934
2935
2936
2937
2938
2939
2940
2941
2942
2943
2944
2945
2946
2947
2948
2949
2950
2951
2952
2953
2954
2955
2956
2957
2958
2959
2960
2961
2962
2963
2964
2965
2966
2967
2968
2969
2970
2971
2972
2973
2974
2975
2976
2977
2978
2979
2980
2981
2982
2983
2984
2985
2986
2987
2988
2989
2990
2991
2992
2993
2994
2995
2996
2997
2998
2999
3000
3001
3002
3003
3004
3005
3006
3007
3008
3009
3010
3011
3012
3013
3014
3015
3016
3017
3018
3019
3020
3021
3022
3023
3024
3025
3026
3027
3028
3029
3030
3031
3032
3033
3034
3035
3036
3037
3038
3039
3040
3041
3042
3043
3044
3045
3046
3047
3048
3049
3050
3051
3052
3053
3054
3055
3056
3057
3058
3059
3060
3061
3062
3063
3064
3065
3066
3067
3068
3069
3070
3071
3072
3073
3074
3075
3076
3077
3078
3079
3080
3081
3082
3083
3084
3085
3086
3087
3088
3089
3090
3091
3092
3093
3094
3095
3096
3097
3098
3099
3100
3101
3102
3103
3104
3105
3106
3107
3108
3109
3110
3111
3112
3113
3114
3115
3116
3117
3118
3119
3120
3121
3122
3123
3124
3125
3126
3127
3128
3129
3130
3131
3132
3133
3134
3135
3136
3137
3138
3139
3140
3141
3142
3143
3144
3145
3146
3147
3148
3149
3150
3151
3152
3153
3154
3155
3156
3157
3158
3159
3160
3161
3162
3163
3164
3165
3166
3167
3168
3169
3170
3171
3172
3173
3174
3175
3176
3177
3178
3179
3180
3181
3182
3183
3184
3185
3186
3187
3188
3189
3190
3191
3192
3193
3194
3195
3196
3197
3198
3199
3200
3201
3202
3203
3204
3205
3206
3207
3208
3209
3210
3211
3212
3213
3214
3215
3216
3217
3218
3219
3220
3221
3222
3223
3224
3225
3226
3227
3228
3229
3230
3231
3232
3233
3234
3235
3236
3237
3238
3239
3240
3241
3242
3243
3244
3245
3246
3247
3248
3249
3250
3251
3252
3253
3254
3255
3256
3257
3258
3259
3260
3261
3262
3263
3264
3265
3266
3267
3268
3269
3270
3271
3272
3273
3274
3275
3276
3277
3278
3279
3280
3281
3282
3283
3284
3285
3286
3287
3288
3289
3290
3291
3292
3293
3294
3295
3296
3297
3298
3299
3300
3301
3302
3303
3304
3305
3306
3307
3308
3309
3310
3311
3312
3313
3314
3315
3316
3317
3318
3319
3320
3321
3322
3323
3324
3325
3326
3327
3328
3329
3330
3331
3332
3333
3334
3335
3336
3337
3338
3339
3340
3341
3342
3343
3344
3345
3346
3347
3348
3349
3350
3351
3352
3353
3354
3355
3356
3357
3358
3359
3360
3361
3362
3363
3364
3365
3366
3367
3368
3369
3370
3371
3372
3373
3374
3375
3376
3377
3378
3379
3380
3381
3382
3383
3384
3385
3386
3387
3388
3389
3390
3391
3392
3393
3394
3395
3396
3397
3398
3399
3400
3401
3402
3403
3404
3405
3406
3407
3408
3409
3410
3411
3412
3413
3414
3415
3416
3417
3418
3419
3420
3421
3422
3423
3424
3425
3426
3427
3428
3429
3430
3431
3432
3433
3434
3435
3436
3437
3438
3439
3440
3441
3442
3443
3444
3445
3446
3447
3448
3449
3450
3451
3452
3453
3454
3455
3456
3457
3458
3459
3460
3461
3462
3463
3464
3465
3466
3467
3468
3469
3470
3471
3472
3473
3474
3475
3476
3477
3478
3479
3480
3481
3482
3483
3484
3485
3486
3487
3488
3489
3490
3491
3492
3493
3494
3495
3496
3497
3498
3499
3500
3501
3502
3503
3504
3505
3506
3507
3508
3509
3510
3511
3512
3513
3
```


图 1.1 中设备详细数据配置信息介绍见下图, SD 接口配置信息见图 1.4, flash 配置信息图 5。

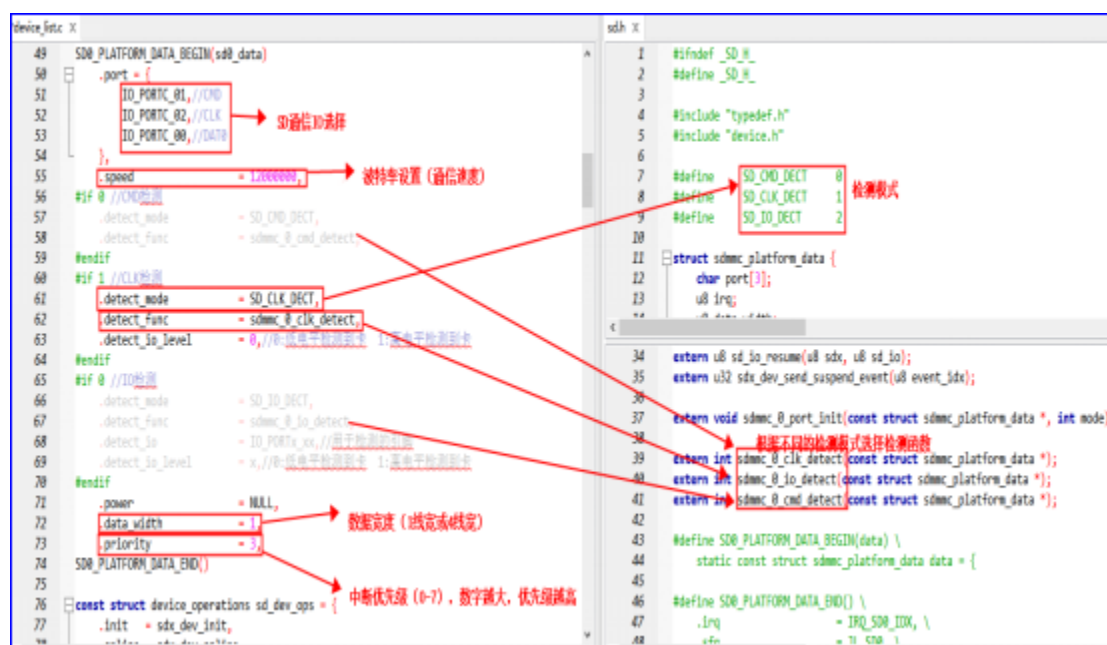


图 1.4 SD 接口配置信息

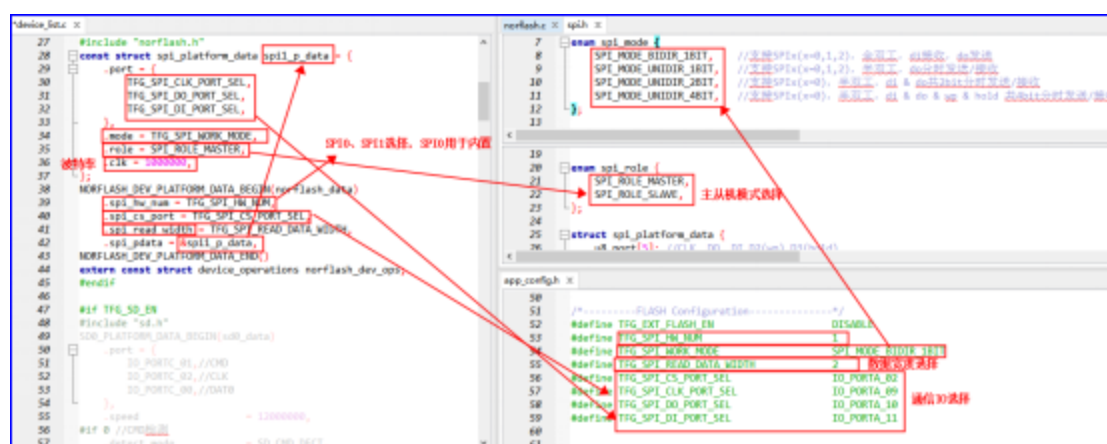


图 1.5 外挂数据 flash 配置信息

7.2 设备函数

7.2.1 函数 int devices_init_api()

应用层函数, 该函数会将 device_table 设备表中注册的设备全部初始化, 这一步一般在开机时进行。其内部会调用驱动层函数 devices_init()。

7.2.2 函数 void *dev_open(u32 index)

应用层函数，此函数会根据传入的设备号，打开对应的设备并返回设备句柄，其内部会调用驱动层函数 dev_open()，其中参数：

- 1、index：dev_open函数返回的设备句柄；
 - ① SD0_INDEX：SD卡设备号；
 - ② UDISK_INDEX：U盘设备号；
- 2、返回值：
 - 成功，返回对应设备句柄；
 - 失败，返回NULL；

7.2.3 函数 u32 device_close(u32 index)

应用层函数，此函数会根据传入的设备号，关闭对应的设备，其内部会调用驱动层函数 dev_close()，其中参数。

- 1、index：dev_open函数返回的设备句柄；
 - ① SD0_INDEX：SD卡设备号；
 - ② UDISK_INDEX：U盘设备号；
- 2、返回值：
 - 成功，返回对应设备句柄；
 - 失败，返回NULL；

7.2.4 函数 void *device_obj(u32 index)

应用层函数，此函数会根据传入的设备号，返回对应的设备句柄，其中参数。

- 1、index：dev_open函数返回的设备句柄；
 - ① SD0_INDEX：SD卡设备号；
 - ② UDISK_INDEX：U盘设备号；
- 2、返回值：
 - 成功，返回对应设备句柄；
 - 失败，返回NULL；

7.2.5 函数 u32 device_online(void)

应用层函数，此函数会返回一个 u32 变量，每一个 bit 置 1 代表对应设备在线。

7.2.6 函数 u32 device_status(u32 index, bool mode)

应用层函数，此函数会根据传入的设备号，返回对应的设备状态，根据 mode 参数选择当发现设备不在线时是否关闭对应设备模块，其中参数：

- 1、index：dev_open函数返回的设备句柄；
 - ① SD0_INDEX：SD卡设备号；
 - ② UDISK_INDEX：U盘设备号；
- 2、mdde：查询模式；
 - ① 0：只查询状态；
 - ② 1：查询到对应设备不在线时，关闭对应设备模块；

(续下页)

(接上页)

- 3、返回值：
 - 设备在线，返回0；
 - 设备离线，返回响应错误值；

7.2.7 函数 int dev_bulk_read(void *_device, void *buf, u32 offset, u32 len)

本函数实现从设备某个地址 offset 读取长度为 len 的数据到 buf 的功能。

- 1、_device: dev_open函数返回的设备句柄；
- 2、buf: 缓存，保存读出的数据，需自定义；
- 3、offset: 读取起始设备地址；
- 4、len: 需要读取的长度；
- 5、返回值: 正常情况返回长度即形参Len，错误时返回0；

7.2.8 函数 int dev_bulk_write(void *_device, void *buf, u32 offset, u32 len)

本函数实现把长度为 len 的 buf 数据写入设备的功能，写入起始地址为 offset。

- 1、_device: dev_open函数返回的设备句柄；
- 2、Buf : 缓存，保存着需写入的数据；
- 3、Offset : 写入起始设备地址；
- 4、Len : 需要写入的长度。
- 5、返回值: 正常情况返回长度即形参Len，错误时返回0
- *注意: 形参offset、len范围要在图3.
 ↪7定义的起始地址和大小范围内才能正常操作设备。*

7.2.9 函数 int dev_ioctl(void *device, int cmd, u32 arg)

本函数实现获取设备信息或设置设备信息，擦除设备等功能。

- 1、device: 设备句柄；
- 2、cmd: 命令，include_lib/dev_mg/ioctl_cmds.h内能查到定义；详细命令表参见图2.
 ↪1。
 常用命令如下表，（注：不同设备支持命令会有差别，并不是支持所有命令）
- 3、arg: 获取信息时：用作保存返回的设备信息如容量、ID等；
 擦出时：用作带入要擦除的地址。
- 4、返回值: 反应函数运行状态。
 0: 表示正常
 其他值: 出错

cmd 命令	命令含义
IOCTL_CMD_SUSPEND	设备挂起
IOCTL_CMD_RESUME	设备解挂
IOCTL_GET_ID	获取设备 ID
IOCTL_GET_SECTOR_SIZE	获取设备扇区大小
IOCTL_GET_BLOCK_SIZE	获取设备块大小
IOCTL_GET_CAPACITY	获取设备容量
IOCTL_GET_STATUS	获取设备状态
IOCTL_ERASE_SECTOR	擦除设备扇区
IOCTL_ERASE_BLOCK	擦除设备块
IOCTL_ERASE_CHIP	对设备整片擦除

<code>#include lib/dev_mg/ioctl_cmds.h</code>		
<code>#define IOCTL_SET_IRQ_NUM</code>	<code>1</code>	<code>#define IOCTL_CMD_RESUME</code> 39
<code>#define IOCTL_SET_PRIORITY</code>	<code>2</code>	<code>#define IOCTL_CMD_SUSPEND</code> 40
<code>#define IOCTL_SET_DATA_WIDTH</code>	<code>3</code>	<code>#define IOCTL_SET_BASE_ADDR</code> 41
<code>#define IOCTL_SET_SPEED</code>	<code>4</code>	<code>#define IOCTL_SET_ASYNC_MODE</code> 42
<code>#define IOCTL_SET_DETECT_MODE</code>	<code>5</code>	
<code>#define IOCTL_SET_DETECT_FUNC</code>	<code>6</code>	<code>#define IOCTL_GET_ID</code> 100
<code>#define IOCTL_SET_DETECT_TIME_INTERVAL</code>	<code>7</code>	<code>#define IOCTL_GET_SECTOR_SIZE</code> 101
<code>#define IOCTL_SET_PORT</code>	<code>8</code>	<code>#define IOCTL_GET_BLOCK_SIZE</code> 102
<code>#define IOCTL_SET_PORT_FUNC</code>	<code>9</code>	<code>#define IOCTL_GET_CAPACITY</code> 103
<code>#define IOCTL_SET_CS_PORT_FUNC</code>	<code>10</code>	<code>#define IOCTL_GET_WIDTH</code> 104
<code>#define IOCTL_SET_READ_MODE</code>	<code>11</code>	<code>#define IOCTL_GET_HEIGHT</code> 105
<code>#define IOCTL_SET_WRITE_MODE</code>	<code>12</code>	<code>#define IOCTL_GET_BLOCK_NUMBER</code> 106
<code>#define IOCTL_SET_WRITE_PROTECT</code>	<code>13</code>	<code>#define IOCTL_CHECK_WRITE_PROTECT</code> 107
<code>#define IOCTL_SET_START_BIT</code>	<code>14</code>	<code>#define IOCTL_GET_STATUS</code> 108
<code>#define IOCTL_SET_STOP_BIT</code>	<code>15</code>	<code>#define IOCTL_GET_TYPE</code> 109
<code>#define IOCTL_FLUSH</code>	<code>16</code>	<code>#define IOCTL_GET_MAX_LUN</code> 110
<code>#define IOCTL_REGISTER_IRQ_HANDLER</code>	<code>17</code>	<code>#define IOCTL_GET_CUR_LUN</code> 111
<code>#define IOCTL_UNREGISTER_IRQ_HANDLER</code>	<code>18</code>	<code>#define IOCTL_SET_CUR_LUN</code> 112
<code>#define IOCTL_GET_SYS_TIME</code>	<code>19</code>	<code>#define IOCTL_SET_FORCE_RESET</code> 113
<code>#define IOCTL_SET_SYS_TIME</code>	<code>20</code>	<code>#define IOCTL_SET_CAPACITY</code> 114
<code>#define IOCTL_GET_ALARM</code>	<code>21</code>	<code>#define IOCTL_GET_NORFS_INFO</code> 115
<code>#define IOCTL_SET_ALARM</code>	<code>22</code>	
<code>#define IOCTL_SET_CAP_LOWSPEED_CARD</code>	<code>23</code>	
<code>#define IOCTL_SET_VDD50_EN</code>	<code>30</code>	<code>#define IOCTL_ERASE_SECTOR</code> 200
<code>#define IOCTL_GET_WEEKDAY</code>	<code>32</code>	<code>#define IOCTL_ERASE_BLOCK</code> 201
<code>#define IOCTL_CLR_READ_MODE</code>	<code>33</code>	<code>#define IOCTL_ERASE_CHIP</code> 202
<code>#define IOCTL_SET_READ_CRC</code>	<code>34</code>	<code>#define IOCTL_SET_ENC_END</code> 203
<code>#define IOCTL_GET_READ_CRC</code>	<code>35</code>	<code>#define IOCTL_ERASE_PAGE</code> 204
<code>#define IOCTL_GET_VOLUME</code>	<code>36</code>	
<code>#define IOCTL_SET_VOLUME</code>	<code>37</code>	
<code>#define IOCTL_SET_ALARM_ENABLE</code>	<code>38</code>	<code>#define IOCTL_SET_DATA_CALLBACK</code> 301

图 2.1 cmd 可选择的命令

7.2.10 函数 `int dev_close(void *device)`

本函数实现关闭设备的功能，对于一些设备调用此函数能省电。

- 1、`device`: `dev_open`返回的设备句柄；
 - 2、返回值：暂无意义

7.2.11 设备管理常见问题介绍

外挂 flash 有两套 OPS 接口函数

默认 SDK 下，外挂 flash 使用的是 `norfs_dev_ops` 接口，
在 `app_config.h` 里打开 `TCFG_USB_EXFLASH_UDISK_ENABLE` 宏后，外挂 flash 使用的是 `norflash_dev_ops` 接口。

备注：

`norfs_dev_ops` 接口在读写接口里只有 **字节读**和 **字节写**。

`norflash_dev_ops` 接口在读写接口里只有 **块读**和 **块写**。

使用不同文件系统时，要求的读写接口不一样，如：

syd 和 **norfs** 文件系统需要 **字节读**和 **字节写**接口。

fat 文件系统需要 **块读**和 **块写**接口。

因此在默认的 SDK 下使用不同的接口时，支持的文件系统不同。

AD16N FLASH 方案和 AW30N mbox_flash 方案支持只读 sydfs、录音读写 norfs 以及 fatfs 文件系统；

OTP 方案只支持只读 fatfs 文件系统，且不使用 vfs 文件系统管理，直接调用 fat 文件系统接口，可节约代码空间；

文件系统涉及的文件：

- 1、apps/app/bsp/common/fs 目录下的所有 c 文件；
- 2、include_lib/fs 目录下的所有头文件
- 3、errno-base.h 文件。

支持的文件系统列表：

文件系 统	OTP 方 案	flash 方 案	详情
sydfs	不支持	支持	只读文件系统，用于存放资源文件到 norflash；
norfs	不支持	支持	循环覆盖文件系统，可读写；针对 norflash 设计；
sim- ple_fat	无	无	简单 fat 文件系统驱动，占用资源少，不支持搜索；支持 FAT12/16/32, EXFAT；
fatfs	支持	支持	完整的 fat 文件系统驱动，占用资源多，支持搜索；支持 FAT12/16/32, EXFAT；

fs 有以下主要函数：

```
void fs_init(void);
u32 fs_mount(void **ppvfs, void *device, void *type);
u32 fs_openbypath(void *pvfs, void **ppvfile, const char *path);
int fs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode,
↪int arg);
u32 fs_read(void *pvfile, void *buf, u32 len);
u32 fs_seek(void *pvfile, u32 offset, u32 mode);
u32 fs_file_close(void **ppvfile);
```

(续下页)

(接上页)

```

u32 fs_fs_close(void **ppvfs);
u32 fs_file_name(void *pvfile, void *name, u32 len);
int fs_get_attr(void *pvfile, void *pvfs_attr);
int fs_ioctl(void *pvfile, int cmd, int arg);
int fs_ftell(void *pvfile, void *parm);
int fs_get_fsize(void *pvfile, void *parm);
struct vfscan *fs_fscan_new(void *pvfs, const char *path, const char *arg,
    ↪ u8 max_deepth, int (*callback)(void), struct vfscan *fsn, struct vfscan_
    ↪ reset_info *info);
int fs_fscan_release(void *pvfs, struct vfscan *fs);
int vs_get_folderinfo(void **pvfile, struct vfscan *fs, int *start_num, int_
    ↪ *end_num);
int fs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num,
    ↪ u32 *total_num);

```

8.1 文件系统介绍

SDK 中支持的文件系统有：只读 sydfs、录音读写 norfs 以及 fatfs；可以在 app/bsp/fs 目录下查看各文件系统的文件以及支持的接口；

本小节将介绍适用于各个文件系统的使用流程和相关接口。

8.2 文件系统使用一般流程

如图，fs 运行大致流程如下；

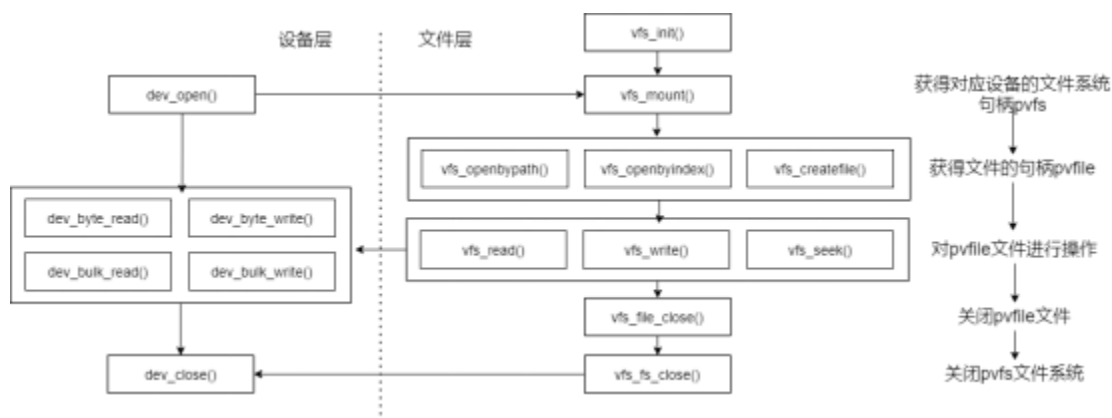


图 2.1 fs 运行大致流程

通用的文件系统使用流程

- 1、文件系统初始化：
调用函数 `void fs_init(void)`，完成对所有文件系统驱动的初始化；
- 2、mount 挂载：

(续下页)

(接上页)

调用u32 fs_mount(void **ppvfs, void *device, void_
 ↳*type), 传递指定设备句柄, 并完成对具体文件系统的挂载, 并获取文件系统的句柄;

3、打开文件:
 调用u32 fs_openbypath(void *pvfs, void **ppvfile, const char *path)或u32_
 ↳fs_select(void *fsn, void *pfs, int sel_mode, void **ppfile, int_
 ↳arg)打开文件, 并获取文件句柄;

4、读取文件数据:
 调用u32 fs_read(void *pvfile, void *buf, u32 len)对文件进行读操作;

5、对文件进行不同的命令操作:
 调用int fs_ioctl(void *pvfile, int cmd, int_
 ↳arg)使用不同的命令进行不同的操作, 如获取文件号、获取文件总数、获取文件地址、获取文件名字等;

6、关闭文件:
 调用u32 fs_file_close(void **ppvfile)关闭文件, 并释放文件句柄。

7、关闭文件系统:
 调用u32 fs_fs_close(void **ppvfs)关闭文件系统, 并释放文件系统句柄。

可以通过调用接口得到的返回值查看是否调用成功, 以及失败的原因; 失败原因可在include_
 ↳lib/common/errno-base.h中查看。

8.3 通用文件系统管理函数

该小节介绍适用于各个文件系统的接口。

8.3.1 函数 void fs_init(void)

此函数实现对所有文件系统驱动的初始化; 在开机调用一次即可。

8.3.2 函数 u32 fs_mount(void **ppvfs, void *device, void *type)

此函数实现对指定设备挂在指定类型的文件系统, 所挂载的系统必须设备自身支持; 其中参数:

1、ppvfs: 文件系统句柄指针, 若句柄指向为NULL, 函数会自行申请空间;
 2、device: 设备句柄;
 3、type: 文件系统类型的字符串;
 ① “sydfs”: 挂载sydfs文件系统;
 ② “norfs”: 挂载norfs文件系统;
 ③ “fat”: 挂载fatfs文件系统;
 ↳
 ↳④ “NULL”: 将在现有的文件系统, 轮询哪个文件系统可以使用 (除去norfs文件系统), 默认即使用syd
 4、返回值:
 成功 0;
 失败 错误请查看errno-base.h。

(续下页)

(接上页)

示例1:

```
void *device = device_open(SD0_INDEX); // 获取SD卡设备句柄
void *pfs = NULL;
u32 err = fs_mount(&pfs, device, "fat"); // SD卡挂载fatfs系统
if (pfs != NULL && err == 0) {
    // SD卡挂载fatfs成功
}
```

示例2:

```
void *device = device_open(UDISK_INDEX); // 获取U盘卡设备句柄
void *pfs = NULL;
u32 err = vfs_mount(&pfs, device, "fat"); // U盘挂载fatfs系统
if (pfs != NULL && err == 0) {
    // U盘挂载fatfs成功
}
```

8.3.3 函数 u32 fs_openbypath(void *pvfs, void **ppvfile, const char *path)

此函数实现打开指定路径的文件，其中参数：

- 1、pvfs：已挂载的文件系统句柄；
- 2、ppvfile：文件句柄指针，若句柄指向为NULL，函数会自行申请空间；
- 3、path：路径字符串；
 - ① 如果字符串只是包含文件夹名，那么会打开文件夹；
 - ② 如果字符串里面包含了完整的文件名，那么会打开对应的文件；
 - ③ 如果字符串为空，则返回错误；
- 4、返回值：成功 0；
失败 错误请查看errno-base.h。

示例：

```
void *pfile = NULL;
u32 err = vfs_openbypath(pfs, &pfile, "dir_song/so001.f1a");
if (pfile != NULL && err == 0) {
    // 成功打开dir_song文件夹里的so001.f1a文件
}
```

8.3.4 函数 u32 fs_read(void *pvfile, void *buf, u32 len)

此函数实现对指定文件的读操作。其中参数：

- 1、pvfile：已打开的文件句柄；
- 2、buf：用来获取数据的buff；
- 3、len：需要读取的字节长度；
- 4、返回值：
 - 成功 读取的字节数；
 - 失败 0。

示例：

```
u8 read_buf[32];
u32 read_len = fs_read(pvfile, (void *)read_buf, 32);
if (read_len != 0) {
    // 成功读取pvfile中read_len长度的数据到read_buf里
}
```


8.3.5 函数 u32 fs_seek(void *pvfile, u32 offset, u32 mode)

此函数实现改变指定文件中读写指针的偏移。其中参数：

1、pvfile：已打开的文件句柄；

2、offset：偏移量；

3、mode：seek模式选择。如图3.1，seek模式有3种，分别为：

① SEEK_

→SET：从文件起始位置开始向后偏移，首先会判断偏移量是否大于文件长度，若小于则使文件读写指针rw_

→p = offset；

② SEEK_CIR：从文件读写指针位置开始向后偏移，使rw_p = rw_p + offset；

③ SEEK_

→END：从文件末尾位置向前偏移，首先会判断偏移量是否大于文件长度，若小于则使rw_

→p = 文件长度 - offset；

4、返回值：0。

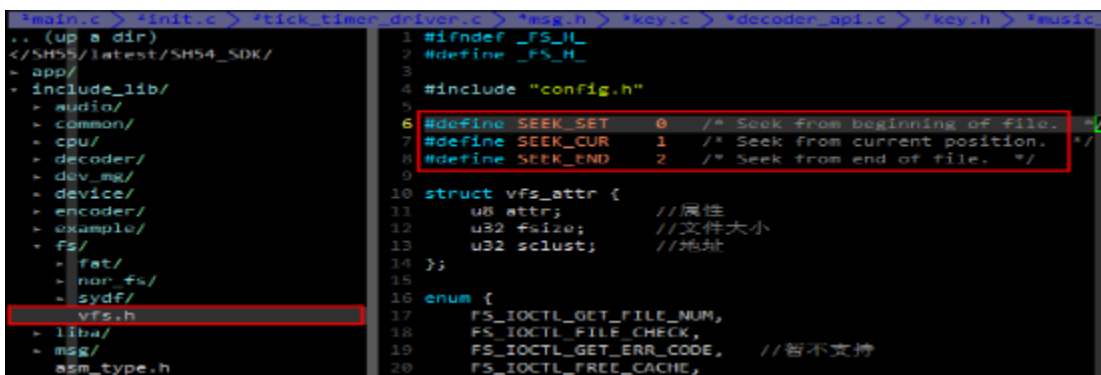


图 3.1 seek 模式选择

8.3.6 函数 u32 fs_file_close(void **ppvfile)

此函数实现关闭文件并释放句柄。其中参数：

1、ppvfile：已打开的文件句柄指针；

2、返回值：0。

示例：

```

vfs_file_close(&pfile);
if (pfile == NULL) {
    //成功关闭pfile文件
}

```

8.3.7 函数 u32 fs_fs_close(void **ppvfs)

此函数实现关闭文件系统并释放相关句柄。其中参数：

- 1、ppvfs：已挂载的文件系统句柄指针；
- 2、返回值：NULL。

示例：

```
fs_fs_close(&pfs);
if (pfs == NULL) {
    //成功关闭pfs文件系统
}
```

8.3.8 函数 u32 fs_file_name(void *pvfile, void *name, u32 len)

此函数实现获取指定文件的文件名。其中参数：

- 1、pvfile：已打开的文件句柄；
- 2、name：存放文件名的buf；
- 3、len：获取文件名的长度；
- 4、返回值：成功 返回获取文件名的长度；

备注：sydfs 和 norfs 文件系统文件名只支持小于 16 个 byte（包含后缀名）fatfs 小于等于 16byte 为短名，默认支持；大于 16byte 为长名，需要打开长名功能（详情请看）；

8.3.9 函数 int fs_get_attr(void *pvfile, void *pvfs_attr)

此函数实现获取文件的相关属性，包括文件大小和地址，nor_fs 无法使用此功能。其中参数：

- 1、pvfile：已打开的文件句柄；
- 2、pvfs_attr：文件系统属性，需要传入vfs_attr结构体类型的指针；
- 3、返回值：0。

示例：

```
struct vfs_attr file_attr;
fs_get_attr(pfile, &file_attr);
//文件的flash地址和大小分别保存在file_attr.sclust和file_attr.fsize中
```

8.3.10 函数 int fs_ioctl(void *pvfile, int cmd, int arg)

此函数实现对指定文件的相关操作，各文件系统具体的 cmd 命令请查看《1 文件系统介绍》小节。其中参数：

- 1、pvfile：已打开的文件句柄；
- 2、cmd：命令；
- 3、arg：参数；
- 4、返回值：
 - 成功 0；
 - 失败 错误请查看errno-base.h。

sydfs 文件系统

Sydfs 是系统数据运行的 只读文件系统，可以通过 sdk 里下载目录里的 download_bat.c 工具加上 -res 命令来下载存放资源文件，并于系统运行时读取；



```
22 copy /b %ELF_NAME%.bin+data.bin+cache_ram.bin app.bin
23 #else
24 copy /b %ELF_NAME%.bin+data.bin app.bin
25 #endif
26
27
28 @echo *****
29 @echo BD49 mbox_flash
30 @echo *****
31 @echo % date %
32 cd / d % ~dp0
33
34 ::isd_download.exe -tonorflash -dev bd49 -boot 0x3f01400 -div8 -wait 300 -uboot uboot.boot -app app.bin 0x20000 -res
35 dir_song dir_eng dir_poetry dir_story
36 isd download.exe -tonorflash -dev bd49 -boot 0x3f01400 -div8 -wait 300 -uboot uboot.boot -app app.bin 0x80000 -res
37 dir_a dir_song dir_eng dir_poetry dir_story dir_midi midi_cfg dir_notice dir_bin_flx
38 :-format all
39 @REM 常用命令说明
40 @rem - format vm // 擦除VM 区域
```

图 1.1 下载目录下内置 flash 资源

该文件系统能使用的接口图 1.1 所示：

```

177 const struct vfs_operations sydf_vfs_ops sec_used(.vfs_operations) = {
178     .fs_type = "sydf",
179     .init     = sydf_init_api,
180     /* .init     = NULL, */
181     .mount     = sydf_monut_api,
182     .openbypath = sydf_open_api,
183     .openbyindex = sydf_openbyindex_api,
184     .openbyfile = sydf_openbyfile_api,
185     .createfile = NULL,
186     .read       = sydf_read_api,
187     .write       = NULL,
188     .seek       = sydf_seek_api,
189     .close_fs    = sydf_fs_close_api,
190     .close_file  = sydf_file_close_api,
191     .fget_attr   = sydf_attr_api,
192     .name        = sydf_file_name_api,
193     .ioctl       = sydf_ioctl_api,
194 };

```

图 1.2 sydf 文件系统接口

备注:

自 **aw30n_v1.3.0** 版本开始, sydf 文件系统可以直接读取根目录下的文件。
 即 **download_bat.c** 里的资源文件可以不用打包也能通过 sydf 接口读取文件数据。
 该功能通过 **cpu_config.c** 里的 **sydf_read_root_doc** 变量控制。

- 1: 打开;
- 0: 关闭;

其中 **ioctl** 支持的 **cmd** 命令有:

① **FS_IOCTL_FILE_TOTAL**:

示例:

```

int file_total;
vfs_ioctl(pfile, FS_IOCTL_FILE_TOTAL, (int)&file_total);
file_total即为pfile文件所在文件系统的总文件数;

```

② **FS_IOCTL_FS_TOTAL**:

示例:

```

int file_total;
vfs_ioctl(pfs, FS_IOCTL_FILE_TOTAL, (int)&file_total);
file_total即为pfs文件系统的总文件数;

```

③ **FS_IOCTL_FILE_ATTR**:

示例:

```

struct vfs_attr attr;
vfs_ioctl(pfile, FS_IOCTL_FILE_ATTR, (int)&attr);
attr即可获取pfile文件的位置和大小;

```

- 注意: 通过 **ioctl** 获取的文件位置仅为该 **pfile** 文件在其上层目录下的相对偏移; 若要获取该文件相对于 **flash** 的偏移地址, 需要调用 **vfs_get_fsize()**;

Norfs 是一个循环管理的可读写文件系统，内部空间环形管理，空间不足时新写入的文件会覆盖之前的文件；

norfs 文件系统只通过文件号进行管理，创建新文件后文件号会累加；

该文件系统一般录音在内置 flash 或外置 flash 时使用；

该文件系统能使用的接口如图所示：

```
104 const struct vfs_operations norfs_vfs_ops sec_used(.vfs_operations) = {
105     .fs_type = "norfs",
106     /* .init      = norfs_init_api, */
107     .init      = NULL,
108     .mount      = norfs_monut_api,
109     .openbypath = NULL,
110     .openbyindex = norfs_openbyindex_api,
111     .createfile = norfs_createfile_api,
112     .read       = norfs_read_api,
113     .write      = norfs_write_api,
114     .seek       = norfs_seek_api,
115     .close_fs   = NULL,
116     .close_file = norfs_file_close_api,
117     .fget_attr  = NULL,
118     .name       = norfs_name_api,
119     .ioctl      = norfs_ioctl_api,
120 };
```

图 1.2 norfs 文件系统支持的接口

其中 ioctl 支持的 cmd 命令有：

- ①FS_IOCTL_FILE_TOTAL，同上
- ②FS_IOCTL_FS_TOTAL，同上
- ③FS_IOCTL_FILE_ATTR，同上
- ④FS_IOCTL_FILE_SYNC:

示例:

```
u32 flen = 0;
vfs_ioctl(pfile, FS_IOCTL_FILE_SYNC, (int)&flen);
```

↪ 该命令一般在录音结束时调用，用于更新norfs录音文件头，flen传入时为需要舍弃的录音末尾字节数。

④FS_IOCTL_FILE_INDEX:

示例:

```
int file_index;
vfs_ioctl(pfile, FS_IOCTL_FILE_INDEX, (int)&file_index);
file_index即为pfile文件所在norfs文件系统的最新文件序号;
```

⑤FS_IOCTL_FS_INDEX:

示例:

```
int file_index;
vfs_ioctl(pfs, FS_IOCTL_FS_INDEX, (int)&file_index);
file_index即为pfs所指的norfs文件系统的最新文件序号;
```

FatFs 是一个通用的文件系统，在本机中，支持对 FAT12/16/32、exFAT 文件系统进行读写操作；FAT 文件系统能使用的接口如图 1.3 所示：

```
392
393 #if (HAS_VFS_EN && FATFS_EN)
394
395 const struct vfs_operations fat_vfs_ops sec_used(.vfs_operations) = {
396     .fs_type = "fat",
397     .mount    = fat_monut_api,
398     .openbypath = fat_openR_api,
399     #if FS_WRITE_EN
400     .createfile = fat_openW_api,
401     .write      = fat_write_api,
402     .fdelete    = fat_delete_file_api,
403     .format     = fat_format_api,
404     #endif
405     .read       = fat_read_api,
406     .seek       = fat_seek_api,
407     .close_fs   = fat_fs_close_api,
408     .close_file = fat_file_close_api,
409     .fget_attr  = fat_attr_api,
410     .name       = fat_file_name_api,
411     .flen       = fat_get_fsize_api,
412     .ftell      = fat_get_pos_api,
413     .openbyindex = fat_openbyindex_api,
414     .openbyclust = fat_openbyclust_api,
415     .openbyfile  = fat_openbyfile_api,
416     .fscan_interrupt = fat_fscan_interrupt_api,
417     .fscan_release = fat_fscan_release_api,
418     .fsel         = fat_sel_api,
419     .ioctl        = fat_ioctl_api,
420 };
421
422 #endif
423
424 NORMAL → /apps/app/bsp/common/fs/fat/fat_resource.c
```

图 1.3 fat 文件系统支持的接口

11.1 FAT 文件系统管理函数

下面的函数为仅 FAT 使用的接口，在 `appbbspcommonfsfatfat_resource.c` 中的 `fat_demo()` 接口，可以找到下列函数的使用示例：

11.1.1 函数 int fs ftell(void *pvfile, void *parm)

此函数实现获取文件读写指针，仅 FAT 可使用该功能。其中参数：

- 1、pvfile: 已打开的文件句柄;
- 2、param: 用于保存文件系统读写指针;
- 3、返回值:
 - 成功 0;
 - 失败 错误请查看errno-base.h。

11.1.2 函数 int vfs_get_fsize(void *pvfile, void *parm)

此函数实现获取文件大小，仅 FAT 可使用该功能。其中参数：

- 1、pvfile：已打开的文件句柄；
- 2、param：用于保存文件大小的指针；
- 3、返回值：
成功 0；
失败 错误请查看errno-base.h。

11.1.3 函数 struct vfscan *vfs_fscan_new(void *pvfs, const char *path, const char *arg, u8 max_depth, int (*callback)(void), struct vfscan *fsn, struct vfscan reset_info *info)

此函数实现 FAT 文件系统的文件扫描配置，会返回文件扫描句柄供其他扫描接口使用，与 `vfs_fscan_release()` 搭配调用，可在 `ac104n sdk` 中的 `play_file.c` 中查看该函数具体使用示例。其中参数：

- 1、pvfs: 已挂载的文件系统句柄;
 - 2、path: 文件扫描的起始路径;
 - 3、arg: 文件扫描配置参数;
- 例: `static const char scan_parm_test[] =` “-t”
“MP1MP2MP3WAVTXT”
“-sn -r”
“-r”

- ① 首行“-”
→t”表示需要扫描的文件类型，后面“MP1MP2MP3WAVTXT”即为扫描的后缀名，三个字符为一种类型，最多不
- ② 第三行“-sn”表示扫描文件的顺序是按照文件号排序，另一种组合“-”
→st”表示按照时间排序；
- ③ 第三行“-r”表示搜索会包含文件夹子目录；
- ④ 各种扫描参数配置间，需要以空格隔开，其他配置有：
a)扫描是否将文件夹算入总数，可配置“-d”；

(续下页)

(接上页)

b) 扫描只读文件配置” -ar”，扫描非读文件配置” -a/
 →r”，不配置默认扫描所有文件；
 4、max_deep：扫描深度，最大为8；
 5、callback：扫描回调；
 6、fsn：文件扫描句柄；
 7、info：扫描结果信息，包含文件总数和文件夹总数等信息；
 8、返回值：
 成功 返回文件扫描句柄；
 失败 错误请查看errno-base.h。

11.1.4 函数 void fs_fscan_release(void *pvfs, struct vfscan *fs)

此函数实现释放文件扫描句柄，仅 FAT 可使用该功能，与 vfs_fscan() 搭配调用。其中参数：

1、pvfs：已挂载的文件系统句柄；
 2、fs：文件扫描句柄；
 3、返回值：无。

11.1.5 函数 int fs_get_folderinfo(void **pvfile, struct vfscan *fs, int *start_num, int *end_num)

此函数实现文件夹扫描，需使用 vfs_fscan() 返回的文件扫描句柄，仅 FAT 可使用该功能。其中参数：

1、pvfile：需要扫描的设备里的文件；
 2、fs：文件扫描句柄；
 3、Start_num：用于保存第一个文件夹的第一个文件号；
 4、End_num：用于保存最后一个文件夹的最后一个文件号；
 5、返回值：
 成功 0；
 失败 错误请查看errno-base.h。

11.1.6 函数 int fs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num)

此函数实现获取录音文件夹信息，包括录音文件总数，仅 FAT 可使用该功能。其中参数：

1、pvfs：已挂载的文件系统句柄；
 2、folder：文件夹路径，如” /JL_REC”；
 3、ext：录音文件的后缀名，如” MP3”；
 4、last_num：第一个可以使用的录音文件的文件序号；
 5、total_num：总录音文件数；
 6、返回值：
 成功 0；
 失败 错误请查看errno-base.h。

11.1.7 函数 u32 vfs_file_name(void *pvfile, void *name, u32 len)

此函数实现获取文件的名字。

该函数在获取文件名小于 16byte(带后缀名) 时, 返回的为 u8 字符;

在获取大于 16byte (带后缀名) 时, 返回的是 unicode 码, 且不会返回完整的文件名。

具体需要获取文件长名并最终转化为 u8 字符进行打印出来, 可参考该 issue 步骤:

FAQ: FAT 文件系统下如何获取函数文件名(短文件名 / 长文件名)?

- 1、pvfile: 已打开的文件句柄;
- 2、name: 获取文件名指针。
- 3、len: 获取文件名长度。
- 4、返回值: 文件名长度;

11.1.8 函数 int fs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg)

此函数实现 FAT 文件的文件查找, 支持多种查找方式, 具体示例可查看 fat_demo(void) 接口。
其中参数:

- 1、pvfs: 已挂载的文件系统句柄;
- 2、ppvfile: 文件句柄指针, 若句柄指向为 NULL, 函数会自行申请空间;
- 3、fs: 文件扫描句柄;
- 4、sel_mode: 文件查找方式:
 - ① FSEL_BY_NUMBER: 按文件号查找;
 - ② FSEL_BY_SCLUST: 按簇号查找;
 - ③ FSEL_BY_PATH: 按路径查找;
- 5、arg: 文件查找参数;
 - ① 按文件号查找时传入文件号;
 - ② 按簇号查找时传入簇号;
 - ③ 按路径查找时传入相应路径, 支持通配符查找;
- 6、返回值:
 - 成功 0;
 - 失败 错误请查看errno-base.h。

11.1.9 函数 int vfs_mk_dir(void *pvfs, char *folder, u8 mode)

此函数实现获取创建文件夹, 仅 FAT 可使用该功能。其中参数:

- 1、pvfs: 已挂载的文件系统句柄;
- 2、folder: 文件夹路径, 如 " /JL_REC" ;
- 3、mode: 目录属性;
 - 1: 设置为隐藏属性;
 - 0: 不设置;
- 4、返回值:
 - 成功 0;
 - 失败 错误请查看errno-base.h。

11.1.10 函数 int vfs_delete_dir(void *pvfs, char *path)

此函数实现获取删除文件夹，仅 FAT 可使用该功能。其中参数：

- 1、pvfs：已挂载的文件系统句柄；
- 2、path：文件夹路径，如” /JL_REC” ；
- 3、返回值：
 - 成功 0；
 - 失败 错误请查看errno-base.h。

使用 demo：

```
log_info("vfs_delete_dir\n");
err = vfs_delete_dir(pvfs, "/test_dir/adir");
if (err != 0) {
    log_error("vfs_delet_dir 0x%x\n", err);
    goto __vfs_delete_fs_close;
}
```

11.1.11 函数 int vfs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num)

此函数实现获取录音文件夹信息，包括录音文件总数，仅 FAT 可使用该功能。其中参数：

- 1、pvfs：已挂载的文件系统句柄；
- 2、folder：文件夹路径，如” /JL_REC” ；
- 3、ext：录音文件的后缀名，如” MP3” ；
- 4、last_num：第一个可以使用的录音文件的文件序号；
- 5、total_num：总录音文件数；
- 6、返回值：
 - 成功 0；
 - 失败 错误请查看errno-base.h。

11.1.12 函数 u32 vfs_file_delete(void *pvfile)

此函数实现文件的删除，仅 FAT 可使用该功能。其中参数：

- 1、pvfile：已打开的文件句柄；
- 2、返回值：
 - 成功 0；
 - 失败 错误请查看errno-base.h。

11.1.13 函数 `int vfs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg)`

此函数实现 FAT 文件系统的文件查找，支持多种查找方式，具体示例可查看 `fat_demo(void)` 接口。其中参数：

- 1、pvfs：已挂载的文件系统句柄；
- 2、ppvfile：文件句柄指针，若句柄指向为 NULL，函数会自行申请空间；
- 3、fs：文件扫描句柄；
- 4、sel_mode：文件查找方式：
 - ① FSEL_BY_NUMBER：按文件号查找；
 - ② FSEL_BY_SCLUST：按簇号查找；
 - ③ FSEL_BY_PATH：按路径查找；
- 5、arg：文件查找参数；
 - ① 按文件号查找时传入文件号；
 - ② 按簇号查找时传入簇号；
 - ③ 按路径查找时传入相应路径，支持通配符查找；
- 6、返回值：
 - 成功 0；
 - 失败 错误请查看 `errno-base.h`。

11.1.14 函数 `int vfs_format(void **ppvfs, const char *dev_name, const char *type, u32 clust_size, u8 create_new)`

此函数实现 FAT 文件系统格式化。其中参数：

- 1、ppvfs：已挂载的文件系统句柄；
- 2、dev_name：目标设备；
- 3、type：格式化后的文件系统；
- 4、clust_size：格式化后文件系统的簇大小，单位为k
- 5、create_new
 - new：设置为1为格式化成新的文件系统是由内部格式化接口自行判断当前设备容量适合哪种fat文件系统。
- 6、返回值：成功为0；有值为失败

使用 demo：

```
ret = vfs_format(&format_pfs, "udisk0", "fat", 64 * 1024, 1);    //
→格式化u盘成fat文件系统，簇大小为64*1024k
ret = vfs_format(&format_pfs, "ext_flsh", "fat", 64 * 1024, 1); //
→格式化外挂flash成fat文件系统，簇大小为64*1024k
ret = vfs_format(&format_pfs, "ext_flsh", "fat", 512, 1);       //
→格式化外挂flash成fat文件系统，簇大小为512k
```

备注：

- 1、若 ppvfs 指针为空，则 create_new 参数不能传 0；
- 2、格式化前最好先 mount 一下目标设备是否里面存在文件系统再去格式化。
- 3、格式化成功后，如需继续使用该设备，需要再重新 mount；

11.1.15 函数 int vfs_ioctl(void *pfile, int cmd, int arg)

fat 文件系统支持多种命令操作，CMD 命令如下：

1. 创建文件夹 FS_IOCTL_MK_DIR
2. 获取文件夹序号和文件夹内文件数目 FS_IOCTL_GET_FOLDER_INFO_3
3. 获取录音文件信息 FS_IOCTL_GET_ENCFOLDER_INFO
4. 重置扫描句柄 FS_IOCTL_RESET_VFSCAN
5. 设置卷标 FS_IOCTL_SET_VOL
6. 获取分区信息，簇大小，容量 FS_IOCTL_GET_PARTITION_INFO
7. 获取剩余空间 FS_IOCTL_GET_FREE_SPACE
8. 获取当前文件的相对路径和绝对路径 FS_IOCTL_GET_PATH

获取当前文件的相对路径和绝对路径 demo：

```
len = vfs_fget_path(pfile, fsn, (u8 *)name, sizeof(name), 0);    //
↪ 获取绝对路径
len = vfs_fget_path(pfile, fsn, (u8 *)name, sizeof(name), 1);    //
↪ 获取相对路径
/*如果路径里的文件夹大于16byte，获取出来的是Unicode码，小于16byte的则为utf8*/
/*返回值为路径长度，name即为获取的路径*/
```

获取剩余空间 demo：

```
u32 cap = 0;
res = vfs_ioctl(pvfs, (int)FS_IOCTL_GET_FREE_SPACE, (int)&cap);
log_info("cap 0x%x\n", res, cap);    //
↪ cap为获取到fat文件系统的剩余容量，单位为k
```

获取分区信息，簇大小，容量 demo：

```
u32 res = 0;
FS_PARTITION_INFO part_info = {0};
res = vfs_ioctl(pvfs, (int)FS_IOCTL_GET_PARTITION_INFO, (int)&part_info);
log_info("0x%x 0x%x 0x%x \n", part_info.clust_size, part_info.total_size, part_
↪ info.fs_type);
/*分别对应簇大小，文件系统总容量大小，文件系统类型*/
```


解码接口

所有解码被统一管理。解码时系统会根据文件采样率与 DAC 采样率进行变采样。

只有一路解码时，可不进行变采样，使 DAC 随着音频文件的采样率调整输出的采样率。

目前 SDK 默认支持的多个解码通路:

```
1、2*f1a/b/c+a
2、ump3+a
3、midi+a
3、标准MP3+a
```

解码管理的接口函数:

```
void decoder_init(void);
dec_obj *decoder_io(void *pfile, u32 dec_ctl, dp_buff * dbuff,u8_
↳loop);
dec_obj *decoder_list(dec_data_stream *p_strm, u32 dec_ctl, dp_buff_
↳*dbuff, u8 loop, u32 output_sr, u16 *p_max_input);
void irq_decoder_ret(dec_obj *obj,u32 ret);
```

解码功能控制函数:

```
void decoder_stop(dec_obj *obj, DEC_STOP_WAIT wait);
void decoder_pause(dec_obj *obj);
bool decoder_ff(dec_obj *obj, u8 step);
bool decoder_fr(dec_obj *obj, u8 step);
u32 decoder_get_flen(void *priv);
int decoder_time(dec_obj *p_dec);
```

各个格式的解码接口函数:

```
u32 midi_decode_api(void *strm, void **ppdec, void *p_dp_buf);
u32 mp3_st_decode_api(void *strm, void **p_dec, void *p_dp_buf);
u32 ump3_decode_api(void *strm, void **p_dec, void *p_dp_buf);
```

(续下页)

(接上页)

```
u32 wav_decode_api(void *strm, void **p_dec, void *p_dp_buf);
u32 a_decode_api(void *strm, void **p_dec, void *p_dp_buf);
u32 f1a_decode_index(void *strm, u32 index, dec_obj **p_dec, void *p_
↪dp_buf, flx_data *p_flx_data);
u32 opus_decode_api(void *strm, void **p_dec, void *p_dp_buf);
u32 ima_decode_api(void *strm, void **p_dec, void *p_dp_buf);
u32 sbc_decode_api(void *strm, void **p_dec, void *p_dp_buf);
u32 speex_decode_api(void *strm, void **p_dec, void *p_dp_buf);
u32 jla_lw_decode_api(void *strm, void **p_dec, void *p_dp_buf);
```

12.1 解码使用一般流程

在开机时需要调用函数 `void decoder_init(void)`，此函数实现对解码管理器进行初始化，会初始化解码控制器里面的一些变量。

12.1.1 在解码之前的设备以及文件的初始化

在解码之前要确保已经完成了需要解码的设备，以及设备中的文件进行了初始化。获取到了文件句柄，初始化 DAC 之后便可以开始正常的解码流程：

1、调用 `decoder_io` 实现对解码器的控制，此函数的传参可以控制：

- a. 对文件在指定的几种解码格式中解析，自动选取正确的格式进行解码；
- b. 控制解码输出的 PCM 数据结果是否经过音效处理，目前阶段主要是 EQ、SRC；
- c. 输入一断点 buff，有了此 buff 才能实现续播功能；
- d. 控制续播的次数；

2、经过上面的一步已经完成了对解码的启动，此时可以调用 `decoder_pause`、`decoder_stop` 实现暂停解码与停止解码的功能；

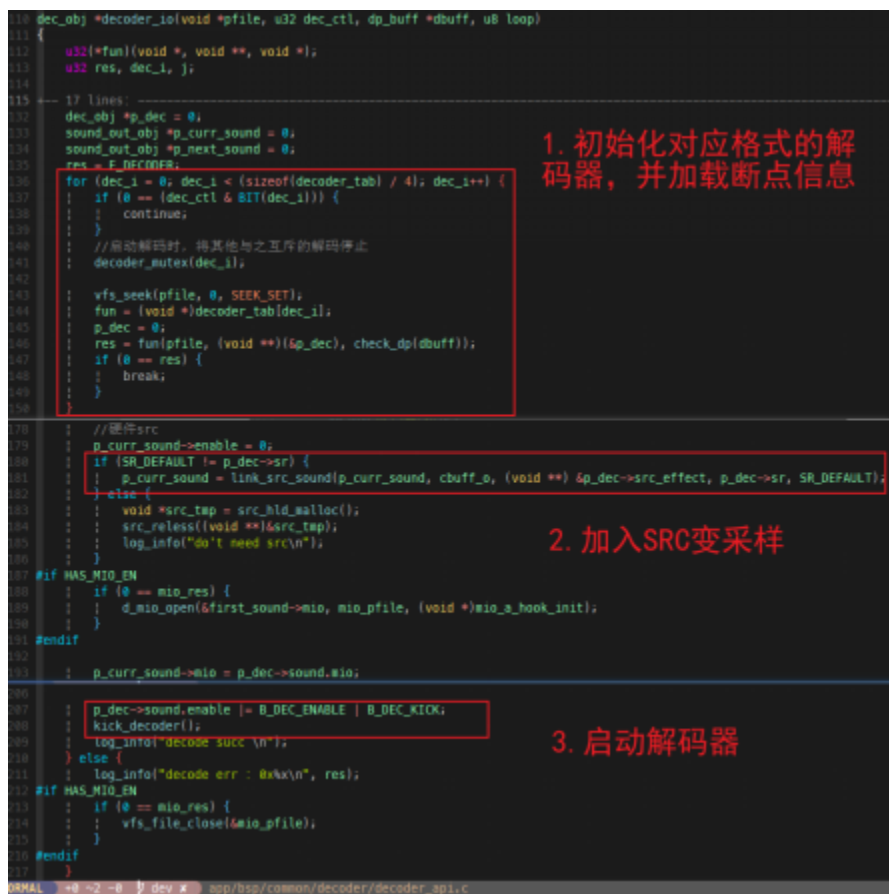


图 1.1 解码初始化流程

12.1.2 数据流获取

所有解码格式初始化时，会注册数据输入、输出接口给解码器进行回调。用户可根据需求自行修改数据输入、输出接口。

涉及文件 apps/app/bsp/common/decoder/mp_io.c，涉及的结构体如下：

```
struct if_decoder_io {
    void *priv ;
    int (*input)(void *priv, u32 addr, void *buf, int len, u8 type);
    int (*check_buf)(void *priv, u32 addr, void *buf);
    u32(*output)(void *priv, void *data, int len);
    u32(*get_lslen)(void *priv);
    u32(*store_rev_data)(void *priv, u32 addr, int len);
};
```

结构体中注册的函数原型：

```
int mp_input(void *priv, u32 addr, void *buf, int len, u8 type)
{
    dec_obj *obj = priv;
    int rlen = 0;
    fs_seek(obj->p_file, addr, SEEK_SET);    //
```

(续下页)

(接上页)

```

    ↪addr为相对文件起始位置的偏移, len为多少个byte
        rlen = fs_read(obj->p_file, buf, len);
        return rlen;
    }

    /*
    ↪解码开始时调用input接口读取音频文件数据, 其中参数addr为文件内的偏移地址, len为读取长度, 返回
    ↪*/

int mp_output(void *priv, void *data, int len)
{
    dec_obj *obj = priv;
    return sound_output(&obj->sound, data, len);
}

/*
    ↪数据解码完成后解码器会调用output接口将解码后的pcm数据写入dac的buff, 其中参数data为数据输出的起
    ↪*/

```

12.1.3 解码流程

1、主循环与解码、AUDIO 中断

如图 4.1, 系统初始化 DAC 后, AUDIO 中断会定时启动并检测是否有解码通道; 未注册解码器时, DAC 模块空转无声。

当解码器启动时, 解码后的 pcm 数据会写入缓存 buff 中, AUDIO 中断则会不断地消耗缓存 buff 中的 pcm 数据, 处理后 DAC 模块出声。

当解码器停止时, AUDIO 中断检测到没有解码通道, DAC 模块空转无声, 重复上述流程。

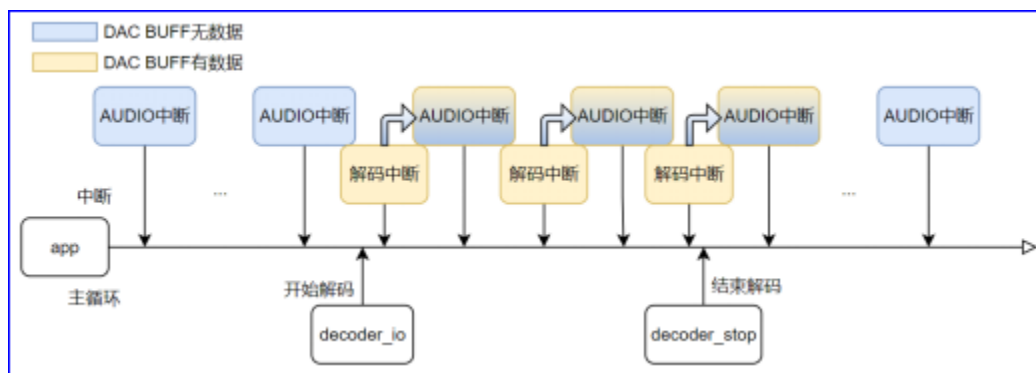


图 1.3 解码启动与结束的中断行为

2、解码中断运行具体流程

如图 1.4, 解码器启动后, 首先会启动一次解码中断;

- 1) 解码中断会通过 input 接口读取音源数据进行解码运算;
- 2) 运算完成后调用 output 接口将解码后的 pcm 数据写入到缓存 buff 中;
- 3) AUDIO 中断会检测已注册的解码通道中是否有数据, 有则将数据写入 DAC 物理 buff 并融合通道数据、调整音量;
- 4) DAC 消耗 pcm 数据, 并启动下一轮解码中断, 循环上述过程直到解码停止或结束。

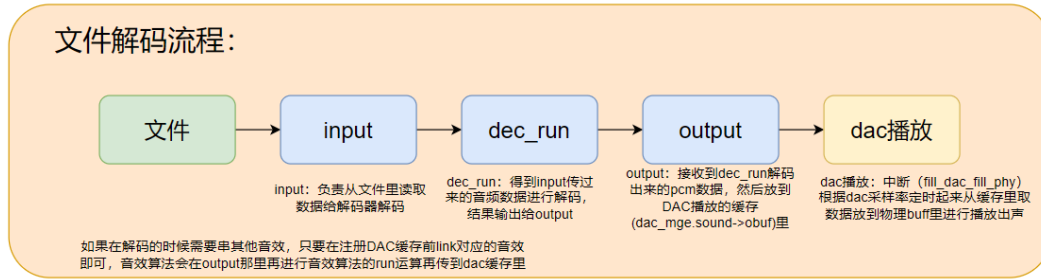


图 1.4 解码运行具体流程

12.1.4 DAC 数据处理说明

DAC 中断会调用函数 fill_dac_fill_phy() 对解码后的 pcm 数据进行相应处理并填充到 DAC 的物理 buff, 该函数处理数据过程如下:

1、解码通道数据轮询

查询各个解码通道是否有被注册、是否有有效数据, 若通道已注册且有有效数据, 则获取该通道 pcm 数据的起始地址与长度。

```

82 active_flag = 0;
83 for (i = 0; i < AUDAC_CHANNEL_TOTAL; i++) {
84     if (0 == (dac_mge.ch & BIT(i))) {
85         continue;
86     }
87     if (false == dac_cbuff_active(dac_mge.sound[i])) {
88         continue;
89     }
90     rptr[i] = cbuf_read_alloc(dac_mge.sound[i]->p_obuf, &olen[i]);
91     if (0 == olen[i]) {
92         log_char('z');
93         rptr[i] = 0;
94     }
95     fifo_dac_fill_kick(i, sp_number / DAC_TRACK_NUMBER); //m10依赖单声道数据量
96 }
  
```

图 1.5 解码通道数据轮询

2、多通道数据融合

获取每个通道的样点数据, 并进行叠加融合。

```

099 for (sp_cnt = 0; sp_cnt < sp_number; sp_cnt++) {
100     t_sp = 0;
101     for (i = 0; i < AUDAC_CHANNEL_TOTAL; i++) {
102         if (0 == rptr[i]) {
103             continue;
104         }
105         if (0 == olen[i]) {
106             rptr[i] = cbuf_read_alloc(dac_mge.sound[i] -> p_obuf, 60len[i]);
107             if (0 == olen[i]) {
108                 log_char('x');
109             }
110         }
111         continue;
112     }
113     t_sp += *rptr[i];
114     if ((0 == (B_STEREO & dac_mge.sound[i] -> info)) && \
115         (2 == DAC_TRACK_NUMBER)) {
116         if (1 == (sp_cnt & 0x01)) {
117             p_cnt[i]++;
118             rptr[i]++;
119         }
120     } else {
121         p_cnt[i]++;
122         rptr[i]++;
123     }
124 }
125
126 //uc03 是立体声
127 /* audio_dac_stereo_cnt(\ */
128 /* (B_STEREO & dac_mge.sound[i] -> info), \ */
129 /* sp_cnt, \ */
130 /* rptr[i]) */
131
132 if ((p_cnt[i] * 2) >= olen[i]) {
133     cbuf_read_update(dac_mge.sound[i] -> p_obuf, p_cnt[i] * 2);
134     /* rptr[i] = 0; */
135     olen[i] = 0;
136     p_cnt[i] = 0;
137     rptr[i] = cbuf_read_alloc(dac_mge.sound[i] -> p_obuf, 60len[i]);
138     if (0 == olen[i]) {
139         log_char('y');
140     }
141 }
142
143 }
144 }

```

图 1.6 多通道数据融合

3、音量限幅

通道融合后的数据需要进行限幅，并将处理后的数据更新到 DAC 的物理 buff。

```

151 t_sp += sp_buf[sp_cnt];
152 if (t_sp > 32767) {
153     t_sp = 32767;
154 } else if (t_sp < -32768) {
155     t_sp = -32768;
156 }
157 sp_buf[sp_cnt] = t_sp;
158 all_cnt++;
159 }
160 }
161 }
162 }
163 }
164 }

```

图 1.7 音量调节

4、启动下一次解码中断

更新缓存 buff 的信息，减去本次消耗的 pcm 数据长度，并启动下一次解码中断。

```

167
168     for (i = 0; i < AUDAC_CHANNEL_TOTAL; i++) {
169         if (0 == (dac_mge.ch & BIT(i))) {
170             continue;
171         }
172         if (0 != rptr[i]) {
173             cbuf_read_updata(dac_mge.sound[i]->p_obuf, p_cnt[i] * 2);
174         }
175         dac_kick(dac_mge.sound[i], dac_mge.kick[i]);
176     }

```

uc03_dev <ommon/audio/fill_audac.c c utf-8

图 1.8 启动下一次解码中断

12.2 解码管理接口函数

12.2.1 函数 `dec_obj *decoder_io(void *pfile, u32 dec_ctl, dp_buff * dbuff, u8 loop)`

此函数实现对以及获取到的文件进行解码，其中参数：

- 1、void *pfile，已经获取到的文件句柄；
- 2、u32 dec_ctl，解码控制参数，通过对不BIT的置一操作实现对相应功能的控制：
 - ① BIT_MP3_ST：置上此位为可以将MP3解码加入到解码轮询中；
 - ② BIT_WAV：置上此位为可以将WAV解码加入到解码轮询中；
- 3、dp_buff * dbuff，断点缓存，用于续播功能：
 - ① NULL，不能实现续播功能，
 - ② 非NULL，会使用给定的buff实现续播功能
- 4、u8 loop，续播次数控制(0~255)：
 - ① 0，播放完成后不续播；
 - ② 1~244，续播次数；
 - ③ 255，一直持续续播；
- 5、返回值，解码句柄，利用解码句柄可以实现对当前解码通道进行控制（主要是暂停和播放）。

```

167
168     for (i = 0; i < AUDAC_CHANNEL_TOTAL; i++) {
169         if (0 == (dac_mge.ch & BIT(i))) {
170             continue;
171         }
172         if (0 != rptr[i]) {
173             cbuf_read_updata(dac_mge.sound[i]->p_obuf, p_cnt[i] * 2);
174         }
175         dac_kick(dac_mge.sound[i], dac_mge.kick[i]);
176     }

```

uc03_dev <ommon/audio/fill_audac.c c utf-8

图 1.8 启动下一次解码中断

```

decoder_msg_tab.h x
22     INDEX_MIDI_CTRL,
23     #endif
24     #if DECODER_WAV_EN
25         INDEX_WAV,
26     #endif
27     #if DECODER_MP3_ST_EN
28         INDEX_MP3_ST,
29     #endif
30     INDEX_E_SPEED = 12,
31 };
32
33 #define BIT_SPEED BIT(INDEX_E_SPEED)
34
35 #if DECODER_MP3_ST_EN
36 #include "mp3_standard_api.h"
37 #define MP3_ST_HLD (u32)&dec_mp3_st_hld
38 #define MP3_ST_LST MP3_ST_HLD,
39 #define MP3_ST_API (u32)mp3_st_decode_api,
40 #define MP3_ST_MUT_TAB (u32)mp3_st_buff_api,
41 #define BIT_MP3_ST BIT(INDEX_MP3_ST)
42 #else
43 #define MP3_ST_HLD (u32)NULL
44 #define MP3_ST_LST
45 #define MP3_ST_API
46 #define MP3_ST_MUT_TAB
47 #define BIT_MP3_ST 0
48 #endif

```

图 2.1 解码控制参数

12.2.2 dec_obj *decoder_list(dec_data_stream *p_strm, u32 dec_ctl, dp_buff *dbuff, u8 loop, u32 output_sr)

此函数实现对获取到的数据源进行解码初始化，其中参数：

- 1、dec_data_stream *pfile，已经获取到的文件句柄；
- 2、u32 dec_ctl，解码控制参数，通过对BIT的置一操作实现对相应功能的控制：
 - ① BIT_MP3_ST：置上此位为可以将MP3解码加入到解码轮询中；
 - ② BIT_WAV：置上此位为可以将WAV解码加入到解码轮询中；
- 3、dp_buff * dbuff，断点缓存，用于续播功能：
 - ① NULL，不能实现续播功能，
 - ② 非NULL，会使用给定的buff实现续播功能
- 4、u8 loop，续播次数控制(0~255)：
 - ① 0，播放完成后不续播；
 - ② 1~244，续播次数；
 - ③ 255，一直持续续播；
- 5、u32 output_sr，音频解码输出采样率；
- 6、返回值，解码句柄，利用解码句柄可以实现对当前解码通道进行控制（主要是暂停和播放）。

```

167
168     for (i = 0; i < AUDAC_CHANNEL_TOTAL; i++) {
169         if (0 == (dac_mge.ch & BIT(i))) {
170             continue;
171         }
172         if (0 != rptra[i]) {
173             cbuf_read_updata(dac_mge.sound[i]->p_obuf, p_cnt[i] * 2);
174         }
175         dac_kick(dac_mge.sound[i], dac_mge.kick[i]);
176     }

```

uc03_dev <ommon/audio/fill_audac.c c utf-8

图 1.8 启动下一次解码中断

```

decoder_msg_tab.h x
22     INDEX_MIDI_CTRL,
23     #endif
24     #if DECODER_WAV_EN
25         INDEX_WAV,
26     #endif
27     #if DECODER_MP3_ST_EN
28         INDEX_MP3_ST,
29     #endif
30     INDEX_E_SPEED = 12,
31 };
32
33 #define BIT_SPEED BIT(INDEX_E_SPEED)
34
35 #if DECODER_MP3_ST_EN
36 #include "mp3_standard_api.h"
37 #define MP3_ST_HLD (u32)&dec_mp3_st_hld
38 #define MP3_ST_LST MP3_ST_HLD,
39 #define MP3_ST_API (u32)mp3_st_decode_api,
40 #define MP3_ST_MUT_TAB (u32)mp3_st_buff_api,
41 #define BIT_MP3_ST BIT(INDEX_MP3_ST)
42 #else
43 #define MP3_ST_HLD (u32)NULL
44 #define MP3_ST_LST
45 #define MP3_ST_API
46 #define MP3_ST_MUT_TAB
47 #define BIT_MP3_ST 0
48 #endif

```

图 2.1 解码控制参数

12.2.3 函数 void irq_decoder_ret(dec_obj *obj,u32 ret)

此函数是解码器在解码过程中的回调函数，不能再这里做长时间的操作，因为这样会卡慢解码的速度，其中参数：

- 1、dec_obj *obj，解码句柄
- 2、ret，解码运行的结果，0为成功，其他如表2.1和图2.2
- 3、返回值，无。

irq_decoder_ret 返回值	含义
MAD_ERROR_FILE_END	解码文件结束
MAD_ERROR_FILESYSTEM_ERR	未使用
MAD_ERROR_DISK_ERR	未使用
MAD_ERROR_SYNC_LIMIT	文件错误
MAD_ERROR_FF_FR_FILE_END	快进到头
MAD_ERROR_FF_FR_END	未使用
MAD_ERROR_FF_FR_FILE_START	快退到头
MAD_ERROR_LIMIT	未使用
MAD_ERROR_NODATA	未使用
MAD_ERROR_STREAM_NODATA	本轮 run 读不到数据
MAD_CANNOT_SYNC_TSLOOP	本轮没有找到同步字
MAD_THISREAD_LT_NEEDSZ	本次读数据没读够
MAD_FRAMELEN_GT_BUFFSZ	压缩帧长超出范围

表 2.1 irq_decoder_ret 返回值

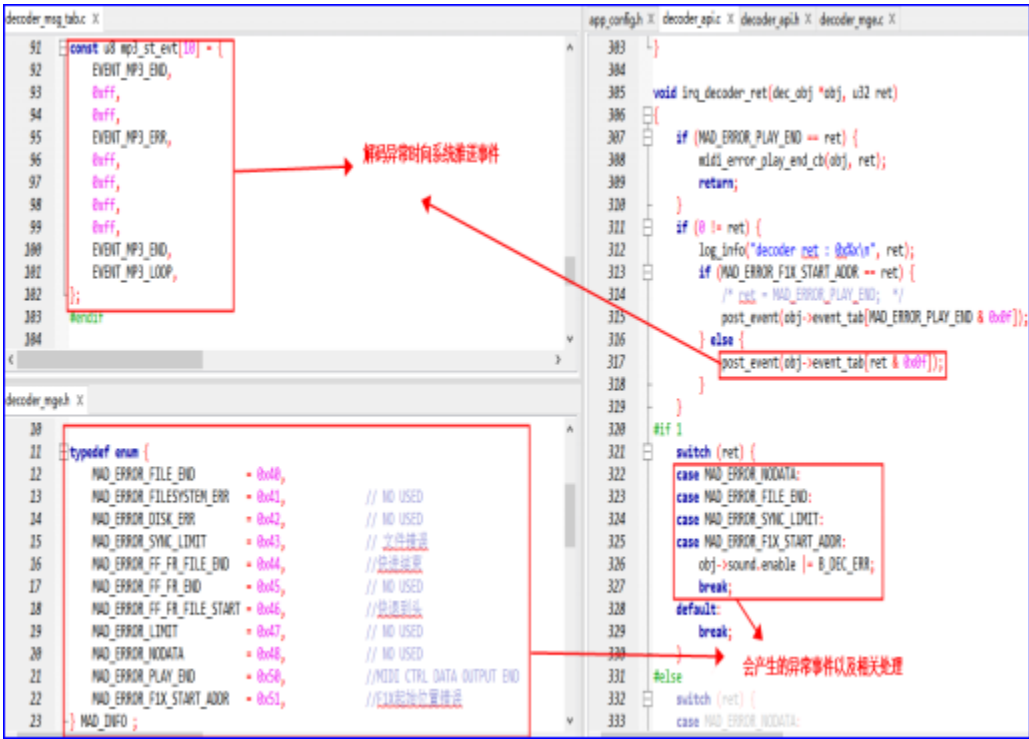


图 2.2 解码回调函数的相关处理

12.3 解码功能控制函数

12.3.1 函数 void decoder_pause(dec_obj *obj)

此函数实现在解码过程中对当前解码进行暂停播放与继续播放的控制，其中参数：

1、dec_obj *obj，解码句柄，启动解码时从函数decoder_io获取的返回值；

12.3.2 函数 void decoder_stop(dec_obj *obj, DEC_STOP_WAIT wait)

此函数实现停止当前解码的功能，其中参数：

- 1、obj，解码句柄，启动解码时从函数decoder_io获取的返回值；
- 2、wait，是否等待DAC消耗完解码输出的PCM数据后再停止解码；
 - ① NO_WAIT，直接停止解码
 - ② NO_WAIT，等待DAC消耗完全部的解码输出数据后再停止解码

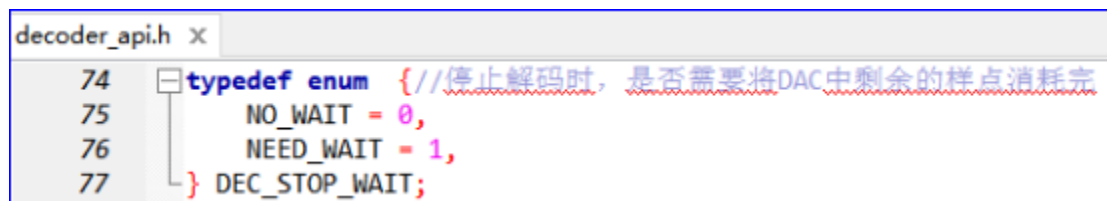


图 3.1 停止解码等待变量的 DEC_STOP_WAIT

12.3.3 函数 bool decoder_ff(dec_obj *obj, u8 step)

此函数实现解码快进功能，仅播放标准 MP3 和 WAV 有效，其中参数：

- 1、obj：解码句柄，启动解码时从函数decoder_io获取的返回值；
- 2、step：快进步进，单位为秒；

12.3.4 函数 bool decoder_fr(dec_obj *obj, u8 step)

此函数实现解码快退功能，仅播放标准 MP3 和 WAV 有效，其中参数：

- 1、obj：解码句柄，启动解码时从函数decoder_io获取的返回值；
- 2、step：快退步进，单位为秒；

12.3.5 函数 int decoder_time(dec_obj *p_dec)

此函数实现获取当前播放时间位置功能，其中参数：

- 1、obj：解码句柄，启动解码时从函数decoder_io获取的返回值；
- 2、返回值：当前正在播放的时间位置，单位为秒；

12.4 各个格式的解码函数

各种格式的解码启动函数统一被 decoder_io 所调用，其一般名称为：XXX_decode_api。XXX 为其格式名称。

```
u32 mp3_decode_api(void *p_file, void **p_dec, void *p_dp_buf)
u32 wav_decode_api_1(void *p_file, void **p_pdec, void *t_dp_buf)
```

解码 MP3 音频时会进行 MP3 格式检查，格式检查可能需要读取多段音源文件数据，对于部分读取速度较慢的设备，可能会无法清狗而导致看门狗复位；故在 mp3_standard_api.c 文件中有 FORMAT_CHECK_TIMEOUT_10MS 宏定义，当格式检查超过一定时间时，强制退出格式检查并返回错误。

```
#define MP3_FORMAT_CHECK_TIMEOUT_10MS 400
//MP3格式检查超时时间，单位10ms，400即4秒，一般不超过看门狗复位时间的一半
```

12.4.1 统一介绍 decode_api

函数 u32 XXX_decode_api(void *p_file, void **p_dec, void *p_dp_buf):

- ① void *p_file, 文件句柄
- ② void **p_dec, 存放解码句柄指针的指针
- ③ void *p_dp_buf, 断点buff, 提供给续播使用

12.5 解码常见问题

12.5.1 播歌有卡音或者杂音如何查找问题

播歌有卡音或者杂音通常有三种情况：

函数 fill_dac_fill() 从缓存 buff 中取不到数据

这种情况可以通过查看调用函数 cbuf_read_alloc() 后打印的字符 x、y 和 z 来判断，通常卡音与字符打印会同时出现；该问题出现的原因可能是解决被高优先级中断打断或者速度不够解不过来，可以尝试提高系统时钟频率。

```

75
76     for (sp_cnt = 0; sp_cnt < sp_number; sp_cnt++) {
77         t_sp = 0;
78         for (i = 0; i < AUDAC_CHANNEL_TOTAL; i++) {
79             if (0 == rptr[i]) {
80                 continue;
81             }
82             if (0 == olen[i]) {
83                 rptr[i] = cbuf_read_alloc(dac_mge.sound_later[i]->p_obuf, &olen[i]);
84                 if (0 == olen[i]) {
85                     log_char('x');
86                 }
87                 continue;
88             }
89         }
90         t_sp += *rptr[i];
91         apps/app/bsp/common/audio/fill_audac.c
116
117         if ((p_cnt[i] * 2) >= olen[i]) {
118             cbuf_read_update(dac_mge.sound_later[i]->p_obuf, p_cnt[i] * 2);
119             /* rptr[i] = 0; */
120             olen[i] = 0;
121             p_cnt[i] = 0;
122             rptr[i] = cbuf_read_alloc(dac_mge.sound_later[i]->p_obuf, &olen[i]);
123             if (0 == olen[i]) {
124                 log_char('y');
125             }
126         }
127     }
128 }
129
130 if (dac_mge.flag & B_DAC_DIG_VOL) {
131     u16 max_phy_vol = get_dac_max_phy_vol();
132     if (max_phy_vol != dac_mge.vol_l_phy) {
133         /* 使用数字音量默认以左声道音量调整样点 */
134         apps/app/bsp/common/audio/fill_audac.c
135         continue;
136     }
137     /* if ((cbuf_get_data_size(dac_mge.sound[i]->p_obuf) >= len) || \ */
138     /*      (!{dac_mge.sound[i]->enable & B_DEC_RUN_EN})) { */
139     /*      rptr[i] = cbuf_read_alloc(dac_mge.sound[i]->p_obuf, &olen[i]); */
140     /*  } */
141     rptr[i] = cbuf_read_alloc(dac_mge.sound_later[i]->p_obuf, &olen[i]);
142     if (0 == olen[i]) {
143         log_char('z');
144         rptr[i] = 0;
145     }
146     fifo_dac_fill_kick(i, sp_number / DAC_TRACK_NUMBER); //mio依赖单声道数据量
147 }
148
149 for (sp_cnt = 0; sp_cnt < sp_number; sp_cnt++) {
150     t_sp = 0;
151     for (i = 0; i < AUDAC_CHANNEL_TOTAL; i++) {
152         apps/app/bsp/common/audio/fill_audac.c

```

图 5.1 AUDIO_DAC 取不到数据

DAC 中断被打断导致没有定时起中断

AW30N 在使用 APA 输出时，可以通过在 dac 中断函数中调用到的 `fifo_dac_fill()` 函数中翻转 I/O 口，检查进入函数的频率是否固定；同时检查是否有更高优先级的中断挡住 DAC 中断或者关总中断时间过长的地方。

```
201
202 AT(.audio_isr_text)
203 u32 fifo_dac_fill(u8 *buf, u32 len, AUDIO_TYPE type)
204 {
205     JL_PORTA->DIR &= ~BIT(11);  翻转PA11，查看IO翻转频率是否正常
206     JL_PORTA->OUT |= BIT(11);
207     JL_PORTA->OUT &= ~BIT(11);
208     return fill_dac_fill_phy(buf, len);
209 }
```

NORMAL → apps/app/bsp/common/audio/fill_audac.c

图 5.2 DAC 中断

如果使用的是 RDAC 输出，可以通过以上方法翻转 IO，但是一次 dac 中断可能会二次调用 `fifo_dac_fill()` 函数，所以需要看整体的时间是否固定。

音源本身有杂音

这种情况可以通过将音源放在电脑端播放验证。

12.5.2 解码开始后声音变调问题

这种情况通常是因为音频文件的采样率与 DAC 的采样率不一致，需要进行 SRC 变采样；而工程占用了太多 ram 资源，SRC 变采样模块申请不到空间不起作用导致播放变调；可以通过查看打印中是否有 “src init fail” 来确定该问题。

各解码格式配置说明

AD16N、AW30N 同时支持的解码格式有：F1A/B/C/X、UMP3、MIDI、A、标准 MP3、WAV 六种格式；

AW30N 还支持 OPUS、ADPCM-IMA、SBC/MSBC、SPEEX 解码格式。

AD16N 和 AW30N 各 SDK 对解码格式支持情况如下表：

解码格式\芯片对应 SDK	AD16N otp 方案	AD16N flash 方案	AW30N 方案
f1a	不支持	支持	支持
midi 解码 & midi 琴	不支持	支持	支持
A 文件解码	不支持	支持	支持
标准 MP3	支持	支持	支持
WAV	支持	支持	支持
UMP3	不支持	支持	支持
OPUS	不支持	不支持	支持
ADPCM-IMA	不支持	不支持	支持
SBC/MSBC/USBC	不支持	不支持	支持
SPEEX	不支持	不支持	支持
jla_lw	不支持	不支持	支持

下面简要介绍一下各个格式配置：

文件流解码格式：

1. “f1a 解码格式”
2. “midi 解码和 midi 琴解码格式”
3. “a 解码格式”
4. “标准 MP3 解码格式”
5. “WAV 解码格式”

音频数据流解码格式：

6. “OPUS 解码格式”
7. “ADPCM-IMA 解码格式”
8. “SBC/MSBC/USBC 解码格式”
9. “speex 解码格式”
- 10.” jla_lw 解码格式”

同时支持文件流解码和数据流解码格式：

11. “ump3 解码格式”

13.1 f1a 解码格式

f1a 解码格式为我司特有的音频解码格式。

具体内容可以了解 f1a_api.c 和 f1a_api.h

格式	后缀名	AD16N otp 方案	flash 方案
f1a	.f1a .f1b .f1c .f1x	不支持	支持

- 解码接口函数：

```
u32 f1a_decode_index(void *strm, u32 index, dec_obj **p_dec, void *p_dp_buf, f1x_data
*p_f1x_data);
```

13.2 ump3 解码格式

ump3 解码格式为我司特有的音频解码格式。

具体内容可以了解 ump3_api.c 和 ump3_api.h

格式	后缀名	AD16N otp 方案	flash 方案
ump3	.ump3	不支持	支持

- 解码接口函数：

```
u32 ump3_decode_api(void *strm, void **p_dec, void *p_dp_buf);
```

13.3 midi 解码和 midi 琴解码格式

midi 解码和 midi 琴解码格式

MIDI 解码具体内容可以了解 midi_api.c 和 MIDI_DEC_API.H

MIDI 琴解码具体内容可以了解 midi_ctrl_api.c 和 MIDI_CTRL_API.H

midi 解码用户可参考 midi_decode_app 模式。

midi 琴用户可参考 midi_keyboard_app 模式。

详细 midi 功能及说明请看 SDK 包里的《杰理 AD1x-45678_MIDI 应用说明文档.pdf》文档。

(注：该文档实际函数以 AD16N_flash_v1.4.0 工程为准，其余芯片 SDK 实际函数名会有出入，但实现原理不变)

AD16N 从 V1.4.0 版本开始开源 2byte 和 4byte 地址版本的 midi 中间层代码。

备注：

midi 功能常涉及两部分文件，一个是 **音色库**，一个是用户自己的 **midi 乐曲文件**。

1. **音色库**只能烧写放入内置 flash 里，不支持放在其他设备!!!
2. **midi 乐曲文件**用户可以任意放在其他设备。

13.3.1 midi 驱动版本选择说明

AD16N 从 V1.4.0 版本开始开源 2byte 和 4byte 地址版本的 midi 中间层代码，用户可使用位于 SDK 根目录的工具来选择编译链接 2byte 或 4byte 版本的中间层代码、头文件以及算法库。

注：该工具是通过修改 cbp 文件信息来选择编译对应的驱动的版本，不会对 makefile 做出任何改动，因此只能用于 codeblocks 等依赖 cbp 文件信息来编译的开发平台。

midi 2byte 和 4byte 版本的基本差异如下表：

midi 版本	乐 谱 后 缀 名	音 色 库 后 缀 名	算 法 库 名 (已 开 源 的 库 名)	音 色 库 支 持 的 PCM 样 点 数	音 色 库 支 持 的 声 道 数	算 法 输 出 支 持 的 声 道 数
2byte	.mfa	.mda	lib_midi_synth.a	<=65535 个点	单声道	单声道
4byte	.mfa	.mdb	lib_midi_synth_pi	<=1900000	单声道、双声道	单声道、双声道

ad16n_flash-release_v1.4.0 >				
名称	修改日期	类型	大小	
.vscode	2025/9/10 10:29	文件夹		
apps	2025/9/10 10:29	文件夹		
obj	2025/9/10 10:33	文件夹		
tools	2025/9/10 10:29	文件夹		
AD16N_mbox_flash.cbp	2025/9/9 11:45	project file	37 KB	
AD16N_mbox_flash.depend	2025/9/10 10:34	DEPEND 文件	83 KB	
AD16N_mbox_flash.layout	2025/9/10 11:02	LAYOUT 文件	1 KB	
make_prompt.bat	2025/9/9 11:45	Windows 批处理...	1 KB	
Makefile	2025/9/9 11:45	文件	19 KB	
midi_2byte.bat	2025/9/9 11:45	Windows 批处理...	1 KB	
midi_4byte.bat	2025/9/9 11:45	Windows 批处理...	1 KB	

midi 驱动版本选择工具

在上述工具中，双击 midi_4byte.bat 就是选择编译 4byte 版本的 midi 驱动，双击 midi_2byte.bat 则选择编译 2byte 版本的 midi 驱动。

双击后 4byte 的工具成功后会有以下打印，然后用文本编辑器点开 AD16N_mbox_flash.cbp 检查 2byte 的目录是否被替换成 4byte，2byte 算法库是否有被替换成 4byte 的算法库

```

C:\Windows\system32\cmd.exe
完成!!!!!!
midi音色库4byte地址版本驱动替换完成，能支持立体声！
请按任意键继续...

```

midi 驱动版本选择 4byte 版本

```

AD16N_mbox_flash.cbp
53 <Unit filename="apps/app/bsp/common/key/key_touch.c"><Option compilerVer="CC"/></Unit>
54 <Unit filename="apps/app/bsp/common/key/key_touch.h" />
55 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/MIDIDefs.h" />
56 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/midi_dec.c"><Option compilerVer="CC"/></Unit>
57 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/midi_dec.h" />
58 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/midi_event.c"><Option compilerVer="CC"/></Unit>
59 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/midi_play.c"><Option compilerVer="CC"/></Unit>

```

midi 编译 4byte 版本驱动

```

AD16N_mbox_flash.cbp
43 <Add directory="apps/app/src/mbox_flash/remotecontrol" />
44 <Add directory="apps/include/lib/liba/ARCH/pi32v2_lto_r1/header" />
45 <Add directory="apps/include/lib/liba/ARCH/pi32v2_lto_r1/header/midi_4byte" />
46 <Add directory="apps/app/src/mbox_flash/test_dir" />

```

midi 头文件依赖选择 4byte 版本

```

AD16N_mbox_flash.cbp
71 <Add option="apps/include/lib/liba/ARCH/pi32v2_lto_r1/lib_a_decode.a" />
72 <Add option="apps/include/lib/liba/ARCH/pi32v2_lto_r1/ump3_dec.lib.a" />
73 <Add option="apps/include/lib/liba/ARCH/pi32v2_lto_r1/lib_midi_synth_pro.a" />
74 <Add option="apps/include/lib/liba/ump3/mbox_flash/encoder_midi.lib.a" />

```

链接 4byte 版本的 midi 算法库

然后打开 SDK，在 app_modules.h 选择 4byte 版本的配置（该步骤影响 midi 的初始化流程、运算 buf 大小、音频数据流对 midi 算法适配的声道数）


```

*app_modules.h x
43
44 // MIDI版本选择
45 #define MIDI_VER_2BYTE 1
46 #define MIDI_VER_4BYTE 2
47 #define MIDI_VER_SELECT MIDI_VER_4BYTE
48

```

SDK 配置 midi 使用 4byte 版本驱动

最后需要准备好适应 4byte 版本的 midi 音色库，sdk 下载目录中已有测试用的音色库 (midi_cfg_ster1)，将其加入 download_bat.c，编译下载并启动 SDK 之后，midi 初始化会打开此文件。

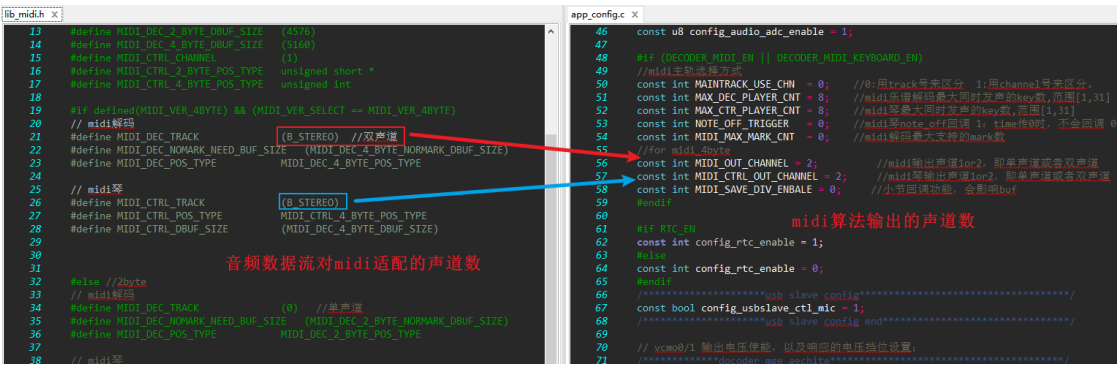
```

midi_api.c x
184 }
185
186 u32 midi_dec_confing_api(dec_obj *obj, u32 cmd, void *parm)
187 {
188     audio_decoder_ops *ops = (audio_decoder_ops *)obj->dec_ops;
189     if (NULL == obj || NULL == ops) {
190         return -1;
191     }
192     u32 ret = ops->dec_confing(obj->p_dbuf, cmd, parm);
193     return ret;
194 }
195
196 int midi_decode_cfg_init(void)
197 {
198     void *pvfs = 0;
199     void *pvfile = 0;
200     u32 err = 0;
201
202     err = vfs_mount(&pvfs, (void *)NULL, (void *)NULL);
203     if (err != 0) {
204         return E_MOUNT;
205     }
206
207     #if defined(MIDI_VER_4BYTE) && (MIDI_VER_SELECT == MIDI_VER_4BYTE)
208     /* 4byte的音色库文件后缀是mdb */
209     err = vfs_openbypath(pvfs, &pvfile, "/midi_cfg_ster1/00_MIDI.mdb");
210     #else
211     err = vfs_openbypath(pvfs, &pvfile, "/midi_cfg/00_MIDI.mda");
212     #endif
213     if (err != 0) {

```

midi 解码初始化打开音色库文件

声道配置注意：midi 算法输出数据的声道数需要和音频数据流的配置适配，否则会出现播放声音异常的问题



13.3.2 midi 相关资源文件说明

midi 文件后缀名详情：

工具版本	乐谱后缀名	音色库后缀名
旧版无压缩	.mid	.mid
旧版有压缩	.mid	.mda
新版 (2byte)	.mfa	.mda
新版 (4byte)	.mfa	.mdb

13.3.3 midi 相关功能配置说明

MIDI 解码基本功能以及对应的命令如下：

- 1. 配置 midi 使用模式 (CMD_MIDI_CTRL_MODE)
- 2. 配置 OKON 模式发声 (CMD_MIDI_GOON / CMD_MIDI_OKON_MODE)
- 3. 切换主旋律乐器 (CMD_MIDI_SET_PROG)
- 4. 配置主通道乐器或者所以通道的乐器 (CMD_MIDI_SET_CHN_PROG)
- 5. mark 回调播放功能 (CMD_MIDI_SET_MARK)
- 6. 小节回调 (CMD_MIDI_SEEK_BACK_N)
- 7. 调节 midi 外部音量功能 (CMD_MIDI_SET_EX_VOL)
- 8. 移半音功能 (CMD_MIDI_SET_SEMITONE)
- 9. 更改节奏和衰减功能 (CMD_MIDI_CTRL_TEMPO)
- 10. 配置限幅器 (CMD_MIDI_LIMITER_TRIGGER)

MIDI 琴基本功能与对应的命令如下：

- 1. 动态修改音色相关参数
- 2. 更改节奏和衰减功能 (CMD_MIDI_CTRL_TEMPO)
- 3. 调节 midi 外部音量功能 (CMD_MIDI_SET_EX_VOL)
- 4. 移半音功能 (CMD_MIDI_SET_SEMITONE)
- 5. 配置限幅器 (CMD_MIDI_LIMITER_TRIGGER)

- 6. 琴键颤音功能
- 7. 弯音轮功能
- 8. 动态修改当前播放的乐器通道
- 解码接口函数:

```
u32 midi_decode_api(void *strm, void **ppdec, void *p_dp_buf);
u32 midi_ctrl_decode_api(void *strm, void **ppdec, void *p_dp_buf)
```

13.4 a 解码格式

- a 解码格式为我司特有的音频解码格式。
- a 解码具体内容可以了解 a_api.c 和 a_api.h

格式	后缀名	AD16N otp 方案	flash 方案
a	.a .b .e	不支持	支持

- 解码接口函数:
- ```
u32 a_decode_api(void *strm, void **p_dec, void *p_dp_buf);
```

## 13.5 标准 MP3 解码格式

- 标准 MP3 解码格式
- MP3 解码具体内容可以了解 mp3\_standard\_api.c 和 a\_amp3\_standard\_apipi.h

| 格式     | 后缀名            | AD16N otp 方案 | flash 方案 |
|--------|----------------|--------------|----------|
| 标准 MP3 | .mp3 .mp2 .mp1 | 支持           | 支持       |

- 解码接口函数:
- ```
u32 mp3_st_decode_api(void *strm, void **p_dec, void *p_dp_buf);
```

13.6 WAV 解码格式

- WAV 解码格式
- WAV 解码具体内容可以了解 wav_api.c 和 wav_api.h

格式	后缀名	AD16N otp 方案	flash 方案
WAV	.wav	支持	支持

- 解码接口函数:
u32 wav_decode_api(void *strm, void **p_dec, void *p_dp_buf);

13.7 OPUS 解码格式

OPUS 解码格式为数据流解码格式。

目前的 OPUS 解码仅支持百度格式。

OPUS 解码具体内容可以了解 opus_api.c 和 opus_api.h

- 解码接口函数:
u32 opus_decode_api(void *strm, void **p_dec, void *p_dp_buf);

13.8 ADPCM-IMA 解码格式

ADPCM-IMA 解码格式为数据流解码格式。

ADPCM-IMA 解码具体内容可以了解 ima_api.c 和 ima_api.h

- 解码 run 返回值

- 1、0x0: 正常情况;
- 2、0x60: 读不到数据;

- 输入输出量控制

ima_dec_stream_max_input: 一次算法 run input 处理最大数据量 (单位: byte)

ima_dec_stream_max_output: 一次算法 run output 解码出来的最大数据量 (单位: 样点数)

两者会影响 ADPCM-IMA 里运行 buff 的大小。

```

23  u32 obuf_ima[IMA_OBUF_SIZE / 4]      AT(.ima_dec_data);
24  u32 ima_decode_buff[0x21c / 4]      AT(.ima_dec_data); //受input、output大小
25  u32 ima_decode_buff[0x21c / 4]      AT(.ima_dec_data);
26  static u32 ima_dp_buff[24 / 4]      AT(.ima_dec_data);
27
28  struct if_decoder_io ima_dec_io0     AT(.ima_dec_data);
29
30  #define IMA_CAL_BUF ((void *)&ima_decode_buff[0]) //运行buff
31
32  const int ima_dec_stream_max_input = 80; //byte
33  const int ima_dec_stream_max_output = 160; //样点数
34
35  u32 ima_decode_api(void *strm, void **p_dec, void *p_dp_buf)
36  {
37      log_info("ima_decode_api");
38      dec_data_stream *p_strm = strm;
39      u32 buff_len, i = 0;
40      decoder_ops_t *ops;
NORMAL> p uc03_toy_flash </apps/app/bsp/common/decoder/list/ima_api.c c

```

adpcm-ima 输入输出量控制

- 预测值与索引配置接口函数:

与 adpcm_ima 编码输出 header 功能对应, 用户在使用前需要先用 config 配置

CMD_SET_PRE_IDX 命令

config 配置说明: CMD_SET_PRE_IDX //配置每包预测值与步长表索引, 在每包解码前配置

使用方式 ima_test->dec_confing(ima_work_buf, CMD_SET_PRE_IDX, &prevalue_index_info)

prevalue_index_info 是 ima_dec_stream_pre_idx 变量

其中 prevalue_index_info.valpred = head[0] << 8 | head[1];

prevalue_index_info.index = head[2];

head 是 unsigned char 类型

注意: 在配置之前需要暂停解码, 配置之后才能开始解码 (保证解码当前包开始, 是使用的头中的 valpred 与 index)

- 解码接口函数:

u32 ima_decode_api(void *strm, void **p_dec, void *p_dp_buf);

13.9 SBC/MSBC/USBC 解码格式

SBC/MSBC/USBC 解码格式为数据流解码格式。

sbc / msbc / usbc 共用一个算法库, 用户需要注意的是选择 sbc 解码、msbc 解码还是 usbc 解码, 三者不可混用。

即 sbc 的解码格式只能解码 sbc 的编码数据, msbc 的解码格式只能解码 msbc 的编码数据, usbc 的解码格式只能解码 usbc 的编码数据;

sbc/msbc/usbc 解码具体内容可以了解 sbc_api.c 和 sbc_stddec_api.h

- sbc / msbc / usbc 解码选择



```

1  #include "app_modules.h"
2
3  #if defined(DECODER_SBC_EN) && (DECODER_SBC_EN)
4  #include "circular_buf.h"
5  #include "typedef.h"
6  #include "errno-base.h"
7  #include "if_decoder_ctrl.h"
8  #include "decoder_api.h"
9  #include "decoder_mge.h"
10 #include "audio_dac_api.h"
11
12 #include "sbc_api.h"
13 #include "sbc_stddec_api.h"
14
15 #define LOG_TAG_CONST      NORM
16 #define LOG_TAG           "[sbc_api]"
17 #include "log.h"
18
19 #define DEC_SBC           0
20 #define DEC_MSBC          1
21 #define DEC_USBC          2
22 #define DEC_SBC_TYPE      DEC_SBC //决定解码格式 0: SBC; 1: MSBC 2: USBC
23
24 #define SBC_OBUF_SIZE     (DAC_DECODER_BUF_SIZE * 4)
25 #define SBC_KICK_SIZE     (SBC_OBUF_SIZE - (SBC_DEC_OUTPUT_MAX_SIZE * 2))
26
27

```

sbc/msbc/usbc 解码格式选择

- 解码 run 返回值

- 1、0x0: 正常读到数据。
- 2、0x60: 算法 input 读不到数据, 即读到数据长度为 0
- 3、0x61: 错误的采样率/声道模式信息
- 4、0x62: 错误的 bitpool/subband 信息
- 5、0x63: 错误的 bitpool+subband+channel mode 组合
- 6、0x64: 目标帧长超出范围

- 解码接口函数:

```
u32 sbc_decode_api(void *strm, void **p_dec, void *p_dp_buf);
```

备注:

选择 usbc 做解码格式时, 只能解码对应参数的 usbc 编码。
所以用户需要配置 usbc 的参数和编码端的参数一致。

```

71 ops->open(SBC_CAL_BUF, &sbc_dec_io0, NULL);
72
73 AUDIO_DEC_CH_OPUT_PARA sbcset;
74 sbcset.mode = 3;
75 sbcset.pl = 8192;
76 sbcset.pR = 8192;
77 ops->dec_confing(SBC_CAL_BUF, SET_DEC_CH_MODE, &sbcset);
78
79 AUDIO_DECODE_PARA sbco_m; //设置是否完全输出方式
80 sbco_m.mode = 1;
81 ops->dec_confing(SBC_CAL_BUF, SET_DECODE_MODE, &sbco_m);
82 /*****
83
84 #if (DEC_SBC_TYPE == DEC_MSBC)
85  /* msbc */
86  ret = ops->dec_confing(SBC_CAL_BUF, SET_FORMAT_MSBC, &sbco_m); //msbc时需要配置此参数
87  if (ret != 0) {
88      log_info("init msbc err\n");
89  } else {
90      log_info("init msbc succ\n");
91  }
92 #endif
93
94 #if 1// (DEC_SBC_TYPE == DEC_USBC)
95  /* usbc */
96  /* usbc解码的时候需要配置和编码一样的参数 */
97  USBC_DECODE_PARA usbc_para;
98  usbc_para.sr = 16000;
99  usbc_para.bitpool = 31;
100  usbc_para.subbands = 8;
101  usbc_para.blocks = 16;
102  usbc_para.snr = 1;
103  usbc_para.nch = 1;
104  usbc_para.joint = 0;
105  usbc_para.dual = 1;
106
107  ret = ops->dec_confing(SBC_CAL_BUF, SET_FORMAT_USBC, &usbc_para); //usbc时需要配置此参数
108  if (ret != 0) {
109      log_info("init usbc err\n");
110  } else {
111      log_info("init usbc succ\n");
112  }
113 #endif
114
115 *p_dec = (void *)&dec_sbc_hld;
116 return 0;
117 }
118

```

usbc 参数配置

13.10 speex 解码格式

speex 解码格式为数据流解码格式，该格式主要用于人声场景。

speex 解码具体内容可以了解 `speex_api.c` 和 `speex_api.h`

- 解码接口函数:

```
u32 speex_decode_api(void *strm, void **p_dec, void *p_dp_buf);
```

13.11 jla_lw 解码格式

jla_lw 解码格式为数据流解码格式

JLA_LW 编解码共用同一个算法库，解码的采样率和码率需要与编码一致。

该格式目前主要应用在对讲机应用上，用来和蓝牙连接间隔对齐。

jla_lw 解码具体内容可以了解 `jla_lw_api.c` 和 `jla_lw_api.h` 和 `jla_lw_codec_ctrl.h`

- 丢包修复

JLA_LW 解码格式自带有丢包修复功能。

即该格式 10ms 处理一包数据，若该 10ms 的数据丢失，则会根据前文数据生成一包新的数据来弥补这 10ms 的音频。

//具体修复多少包数据可以查看 `trans_unpacket.c` 里的 `fill_empty_frames()` 函数。

//代码实现为通过计算蓝牙传输每包数据的 index 序号，根据本包数据的 index 序号与上一包数据的 index 序号相差多少就填以 0x55aa 为头的空包进入解码器（目前应用层当设置丢失超过 5 包时，只填充 5 包数据）

- 淡入淡出功能

当蓝牙信号弱时出现丢包现象，如果打开了淡入淡出功能，JLA_LW 格式除了在库里面会做丢包修复，还会使得丢包处的声音尽量平滑过度不至于出现明显的卡音。

```
22
23 /* 淡入淡出参数 */
24 const int JLA_LW_PLC_EN = 0; //0:关闭; 1:打开
25 const int JLA_LW_PLC_FADE_OUT_START_POINT = 1200; //淡入淡出功能从第几个样点开始有效
26 const int JLA_LW_PLC_FADE_OUT_POINTS = 120 * 2; //淡出样点数
27 const int JLA_LW_PLC_FADE_IN_POINTS = 120 * 2; //淡入样点数
28
```

jla_lw 淡入淡出配置

- jla_lw_br_level 压缩档位 (位于 `app_config.c`)

0 档位适用于:

16k 采样率，36kbps，每帧编码间隔为 20ms；
32k 采样率，72kbps，每帧编码格式为 10ms；

1 档位适用于:

16k 采样率，72kbps，每帧编码间隔为 10ms；

```
80
81 ///////////////////////////////////////////////////////////////////
82 #if ((defined(DECODER_JLA_LW_EN) && (DECODER_JLA_LW_EN)) || (defined(ENCODER_JLA_LW_EN) && (ENCODER_JLA_LW_EN)))
83 const int JLA_LW_BITSTREAM_WEIGHT_TAB[8] = {-2, 0, 0, 0, 0, 0, 0, 1};
84
85 /**@note jla_lw_br_level
86  @brief 码率压缩档位:0 ~ 1
87  @note1 0档位: 适用于32k采样率(10ms)、16k采样率(20ms)
88  @note2 1档位: 适用于16k采样率(10ms)
89  @note3 编解码需要采用同样的档位
90  */
91
92 const u8 jla_lw_br_level = 0;
93 #endif
94
95 /**
96  * @brief Bluetooth Controller Log
97  */
```

JLA 格式的 jla_lw_br_level 配置

备注:

在 v1.4.0 版本前, JLA 格式只支持 0 档位的配置 (该压缩档位没有开放出来)

在 v1.4.0 版本后, JLA 格式才支持 0 和 1 档位的配置;

该配置需要编解码双方一致!!!!!!

- 解码接口函数:

```
u32 jla_lw_decode_api(void *strm, void **p_dec, void *p_dp_buf);
```


编码接口

AW30N 目前支持的编码格式包含：UMP3、A、OPUS、标准 MP2、ADPCM_IMA、sbc、msbc、speex 和 JLA_LW 格式。所有编码格式被统一管理。

编码前需要配置编码采样率以及码率等相关配置，各编码格式支持的采样率以及码率如下：

编码划分为：

- 文件编码 (encoder_file.c)
- 音频传输编码 (encoder_stream.c)

14.1 编码功能控制函数

编码功能控制函数：

```
enc_obj \*encoder_io(u32(*fun)(void *, void *, void *), void \*input_func, void \*output_func, void \*pfile)
void stop_encode(void \*pfile, u32 dlen);
```

各个格式的编码接口函数：

```
u32 mp3_encode_api(void \*p_file, void \*input_func, void \*output_func);
u32 ump3_encode_api(void \*p_file, void \*input_func, void \*output_func);
u32 a_encode_api(void \*p_file, void \*input_func, void \*output_func);
u32 opus_encode_api(void \*p_file, void \*input_func, void \*output_func);
u32 ima_encode_api(void \*p_file, void \*input_func, void \*output_func);
u32 sbc_encode_api(void \*p_file, void \*input_func, void \*output_func);
u32 speex_encode_api(void \*p_file, void \*input_func, void \*output_func);
u32 jla_lw_decode_api(void \*strm, void \*p_dec, void \*p_dp_buf);
```

14.1.1 编码使用一般流程

在开始编码前需要调用函数 `u32 audio_adc_init_api(u32 sr, AUDIO_ADC_MODE mode, u32 ch)` 配置 AUDIO_ADC 采样率；

在编码之前要确保已经打开存放编码文件的设备，并创建新的文件进行编码，指定需要编码的格式，即可以按照预先设定的采样率以及码率进行编码流程：

调用 `encoder_io` 实现对编码器的控制，此函数的传参可以控制：

- a. 初始化指定编码格式；
- b. 将编码数据，写入新创建的文件中；

经过上面的一步已经完成了对编码的启动，此时可以调用 `stop_encoder` 实现停止编码的功能；

14.1.2 文件编码功能控制接口函数

14.2 函数 `void encoder_file(u32(*fun)(void *, void *, void *), void *pfile)`

此函数实现初始化编码器并启动编码，其中参数：

- 1、`(*fun)(void *, void *, void *)`：编码器初始化回调，传入指定解码器格式并进行初始化；
- 2、`void *pfile`，需要写入编码数据的文件句柄；

14.3 函数 `void stop_encode_file(enc_obj *p_enc_obj, u32 dlen)`

此函数实现停止编码器，其中参数：

- 1、`enc_obj *p_enc_obj`：当前使用的编码句柄；
- 2、`dlen`：对于 `nor_fs` 文件系统编码，该参数指定编码文件末尾舍弃的字节数；

14.3.1 音频传输编码功能控制接口函数

14.4 函数 `enc_obj *audio2rf_encoder_io(u32(*enc_fun)(void *, void *, void *), void *p_queue_ops, void *ops)`

备注：该函数实现初始化编码器，用于音频传输应用；

- 初始化编码器后，再将编码链路挂载到 AUDIO_ADC 中断；
- 调用该函数前需确保 AUDIO_ADC 已工作

1. **enc_fun**：编码器选择，可传入 `ump2` 或者 `opus` 等编码器初始化接口；

- `ump3_encode_api`：ump2 编码器初始化 api；
- `opus_encode_api`：opus 编码器初始化 api；

- sbc_encode_api: sbc/msbc/usbc 编码器初始化 api;
 - speex_encode_api: speex 编码器初始化 api;
 - ima_encode_api: ima 编码器初始化 api;
 - jla_lw_encode_api: jla_lw 编码器初始化 api;
2. p_queue_ops: 编码输出使用的队列对应句柄指针;
 3. void *ops: 蓝牙发送接口集合指针, 如果 ops 为非空, 将会注册的发送队列中断;
 4. 返回值: 编码句柄

14.5 函数 u32 audio2rf_encoder_start(enc_obj *obj)

备注: 该函数实现启动音频编码发送; 1. 返回值: 启动成功 0;

14.6 函数 u32 rf_enc_output(void *priv, u8 *data, u16 len)

备注: 该函数实现编码器输出数据并进行无线传输, 由编码器回调;

- 无线传输模块出错导致无法正常发送时停止编码, 发送繁忙时等待下次编码输出
 - 该函数在启动编码器时传入编码器 output 参数;
1. priv: 私有结构体, 此处为编码句柄;
 2. data: 输出数据 buff;
 3. len: 输出数据长度;
 4. 返回值: 本次消耗的编码数据字节长度;

14.7 函数 void audio2rf_encoder_stop(queue_obj *p_queue, ENC_STOP_WAIT wait)

备注: 该函数实现停止编码和数据发送, 并发送停止包给远端;

1. p_queue: 编码输出使用的发送队列指针;
2. wait: 是否等待编码数据消耗完成;
3. 返回值: 无

各编码格式配置说明

AD16N、AW30N 同时支持的编码格式有：标准 MP2、UMP2、A 格式三种格式；

AW30N 还支持 OPUS、ADPCM-IMA、SBC/MSBC/USBC、SPEEX 编码格式。

AD16N 和 AW30N 各 SDK 对解码格式支持情况如下表：

编码格式\芯片对应 SDK	AD16N otp 方案	AD16N flash 方案	AW30N 方案
A 文件编码	不支持	支持	支持
标准 MP2	不支持	支持	支持
UMP2	不支持	支持	支持
OPUS	不支持	不支持	支持
ADPCM-IMA	不支持	不支持	支持
SBC/MSBC/USBC	不支持	不支持	支持
SPEEX	不支持	不支持	支持
jla_lw	不支持	不支持	支持

下面介绍一下各个格式配置：

文件流解码格式：

1. “标准 MP2 编码”
2. “A 格式编码”

数据流解码格式：

3. “OPUS 格式编码”
4. “ADPCM_IMA 格式编码”
5. “sbc/msbc/usbc 编码格式”
6. “speex 编码格式”
7. “JLA_LW 编码格式”

同时支持文件流解码和数据流解码格式：

8. “ump2 编码”

15.1 标准 MP2 编码

具体内容可以了解 mp3_encoder.c 和 mp3_encoder.h
(注意文件名是 MP3, 实际的编码格式是 mp2)

标准 MP2 编码码率 (BR) / 采样率 (SR) 对应关系								
采样率 (sr)	8000	12000	16000	22050	24000	32000	44100	48000
码率 (BR)	8kbps					32kbps		
	16kbps					48kbps		
	24kbps					56kbps		
	32kbps					64kbps		
	40kbps					80kbps		
	48kbps					96kbps		
	56kbps					112kbps		
	64kbps					128kbps		
	80kbps					160kbps		
	88kbps					160kbps		
	96kbps					224kbps		
	112kbps					256kbps		
	128kbps					320kbps		
	160kbps					384kbps		

15.2 ump2 编码

具体内容可以了解 ump3_encoder.c 和 ump3_encoder.h

ump2 编码采样率 (sr) / 压缩比 (cr) 对应关系								
采样率 (SR)	8000	12000	16000	22050	24000	32000	44100	48000
压缩比 (CR)	3							
	4							
	5							
	6							
	7							
	8							
注: br 的范围时: $sr \times 16 / \text{压缩比 (CR)}$, 压缩比的范围时 3~8; 例如 SR=24K, 那么 BR 的范围时: $(24 \times 16 / 8) \sim (24 \times 16 / 3)$								

15.3 A 格式编码

具体内容可以了解 a_encoder.c 和 a_encoder.h

A 格式编码码率 (BR) /采样率 (SR) 对应关系					
采样率 (SR)	8000	12000	16000	24000	32000
码率 (BR)	32kbps	48kbps	64kbps	96kbps	128kbps

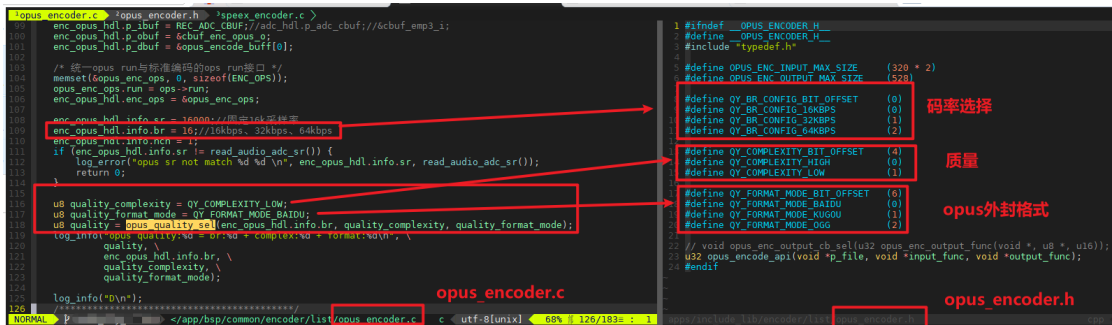
15.4 OPUS 格式编码

OPUS 格式编码目前只包含三种封装格式：分别是百度、酷狗和 OGG

具体内容可以了解 opus_encoder.c 和 opus_encoder.h

OPUS 格式编码码率 (BR) /采样率 (SR) 对应关系
采样率 (SR) 16000

码率 (BR)	16kbps	32kbps	64kbps
---------	--------	--------	--------



opus 相关参数配置

15.5 ADPCM_IMA 格式编码

具体内容可以了解 ima_encoder.c 和 ima_encoder.h

- 压缩比：4:1
- ADPCM-IMA 采样率
没有限制。
- ADPCM-IMA 输出 header 控制

ima_enc_stream_header: 编码每次输出是否包含 header 0: 不包含 1: 包含

输出的 header 占 3 个 byte。前两个 byte 为 prev pred，最后一个 byte 为 index。输出时的长度包含在编码输出的长度中

- ADPCM-IMA 输入/输出量控制

ima_enc_stream_max_input: 一次算法 run input 处理最大数据量 (单位: 样点数)

ima_enc_stream_max_output: 一次算法 run output 编码出来的最大数据量 (单位: byte)

两者影响算法库内运算 buff 大小。

```

13
14 #define LOG_TAG_CONST      NORM
15 #define LOG_TAG            "[ima_enc]"
16 #include "log.h"
17
18 enc_obj enc_ima_hdl;
19 static cbuffer_t cbuf_enc_ima_o      AT(.enc_ima_data);
20 static u32 obuf_eima_o[1024 / 4]    AT(.enc_ima_data);
21 static u32 ima_encode_buff[820 / 4] AT(.enc_ima_data);
22
23 static EN_FILE_IO ima_enc_io        AT(.enc_ima_data);
24
25 const int ima_enc_stream_max_input = 160; //样点数 320 byte
26 const int ima_enc_stream_max_output = 40 * 2; //byte
27
28 u32 ima_encode_api(void *p_file, void *input_func, void *output_func)
29 {

```

NORMAL → /app/bsp/common/encoder/list/ima_encoder.c

adpcm-ima 输入输出量控制

15.6 sbc/msbc/usbc 编码格式

sbc / msbc / usbc 编码共用同一个算法库，但是三者编解码不可混用，用户需要注意做好区分。

即 sbc 的解码格式只能解码 sbc 的编码数据，msbc 的解码格式只能解码 msbc 的编码数据，usbc 的解码格式只能解码 usbc 的编码数据；

具体内容可以了解 sbc_encoder.c 和 sbc_encoder.h

- sbc / msbc / usbc 编码选择


```

1  #include "app_modules.h"
2
3  #if 1//ENCODER_SBC_EN
4
5  #include "sbc_encoder.h"
6  #include "encoder_mge.h"
7  #include "cpu.h"
8  #include "config.h"
9  #include "typedef.h"
10 #include "hwi.h"
11 #include "circular_buf.h"
12
13 #include "sbc_stdenc_api.h"
14
15 #define LOG_TAG_CONST    NORM
16 #define LOG_TAG          "[sbc_enc]"
17 #include "log.h"
18
19 #define ENC_SBC          0
20 #define ENC_MSBC         1
21 #define ENC_USBC         2
22 #define ENC_SBC_TYPE    ENC_SBC
23
24
25 enc_obj enc_sbc_hdl;
26 static cbuffer_t cbuf_enc_sbc_o    AT(.enc_sbc_data);
27 static u32 cbuf_esbc_o[1024/4]    AT(.enc_sbc_data);

```

sbc / msbc / usbc 编码选择

- sbc/msbc/usbc 相关参数配置

用户可以在 sbc_stdenc_api.h 头文件里查看相应参数配置信息。

注意在选择 msbc 编码后, 相关参数设置无效, 具体可以看头文件说明。

```

10 #endif
11
12 //ERROR_CODE for sbc.m enc
13 #define ERR_ERROR_MODATA 0x60 //读不越数据
14 #define ERR_SET_INFO_SR_CHM 0x61 //设置码率/声道模式信息
15 #define ERR_BITPOOL_SUBBAND 0x62 //错误的bitpool/subband信息
16 #define ERR_BSC_COMBINATION 0x63 //错误的bitpool+subband+channel mode组合
17 #define ERR_GOALFLEN_OVERFLOW 0x64 //目标帧长超出范围
18 //编码opend时所需数据是否满足需求 返回0成功,
19 //run 失败返回错误码.
20
21
22
23
24 typedef struct _SBC_ENC_FILE_IO_ {
25     void *priv;
26     u16 (*input_data)(void *priv, s16 *buf, u8 channel, u16 len); //short
27     u32 (*output_data)(void *priv, u8 *buf, u16 len); //bytes
28 } SBC_ENC_FILE_IO;
29
30
31 typedef struct _SBC_ENC_PARA_ {
32     u32 sr; //sbc/sbc/usbc/sr: 16000/32000/44100/48000; msbc: 16000
33     u8 msbc; //sbc 格式为sbc; 设为1, 格式为msbc; 其他参数无效; msbc_sr=16000, nch=1, bitpool=26, subbands=8, blocks=15; 设为2, 格式为usbc
34     u8 bitpool; //4~16*subbands; bitpool数据范围; 蓝牙常用单声道36 双声道51/53 at subbands=8; subbands=4时参数应减半;
35     u8 subbands; //4/8 声道数;
36     u8 blocks; //sbc 4/8/12/16; msbc 15; usbc 4~16; 块长度;
37     u8 snr; //0/1 根据内部资源分配; allocation;
38     u8 nch; //0/1 2 channels;
39     u8 joint; //0/1 joint_stereo for nch=2;
40     u8 dual; //0/1 dualchannel for nch=2; 设1码率加倍; joint和dual不能同时为1;
41     //每个声道输入pcm点数为 subbands*blocks; GET_ENC_PCMPOINT_PERCH
42 } SBC_ENC_PARA;
43 //add msbc = 2 for usbc; blocks参数支持4~16;
44
45

```

sbc/msbc/usbc 参数配置

- 采样率和码率之间的关系

SBC / USBC 格式编码码率 (BR) /采样率 (SR) 对应关系				
采样率 (SR)	16000	32000	44100	48000
码率 (BR)	$(8 * \text{outdata_size} * \text{sr}) / (\text{subbands} * \text{blocks} * 1000);$			

MSBC 格式编码码率 (BR) /采样率 (SR) 对应关系	
采样率 (SR)	16000
码率 (BR)	62kbps

备注: 用户可以根据这两个信息算出具体压缩比为多少

```
63
64 sbc_enc_io.priv      = &enc_sbc_hdl;
65 sbc_enc_io.input_data = input_func;
66 sbc_enc_io.output_data = output_func;
67 if (ops->open((void *)&sbc_encode_buff[0], &sbc_enc_io, &sbc_para_set)) { //传入io接口
68     log_error("open fail\n");
69     return 0;
70 } else {
71     log_info("sbc open succ\n");
72 }
73
74 /* 获取输入pcm点数和压缩后的帧长需要在open后获取 */
75 enc_sbc_hdl.indata_kick_size = ops->config((void *)&sbc_encode_buff[0], GET_ENC_PCMPOINT_PERCH, 0); //获取算法输入pcm点数
76
77 /* u32 outdata_size = ops->config((void *)&sbc_encode_buff[0],GET_ENCODE_FRLLEN,0); //获取算法输出帧长 */
78 /* log_info("indata_size %d, outdata_size %d\n", enc_sbc_hdl.indata_kick_size, outdata_size); */
79 return (u32)&enc_sbc_hdl;
80 }
81 #endif
82
83
```

获取算法输入pcm点数

获取算法输出帧长

sbc/msbc/usbc 获取算法相关信息

备注:

msbc 编码时, 其余参数配置无效, 固定配置为:

```
sr=16000;
nch=1;
bitpool=26;
subbands=8;
blocks=15;
```

15.7 speex 编码格式

speex 编解码共用同一个算法库, 主要应用于人声场景。
具体内容可以了解 speex_encoder.c 和 speex_encoder.h

- **SPEEXENC_SPEEDUP_EN:** 配置 speex 编码加速
该值只对编码效果有影响, 提高提速越多, 效果会变差
- **sr:** 采样率
speex 支持的采样率为 8k / 16k。
- **quality:** 音频质量
quality: 0 ~ 9。
- **sr、br、quality 之间的关系**

br \ quality	0	1	2	3	4
采样率: 8000	2kbps	4kbps	6kbps	8kbps	8kbps
采样率: 16000	4kbps	6kbps	8kbps	10kbps	12kbps

br \ quality	5	6	7	8	9
采样率: 8000	11kbps	11kbps	15kbps	15kbps	18kbps
采样率: 16000	16kbps	20kbps	24kbps	28kbps	34kbps

15.8 JLA_LW 编码格式

JLA_LW 编解码共用同一个算法库。

该格式目前主要应用在对讲机应用上，用来和蓝牙连接间隔对齐。

具体内容可以了解 jla_lw_encoder.c 和 jla_lw_encoder.h 和 jla_lw_codec_ctrl.h

- jla_lw_br_level 压缩档位 (位于 app_config.c)

0 档位适用于：

16k 采样率，36kbps，每帧编码间隔为20ms；
32k 采样率，72kbps，每帧编码格式为10ms；

1 档位适用于：

16k 采样率，72kbps，每帧编码间隔为10ms；

```

80
81 ///////////////////////////////////////////////////////////////////
82 #if ((defined(DECODER_JLA_LW_EN) && (DECODER_JLA_LW_EN)) || (defined(ENCODER_JLA_LW_EN) && (ENCODER_JLA_LW_EN)))
83 const int JLA_LW_BITSTREAM_WEIGHT_TAB[8] = {-2, 0, 0, 0, 0, 0, 0, 1};
84
85 /**@note jla_lw_br_level
86  @brief 码率压缩档位:0 ~ 1
87  @note1 0档位: 适用于32k采样率(10ms)、16k采样率(20ms)
88  @note2 1档位: 适用于16k采样率(10ms)
89  @note3 编解码需要采用同样的档位
90 */
91 ///////////////////////////////////////////////////////////////////
92 const u8 jla_lw_br_level = 0;
93 #endif
94
95 /**
96  * @brief Bluetooth Controller Log
97  */

```

JLA 格式的 jla_lw_br_level 配置

备注：

在 AW30N_v1.4.0 版本前，JLA 格式只支持 0 档位的配置（该压缩档位没有开放出来）

在 AW30N_v1.4.0 版本后，JLA 格式才支持 0 和 1 档位的配置；

该配置需要编解码双方一致!!!!!!

- MIDI 全称是乐器数字接口 (Musical Instrument Digital Interface)，是 20 世纪 80 年代初为解决电声乐器之间的通信问题而提出的，编曲界最广泛的音乐标准格式。
- 默认 SDK 提供的 midi 功能分为 midi 解码和 midi 琴。
 - midi 解码**用户可参考 `midi_decode_app` 模式。
 - midi 琴**用户可参考 `midi_keyboard_app` 模式。
- 详细 midi 功能及说明请看 SDK 包里的《杰理 AD1x-45678_MIDI 应用说明文档.pdf》文档。

(注：该文档实际函数以 `AD16N_flash_v1.4.0` 工程为准，其余芯片 *SDK* 实际函数名会有出入，但实现原理不变)
- AD16N 从 V1.4.0 版本开始开源 2byte 和 4byte 地址版本的 midi 中间层代码。

16.1 midi 驱动版本选择说明

AD16N 从 V1.4.0 版本开始开源 2byte 和 4byte 地址版本的 midi 中间层代码。

用户可运行位于 SDK 根目录的批处理来选择编译链接 2byte 或 4byte 版本的中间层代码、头文件以及算法库。批处理会更改：

1、文件 `AD16N_mbox_flash.cbp` 的配置 2、`appsappsrcmbox_flashuc03app_modules.h` 的 `MIDI_VER_SELECT` 宏定义

注：该工具是通过修改 `cbp` 文件信息来选择编译对应的驱动的版本，不会对 `makefile` 做出任何改动，因此只能用于 `codeblocks` 等依赖 `cbp` 文件信息来编译的开发平台。

ad16n_flash-release_v1.4.0

名称	修改日期	类型	大小
.vscode	2025/9/19 14:45	文件夹	
apps	2025/9/19 14:45	文件夹	
obj	2025/9/19 14:48	文件夹	
tools	2025/9/19 14:45	文件夹	
AD16N_mbox_flash.cbp	2025/9/19 14:49	project file	38 KB
AD16N_mbox_flash.layout	2025/9/19 14:50	LAYOUT 文件	1 KB
make_prompt.bat	2025/9/19 14:45	Windows 批处理...	1 KB
Makefile	2025/9/19 14:45	文件	19 KB
midi_2byte.bat	2025/9/19 14:45	Windows 批处理...	1 KB
midi_4byte.bat	2025/9/19 14:45	Windows 批处理...	1 KB
MIDI_VER_SELECT.bat	2025/9/19 14:45	Windows 批处理...	2 KB

运行此批处理，工程使用2字节的midi驱动

运行此批处理midi使用4字节的midi驱动

midi 批处理

midi 2byte 和 4byte 版本的基本差异如下表：

midi 版本	乐 谱 后 缀 名	音 色 库 后 缀 名	算 法 库 名 (已 开 源 的 库 名)	音 色 库 支 持 的 PCM 样 点 数	音 色 库 支 持 的 声 道 数	算 法 输 出 支 持 的 声 道 数
2byte	.mfa	.mda	lib_midi_synth.a	<=65535 个 点	单声道	单声道
4byte	.mfa	.mdb	lib_midi_synth_pi	<=1900000	单声道、双声道	单声道、双声道

16.1.1 midi 驱动版本选择步骤

公版 SDK 默认使用 2byte 版本的 midi 驱动，执行 midi_4byte.bat 会替换成 4byte 版本，下面详细介绍：

step1: 选择编译 midi 版本驱动

ad16n_flash-release_v1.4.0				
名称	修改日期	类型	大小	
.vscode	2025/9/10 10:29	文件夹		
apps	2025/9/10 10:29	文件夹		
obj	2025/9/10 10:33	文件夹		
tools	2025/9/10 10:29	文件夹		
AD16N_mbox_flash.cbp	2025/9/9 11:45	project file	37 KB	
AD16N_mbox_flash.depend	2025/9/10 10:34	DEPEND 文件	83 KB	
AD16N_mbox_flash.layout	2025/9/10 11:02	LAYOUT 文件	1 KB	
make_prompt.bat	2025/9/9 11:45	Windows 批处理...	1 KB	
Makefile	2025/9/9 11:45	文件	19 KB	
midi_2byte.bat	2025/9/9 11:45	Windows 批处理...	1 KB	
midi_4byte.bat	2025/9/9 11:45	Windows 批处理...	1 KB	

midi 驱动版本选择工具

在上述工具中，双击 midi_4byte.bat 就是选择编译 4byte 版本的 midi 驱动，双击 midi_2byte.bat 则选择编译 2byte 版本的 midi 驱动。

双击后 4byte 的工具成功后会有以下打印，然后用文本编辑器点开 AD16N_mbox_flash.cbp 检查 2byte 的目录是否被替换成 4byte，2byte 算法库是否有被替换成 4byte 的算法库

```

C:\Windows\system32\cmd.exe
完成! ! ! ! !
midi 音色库 4byte 地址版本驱动替换完成，能支持立体声！
请按任意键继续。 . . .

```

midi 驱动版本选择 4byte 版本**step2: 检查 cbp 文件**

在成功运行批处理后，cbp 文件的配置会有下面的这些修改，客户也可以如下图手动修改：

```

AD16N_mbox_flash.cbp
53 <Unit filename="apps/app/bsp/common/key/key_touch.c"><Option compilerVer="CC"/></Unit>
54 <Unit filename="apps/app/bsp/common/key/key_touch.h" />
55 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/MIDIDefs.h" />
56 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/midi_dec.c"><Option compilerVer="CC"/></Unit>
57 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/midi_dec.h" />
58 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/midi_event.c"><Option compilerVer="CC"/></Unit>
59 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/midi_play.c"><Option compilerVer="CC"/></Unit>
60 <Unit filename="apps/app/bsp/common/midi_open/midi_4byte/midi_play.h" />

```

midi 编译 4byte 版本驱动

```

AD16N_mbox_flash.cbp
43 <Add directory="apps/app/src/mbox_flash/remotecontrol" />
44 <Add directory="apps/include/lib/liba/ARCH/pi32v2_lto_r1/header" />
45 <Add directory="apps/include/lib/liba/ARCH/pi32v2_lto_r1/header/midi_4byte" />
46 <Add directory="apps/app/src/mbox_flash/test_dir" />

```

midi 头文件依赖选择 4byte 版本

```
AD16N_mbox_flash.cbp
71 <Add option="apps/include_lib/liba/ARCH/pi32v2_lto_r1/lib_a_decode.a" />
72 <Add option="apps/include_lib/liba/ARCH/pi32v2_lto_r1/ump3_dec_lib.a" />
73 <Add option="apps/include_lib/liba/ARCH/pi32v2_lto_r1/lib_midi_synth_pro.a" />
74 <Add option="apps/include_lib/liba/uc03/mbox_flash/encoder_midi_lib.a" />
```

链接 4byte 版本的 midi 算法库

step3: 检查 app_modules.h 文件的宏定义

在成功运行批处理后，app_modules.h 的宏定义 “#define MIDI_VER_SELECT” 会被修改，客户也可以如下图手动修改：（该处修改会影响 midi 的初始化流程、运算 buf 大小、音频数据流对 midi 算法适配的声道数）

```
*app_modules.h x
43
44 // MIDI版本选择
45 #define MIDI_VER_2BYTE 1
46 #define MIDI_VER_4BYTE 2
47 #define MIDI_VER_SELECT MIDI_VER_4BYTE
48
```

SDK 配置 midi 使用 4byte 版本驱动

step4: 检查 SDK 音色库是否应用成功

在 v1.4.0 的 SDK 中,4 字节 midi 和 2 字节, midi 音色库是不一样的，音色库位于目录 apps/app/post_build/uc03 中，下表为音色库文件名信息：

type	name
2byte	midi_cfg
4byte	midi_cfg_ster1

在编译成功后，目录 apps/app/post_build/uc03 中会生成 download.bat 文件，此文件会下载如下如所示不同的 midi 音色库：

```
27
28
29
30
31 isd_download.exe -tonorflash -dev uc03 -boot 0x101600 -div8 -wait 300
-otp usbboot.bin -uboot uboot.boot -app app.bin -res dir_a_dir_song
dir_eng_dir_poetry_dir_story_dir_bin_fix_dir_notice_dir_midi_midi_cfg
dir_eq -output-fw output_flash.fw
32
33
34 ::-ex_flash
35 ::-format all
36 REM 常用命令说明
37 REM - format vm
38 REM - format all
39 REM - reboot 500
40
41 copy /y /b output_flash.fw jl_isd.fw
42 fw_add.exe -noenc -fw jl_isd.fw -add ota.bin -type 100 -out jl_isd.fw
43 ufw_maker.exe -fw_to_ufw jl_isd.fw
44 copy jl_isd.ufw update.ufw
45 del jl_isd.ufw
46
```

2字节midi音色库

```
26
27
28
29 isd_download.exe -tonorflash -dev uc03 -boot 0x101600 -div8
-wait 300 -otp usbboot.bin -uboot uboot.boot -app app.bin -res
dir_a_dir_song_dir_eng_dir_poetry_dir_story_dir_bin_fix
dir_notice_dir_midi_midi_cfg_ster1_dir_eq -output-fw
output_flash.fw
30
31
32
33
34 ::-ex_flash
35 ::-format all
36 REM 常用命令说明
37 REM - format vm
38 REM - format all
39 REM - reboot 500
40
41 copy /y /b output_flash.fw jl_isd.fw
42 fw_add.exe -noenc -fw jl_isd.fw -add ota.bin -type 100 -out
jl_isd.fw
43 ufw_maker.exe -fw to ufw jl_isd.fw
```

4字节音色库

download.bat文件对比

2 字节与 4 字节 midi 音色库文件下载区别

注 1: download.bat 是由 apps/app/post_build/uc03/mbox_flash/download_bat.c 在工程编译时自动生成

注 2: download_bat.c 中的宏定义 HAS_midi_4byte_USED 控制最终生成 download_bat.c 下载何种音色库

编译下载并启动 SDK 之后, midi 初始化会打开此文件。

```

midi_api.c x
184 }
185
186 u32 midi_dec_confing_api(dec_obj *obj, u32 cmd, void *parm)
187 {
188     audio_decoder_ops *ops = (audio_decoder_ops *)obj->dec_ops;
189     if (NULL == obj || NULL == ops) {
190         return -1;
191     }
192     u32 ret = ops->dec_confing(obj->p_dbuf, cmd, parm);
193     return ret;
194 }
195
196 int midi_decode_cfg_init(void)
197 {
198     void *pvfs = 0;
199     void *pvfile = 0;
200     u32 err = 0;
201
202     err = vfs_mount(&pvfs, (void *)NULL, (void *)NULL);
203     if (err != 0) {
204         return E_MOUNT;
205     }
206
207     #if defined(MIDI_VER_4BYTE) && (MIDI_VER_SELECT == MIDI_VER_4BYTE)
208     /* 4byte的音色库文件后缀是mdb */
209     err = vfs_openbypath(pvfs, &pvfile, "/midi_cfg_ster1/00_MIDI.mdb");
210     #else
211     err = vfs_openbypath(pvfs, &pvfile, "/midi_cfg/00_MIDI.mda");
212     #endif
213     if (err != 0) {

```

midi 解码初始化打开音色库文件

step5: 按照自己的需求, 选择输出的声道数

声道配置注意: midi 算法输出数据的声道数需要和音频数据流的配置适配, 否则会出现播放声音异常的问题

```

13 #define MIDI_DEC_2_BYTE_DBUF_SIZE (4096)
14 #define MIDI_DEC_4_BYTE_DBUF_SIZE (8192)
15 #define MIDI_CTRL_CHANNEL (1)
16 #define MIDI_CTRL_2_BYTE_POS_TYPE unsigned short *
17 #define MIDI_CTRL_4_BYTE_POS_TYPE unsigned int
18
19 #if 0 == (MIDI_VER_4BYTE) && (MIDI_VER_SELECT == MIDI_VER_4BYTE)
20 // midi解码
21 #define MIDI_DEC_TRACK (8 STEREO) //双声道
22 #define MIDI_DEC_NEED_BUF_SIZE (MIDI_DEC_4_BYTE_NORMARK_DBUF_SIZE)
23 #define MIDI_DEC_POS_TYPE MIDI_DEC_4_BYTE_POS_TYPE
24
25 // midi琴
26 #define MIDI_CTRL_TRACK (8 STEREO) //双声道
27 #define MIDI_CTRL_POS_TYPE MIDI_CTRL_4_BYTE_POS_TYPE
28 #define MIDI_CTRL_DBUF_SIZE (MIDI_DEC_4_BYTE_DBUF_SIZE)
29
30 // midi琴
31 // midi解码
32 #define MIDI_DEC_TRACK (8) //单声道
33 #define MIDI_DEC_NEED_BUF_SIZE (MIDI_DEC_2_BYTE_NORMARK_DBUF_SIZE)
34 #define MIDI_DEC_POS_TYPE MIDI_DEC_2_BYTE_POS_TYPE
35
36 // midi琴
37 // midi解码
38

```

音频数据流对midi适配的声道数

```

46 const u8 config_audio_adc_enable = 1;
47
48 #if (DECODER_MIDI_IN == DECODER_MIDI_KEYBOARD_IN)
49 //midi主轨选择方式
50 const int MAINTRACK_USE_CHN = 0; //0:用track号来区分, 1:用channel号来区分
51 const int MAX_DEC_PLAYER_CNT = 8; //midi乐器解码器最大同时发声的key数,范围[1,31]
52 const int MAX_CTR_PLAYER_CNT = 8; //midi琴最大同时发声的key数,范围[1,31]
53 const int NOTE_OFF_TRIGGER = 0; //midi琴note_off回调: 1:time槽0时, 不会回调, 0:会回调
54 const int MIDI_MAX_MARK_CNT = 0; //midi解码最大支持的mark数
55 //for midi_bytes
56 const int MIDI_OUT_CHANNEL = 2; //midi输出声道1or2, 即单声道或者双声道
57 const int MIDI_CTRL_OUT_CHANNEL = 2; //midi琴输出声道1or2, 即单声道或者双声道
58 const int MIDI_SAVE_DIV_ENBALE = 0; //小节拍回调函数, 会影响buf
59 #endif
60
61 #if RTT_EN
62 const int config_rtc_enable = 1;
63 #else
64 const int config_rtc_enable = 0;
65 #endif
66
67 //*****usb slave config*****
68 const bool config_usblave_ctl_mic = 1;
69 //*****usb slave config end*****
70
71 // vcm0/1 输出电压使能, 以及相应的电压档位设置。
72 //*****vcm0/1 vcm0/1 vcm0/1*****

```

midi算法输出的声道数

midi 声道数配置

16.2 MIDI 基本功能配置

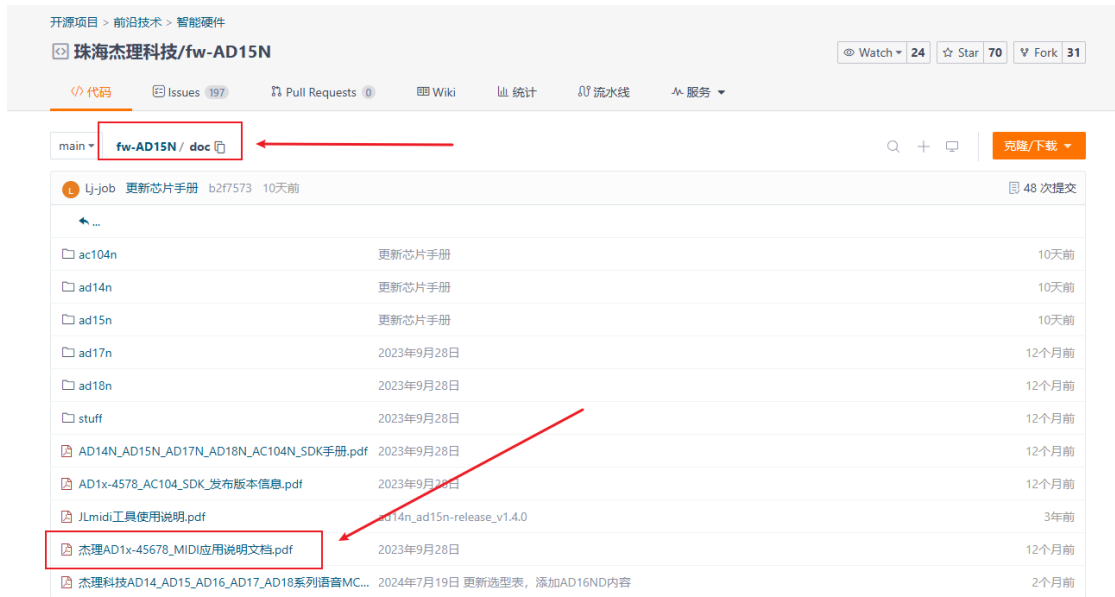
MIDI 解码基本功能以及对应的命令如下：

1. 配置 midi 使用模式 (CMD_MIDI_CTRL_MODE)
2. 配置 OKON 模式发声 (CMD_MIDI_GOON / CMD_MIDI_OKON_MODE)
3. 切换主旋律乐器 (CMD_MIDI_SET_PROG)
4. 配置主通道乐器或者所以通道的乐器 (CMD_MIDI_SET_CHN_PROG)
5. mark 回调播放功能 (CMD_MIDI_SET_MARK)
6. 小节回调 (CMD_MIDI_SEEK_BACK_N)
7. 调节 midi 外部音量功能 (CMD_MIDI_SET_EX_VOL)
8. 移半音功能 (CMD_MIDI_SET_SEMITONE)
9. 更改节奏和衰减功能 (CMD_MIDI_CTRL_TEMPO)
10. 配置限幅器 (CMD_MIDI_LIMITER_TRIGGER)

MIDI 琴基本功能与对应的命令如下：

1. 动态修改音色相关参数
2. 更改节奏和衰减功能 (CMD_MIDI_CTRL_TEMPO)
3. 调节 midi 外部音量功能 (CMD_MIDI_SET_EX_VOL)
4. 移半音功能 (CMD_MIDI_SET_SEMITONE)
5. 配置限幅器 (CMD_MIDI_LIMITER_TRIGGER)
6. 琴键颤音功能
7. 弯音轮功能
8. 动态修改当前播放的乐器通道

详细更多说明，请看 SDK 包里 doc 文件夹里的《杰理 AD1x-45678_MIDI 应用说明文档.pdf》



midi 文档位置

JL_MIO 用于在音频文件中添加一些波形信息，我司芯片环境中可以使用这些波形信息进行相应的控制。mio 文件制作详情见 JLMio 使用说明文档，本章介绍 mio 在 SDK 中的应用。

17.1 JLMio 一般流程

17.1.1 使用 JLMio.exe 创建 mio 文件

详情见 JLMio 使用说明。生成的 mio 文件会在工具目录的 Data 文件夹下；

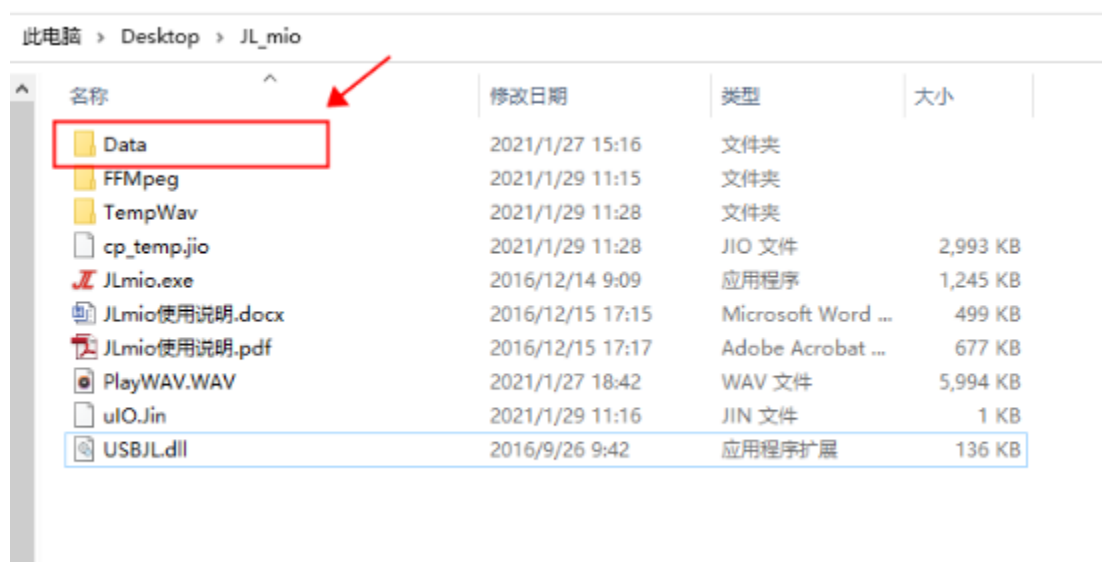


图 1 mio 文件生成位置

17.1.2 使用打包工具将音频文件与所制成的 mio 文件放于同一目录下

打包工具请参考 2.3 打包工具使用，注意：音频文件需要与生成的 mio 文件放于同一目录下；生成目录文件时，不能将 mio 放在目录的第一个文件位置处，否则音乐播放可能会失败！

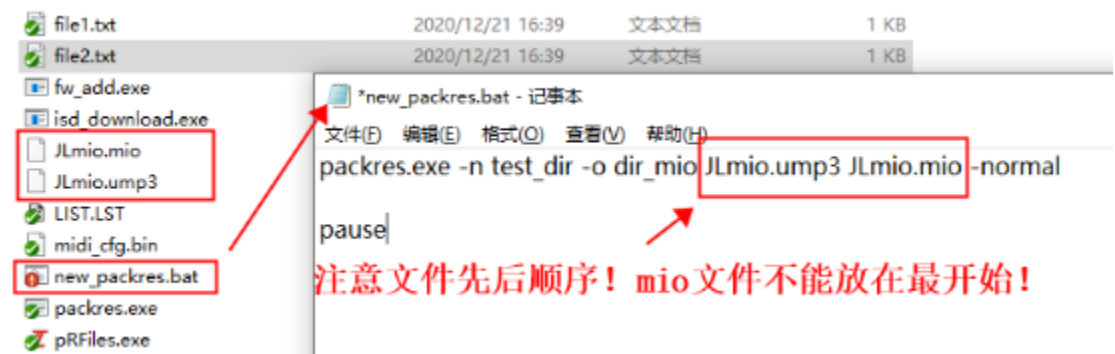


图 2 mio 文件打包位置顺序注意事项

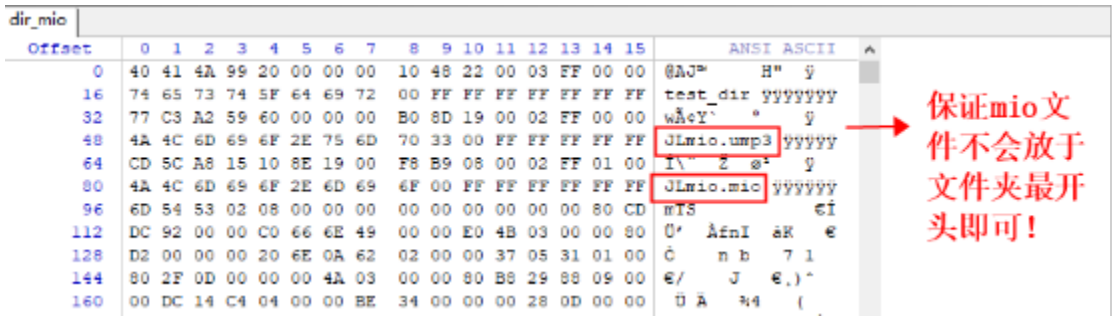


图 3 mio 文件正确打包位置顺序示意图

17.1.3 在 download.bat 中添加相应 mio 文件夹

在 download.bat 中添加所打包的包含音频文件与 mio 文件的文件夹目录 dir_mio，可放在 dir_a 文件夹后的任意位置；



图 4 将 dir_mio 文件夹添加进下载目录中

17.1.4 在 app_music.c 中添加相应 mio 文件夹

在 app_music 中添加所打包的包含音频文件与 mio 文件的文件夹目录，可放在任意位置并用按键进行文件夹切换；

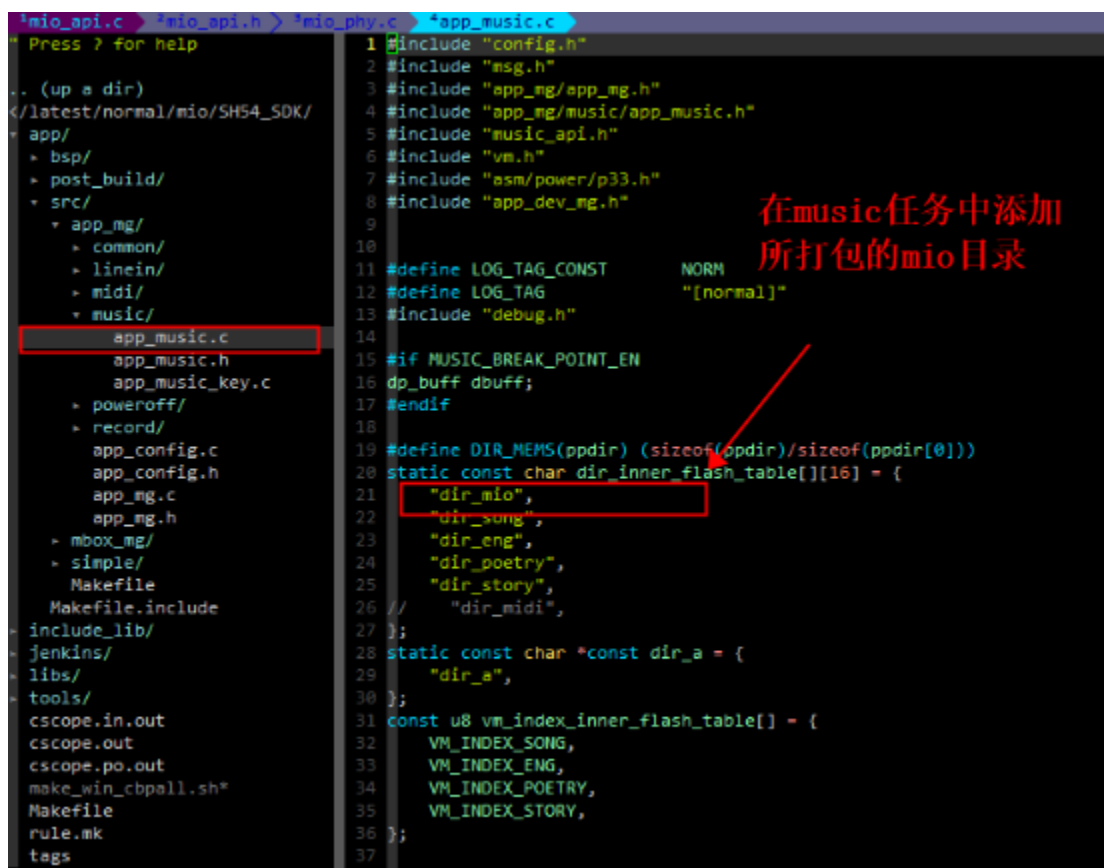


图 5 在 app_music 中添加 mio 文件夹

17.1.5 在 mio_api.c 中将相应宏开关打开

在 mio_api 中将 MIO_EN 宏定义打开即可，SDK 默认使用 1 路 PWM (PA15) 以及 2 路 IO 口 (PA1 与 PA2) 作为示例，用户可自行添加并完善功能；注意：mio 功能会占用相应引脚，若不用 mio 功能需关闭宏开关。

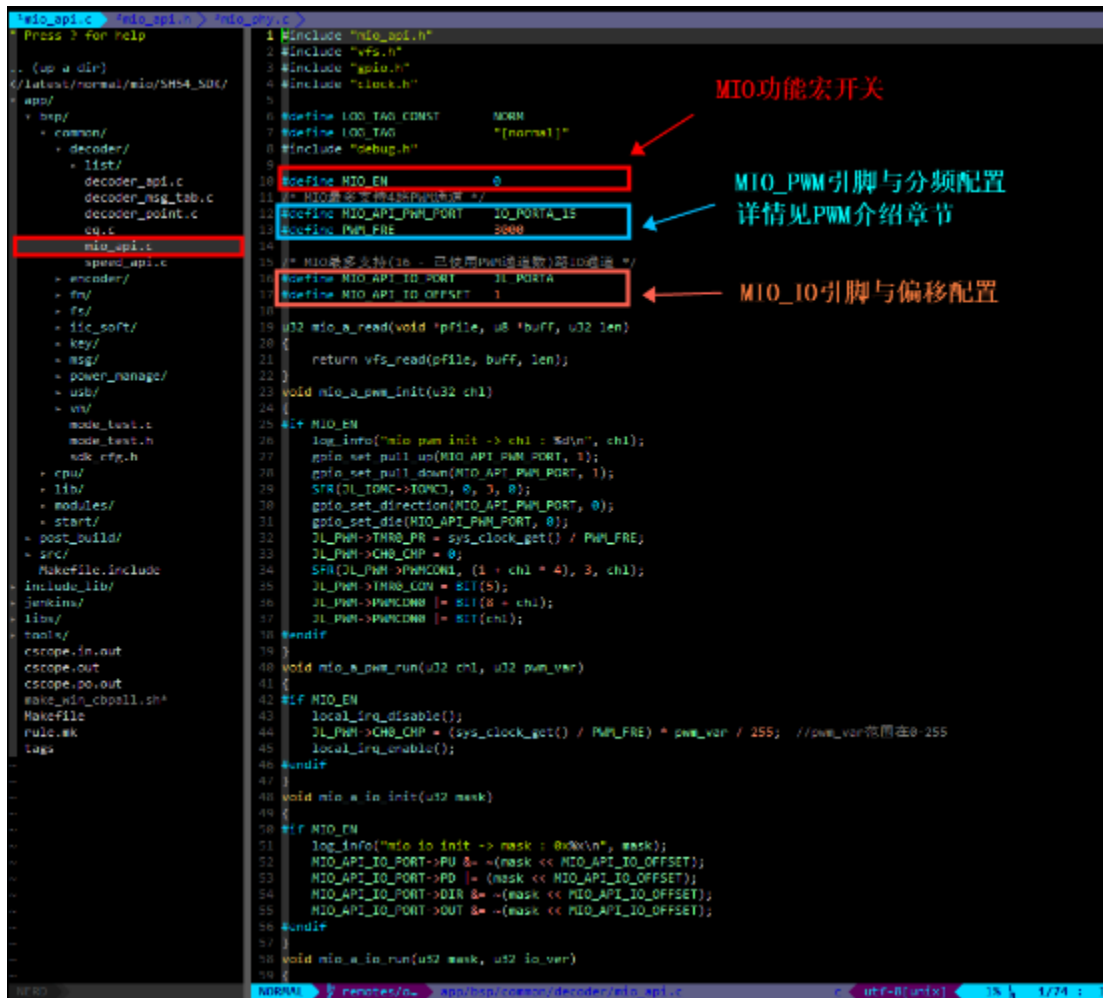


图 6 mio_pwm 与 mio_io 宏定义配置

17.2 JLMio 接口介绍

17.2.1 void mio_a_pwm_init(u32 chl)

此函数实现 mio_pwm 通道输出的初始化，其中参数：

1、chl：需要进行初始化的pwm通道编号（mio最多支持4路pwm通道）；

17.2.2 void mio_a_pwm_run(u32 chl, u32 pwm_var)

此函数实现 mio_pwm 通道输出，其中参数：

- 1、chl：正在输出的pwm通道编号；
- 2、pwm_var：mio文件中读取到的pwm数值，可用于配置pwm输出占空比；

17.2.3 void mio_a_io_init(u32 mask)

此函数实现 mio_io 通道输出的初始化，其中参数：

- 1、mask：需要进行初始化的io通道（mio最多支持（16-pwm通道）路io通道）；
 - ① mio文件中设置的每一个io通道依次对应mask中每个位；
 - ② 如单io通道mask会传入0x01，双io通道mask会传入0x3，以此类推；

17.2.4 void mio_a_io_run(u32 mask, u32 io_ver)

此函数实现 mio_io 通道输出，其中参数：

- 1、mask：需要进行初始化的io通道；
- 2、io_ver：mio文件中读取到的io数值，可用于配置IO口的电平；

OTP 小音箱应用流程

AD16N 的 OTP 应用模式有 Music 模式、USB 从机模式以及 LINEIN 模式。不同的模式间会有资源和按键互斥的关系。应用上面做了一个简单的模式切换管理来管理不同模式的资源和按键。

模式管理的整体管理思路是，进入该模式的时候初始化模式需要使用的资源和按键处理函数，退出该模式的时候关闭该模式需要使用的资源和释放按键处理函数。上电默认进入 LINEIN 模式，在检测到有 USB 线连接小机到电脑时，会自动切换到 USB 从机模式；在检测到有 SD 卡、U 盘等设备插入时，会自动切换到 Music 模式

模式轮询函数:

```
void mbox_main(void)
```

18.1 音乐解码模式

音乐解码模式涉及设备操作、文件读取、音频解码以及音频断点记忆等功能，可实现 SD 卡/U 盘的 MP3/WAV 解码、快进快退、EQ、文件夹循环/单曲循环/随机播放等功能。涉及的文件目录：appsappsrcmboxmusic，涉及的应用函数如下：

18.1.1 函数 `bool music_play_control(MBOX_MUSIC_CMD cmd, u32 index, DEC_STOP_WAIT mode)`

此函数是应用层控制设备或文件进行相应操作的接口，根据 cmd 判断进行设备操作或文件操作，内部完成设备、文件和解码的流转，其中参数：

- 1、cmd：音乐设备或文件操作cmd，如下表；
- 2、index：播放指定设备或指定文件时，传入的设备号或文件号；
- 3、mode：解码停止方式；
 - ① NO_WAIT：直接停止解码；
 - ② NEED_WAIT：停止解码时，等待DAC消耗完剩余样点；

(续下页)

(接上页)

4、返回值： 成功 返回1；
 失败 返回0；

播放控制命令	含义
DEV_CMD_SEL_NEW_DEV	播放指定设备
DEV_CMD_PREV	播放上一设备
DEV_CMD_NEXT	播放下一设备
FILE_CMD_PLAY_BY_INDEX	播放指定文件
FILE_CMD_PREV	播放上一曲
FILE_CMD_NEXT	播放下一曲

表 1 音乐设备或文件操作 cmd

18.1.2 函数 bool mbox_simple_play_file(u8 dindex, u32 findex)

此函数是应用层播放指定设备和指定文件的接口，其中参数：

- 1、dindex： 需要播放的设备号：

① SD0_INDEX：SD卡设备号；
② UDISK_INDEX：U盘设备号；
- 2、findex： 需要播放的文件序号；
- 3、返回值：

成功： 返回1；
失败： 返回0；

18.1.3 函数 u32 device_mount(play_control *ppctl)

此函数是应用层挂载设备和文件系统，并进行扫盘操作的接口。该函数会去读取访问物理设备，设备和文件系统挂载成功后，会将扫盘结果保存在结构体 dev_scan_info 中，失败时会返回错误值并寻找下一有效设备，其中参数：

- 1、ppctl： 音乐播放器句柄指针；
- 2、返回值：

成功： 返回0；
失败： 错误值可查看errno-base.h；

18.1.4 函数 void fs_dev_close(void)

此函数是应用层关闭文件、文件系统以及设备的接口。

18.2 2 USB 从机模式

USB 从机模式可支持 USB Speaker 音频、USB Massstorage 大容量存储器、USB HID 控制等功能，涉及的文件目录：appsappsrcmbboxusb_slave。

该模式下的音量与系统音量保持独立，可在 apps/app/bsp/common/usb/device/descriptor.c 文件中添加、修改设备描述符。

18.3 LINEIN 模式

LINEIN 模式可支持模拟直通模式以及数字 LINEIN 模式；

开发板接线时，模拟直通需将 AUXL、AUXR 分别接到 PC4、PC5 引脚；数字 LINEIN 模式需将 AUXL、AUXR 接到 PA1 引脚；

数字 LINEIN 模式下 AUDIO_ADC 和 AUDIO_DAC 需要配置相同的采样率，数字模式下可调节音量，涉及的文件目录：appsappsrcmbboxlinein。

模拟直通与数字 LINEIN 切换如下：

- 1) 修改宏定义 LINEIN_MODE 为 ANALOG_LINEIN_MODE，即模拟直通 LINEIN；
- 2) 修改宏定义 LINEIN_MODE 为 DIGITAL_LINEIN_MODE，即数字 LINEIN；

FLASH 工程应用流程

AD16N 的 FLASH 工程拥有更大的存储空间，在 OTP 工程的 Music 模式、USB 从机模式以及 LINEIN 模式的基础上，新增了玩具多路解码模式、录音模式、MIDI 解码模式、MIDI 琴模式等应用。

19.1 音乐解码模式

音乐解码模式在 OTP 小音箱 Music 模式的基础上，增加了内置 flash 设备播放。涉及的文件目录：appsappsrcmbx_flashmusic，涉及应用函数请参考《OTP 小音箱应用流程》->《音乐解码模式》。

19.2 USB 从机模式

同《OTP 小音箱应用流程》中的 USB 从机模式。

19.3 LINEIN 模式

同《OTP 小音箱应用流程》中的 LINEIN 模式。

19.4 MIDI 解码 / MIDI 琴模式

MIDI 具体内容请参考《杰理 AD14N_AD15N_MIDI 应用说明文档.pdf》。

19.5 玩具多路解码模式

多路解码模式使用了 SYD 文件系统，能够播放打包成 dir 文件的 F1A/B/C/X、UMP3，并叠加 A 文件解码，同时支持外挂 FLASH 解码。该模式还具备 VM 记忆，可记录解码文件序号与断点。多路解码使用 play_control 结构体，其成员如下：

```
typedef struct __play_control {
    void *pfs;           //文件系统句柄
    void *pfile;         //文件句柄
    dec_obj *p_dec_obj;  //解码器句柄
    void *pdp;           //断点指针
    u32  findex;         //当前播放的文件序号
    u32  ftotal;         //当前文件夹中的文件总数
    u16  dec_type;       //解码器支持的解码类型，如BIT_F1A1
    u8   dev_index;      //当前挂载设备的序号，如INNER_FLASH_R0
    u8   play_mode;      //播放模式(Music模式使用)
#ifdef HAS_SYDFS_EN
    void *device;        //当前挂载设备的句柄
    void *pdir;          //文件夹指针,用于切换文件夹
    u8   *p_vm_tab;      //存放vm项指针,用于记忆不同文件夹的掉电数据
    u8   dir_total;      //文件夹总数
    u8   dir_index;      //当前播放的文件夹序号
    u8   loop;           //播放循环次数
#endif
    // u32  sclust;
    u8   flag;           //标志位(Music模式使用)
} play_control;
```

图 5.1 play_control 结构体成员

涉及的文件目录：appsappsrcmbx_flashsimple_decode，涉及的应用函数如下：

19.5.1 函数 u32 simple_dev_fs_mount(play_control *ppctl)

此函数是多路解码挂载设备和 SYD 文件系统，其中参数：

- 1、ppctl：音乐播放器句柄指针；
- 2、返回值：
 - 成功 返回0；
 - 失败 错误值可查看errno-base.h；

19.5.2 函数 u32 simple_dev_fs_close(play_control *ppctl)

此函数实现多路解码释放文件、文件系统句柄，并关闭设备；释放的句柄分别是结构体 play_control 中的成员 pfile，pfs 以及 device，其中参数：

- 1、ppctl：音乐播放器句柄指针；
- 2、返回值：0；

19.5.3 函数 u32 simple_play_file_byindex(play_control *ppctl)

此函数实现多路解码按文件序号播放指定文件，播放的文件序号使用结构体 play_control 中的成员 findex，其中参数：

- 1、ppctl：音乐播放器句柄指针；
- 2、返回值：0；

19.5.4 5.4 函数 u32 get_decoder_status(dec_obj *obj)

此函数实现获取多路解码当前的状态，分别有播放停止（MUSIC_STOP）、播放暂停（MUSIC_PAUSE）以及播放中（MUSIC_PLAY）三种状态，其中参数：

- 1、obj：解码句柄；
- 2、返回值：返回当前解码器播放状态；

19.5.5 函数 static bool simple_switch_device(play_control *ppctl)

此函数实现多路解码切换下一个设备（内置 flash<-> 外挂 flash），函数中需要停止解码器，关闭上次播放的文件与设备，重新初始化下一个需要播放的设备并开始解码，其中参数：

- 1、ppctl：音乐播放器句柄指针；
- 2、返回值：
 - 成功 true；
 - 失败 false；

19.5.6 函数 static bool simple_next_dir(play_control *ppctl)

此函数实现多路解码切换下一个 dir 文件夹，函数中需要停止解码器，关闭上次播放的文件与设备，重新初始化下一个需要播放的设备并开始解码，其中参数：

- 1、ppctl：音乐播放器句柄指针；
- 2、返回值：
 - 成功 true；
 - 失败 false；

19.6 录音模式

录音模式支持内置 flash、SD 卡/U 盘录音与播放，同时支持变速变调音效。

内置 flash 录音时挂载 NORFS 文件系统，支持录制 UMP3、A 格式音频；SD 卡/U 盘录音时挂载 FAT 文件系统，支持录制标准 MP2 格式音频；各编码格式支持的采样率以及码率范围请参考《第六章编码接口》介绍。

编码使用 Encode_Control 结构体，其成员如下：

```
typedef struct _Encode_Control {
    char fs_name[8];           //编码文件系统名
    void *device;              //编码设备句柄
    void *pfs;                 //编码文件系统句柄
    void *pfile;               //编码文件句柄
    union {
        !   u32 rec_file_index; //nor_fs最新文件号
        !   char *rec_file_path; //fat_fs最新文件路径
    };
    u8 dev_index;              //编码设备序号
    ENC_STATUS enc_status;     //编码器工作状态
} Encode_Control;
```

图 5.1 编码结构体 Encode_Control 说明

涉及的文件目录：appsappsrcmbx_flashrecord，涉及的应用函数如下面几个小节；

19.6.1 函数 int encode_start(Encode_Control *obj)

此函数实现配置 AUDIO_ADC 采样率，并识别当前在线设备进行编码，SD 卡/U 盘挂载 fat 文件系统录制标准 mp2 音频，内置 flash 挂载 norfs 文件系统录制 ump3 或 a 格式音频，其中参数：

- 1、obj：编码器句柄指针；
- 2、返回值：
 - 成功 返回0；
 - 失败 错误值可查看errno-base.h；

19.6.2 函数 void encode_stop(Encode_Control *obj)

此函数实现停止编码器工作，并保存录制完成的文件，其中参数：

- 1、obj：编码器句柄指针；
- 2、返回值：无；

19.6.3 函数 `int fatfs_enc_file_create(Encode_Control *obj)`

此函数实现打开 SD 卡/U 盘设备，并创建固定命名格式的文件，创建的文件名遵循“AD16****.mp3”格式，可录制文件“AD160000.mp3”至“AD169999.mp3”，文件系统能识别最大后缀序号，并自增作为新建文件名，其中参数：

- 1、obj：编码器句柄指针；
- 2、返回值：
 - 成功 返回0；
 - 失败 错误值可查看errno-base.h；

19.6.4 函数 `dec_obj *fatfs_enc_file_decode(Encode_Control *obj, u16 dec_type)`

此函数实现播放 SD 卡/U 盘设备录制的标准 mp2 编码文件，其中参数：

- 1、obj：编码器句柄指针；
- 2、dec_type：解码格式、音效选择；
- 3、返回值：
 - 成功 返回解码器句柄；
 - 失败 返回NULL；

19.6.5 函数 `int norfs_enc_file_create(Encode_Control *obj)`

此函数实现内置 flash 挂载 norfs 文件系统录制 ump3/a 格式音频，norfs 为循环管理的可读写文件系统，写满后会自动覆盖最开始的数据，该文件系统只能用文件序号打开文件，创建新文件时，会返回最新文件序号，其中参数：

- 1、obj：编码器句柄指针；
- 2、返回值：
 - 成功 返回0；
 - 失败 错误值可查看errno-base.h；

19.6.6 函数 `dec_obj *norfs_enc_file_decode(Encode_Control *obj, u16 dec_type)`

此函数实现播放内置 flash 录制的 ump3/a 编码文件，其中参数：

- 1、bj：编码器句柄指针；
- 2、dec_type：解码格式、音效选择；
- 3、返回值：
 - 成功 返回解码器句柄；
 - 失败 返回NULL；

19.7 LoudSpeaker 扩音模式

扩音模式为 mic 采集音频数据，串联一个或多个音效算法并将处理后的数据输出至 DAC 出声的功能，常用于扩音喇叭等应用。

目前可串联的音效有：

- 1) speed 实时变调音效；
- 2) echo 混响音效；
- 3) voice_pich 变音音效；
- 4) notcch_howling 陷波啸叫抑制音效；
- 5) pitchshift_howling 移频啸叫抑制音效；
- 6) EQ 音效；

以上音效可在 speaker_api.c 文件中通过 link 函数形式串联到数据流中；

其中，(1) ~ (3) 音效资源互斥，同时只可开启其中一个。

在 audio DAC 出数过程中，如遇到打印 ‘*’ 号，可能是算法运算时间较长，数据处理不过来，可尝试提高系统时钟解决。

各音效算法的介绍请查看《音效》章节说明。

19.7.1 函数 void audio_adc_speaker_start(u32 sr)

此函数实现启动 mic 直通喇叭功能，其中参数：

1、sr：speaker工作采样率，可选采样率如下表所示；

序号	AUDIO ADC 可选采样率
1	8000
2	11025
3	12000
4	16000
5	22050
6	24000
7	32000
8	44100
9	48000

表 1 AUDIO ADC 可选采样率

19.7.2 函数 void audio_adc_speaker_reless(void)

此函数实现关闭 mic 直通喇叭功能；

AD16N AUDIO 包含 DAC 和 ADC；
其中 DAC 支持立体声和单声道，ADC 支持单声道。

20.1 audio 采样率列表

AD16N AUDIO 采样率支持情况		
采样率	DAC	ADC
8000	yes	yes
11025	yes	yes
12000	yes	yes
16000	yes	yes
22050	yes	yes
24000	yes	yes
32000	yes	yes
44100	yes	yes
48000	yes	yes
64000	yes	null
88200	yes	null
96000	yes	null

表 1 audio 采样率支持情况

20.2 audio 工作模式列表

本小节主要介绍 AUDIO 几种模式的工况。

20.2.1 AUDIO DAC 工作模式

AD16N AUDIO DAC 工作模式				
DAC	立体声	差分	单声道单端 DACL	单声道单端 DACR

表 2 AD16N AUDIO DAC 工作模式

20.2.2 AUDIO ADC - MIC 工作模式

AD16N AUDIO ADC - MIC 工作模式			
ADC MIC	内置偏置电阻	外置偏置电阻	差分

表 3 AD16N AUDIO ADC - MIC 工作模式

20.2.3 AUDIO ADC - LINEIN 工作模式

AD16N AUDIO ADC - LINEIN 工作模式		
LINEIN	模拟直通 DAC	ADC 转数字注册到 DAC 的注册表

表 4 AD16N AUDIO ADC - LINEIN 工作模式

20.3 《app_config.c》中的开机默认配置

如图 1 中 Audio 公共部分和 AUDIO DAC 包含以下配置：

1. vcm 外挂电容；
 2. vcmo0/1 电压；
 3. dac 模拟增益；
 4. 左右声道输出；


```

21 /*****audio dac analog config*****/
22 const bool config_vcm_cap_addon = 0; // 0 :vcm不会外挂电容; 1: vcm会外挂电容;
23
24 /*****update*****/
25 #if TFG SDPG_ENABLE
26 const u8 dev_update_power_io = IO_PORTC_03;
27 #else
28 const u8 dev_update_power_io = -1;
29 #endif
30
31 //升级使用区域, 0:VM区, 1:eeeprom区
32 const u8 dev_update_use_eeprom = 0;
33 //升级时是否保持IO状态
34 const u8 dev_update_keep_io_status = 0;
35 //ufw文件的vid要求: 0:vid相同 1:vid不同
36 const u8 ufw_vid_need_to_be_different = 0;
37
38 /*****audio dac analog config*****/
39 * 以下配置, 作用为系统音频 DAC 模块开机默认配置
40 */
41 // vcm0/1 输出电压使能, 以及响应的电压挡位设置;
42 bool const audio_dac_vcm0_enable = 0;
43 bool const audio_dac_vcm1_enable = 0;
44 AUDIO_VCM0_VOLTAGE const audio_dac_vcm0_voltage = AUDAC_VCM0_1v2;
45 AUDIO_VCM0_VOLTAGE const audio_dac_vcm1_voltage = AUDAC_VCM0_1v2;
46
47 //模拟增益挡位
48 AUDIO_DAC_ANA_VOL const audio_dac_analog_vol_l = AUDAC_1db87;
49 AUDIO_DAC_ANA_VOL const audio_dac_analog_vol_r = AUDAC_1db87;
50
51 // 左右声道音频输出声能
52 bool const audio_dac_lpf_lenable = 1;
53 bool const audio_dac_lpf_renable = 1;
54
55 // 左右声道静音使能
56 bool const audio_dac_lpf_lmute = 0;
57 bool const audio_dac_lpf_rmute = 0;
58
59 #if (DAC_TRACK_NUMBER == 1)
60 const bool config_dac_points_combine = 1; // 单声道融合左右声道数据 1:融合 0:不融合
61 /* 融合公式:L*Lgain/Totalgain + R*Rgain/Totalgain */
62 const u8 config_dac_points_Lgain = 128;
63 const u8 config_dac_points_Rgain = 128;
64 const u8 config_dac_points_Totalgain = 128;
65 #else
66 const bool config_dac_points_combine = 0;
67 #endif
68
69 /*****audio adc analog config*****/
70 * 以下配置, 作用为系统音频 ADC 模块开机默认配置
71 apps/app/src/mbox_flash/uc03/cpu_config.c

```

AUDIO公共部分

AUDIO_DAC部分

图 1 audio config

20.4 Audio 公共配置

const bool config_vcm_cap_addon: vcm 外挂电容配置;

软件默认不外挂电容;

如果软件选择了外挂电容, 硬件却没有电容连接, 会导致音频性能大幅降低;

VCM 正确挂载了电容的情况下, 软件悬着外挂电容会比不外挂电容音频性能更好;

AUDIO DAC

AD16N AUDIO_DAC 支持立体声和单声道，此 DAC 支持 16bit 精度。

各芯片音频性能对比情况如下：

芯片 DAC 性能对比	AC104N	AD16N
Output Swing(空载)	430mVrms	760mVrms
SNR	83dB	97dB
DR	82dB	96dB
THD	-70dB	-82dB
Noise Floor	30uVrms	11uVrms

表 1 AUDIO_DAC 性能对比

AD16N AUDIO_DAC 采样率：

AD16N AUDIO 采样率	采样率支持情况 DAC
8000	yes
11025	yes
12000	yes
16000	yes
22050	yes
24000	yes
32000	yes
44100	yes
48000	yes
64000	yes
88200	yes
96000	yes

表 2 AUDIO_DAC 采样率支持情况

AUDIO DAC 工作模式：

AD16N AUDIO 工作模式				
DAC	立体声	差分	单声道单端 DACL	单声道单端 DACR

表 3 AUDIO_DAC 工作模式

21.1 《app_config.c》中的开机默认配置

Audio 公共部分和 AUDIO DAC 包含以下配置：

1. vcm 外挂电容；
 2. vcmo0/1 电压；
 3. dac 模拟增益；
 4. 左右声道输出；

21.1.1 Audio 公共部分

const bool config_vcm_cap_addon：vcm 外挂电容配置；
软件默认不外挂电容；
如果软件选择了外挂电容，硬件却没有电容连接，会导致音频性能大幅降低；
VCM 正确挂载了电容的情况下，软件悬着外挂电容会比不外挂电容音频性能更好；

21.1.2 AUDIO DAC 部分配置

VCMO0/1 输出电压配置，正确配置后 VCMO0/1 能输出一个和 DAC 工作电压一样的电压；
这样在耳机方案应用中，使用 VCOM0/1 作为公共端，可以省掉 DAC 输出所需的隔直电容；

- bool const audio_dac_vcmo0_enable：vcmo0 是否输出电压；
- bool const audio_dac_vcmo1_enable：vcmo1 是否输出电压；
- AUDIO_VCMO_VOLTAGE const audio_dac_vcmo0_voltage：vcmo0 输出电压挡位，默认为 1.2v；
- AUDIO_VCMO_VOLTAGE const audio_dac_vcmo1_voltage：vcmo1 输出电压挡位，默认为 1.2v；
- AUDIO_DAC_ANA_VOL const audio_dac_analog_vol_l：DAC 模拟增益，可根据枚举变量选择；
- AUDIO_DAC_ANA_VOL const audio_dac_analog_vol_r：DAC 模拟增益，可根据枚举变量选择；
- bool const audio_dac_lpf_lenable：左声道音频输出使能；
- bool const audio_dac_lpf_renable：右声道音频输出使能；
- bool const audio_dac_lpf_lmute：左声道静音使能，使能后开机初始化会默认 mute 住 DACL；
- bool const audio_dac_lpf_rmute：右声道静音使能，使能后开机初始化会默认 mute 住 DACR；

注: VCOM0 / 1 也可作为 AUDIO_ADC 的内置偏置 I/O, DAC 配置 VCOM0 / 1 时需要与 ADC 互斥!

AUDIO_ADC 内置偏置使用 VCOM0 配置: `audio_adc_mic_bias_2_vcmo0`

AUDIO_ADC 内置偏置使用 VCOM1 配置: `audio_adc_mic_bias_2_vcmo1`

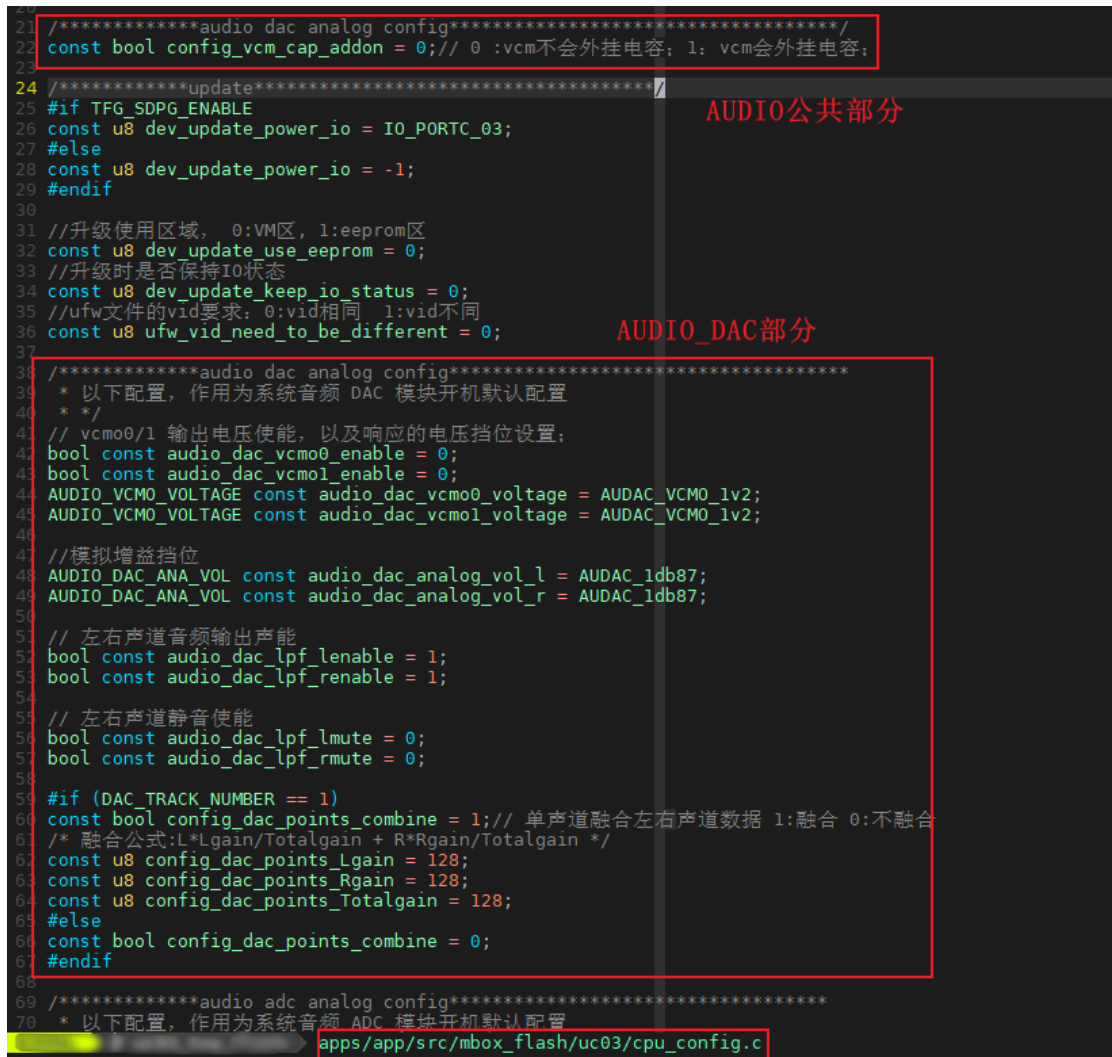


图 1.1 audio config

21.2 《audio_dac.h》中的开机默认配置

在 `audio_dac.h` 的宏定义里面主要有以下:

- 1、四种工作模式, 一种立体声, 三种单声道;
- 2、AUDIO DAC 样点的位宽;

21.2.1 工作模式

模式	详解
A_DAC_MONO_L	此模式 DAC 会将左右声道的数据合并输出到左声道，软件中的数据流依旧是双声道的；
A_DAC_MONO_R	此模式 DAC 会将左右声道的数据合并输出到右声道，软件中的数据流依旧是双声道的；
A_DAC_STERO	标准立体声模式；
A_DAC_DIEF	以差分方式输出信号；

工作模式可根据图 2.1 所示，在《audio_dac.h》文件中的 **AUDAC_OUTPUT_MODE** 来选择：

21.2.2 位宽

目前芯片支持 16 位位宽和 24 位位宽，但是 SDK 只支持了 16 位位宽；

```
50
51 //audio dac work mode
52 #define A_DAC_MONO_L      A_DAC_CH0_EN
53 #define A_DAC_MONO_R      A_DAC_CH1_EN
54 #define A_DAC_STERO        (A_DAC_CH1_EN | A_DAC_CH0_EN)
55 #define A_DAC_DIEF         (A_DIF_EN | A_DAC_CH0_EN)
56
57 // audio dac 声道数
58 #define AUDAC_OUTPUT_MODE A_DAC_STERO
59 #if (A_DAC_STERO != AUDAC_OUTPUT_MODE)
60 #define DAC_TRACK_NUMBER 1
61 #else
62 #define DAC_TRACK_NUMBER 2
63 #endif
64
65 // audio dac 样点位宽
66 #define AUDAC_BIT_WIDE 16 //24
67 #if AUDAC_BIT_WIDE == 16
68 #define A_DAC_SP_BIT 0
69 #endif
70 #if AUDAC_BIT_WIDE == 24
71 #define A_DAC_SP_BIT A_DAC_SP_24B
72 #endif
73
apps/include_lib/cpu/uc03/audio_dac.h
```

图 2.1 audio dac 位宽

21.3 AUDIO DAC 示例

如图 4，有 audio dac 使用的示例，实例中做了以下事情：

- ① 初始化测试数据流，此数据流为 DAC 提供样点数据；
- ② 初始化 AUDIO 公共部分；
- ③ 初始化 AUDIO DAC 模块，这一步完成之后 audio dac 就能起中断；
- ④ 初始化数据流句柄；

- ⑤ 注册数据流句柄；
- ⑥ 循环将 ADC 采样获取的样点数据打印出来。

```
static void audio_dac_demo(void)
{
    // 为DAC准备样点数据
    tab_init(&sine_demo_obj, (void *)&sine_st_demo[0], sizeof(sine_st_demo));
    sine_demo_obj.cnt = -1;

    //初始化AUDIO所需的变量，初始化AUDIO公共部分
    audio_variate_init();
    audio_init();

    //初始化audio dac驱动模块，初始化audio dac硬件
    dac_mode_init(31);
    dac_init_api(32000);

    //初始化数据流句柄
    cbuf_init(&t_cbuf, t_obuf, sizeof(t_obuf));
    memset(&t_sound, 0, sizeof(t_sound));
    t_sound.p_obuf = &t_cbuf;
    t_sound.info |= B_STEREO;
    t_sound.enable |= B_DEC_RUN_EN;
    //将数据流句柄，注册到DAC管理器中
    regist_dac_channel(&t_sound, 0);

    u32 wlen;
    u8 *data;
    while (1) {

        //将样点写入到DAC数据流中
        data = cbuf_write_alloc(t_sound.p_obuf, &wlen);
        tab_read(data, &sine_demo_obj, wlen);
        cbuf_write_updata(t_sound.p_obuf, wlen);

        wdt_clear();
    }
}
```

图 3.1 audio dac demo

21.3.1 开打 AUDIO DAC 硬件

- ① 初始化 AUDIO 公共部分；
- ② 初始化 AUDIO DAC 部分；

```
//初始化AUDIO所需的变量，初始化AUDIO公共部分
audio_variate_init();
audio_init();

//初始化audio dac驱动模块，初始化audio dac硬件
dac_mode_init(31);
dac_init_api(32000);
```

图 3.2 audio dac 初始化

21.3.2 关闭 AUDIO DAC 硬件

- ① 关闭 AUDIO DAC 数字部分;
- ② 关闭 AUDIO DAC 模拟部分;

```
void audio_dac_off_demo(void)
{
    //关闭AUDIO DAC硬件的数字部分
    dac_off_api();
    //关闭AUDIO_ADC硬件的模拟部分
    audac_analog_off();
}
```

图 3.3 关闭 audio dac

21.3.3 关闭 AUDIO DAC 与 AUDIO 硬件

- ① 关闭 AUDIO DAC 数字部分;
- ② 关闭 AUDIO DAC 模拟部分;
- ③ 关闭 AUDIO 硬件的数字部分;
- ④ 关闭 AUDIO 硬件的模拟部分;
- ⑤ 关闭 AUDIO 的时钟;

注：从上面第 ③ 步开始关闭了 AUDIO 公共部分后，包含 AUDIO DAC 和 AUDIO ADC 在内的所有 AUDIO 部分将不能再工作！

```
void audio_off_demo(void)
{
    //关闭AUDIO DAC硬件的数字部分
    dac_off_api();
    //关闭AUDIO_DAC硬件的模拟部分
    audac_analog_off();

    //关闭AUDIO硬件的数字部分
    audio_common_digital_off();
    //关闭AUDIO硬件的模拟部分
    audio_analog_off();
    //关闭AUDIO的时钟
    audio_clock_off();
}
```

图 3.4 关闭 audio dac & audio 公共部分

21.4 AUDIO 全关闭示例

- ① 关闭 AUDIO DAC 数字部分;
- ② 关闭 AUDIO DAC 模拟部分;
- ③ 关闭 AUDIO ADC 数字部分;
- ④ 关闭 AUDIO ADC 模拟部分;
- ⑤ 关闭 AUDIO 硬件的数字部分;
- ⑥ 关闭 AUDIO 硬件的模拟部分;
- ⑦ 关闭 AUDIO 的时钟;

```
void audio_off_demo(void)
{
    //关闭AUDIO DAC硬件的数字部分
    dac_off_api();
    //关闭AUDIO_DAC硬件的模拟部分
    audac_analog_off();

    if (config_audio_adc_enable) {
        //关闭AUDIO ADC硬件的数字部分
        audio_adc_phy_off();
        //关闭AUDIO_ADC硬件的模拟部分
        audio_adc_analog_off();
    }
    //关闭AUDIO硬件的数字部分
    audio_common_digital_off();
    //关闭AUDIO硬件的模拟部分
    audio_analog_off();
    //关闭AUDIO的时钟
    audio_clock_off();
}
```

图 4.1 全面 audio

21.5 AUDIO DAC 管理架构

AD16N 内置 DAC, 此 DAC 支持 16/24bit 精度, 支持立体声和单声道, 最高支持 96k 采样率; DAC 固件管理框架同时支持最多 2 路 DAC 数据流输出。AUDIO DAC 相关的源代码文件:

```
apps/app/bsp/common/audio/audio_dac_api.c
apps/app/bsp/cpu/uc03/audio_dac_cpu.c
apps/include_lib/audio/audio_dac_api.h
apps/include_lib/cpu/uc03/audio_dac.h
```

下图是 Audio DAC 管理框架对应的结构体。

```

typedef struct _DAC_MANAGE {
    /*多路数据流句柄*/
    sound_out_obj *sound[AUDAC_CHANNEL_TOTAL];
    /*多路数据流句柄所对应的触发函数*/
    void (*kick[AUDAC_CHANNEL_TOTAL])(void *);
    u8 ch; /*通路标记*/
    u16 vol_l_phy; /*左声道物理音量*/
    u16 vol_r_phy; /*右声道物理音量*/
    u8 vol_l; /*左声道音量*/
    u8 vol_r; /*右声道音量*/
    u8 flag; /*静音等控制标记*/
    /* u8 track; */
} DAC_MANAGE;

```

apps/app/bsp/common/audio/audio_dac_api.c

图 5.1 audio dac 管理框架

AUDIO DAC 主要有以下函数：

```

void dac_mode_init(u16 vol)
void dac_init_api(u32 sr)
void dac_sr_api(u32 sr)
void dac_off_api(void)
void fifo_dac_fill(u8 *buf, u32 len, AUDIO_TYPE type)
u8 stereo_dac_vol(char set, u8 vol_l, u8 vol_r)
u8 dac_vol(char set, u8 vol)
bool dac_mute(bool mute)
bool regist_dac_channel(void *psound, void *kick)
bool unregist_dac_channel(void *psound)
bool dac_cbuff_active(void *sound_hld)
void dac_kick(void *sound_hld, void *pkick)
void audio_dac_analog_vol(AUDIO_DAC_ANA_VOL vol_l, AUDIO_DAC_ANA_VOL vol_r)
void audio_amux_analog_vol(AUDIO_DAC_AUMX_VOL vol_l, AUDIO_DAC_AUMX_VOL vol_
↪r)

```

21.5.1 AUDIO DAC 驱动模式

AUDIO DAC 主要有下面几种驱动模式；

立体声

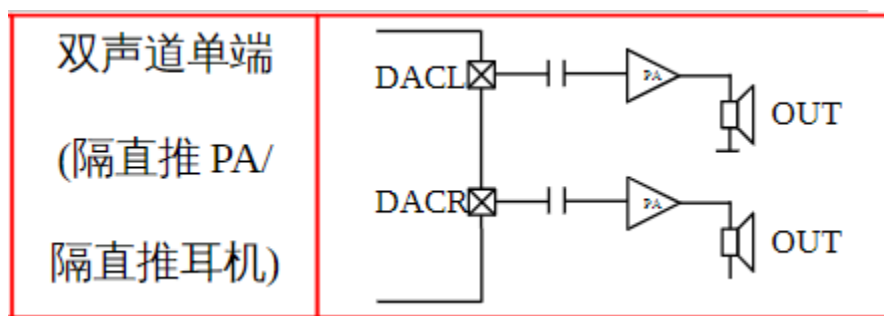


图 5.2 audio dac 立体声

差分单声道

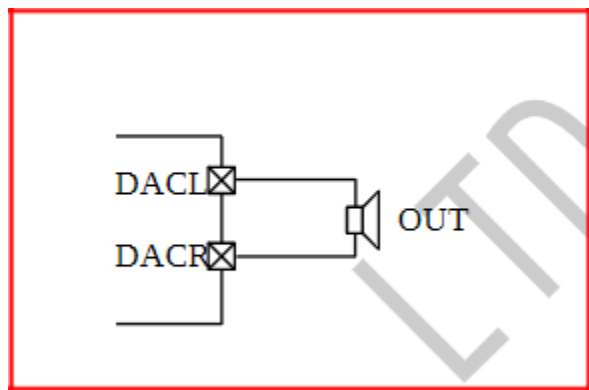


图 5.3 audio dac 差分输出

单声道单端直推

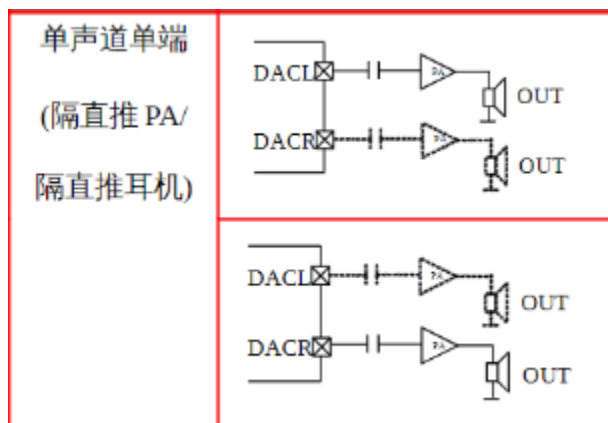


图 5.4 audio dac 单端直推

21.5.2 AUDIO DAC 共 VCM0 模式

AC16N 有 VCM00 和 VCM01 引脚能输出一个和 DAC 工作电压一样的电压；

正确配置后 VCM00/1 能输出一个和 DAC 工作电压一样的电压，这样在耳机方案应用中，使用 VCM00/1 作为公共端，可以省掉 DAC 输出所需的隔直电容，配置方法参考本章的配置章节的内容。

下图是利用 VCM0 做公共端时的连接方式。

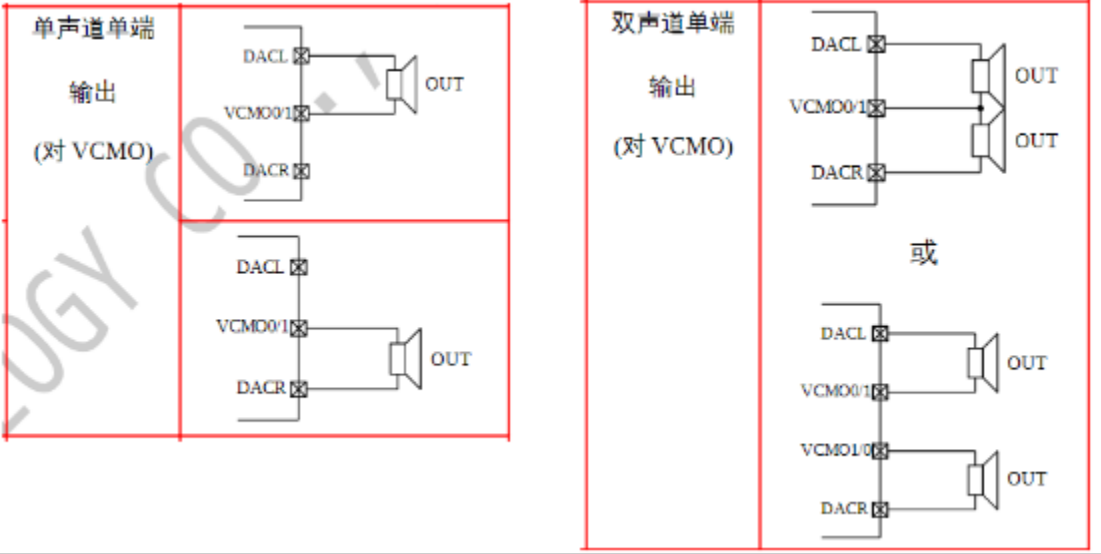


图 5.5 audio dac 共 VCM0 模式

21.6 AUDIO DAC 相关函数

21.6.1 函数 void dac_mode_init(void)

此函数实现对 DAC 管理结构体和相关解码参数进行初始化，开机初始化调用一次即可。

21.6.2 函数 void dac_init_api(u32 sr)

此函数实现对 DAC 硬件资源和相关寄存器进行初始化，此处会根据 dac.h 文件中宏定义 DAC_CURR_MODE 的值来选择开启 DAC 工作模式，开机初始化调用一次即可，其中参数：

1、sr：采样率，具体传参，参考表本章表15.1；

21.6.3 函数 dac_sr_api(u32 sr)

采样率设置函数：

1、sr：采样率，具体传参，参考表本章表15.1；

21.6.4 函数 void dac_off_api(void)

关闭 dac 数字部分的函数。

21.6.5 函数 u8 stereo_dac_vol(char set, u8 vol_l, u8 vol_r)

音量设置函数：

```
1、set：
    ① '+'，增大一格数字音量；
    ② '-'，减小一个数字音量；
    ③ 'r'，返回当前数字音量；
    ④ 0，传入0，直接将数字音量设置另一参数vol的值；
2、vol_l：左声道音量，0-31，当set为0时，直接设置为此音量，最大值为31；
3、vol_r：右声道音量，0-31，当set为0时，直接设置为此音量，最大值为31；
4、返回值：
    当前设置的数字音量值；
```

21.6.6 函数 u8 dac_vol(char set, u8 vol)

音量设置函数：

```
1、set：
    ① '+'，增大一格数字音量；
    ② '-'，减小一个数字音量；
    ③ 'r'，返回当前数字音量；
    ④ 0，传入0，直接将数字音量设置另一参数vol的值；
2、vol：0-31，当set为0时，直接设置为此音量，最大值为31；
3、返回值：
    当前设置的数字音量值；
```

21.6.7 函数 bool dac_mute(bool mute)

静音函数：

```
1、mute：
    ① 1，静音；
    ② 0，解除静音；
2、返回值：
    暂无意义；
```

21.6.8 函数 bool regist_dac_channel(void *psound, void *kick)

注册数据流到 AUDIO DAC 管理框架中：

```
1、psound：sound_out_obj 指向的 sound 数据流；
2、kick：    触发函数；
3、返回值：
    成功 true；
    失败 false；
```

21.6.9 函数 `bool unregist_dac_channel(void *psound)`

将数据流从 AUDIO DAC 中注销掉：

```
1、psound：需要注销的 sound_out_obj 类型的数据流；
2、返回值：
   成功 true；
   失败 false；
```

21.6.10 函数 `void fifo_dac_fill(u8 *buf, u32 len, AUDIO_TYPE type)`

此函数实现将解码后的音频样点数据经过处理后推给 DAC 的缓存 buf 使 DAC 进行转换，其中参数：

```
1、buf：DAC即将要进行转换的解码样点数据缓存buf；
2、len：该buf的大小，默认为1k个byte，存有512个样点；
3、type：数据流来源；
```

该接口实现将解码后的样点数据（存放于 `dac_mge.sound[i]->p_obuf` 中）经过处理后输出至 DAC。DAC 支持 16bit 精度，一个样点为 2 个 byte。其具体操作如下：

- ① DAC 转换一次后，会在转换结束后调用该接口将新的数据样点传给 DAC；
- ② 该接口首先会找到正在解码播放的通道，并读取该解码器解码出来的样点 buf；
- ③ `rptr[i]` 是解码读出的一个原始样点数据，可以读取该数组获得数据；
- ④ `sp_buf[sp_cnt]` 是经过音量处理后的样点数据，可以读取该数组获得数据；
- ⑤ `sp_buf[sp_cnt]` 中的数据为最终 DAC 进行转换的样点数据；
- ⑥ 待样点数据全部处理完后，启动 kick 继续进行下一轮 DAC 转换；

21.6.11 函数 `void audio_dac_analog_vol(AUDIO_DAC_ANA_VOL vol_l, AUDIO_DAC_ANA_VOL vol_r)`

AUDIO DAC 模拟增益：

```
1、vol_l：左声道增益；
2、vol_r：右声道增益；
```

21.6.12 函数 `void audio_amux_analog_vol(AUDIO_DAC_AUMX_VOL vol_l, AUDIO_DAC_AUMX_VOL vol_r)`

AUDIO LINEIN 模拟直通 DAC 的模拟增益：

```
1、vol_l：左声道增益；
2、vol_r：右声道增益；
```

AD16N AUDIO ADC 为一通道，支持单声道，支持模拟 MIC 输入或者 LINEIN 输入。

AD16N AUDIO 采样率支持情况	ADC
8000	yes
11025	yes
12000	yes
16000	yes
22050	yes
24000	yes
32000	yes
44100	yes
48000	yes

表 1 AUDIO_ADC 采样率支持情况

AUDIO ADC - MIC 工作模式

AD16N AUDIO ADC - MIC 工作模式			
ADC MIC	内置偏置电阻	外置偏置电阻	差分

表 2 AD16N AUDIO ADC - MIC 工作模式

AUDIO ADC - LINEIN 工作模式

AD16N AUDIO ADC - LINEIN 工作模式			
LINEIN	模拟直通	DAC	ADC 转数字注册到 DAC 的注册表

表 3 AD16N AUDIO ADC - LINEIN 工作模式

22.1 《app_config.c》中的开机默认配置

如图 1 中 Audio 公共部分和 AUDIO DAC 包含以下配置：

1. vcm 外挂电容；
2. vcmo0/1 电压；
3. dac 模拟增益；
4. 左右声道输出；

22.1.1 Audio 公共部分

`const bool config_vcm_cap_addon`: vcm 外挂电容配置；

软件默认不外挂电容；

如果软件选择了外挂电容，硬件却没有电容连接，会导致音频性能大幅降低；

VCM 正确挂载了电容的情况下，软件悬着外挂电容会比不外挂电容音频性能更好；

22.1.2 AUDIO ADC 部分配置

VCOM0/1 可作为内置偏置方案（MIC_BIAS）的偏置 I/O，该配置与 DAC 使能 VCOM0/1 互斥！

AUDIO_ADC - MIC 配置

- **AUDIO_MIC_RS_MODE** `const audio_adc_mic_rs_mode`: AUDIO_ADC 输入方式选择：
 1. MICBIAS 方案：VCOM0/1 输出内部 MICLDO + 内置可调电阻
 2. MICLDO 方案：PA0 I/O 输出内部 MICLDO + 外置电阻
 3. 外部提供偏置方案：外置电源 + 外置电阻
- `u32 const audio_adc_mic_bias_2_vcmo0`: 使能 VCOM0 MICBIAS 功能；
- `u32 const audio_adc_mic_bias_2_vcmo1`: 使能 VCOM1 MICBIAS 功能；
- **AUDIO_MICBIAS_RS** `const audio_adc_mic_bias_rs`: 使能 MICBIAS 方案时，选择内置偏置电阻大小
- **AUDIO_MICLDO_VS** `const audio_adc_mic_ldo_vs`: 使能 MICLDO 方案时，选择 PA0 输出 LDO 电压大小
- **AUDIO_MIC_INPUT_MODE** `const audio_adc_mic_input_mode`: 选择 MIC 输入 IO，可选择 PA0 / PA1

AUDIO_ADC - MIC 两级增益选择：

- `u32 const audio_adc_mic_pga_6db`: MIC 前级 6dB 增益，0: 不使能 1: 使能
- **AUDIO_MICPGA_G** `const audio_adc_mic_pga_g`: MIC 后级增益档位，根据枚举变量选择；

AUDIO_ADC - LINEIN 一级增益选择：

- **AUDIO_MICPGA_G** `const audio_adc_aux_pga_g`: AUX 增益档位选择，根据枚举变量选择；

使用内部偏置（MICBIAS）方案配置：

22.2 AUDIO ADC 示例

如图 8，有 audio adc 使用的示例，实例中做了以下事情：

- ① 初始化 AUDIO 公共部分；
- ② 初始化 AUDIO ADC 模块，这一步完成之后 audio dac 就能起中断；
- ③ 初始化数据流句柄；
- ④ 注册数据流句柄；
- ⑤ 循环将 ADC 采样到的样点打印出来。

```
static void audio_dac_demo(void)
{
    // 为DAC准备样点数据
    tab_init(&sine_demo_obj, (void *)&sine_st_demo[0], sizeof(sine_st_demo));
    sine_demo_obj.cnt = -1;

    //初始化AUDIO所需的变量，初始化AUDIO公共部分
    audio_variate_init();
    audio_init();

    //初始化audio dac驱动模块，初始化audio dac硬件
    dac_mode_init(31);
    dac_init_api(32000);

    //初始化数据流句柄
    cbuf_init(&t_cbuf, t_obuf, sizeof(t_obuf));
    memset(&t_sound, 0, sizeof(t_sound));
    t_sound.p_obuf = &t_cbuf;
    t_sound.info |= B_STEREO;
    t_sound.enable |= B_DEC_RUN_EN;
    //将数据流句柄，注册到DAC管理器中
    regist_dac_channel(&t_sound, 0);

    u32 wlen;
    u8 *data;
    while (1) {

        //将样点写入到DAC数据流中
        data = cbuf_write_alloc(t_sound.p_obuf, &wlen);
        tab_read(data, &sine_demo_obj, wlen);
        cbuf_write_updata(t_sound.p_obuf, wlen);

        wdt_clear();
    }
}
```

图 2.1 audio adc demo

22.2.1 打开 AUDIO ADC 硬件

- ① 初始化 AUDIO ADC;
- ② 使能启动 AUDIO ADC;

```
//初始化audio dac驱动以及硬件，并启动
audio_adc_init_api(32000, AUDIO_ADC_MIC, 0);
audio_adc_enable();
```

图 2.2 audio adc 初始化

22.2.2 停止 AUDIO ADC 运行

- ① 调用函数 audio_adc_disable 能停止 AUDIO ADC 的运行

```
void audio_adc_stop_demo(void)
{
    //停止AUDIO ADC运行
    audio_adc_disable();
}
```

图 2.3 audio adc stop demo

22.2.3 关闭 AUDIO ADC 硬件

- ① 关闭 AUDIO ADC 数字部分;
- ② 关闭 AUDIO ADC 模拟部分;

```
void audio_dac_off_demo(void)
{
    //关闭AUDIO DAC硬件的数字部分
    dac_off_api();
    //关闭AUDIO_ADC硬件的模拟部分
    audac_analog_off();
}
```

图 2.4 close audio adc

22.2.4 关闭 AUDIO 以及 AUDIO ADC 硬件

- ① 关闭 AUDIO ADC 数字部分;
- ② 关闭 AUDIO ADC 模拟部分;
- ③ 关闭 AUDIO 硬件的数字部分;
- ④ 关闭 AUDIO 硬件的模拟部分;

- ⑤ 关闭 AUDIO 的时钟;

```
void audio_off_demo(void)
{
    //关闭AUDIO DAC硬件的数字部分
    dac_off_api();
    //关闭AUDIO_DAC硬件的模拟部分
    audac_analog_off();

    //关闭AUDIO硬件的数字部分
    audio_common_digital_off();
    //关闭AUDIO硬件的模拟部分
    audio_analog_off();
    //关闭AUDIO的时钟
    audio_clock_off();
}
```

图 2.4 close audio adc & audio

22.3 AUDIO 全关闭示例

- ① 关闭 AUDIO DAC 数字部分;
- ② 关闭 AUDIO DAC 模拟部分;
- ③ 关闭 AUDIO ADC 数字部分;
- ④ 关闭 AUDIO ADC 模拟部分;
- ⑤ 关闭 AUDIO 硬件的数字部分;
- ⑥ 关闭 AUDIO 硬件的模拟部分;
- ⑦ 关闭 AUDIO 的时钟;

```

static void audio_adc_demo(void)
{
    //初始化AUDIO所需的变量，初始化AUDIO公共部分
    audio_variate_init();
    audio_init();

    //初始化audio dac驱动以及硬件，并启动
    audio_adc_init_api(32000, AUDIO_ADC_MIC, 0);
    audio_adc_enable();

    //初始化数据流句柄
    cbuf_init(&t_cbuf, t_obuf, sizeof(t_obuf));
    memset(&t_sound, 0, sizeof(t_sound));
    t_sound.p_obuf = &t_cbuf;
    t_sound.enable |= B_DEC_RUN_EN;
    //将数据流句柄，注册到ADC管理器中
    regist_audio_adc_channel(&t_sound, NULL);

    u32 wlen;
    u8 *data;
    while (1) {

        //将ADC采样而来的样点打印出来
        //data = cbuf_read_alloc(t_sound.p_obuf, &rlen);
        log_info_hexdump(data, rlen)
        cbuf_read_alloc_updata(t_sound.p_obuf, rlen);
        wdt_clear();
    }
}

```

图 3.1 all audio

22.4 AUDIO ADC 管理架构

AD16N 内置 ADC，此 AUDIO_ADC 支持 16bit 精度，单声道，最高支持 48k 采样率；AUDIO_ADC 固件管理框架只支持一路 AUDIO_ADC 输入，最多 2 路将 AUDIO_ADC 采集到的数据流输出。

AUDIO ADC 相关的源代码文件：

```

apps/app/bsp/common/audio/audio_adc_api.c
apps/app/bsp/cpu/uc03/audio_adc_cpu.c
apps/include_lib/audio/audio_adc_api.h
apps/include_lib/cpu/uc03/audio_adc.h

```

下图是 Audio ADC 管理框架对应的结构体。

```
typedef struct _AUDIO_ADC_MANAGE {
    /*多路数据流句柄*/
    sound_out_obj *sound[AUADC_CHANNEL_TOTAL];
    /*多路数据流句柄所对应的触发函数*/
    void (*kick[AUADC_CHANNEL_TOTAL])(void *);
    u8 ch;          /*通路标记*/
    u8 track;       /*声道数目*/
} AUDIO_ADC_MANAGE;

AUDIO_ADC_MANAGE audio_adc_mge ;//AT(.AUDIO_ADC_BUFFER);
```

图 4.1 audio adc 管理框架

AUDIO ADC 主要有以下函数：

```
void audio_adc_mode_init(void)
u32 audio_adc_init_api(u32 sr, AUDIO_ADC_MODE mode, u32 ch)
void audio_adc_off_api(void)
void fill_audio_adc_fill(u8 *buf, u32 len)
bool regist_audio_adc_channel(void *psound, void *kick)
bool unregist_audio_adc_channel(void *psound)
```

22.4.1 AUDIO ADC 驱动模式

单端 MICIN（使用芯片内部偏置电阻）

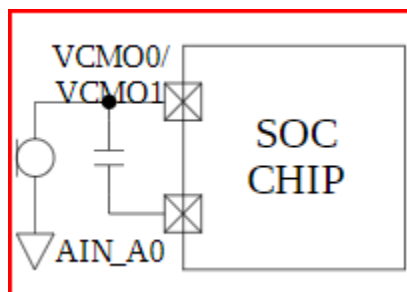


图 4.2 单端 MICIN 使用内部偏置电阻示意图

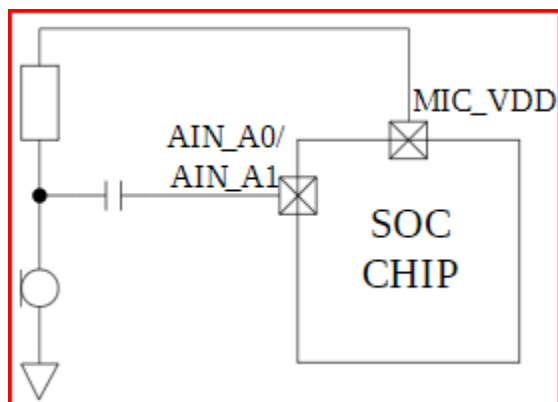
单端 MICIN（使用芯片外部偏置电阻）

图 4.3 单端 MICIN 使用外部偏置电阻示意图

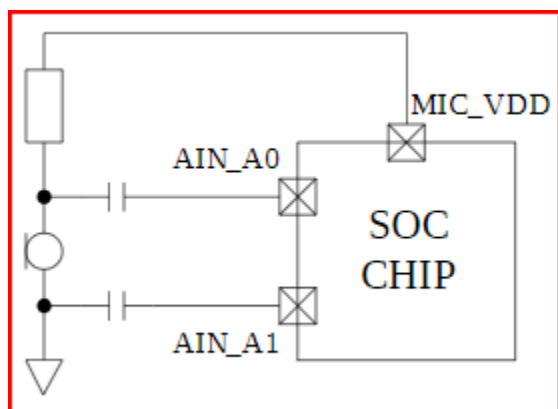
差分 MICIN（使用外部偏置电阻）

图 4.4 差分 MICIN 使用外部偏置电阻示意图

22.5 AUDIO ADC 相关函数

22.5.1 函数 void audio_adc_mode_init(void)

此函数实现对 ADC 管理结构体和相关编码参数进行初始化，开机初始化调用一次即可。

22.5.2 函数 u32 audio_adc_init_api(u32 sr, ADC_MODE mode, u32 throw_sp_num)

此函数实现对 ADC 硬件资源和相关寄存器进行初始化，此处会根据 mode 的值来选择 ADC 模式，其中参数：

```
1、sr：采样率；
    ①、8000；
    ②、11025；
    ③、12000；
    ④、16000；
    ⑤、22050；
    ⑥、24000；
    ⑦、32000；
    ⑧、44100；
    ⑨、48000；

2、mode：ADC模式，默认选择为ADC_MIC；
    ① ADC_MIC：ADUIO_ADC数据来源为MIC采集回来（PA1 或 PA2）
    ② ADC_LINE_IN：AUDIO_ADC数据来源为LINEIN输入（PA1 或 PA2）

3、throw_sp_num：
    第一次启动 AUDIO_ADC 需要抛弃的样点数；

4、返回值：成功 0；
            失败：采样率不匹配；
```

22.5.3 函数 void audio_adc_off_api(void)

此函数实现关闭所有 AUDIO_ADC 模块。

22.5.4 函数 bool regist_audio_adc_channel(void *psound, void *kick)

此函数实现将 sound_out_obj 注册到 AUDIO_ADC 中，其中参数：

```
1、psound：sound_out_obj 指向的 sound 结构体；
2、kick：kick 函数；
3、返回值：
    成功 true；
    失败 false；
```

22.5.5 函数 bool unregist_audio_adc_channel(void *psound)

此函数实现将 AUDIO_ADC 注册列表中指定的 sound_out_obj 移除，用户可将采集到的数据最多分发两路进行处理，其中参数：

```
1、psound：sound_out_obj 指向的 sound 结构体；
2、返回值：
    成功 true；
    失败 false；
```


22.5.6 函数 void fill_audio_adc_fill(u8 *buf, u32 len)

此函数在 ADC 中断中被调用，用于将 ADC 转换出来放在 buf 里的缓存数据推到指定的空间保存，其中参数：

- 1、buf：AUDIO_ADC 转换后样点存放的缓存buf；
- 2、len：该buf的大小，默认为160个byte，存有80个样点；

该接口实现将 ADC 转换完成的数据样点写到相应的空间（存放于 audio_adc_mge.sound[i]->p_obuf）中，其具体操作如下：

- ① AUDIO_ADC_
- 转换一次后，会在中断结束后调用该接口将转换的数据样点取走存放于相应的空间；
- ② 该接口首先会选择正在编码播放的通道，并找到存放样点数据的buf；
- ③ 一次性将buf中的数据写入注册了该 AUDIO_ADC 通道的audio_adc_mge.sound[i]->p_obuf里；
- ④ 若kick函数存在则运行kick函数，为空则该AUDIO_ADC填数函数运行完毕。

软硬件淡入淡出

23.1 淡入淡出效果配置

淡入淡出效果分软件和硬件控制，软件效果时间会持续稍久，硬件效果持续时间微短。

```

76
77 /*-----DAC数字音量设置淡入淡出配置-----*/
78 #define TCFG_DAC_VOL_FADE_EN          DISABLE//硬件DAC淡入淡出
79 #define DAC_SOFT_FADE_EN              DISABLE//软件运算淡入淡出
80 //若为44.1K采样率,音量从最小0->调大最大16384耗时:((16384-0)/step)/(352.8K

```

图 1 软硬件淡入淡出配置宏

硬件淡入淡出效果时间参数由 app_config.h 里的 TCFG_DAC_VOL_FADE_STEP 和 TCFG_DAC_VOL_FADE_SLOW 共同控制；

```

77 /*-----DAC数字音量设置淡入淡出配置-----*/
78 #define TCFG_DAC_VOL_FADE_EN          DISABLE//硬件DAC淡入淡出
79 #define DAC_SOFT_FADE_EN              DISABLE//软件运算淡入淡出
80 //若为44.1K采样率,音量从最小0->调大最大16384耗时:((16384-0)/step)/(352.8K/(slow+1)) = 46.4ms / step * (slow+1)
81 //若为48K/32K采样率,音量从最小0->调大最大16384耗时:((16384-0)/step)/(384K/(slow+1)) = 42.7ms / step * (slow+1)
82 //若为48K/24K采样率,音量从最小0->调大最大16384耗时:((16384-0)/step)/(288K/(slow+1)) = 42.7ms / step * (slow+1)
83 #define TCFG_DAC_VOL_FADE_STEP          1//0和1一样效果,配置范围0~15
84 #define TCFG_DAC_VOL_FADE_SLOW          15//配置范围0~15

```

图 2 硬件淡入淡出时间配置

软件淡入淡出效果时间由 dac_single_fade() 函数决定；

```
57 #if DAC_FADE_ENABLE
58
59 u16 dac_single_fade(u16 vol, u16 vol_phy, u16 *p_cnt)
60 {
61     u16 target_vol = 0;
62     u16 curr_vol = vol_phy;
63     u16 t_fade_step;
64
65     if (curr_vol > 100) {
66         t_fade_step = (curr_vol + 1000) / 60;
67     } else {
68         t_fade_step = 10;
69     }
70 }
```

每级递加/减音量

图 3 软件淡入淡出步长配置

23.2 淡入淡出相关函数

23.2.1 void dac_fade(void)

该函数接口只在软件淡入淡出中生效，主要负责改变音量动作；

23.2.2 dac_fade_in_api()

该接口是执行淡入动作；

23.2.3 dac_fade_out_api()

该接口是执行淡出动作；

接口函数在 audio_dac_fade.h:

```
16 #define DAC_FADE_ENABLE
17 #endif
18
19 #if TCFG_DAC_VOL_FADE_EN
20 void hardware_fade_in(void);
21 void hardware_fade_out(void); 硬件淡入淡出接口
22 void hardware_fade(void);
23 #define dac_fade_api()
24 #define dac_fade_in_api hardware_fade_in
25 #define dac_fade_out_api hardware_fade_out
26 #endif DAC_FADE_ENABLE
27 void dac_fade(void);
28 void dac_fade_in(void); 软件淡入淡出接口
29 void dac_fade_out(u32 delay);
30 #define dac_fade_api dac_fade
31 #define dac_fade_in_api dac_fade_in
32 #define dac_fade_out_api() dac_fade_out(200)
33 #else
34 void dac_fade(void);
35 void dac_fade_in(void);
36 void dac_fade_out(u32 delay);
37 #define dac_fade_api()
38 #define dac_fade_in_api()
39 #define dac_fade_out_api(...)
40 #endif
41
42 #endif
```

图 4 软硬件淡入淡出配置

软件 EQ (PCM_EQ)

AW30N 的 eq 为 pcm_eq。

该功能可通过 app_modules_h.c 里的 PCM_SW_EQ_EN 宏控制开关。

24.1 1 配置参数说明

- **eq_filt_32000**: PCM_EQ 处理 32K 采样率音频的运算数据
该数据由 EQ 工具生成，最多可以有 10 段 EQ 数据 (seg0~seg9)，详情请见 EQ 工具使用说明章节，不同采样率拥有各自的 eq_filt 表；
- **g_work_buf**: PCM_EQ 运算需要的 buff
根据 EQ 使用的频点数 (段数) 决定 buff 大小，可通过 pcm_eq_buf_len() 接口获取大小；

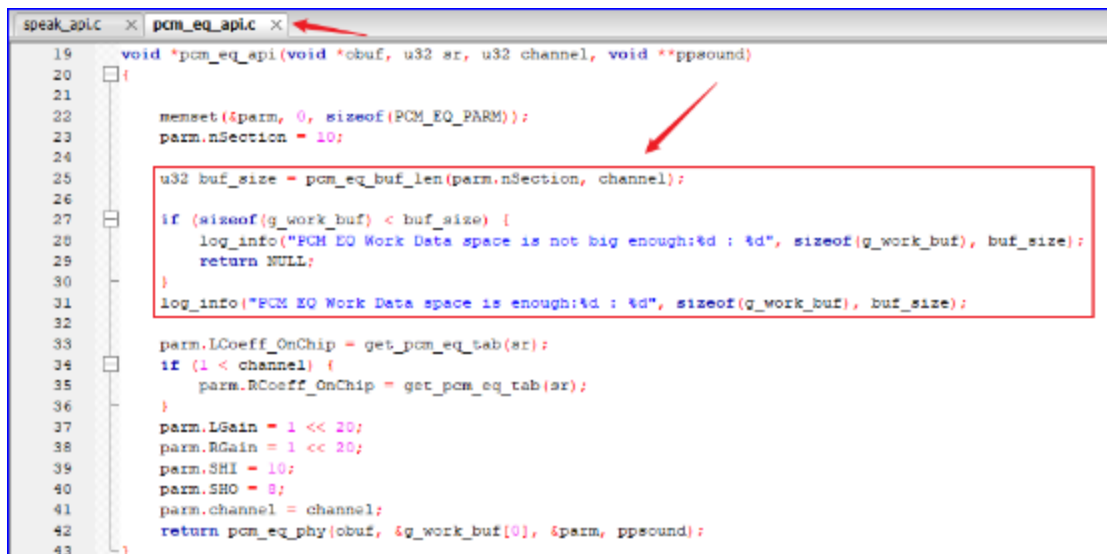


图 1 pcm_eq buff 大小获取

24.2 2 相关函数

24.2.1 2.1 函数 void *link_pcm_eq_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u32 channel)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、sr：当前输入音频的采样率；
- 5、channel：声道数，ad14n和ac104n只有单声道，传1即可；
- 6、返回值：启动之后最新的音频通道句柄；

24.2.2 2.2 函数 void *pcm_eq_api(void *obuf, u32 sr, u32 channel, void **ppsound)

该函数用于初始化 PCM_EQ 的相关参数，由函数 link_pcm_eq_sound() 调用即可，需要关注的其内部参数的初始化：

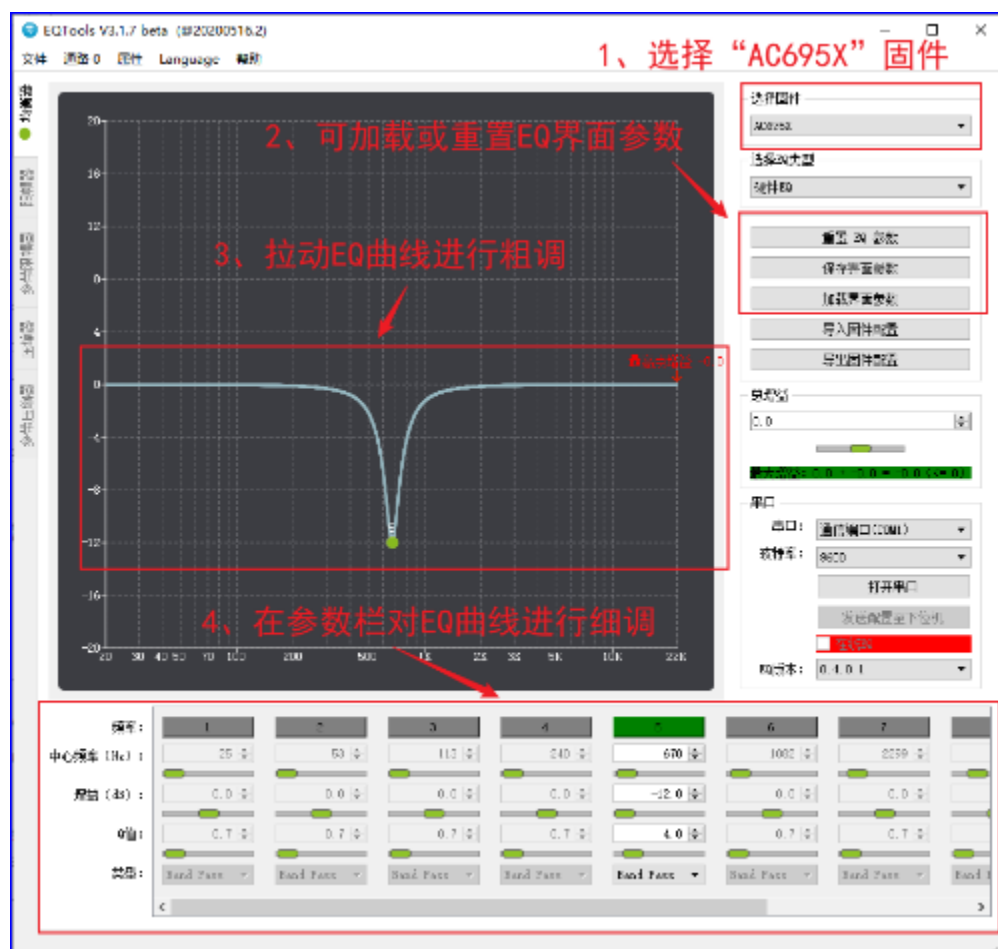
- 1、nSection：EQ频点数（段数），不可超过各采样率的eq_filt表中的段数，范围[1, 10]；
- 2、channel：陷波器带宽，该值越小防啸叫效果越好，但发声啸叫频点误检时音质会更差；

24.3 3 EQ 工具使用

AW30N 可以使用杰理 EQ 工具进行离线 EQ 调试，调试完成后可以导出当前的 EQ 配置，并加载到程序中验证效果。

24.3.1 3.1 EQ 工具使用介绍

1、打开“EQ-3.1.7-beta-20200516.2.exe”工具并选择固件“AC695X”；2、选择需要的频点数，对于不需要的频点可以点击下方频率行对应跑按钮使其变灰；3、拉动 EQ 曲线进行粗调；4、粗调结束后，可在下方参数框对 EQ 曲线进行细调，可设置准确的中心频率、增益、Q 值以及滤波器类型；



5、EQ 调节结束后，点击右边“保存界面参数”按钮对编辑的 EQ 界面进行保存；6、保存后，再点击“导出固件配置”按钮，工具根目录会生成新的“hw_eq_table.h”文件；

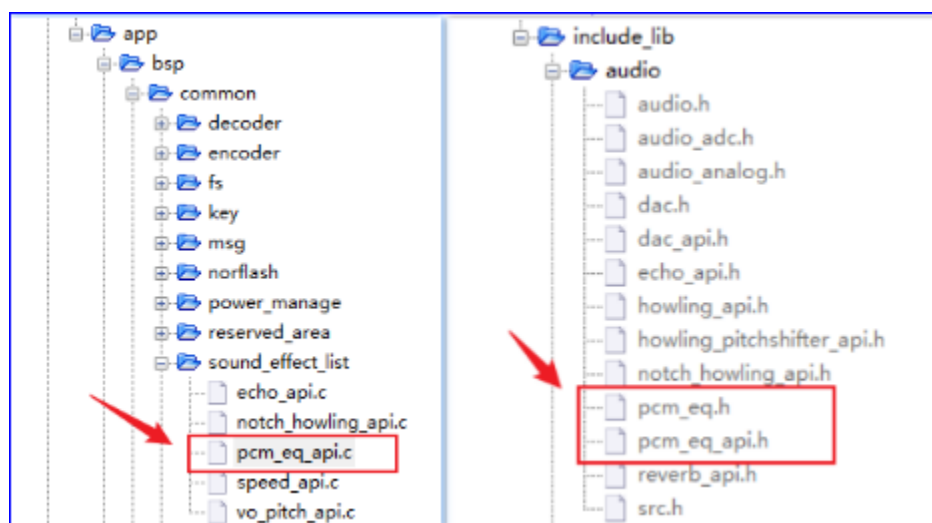
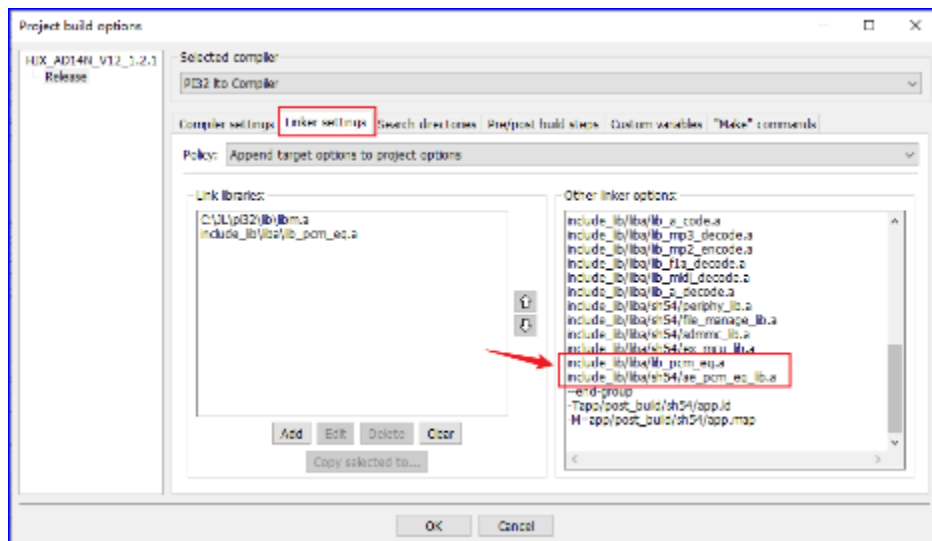


24.3.2 3.2 SDK 配置说明

1、SDK 工程导入 pcm_eq 相关文件；

如：库文件：include_lib/liba/ae_pcm_eq_lib.a

头文件：include_lib/pcm_eq/pcm_eq_api.h 源文件：app/bsp/common/sound_effect_list/pcm_eq_api.c




```

1  /* #include "resample_api.h" */
2  #include "pcm_eq.h"
3  /* #include "resample.h" */
4  #include "decoder_api.h"
5  #include "config.h"
6  #include "sound_effect_api.h"
7
8
9  #define LOG_TAG_CONST    NORM
10 #define LOG_TAG          "[normal]"
11 #include "debug.h"
12
13
14 static u32 g_work_buf[660 / 4] AT(.pcm_eq_data);
15
16 const int *get_pcm_eq_tab(u32 sr);
17
18 PCM_EQ_PARM param;
19 void *pcm_eq_api(void *obuf, u32 sr, u32 channel, void **ppsound)
20 {
21     1、nSection根据EQ工具中选择的频点数确定
22     memset(&param, 0, sizeof(PCM_EQ_PARM));
23     param.nSection = 10;
24
25     u32 buf_size = pcm_eq_buf_len(param.nSection, channel);
26
27     if (sizeof(g_work_buf) < buf_size) {
28         log_info("PCM EQ Work Data space is not big enough:%d : %d", sizeof(g_work_buf), buf_size);
29         return NULL;
30     }
31     log_info("PCM EQ Work Data space is enough:%d : %d", sizeof(g_work_buf), buf_size);
32 }

```

4、打开 decoder_api.c 文件，将 EQ 串入音频解码链路中，运行测试效果；

备注：需要注意 eq 所用的 ram 是否与其他资源复用了，建议不要同时开启多种音效；

```

165 #if AUDIO_SPEED_EN
166 //变速变调
167 if (dec_opt & BIT_SPEED) {
168     p_curr_sound->effect = speed_api(cbuff_o, p_dec->sr, (void **) sp_next_sound);
169     if (NULL != p_curr_sound->effect) {
170         p_curr_sound->enable |= B_DEC_EFFECT;
171         p_curr_sound = p_next_sound;
172         p_curr_sound->p_obuf = cbuff_o;
173         p_next_sound = 0;
174         /* log_info("src link speed\n"); */
175     }
176 }
177
178 //变速变调
179 log_info("001 link pcm_eq_sound!\n");
180 extern void *link_pcm_eq_sound(void *p_sound_buf, void *p_dec_buf, void *p_eff, u32 sr, u32 channel);
181 p_curr_sound = link_pcm_eq_sound(p_curr_sound, cbuff_o, (void **)NULL, 32000, 1);
182 log_info("002 link pcm_eq_sound!\n");
183
184 //变速变调
185 p_curr_sound->enable |= 0;
186 if (32000 != p_dec->sr) {
187     p_curr_sound = link_32000_sound(p_curr_sound, cbuff_o, (void **) sp_dec->src_effect, p_dec->sr, 32000);
188 } else {
189     void *src_tmp = src_hld_malloc();
190     src_release((void **) &src_tmp);
191     //log_info("data need src\n");
192 }

```

完整的浮点 EQ(audio_eq)

AD16N 内置硬件 EQ，最大可以配置 10 段 EQ 参数。

该内置硬件 EQ 可由 `app_modules_h.c` 里的 `AUDIO_HW_EQ_EN` 宏控制开关。

OTP 方案因存储空间限制，只支持单个简单 EQ 音效；

Flash 方案可支持完整的浮点 EQ 音效，支持多路 EQ 同时工作；

本章简要介绍完整的浮点 EQ 音效 (`audio_eq`) 及在最后小节简要说明新版本 EQ 使用流程。

1. “浮点 EQ 配置 (Flash 工程)”
2. “浮点 EQ 相关接口 (Flash 工程)”
3. “在线 EQ 调试助手使用一般流程”

25.1 浮点 EQ 配置 (Flash 工程)

一个浮点 EQ 音效由 EQ 系数表和总增益组成，其中：

1、EQ 系数表默认使用 10 段 EQ 参数，每一段可以单独配置滤波器 (`LOW_PASS`、`HIGH_PASS`、`BAND_PASS`)、中心频点 (20~20kHz)、增益 (-18~18dB) 以及 Q 值 (0.3~30.0)；

2、总增益默认配置范围为 -90~20dB，影响每一段 EQ 系数。

所有 EQ 模式统一由 `eq_type_tab[]` 数组管理，不同 EQ 之间可以任意切换。

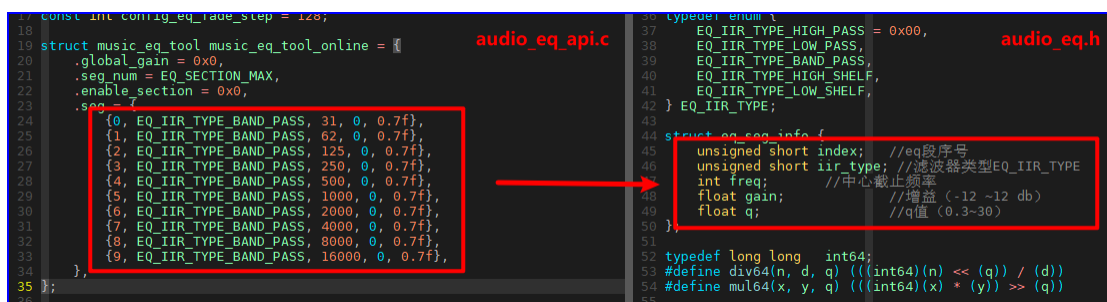


图 1 EQ 系数表对应关系

浮点 EQ 可同时运行多路运算，运算多路 EQ 比运算单路 EQ RAM 资源开销大；

const 变量 config_hw_single_eq = 1 时，仅运行单路 EQ 时，只消耗 344 字节 RAM 空间；

const 变量 config_hw_single_eq = 0 时，可运行多路 EQ，每一路 EQ 消耗 984 字节 RAM 空间；

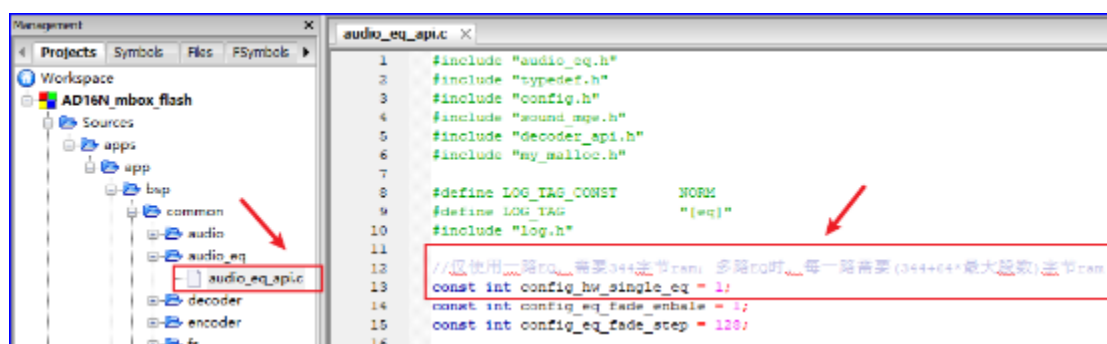


图 2 多路浮点 EQ 配置

(注意：浮点 EQ 在 ad16n_flash-release_v1.3.0 版本以及之后管理模式发生改动，下面简单说明新旧版本 EQ 模式管理区别。)

25.1.1 旧版本 EQ 模式管理

ad16n_flash-release_v1.2.1 版本及往前版本, 所有 EQ 模式统一由 eq_type_tab[] 数组管理, 不同 EQ 之间可以任意切换。

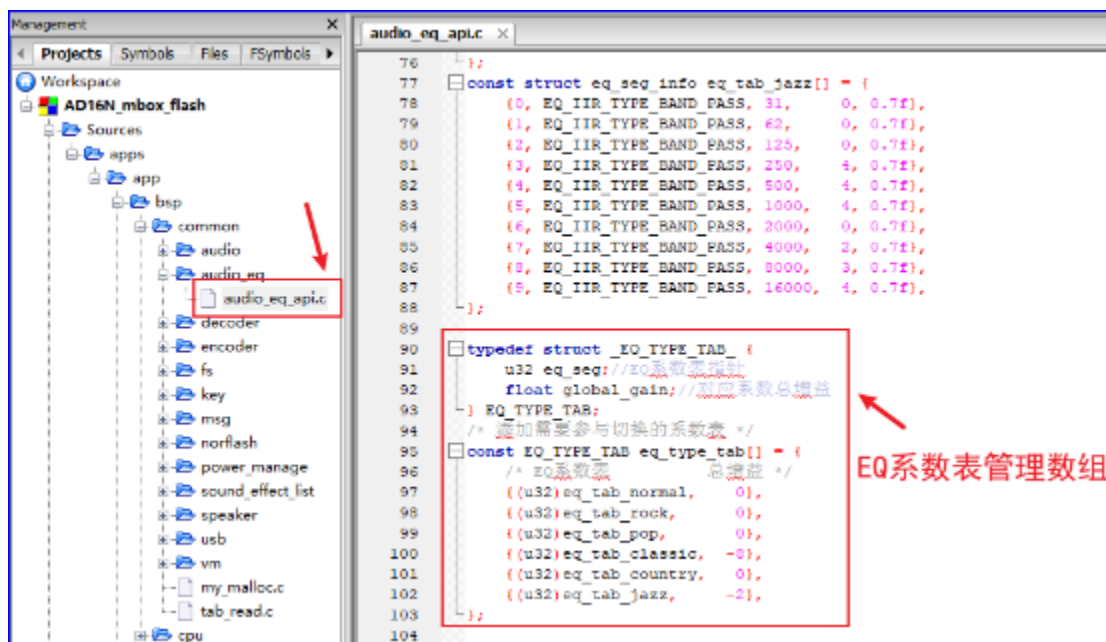


图 3 旧版本 EQ 模式由 eq_type_tab[] 数组管理

25.1.2 新版本 EQ 模式管理

ad16n_flash-release_v1.3.0 版本更新后, EQ 管理模式发生改动, 具体改动原理如下:

- 1、旧版本 EQ 模式切换是由 SDK 里的 eq_type_tab[] 数组管理;
 - 2、新版本 EQ 模式切换是采用 PC 工具的在线 EQ 调试助手生成的 eq_cfg.
- bin 文件进行管理, 方便客户调试 EQ 参数时听取效果;

新版本 EQ 使用时, 需要将工具生成的 bin 文件放置到下载目录里, 并重命名为 eq_cfg.bin, 打包成 dir_eq 文件下载到小机当中。

(注意: SDK 中默认识别 EQ 文件路径名为 dir_eq/eq_cfg.bin, 如果需要更改 bin 文件名可以修改 CFG_EQ_FILE 定义的路径。)

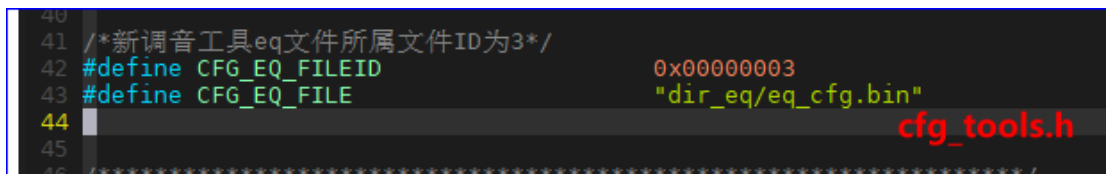


图 4 小机识别 bin 文件路径

目前公版 SDK 里的 eq_cfg.bin 文件已内含 6 中 EQ 模式, 分别为 normal、country、jazz、rock、pop、classic 模式。

具体使用示例可以阅读本章最后章节。

25.2 浮点 EQ 相关接口 (Flash 工程)

25.2.1 函数 void *link_eq_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u8 ch)

此函数只适用于 v1.3.0 版本及往后版本更新的 EQ 初始化。

25.2.2 函数 void *link_eq_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u8 ch)

此函数实现 EQ 模块的初始化。此函数主要实现移频啸叫抑制算法的启动，将抑制器链接到音频链路中：

- 1、p_sound_out：当前算法前级的音频通道句柄；
- 2、p_dac_cbuf：输出到DAC的OBUF；
- 3、pp_effect：存放当前音效算法的控制句柄指针的指针；
- 4、sr：当前输入音频的采样率；
- 5、ch：声道数；
- 6、返回值：启动之后最新的音频通道句柄；

25.2.3 函数 int eq_mode_sw(void *parm)

此函数实现切换下一个 EQ 模式，可在 EQ 运行中调用，其中参数：

- 1、parm：已启动的eq音效句柄；
- 2、返回值：
 - 1，切换失败，EQ未启动；
 - 0，切换成功；

25.2.4 函数 u8 get_eq_mode_total_num(void *parm)

此函数只适用于获取 v1.3.0 版本及往后版本更新的 eq_cfg.bin 文件里的 EQ 模式总数：

- 1、返回值：eq_cfg.bin文件中保存的EQ模式总数

25.2.5 函数 u8 get_eq_mode_total_num(void *parm)

此函数只适用于获取 v1.3.0 版本及往后版本更新的 eq_cfg.bin 文件里的 EQ 模式名字：

- 1、eq_mode：获取第几组EQ模式名字对应的序号；
- 2、返回值：对应序号的EQ模式名字指针；

25.3 在线 EQ 调试助手使用一般流程

下面介绍新版本 EQ 使用在线 EQ 调试助手调试出目标效果后搭配小机使用一般流程。

25.3.1 SDK 里开启 PC 模式和使用在线 EQ 调试工具宏定义

在 app_config.h 里使能 TCFG_PC_ENABLE 和 TCFG_CFG_TOOL_ENABLE。

(注意：开启在线 EQ 调试助手后，PC 模式下需要搭配 CDC 协议（CDC_CLASS）使用，此时与读卡器功能（MASSSTORAGE_CLASS）冲突）

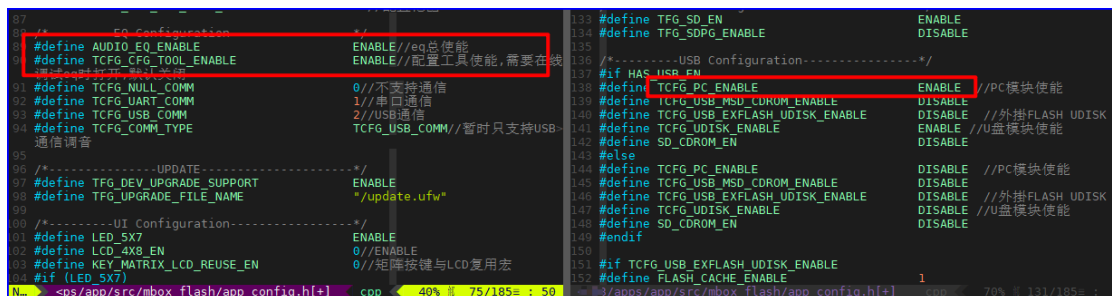


图 5 使用在线 EQ 调试助手前 app_config.h 配置

25.3.2 小机进入 PC 模式后打开使用 AD16N 在线 EQ 调试助手

AD16N 在线 EQ 调试助手工具放在了下载目录里的 eq_tool 文件夹里。

小机进入 PC 模式后，打开在线 EQ 调试工具，在设置界面中的端口选项找到并选中串行设备，打开后会自动跳入到音效调节界面。

常见问题和解决办法：

- 1、无法找到串行设备：检查是否打开上一小节图 5 中的 TCFG_EQ_TOOL_ENABLE 宏；

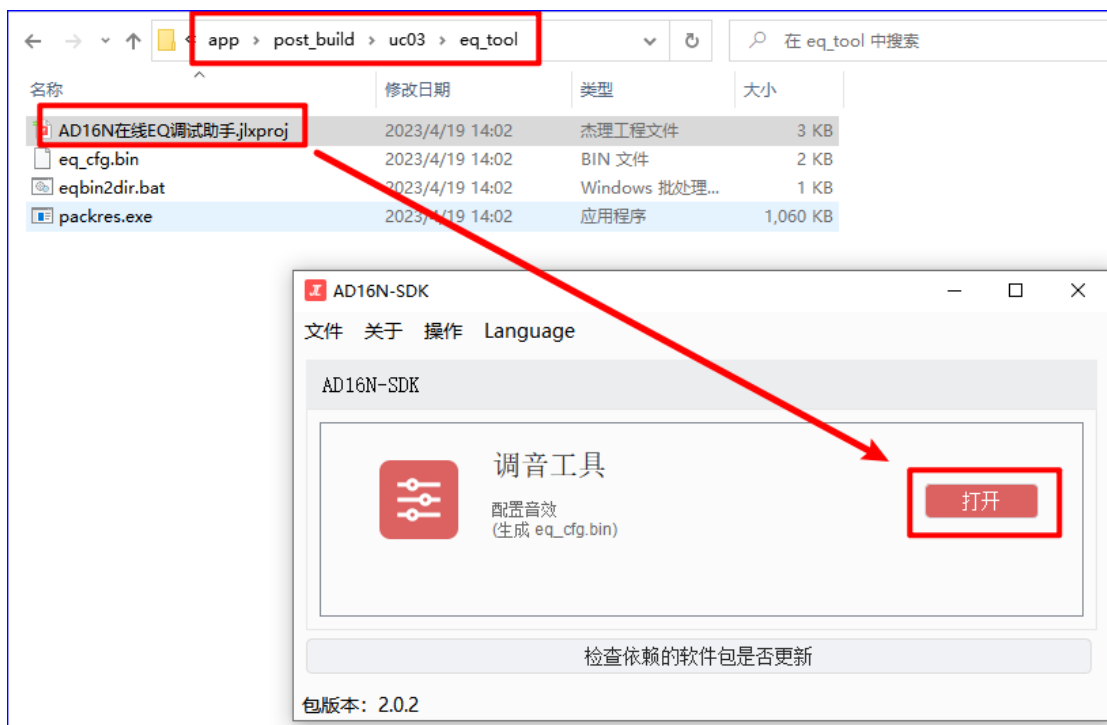


图 6 打开 AD16N 在线 EQ 调试助手工具



图 7 小机与工具进行连接

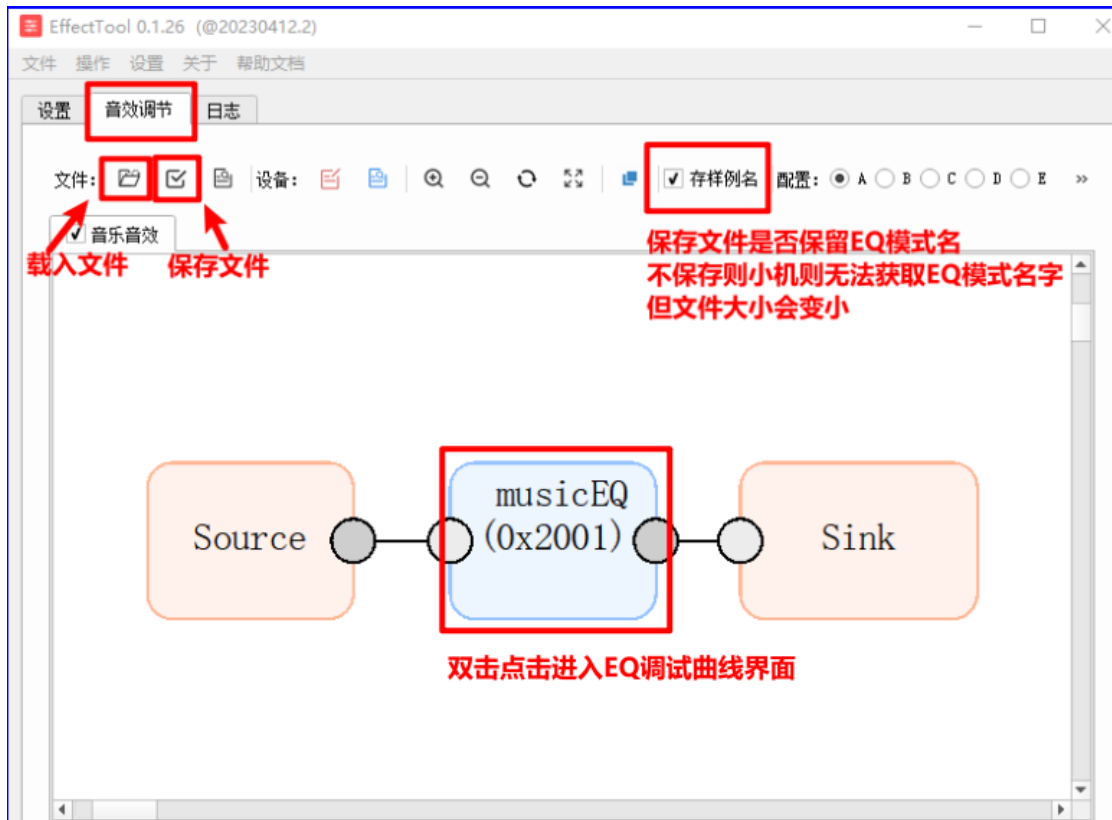


图 8 小机与工具成功连接后自动进入音效调节界面

25.3.3 在音效调节界面双击点击 musicEQ 进入在线调节 EQ 曲线界面

在音效调节界面左上角有帮助文档选项，有具体文档方便用户理解使用该工具。

双击点击进入 musicEQ 后，可以通过拉曲线的方式实时听取小机 EQ 效果；当调试好具体效果后可以点击保存样例，保存当前曲线。

退出 musicEQ 后，在音效调节界面选中保存文件，会生成一个.bin，里面包含了在 musicEQ 里保存样例的各组 EQ 模式。

（注意：SDK 固定最大支持 10 段 EQ（即工具中的 10 个序号）!!!）

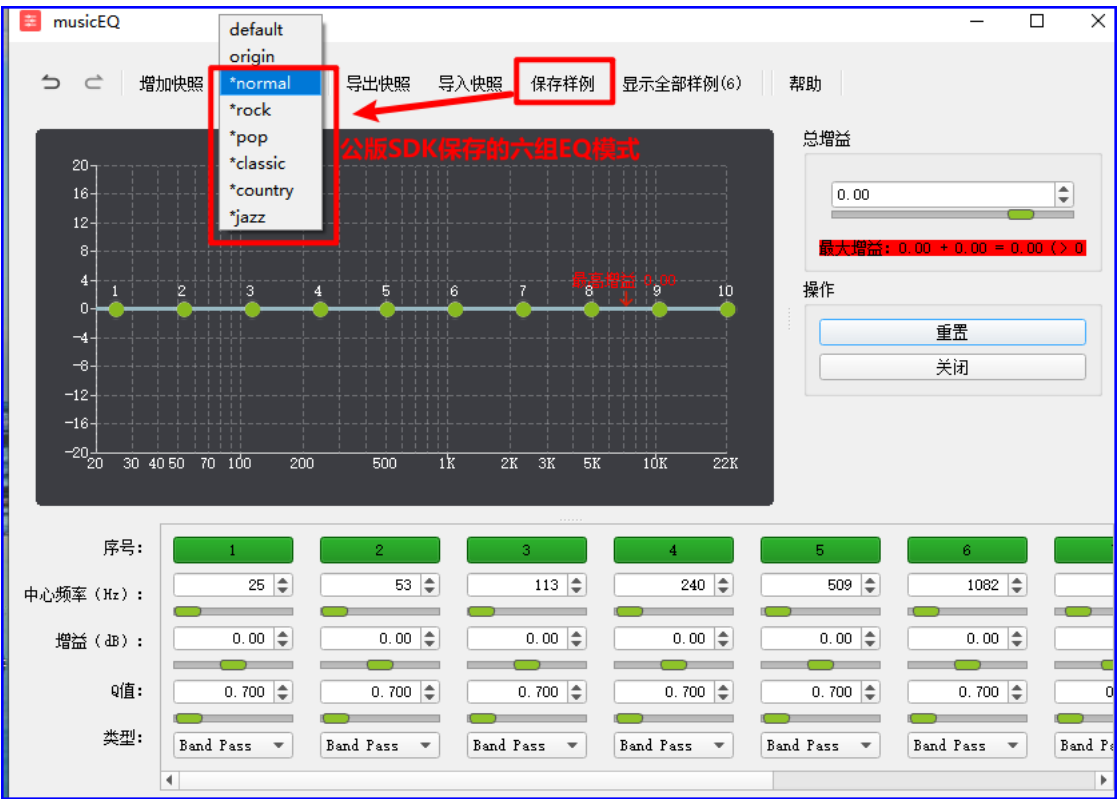


图 9 工具在线调试 EQ 曲线并保存该 EQ 曲线参数

25.3.4 把生成的.bin 文件下载到小机上

把上一小节生成的.bin 文件打包成 dir_eq 放在下载目录（即与 download_bat.c 同一目录下）当中，并把 dir_eq 加入到 download_bat.c 的资源文件下载当中，此时重新下载编译即可。

（注意：下载进入小机的.bin 文件需要命名为 eq_cfg.bin，打包文件需要命名为 dir_eq）

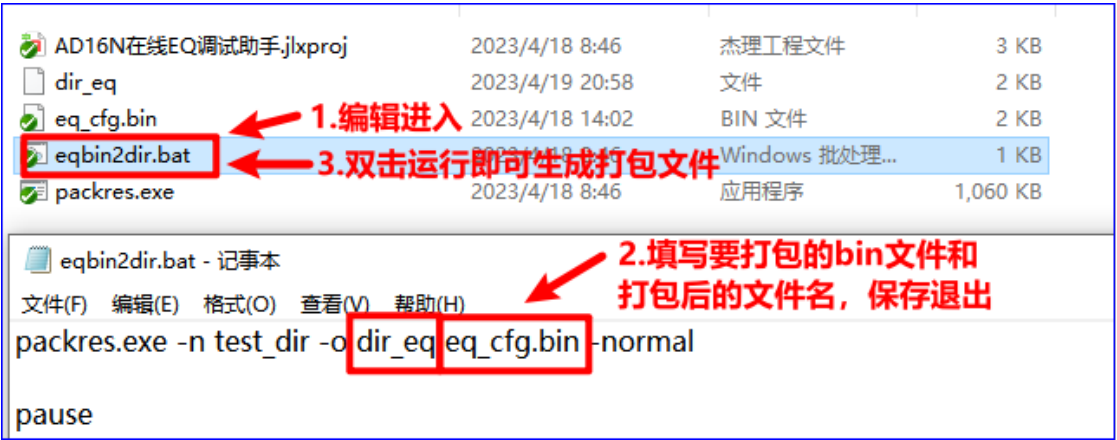


图 10 打包流程



```

set OBJCOPY=C:\JL\pi32\bin\llvm-objcopy.exe

%OBJDUMP% -d -print-imm-hex -print-dbg %ELF_NAME%.elf > %ELF_NAME%.lst
%OBJCOPY% -O binary -j .app_code %ELF_NAME%.elf %ELF_NAME%.bin
%OBJCOPY% -O binary -j .data %ELF_NAME%.elf data.bin
%OBJDUMP% -section-headers %ELF_NAME%.elf
copy /b %ELF_NAME%.bin+data.bin app.bin

@echo *****
@echo UC03 flash
@echo *****
@echo % date %
cd / d % ~dp0

isd_download.exe -tonorflash -dev uc03 -boot 0x101600 -div8 -wait 300 -otp usbboot.bin -uboot uboot.boot -app app.bin
0x2f000 -res dir_eq dir_a dir_song dir_eng dir_poetry dir_story dir_bin_f1x dir_notice dir_midi midi_cfg -output-fw
output_flash.fw

::-ex_flash // 外挂flash烧写命令

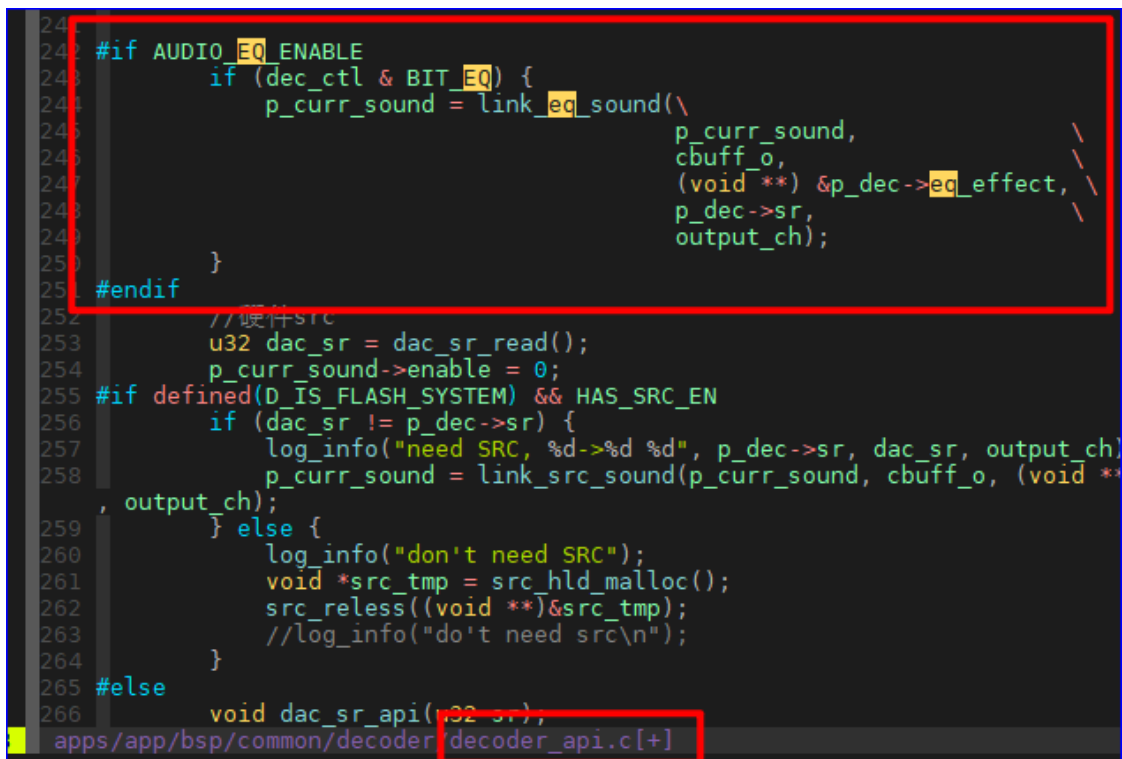
```

图 11 将 dir_eq 文件夹添加进下载目录中

25.3.5 SDK 具体使用 EQ 样例

不再使用调试工具后，SDK 里可以失能 TCFG_CFG_TOOL_ENABLE，以免 PC 模式下无法使用读卡器功能 (MASSSTORAGE_CLASS)。

在公版 SDK 里，音乐模式和 PC 模式都默认开启了 EQ；如在音乐模式下，只需要调用 eq_mode_sw() 函数即可自动切换在图 9 里调试好的 EQ 模式，并打印出切换后 EQ 模式的序号和其名字。



```

242 #if AUDIO_EQ_ENABLE
243     if (dec_ctl & BIT_EQ) {
244         p_curr_sound = link_eq_sound(
245             p_curr_sound,
246             cbuff_o,
247             (void **) &p_dec->eq_effect,
248             p_dec->sr,
249             output_ch);
250     }
251 #endif
252 //硬件SRC
253 u32 dac_sr = dac_sr_read();
254 p_curr_sound->enable = 0;
255 #if defined(D_IS_FLASH_SYSTEM) && HAS_SRC_EN
256     if (dac_sr != p_dec->sr) {
257         log_info("need SRC, %d->%d %d", p_dec->sr, dac_sr, output_ch);
258         p_curr_sound = link_src_sound(p_curr_sound, cbuff_o, (void **)
259             , output_ch);
260     } else {
261         log_info("don't need SRC");
262         void *src_tmp = src_hld_malloc();
263         src_reless((void **) &src_tmp);
264         //log_info("do't need src\n");
265     }
266 #else
267     void dac_sr_api(u32 sr);
268 apps/app/bsp/common/decoder/decoder_api.c[+]

```

图 12 音乐模式下解码器添加 BIT_EQ 选项后解码会连接 EQ

```
[Info] [dec_api]file name:so001.flb
[Info] [fla_api]fla_decode_api
[Info] [fla_api]file name : so001.flb
[Info] [fla_api]file is a flb
[Info] [fla_api]file sr : 32000

[Info] [dec_api]long 0:0
[Info] [dec_api]sr:32000
[Info] [dec_api]br:0
[Info] [dec_api]nch:1
[Info] [eq]eq malloc succ, hdl:0x1082f4 len:344

[Info] [dec_api]need SRC, 32000->48000 1
[Info] [dec_api]decode succ

[Info] [eq]now eq_mode is 1 , eq_mode_name is rock
[Info] [eq]now eq_mode is 2 , eq_mode_name is pop
[Info] [eq]now eq_mode is 3 , eq_mode_name is classic
[Info] [eq]now eq_mode is 4 , eq_mode_name is country
[Info] [eq]now eq_mode is 5 , eq_mode_name is jazz
[Info] [eq]now eq_mode is 0 , eq_mode_name is normal
```

播放歌曲

切换EQ模式

图 13 音乐模式下切换 EQ 模式

```

104 }
105
106 void music_app(void)
107 {
108     vm_write(VM_INDEX_SYSMODE, &work_mode, sizeof(work_mode));
109     u32 dac_sr = dac_sr_read();
110     dac_sr_api(48000);
111     u8 music_vol, dindex, used_device;
112     int msg[2], err;
113
114     music_info_init(&used_device);
115     post_msg(1, MSG_SEL_NEW_DEVICE);
116
117     while (1) {
118         err = get_msg(2, &msg[0]);
119         bsp_loop();
120
121         if (MSG_NO_ERROR != err) {
122             msg[0] = NO_MSG;
123             log_info("get msg err 0x%x\n", err);
124         }
125
126         switch (msg[0]) {
127 #if AUDIO_EQ_ENABLE
128             case MSG_EQ_SW:
129                 eq_mode_sw((void *)((pctl[0].p_dec_obj)->eq_effect));
130                 UI_menu(MENU_EQ);
131                 break;
132 #endif
133             case MSG_PP:

```

图 14 音乐模式下切换 EQ 模式打印

简单浮点 EQ (audio_eq)

AD16N 内置硬件 EQ，最大可以配置 10 段 EQ 参数。

该内置硬件 EQ 可由 app_modules_h.c 里的 AUDIO_HW_EQ_EN 宏控制开关。

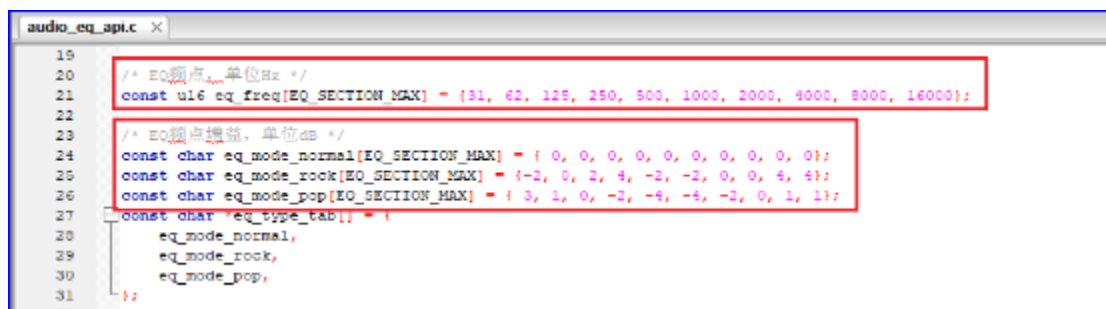
OTP 方案因存储空间限制，只支持单个简单 EQ 音效；

Flash 方案可支持完整的浮点 EQ 音效，支持多路 EQ 同时工作；

本章节介绍单个简单 EQ 音效。

26.1 简单 EQ 配置 (OTP 工程)

SDK 默认使用 10 段 EQ 参数，所有 EQ 模式中心频率统一，每一段 EQ 的增益可配。其中，中心频率由数组 eq_freq[EQ_SECTION_MAX] 来指定，影响所有 EQ 模式；增益则由用于自行定义数组来指定，并加入 EQ 配置列表 eq_type_tab[]。

The image shows a code editor window titled 'audio_eq_api.c'. It contains several lines of C code defining constants for EQ configuration. Lines 21-22 define 'eq_freq' as an array of 10 center frequencies. Lines 24-26 define 'eq_mode_normal', 'eq_mode_rock', and 'eq_mode_pop' as arrays of 10 gain values. Line 27 defines 'eq_type_tab' as an array of 3 elements, each being a pointer to one of the mode arrays. The code is as follows:

```
19
20
21 /* EQ频率, 单位Hz */
22 const ul6 eq_freq[EQ_SECTION_MAX] = {31, 62, 125, 250, 500, 1000, 2000, 4000, 8000, 16000};
23
24 /* EQ增益, 单位dB */
25 const char eq_mode_normal[EQ_SECTION_MAX] = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
26 const char eq_mode_rock[EQ_SECTION_MAX] = {-2, 0, 2, 4, -2, -2, 0, 0, 4, 4};
27 const char eq_mode_pop[EQ_SECTION_MAX] = { 3, 1, 0, -2, -4, -4, -2, 0, 1, 1};
28
29 const char *eq_type_tab[] = {
30     eq_mode_normal,
31     eq_mode_rock,
32     eq_mode_pop,
33 };
```

图 1.1 简单 EQ 配置

26.2 简单 EQ 相关接口 (OTP 工程)

26.2.1 函数 void audio_eq_init_api(void)

此函数实现 EQ 模块的初始化。

26.2.2 函数 void audio_eq_open_api(u8 ch, u16 sr)

此函数实现打开 EQ 模块，解码开始前调用一次，其中参数：

- 1、ch：需要进行EQ的声道数，默认为2；
 - 2、sr：需要进行EQ的采样率，根据文件的采样率设置；

26.2.3 函数 void audio_eq_switch_tab(void)

此函数实现根据 eq_type_tab[] 数组中有的 EQ 模式，切换到下一个 EQ 模式。

26.2.4 函数 void audio_eq_close_api(void)

此函数实现关闭 EQ 模块，解码结束后调用一次。

混响音效 (echo)

AD16N 和 AW30N SDK 均提供混响 (echo) 音效，该功能由 app_modules_h.c 里的 ECHO_EN 宏控制开关。

混响 (echo) 音效一般用于普通扩音，可配置干湿音、回声等效果；一般需要与啸叫抑制算法一起使用。

27.1 配置参数说明

- **DOWN_S_FLAG: 下采参数**

1 代表下采，0 代表不下采；下采申请的 buff 会较小，但是回声部分高频会丢失；

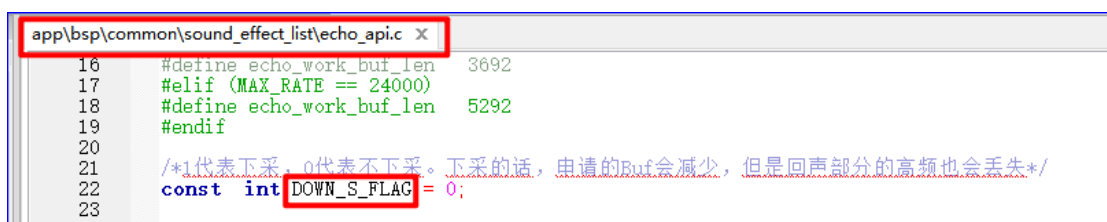


图 1 下采参数

- **buf_len: echo 运算需要的 buff 大小**

根据是否下采以及输入音频采样率决定，可通过 ops->need_buf() 接口获取大小；

```

app\bsp\common\sound_effect_list\echo_api.c X
117 void *echo_phy(void *obuf, ECHO_PARAM *param, void **ppsound)
118 {
119     u32 buf_len;
120     ECHO_FUNC_API *ops;
121     EFFECT_OBJ *echo_obj;
122     sound_in_obj *echo_si;
123     if (!param) {
124         log_error("echo param NULL\n");
125         return NULL;
126     }
127
128     ops = (ECHO_FUNC_API *)get_echo_func_api(); //接口获取
129     buf_len = ops->need_buf(NULL, (EF_REVERB_FIX_PARAM *)param->echo_fix_param); //运算空间获取
130     log_info("echo work buf len %d\n", buf_len);
131     memset(&echo_hdl_save, 0x0, sizeof(ECHO_HDL));
132     ECHO_HDL *echo_hdl = (ECHO_HDL *)&echo_hdl_save;

```

图 2 echo buff 大小获取

- echo 相关参数配置

```

app\bsp\common\sound_effect_list\echo_api.c X
23
24 u32 echo_work_buf[(echo_work_buf_len + 3) / 4] AT(.echo_data); //echo混响buf长度, 采样率改变, buf长度会变
25
26 void *echo_api(void *obuf, u32 sr, void **ppsound)
27 {
28     ECHO_PARAM eparam;
29     memset(&eparam, 0, sizeof(ECHO_PARAM));
30     log_info("echo api\n");
31     eparam.echo_param_obj.decayval = 60; // decay(0~70) 回声衰减比
32     eparam.echo_param_obj.delay = 200; // 回声延时0~300ms :范围0到max_ms, 超过max_ms的话, 会当成max_ms的
33     eparam.echo_param_obj.energy_vad_threshold = 512; // 能量阈值
34     eparam.echo_param_obj.direct_sound_enable = 1; // 如果运算结果需要叠加干声, 则这个置成1, 否则置0
35     eparam.echo_fix_param.max_ms = 100; // 最大延时100ms
36     eparam.echo_fix_param.sr = sr; // 采样率
37     eparam.echo_fix_param.wetgain = 3000; // 湿声增益
38     eparam.echo_fix_param.drygain = 4096; // 干声增益, 如果direct_sound_enable是0, 则这个参数无效, 因为不叠加干声了
39     eparam_ptr = echo_work_buf;
40     eparam_ptr_len = sizeof(echo_work_buf);
41
42     return echo_phy(obuf, &eparam, ppssound);
43 }
44
45

```

图 3 echo 参数配置

27.2 混响音效相关函数

27.2.1 函数 void *link_echo_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、sr：当前输入音频的采样率；
- 5、返回值：启动之后最新的音频通道句柄；

27.2.2 函数 void *echo_api(void *obuf, u32 sr, void **ppsound)

该函数用于初始化混响（echo）的相关参数，由函数 link_echo_sound() 调用即可，需要关注的其内部参数的初始化：

- 1、decayval：回升衰减比，范围：0~70；
- 2、delay：回声延时，范围：0~max_ms，超过max_ms会当场max_ms；
- 3、energy_vad_threshold：mute阈值，默认值为512；低于该能量值的声音输出静音；
→扩音方案中可以用该方式降低底噪；
- 4、direct_sound_enable：干声叠加使能位；
- 5、max_ms：最大回声延时时间，最大值为100；
- 6、sr：输入音频的采样率，支持16K / 24K，会影响混响算法运算buff的大小；
- 7、wetgain：湿声增益，默认为3000；
- 8、drygain：干声增益，默认为4096，干声叠加使能为0时不起作用；

移频啸叫抑制 (pitchshifter_howling)

AD16N 和 AW30N SDK 均提供自适应移频啸叫抑制 (pitchshifter_howling) 音效, 该功能由 app_modules.h 里的 PITCHSHIFT_HOWLING_EN 宏控制开关。

移频啸叫抑制目前与陷波啸叫抑制共同使用, 移频啸叫抑制的抑制效果由变量 FRESHIFT_SPEED_MODE_QUALITY 控制, 可配为 2 至 8 其中一个等级, 等级越高, 抑制效果越好, 但对速度要求越高。

28.1 配置参数说明

- **FRESHIFT_SPEED_MODE_QUALITY: 滤波器音阶数**
滤波器音阶数配置范围 [2, 8], 音阶数越大, 运算量越大;
- **howling_work_buf: 移频啸叫抑制运算需要的 buff**
根据移频方式和滤波器音阶数决定 buff 大小, 可通过 ops->need_buf() 接口获取大小;
- **phparm.fs_parm: 线性移频, 单位 Hz.**
移频方式要选择 FS, 移动多少 Hz 就配多少 Hz, 向高频移取正数, 向低频移取负数, 配置的采样率要和声音输入源的采样率一致。目前 fs_parm 支持的范围约为采样率的 0.15 倍, 比如 16K 采样率下, 移频范围是 [-2400, 2400]
- **phparm.effect_v: 移频方式**
可以为 EFFECT_HOWLING_PS 或者 EFFECT_HOWLING_FS, 或者 EFFECT_HOWLING_FS | EFFECT_HOWLING_PS;

```

22
23 void *pitchshift_howling_api(void *obuf, u32 sr, void **ppsound)
24 {
25     HOWLING_PITCHSHIFT_PARM phparam = {0};
26     /* 等比移频参数, -350到350, 归一化系数为8192 */
27     phparam.ps_parm          = -200;
28
29     /* 线性移频参数, -10到10, 单位Hz */
30     phparam.fs_parm          = 10;
31
32     /* 开关位, 可以为 EFFECT_HOWLING_PS, 或者 EFFECT_HOWLING_FS,
33     ! 或者 EFFECT_HOWLING_FS | EFFECT_HOWLING_PS */
34     phparam.effect_v         = EFFECT_HOWLING_PS;
35
36     apps/app/bsp/common/sound_effect_list/pitchshift_howling_api.c

```

移频啸叫抑制参数配置

图 1 移频啸叫抑制参数配置

28.2 相关函数

28.2.1 函数 void *link_pitchshift_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、sr：当前输入音频的采样率；
- 5、返回值：启动之后最新的音频通道句柄；

28.2.2 函数 void *pitchshift_howling_api(void *obuf, u32 sr, void **ppsound)

该函数用于初始化移频抑制啸叫（pitchshifter_howling）的相关参数，由函数 link_pitchshift_howling_sound() 调用即可，需要关注的其内部参数的初始化：

- 1、effect_v：移频方式，可选EFFECT_HOWLING_PS或EFFECT_HOWLING_FS；
- 2、ps_parm：等比移频，建议范围[-350, 350]，归一化系数为8192，effect_v置上EFFECT_HOWLING_PS有效；
- 3、fs_parm：线性移频，建议范围[-10, 10] (Hz)，effect_v置上EFFECT_HOWLING_FS有效；

自适应陷波啸叫抑制 (notch_howling)

AD16N 和 AW30N SDK 均提供自适应陷波啸叫抑制 (notch_howling) 音效, 该功能由 app_modules_h.c 里的 NOTCH_HOWLING_EN 宏控制开关。

自适应陷波抑制啸叫可以抑制改变环路反馈声音的频率, 环路反馈一直改变, 起到了平衡磨损的作用, 防止某个频率一直放大。

29.1 配置参数说明

-  **notch_howling_work_buf**: 自适应陷波啸叫抑制运算需要的 buff
根据配置参数决定 buff 大小, 可通过 ops->need_buf() 接口获取大小;

```
11
12 void *notch_howling_api(void *obuf, u32 sr, void **ppsound)
13 {
14     NotchHowlingParam nhparam = {0};
15     nhparam.gain               = (int)(-25.0 * (1 << 20)); //陷波器压制度, 越大防啸叫越好, 但发声啸叫频点误检时音质会更差
16     nhparam.Q                 = (int)(0.3 * (1 << 24)); //陷波器带宽, 越小防啸叫越好, 但发声啸叫频点误检时音质会更差
17     nhparam.fade_time         = 1000; //启动时间与释放时间, 越小启动与释放越快, 可能导致杂音出现且音质变差
18     nhparam.threshold         = (int)(20.0 * (1 << 15)); //频点啸叫判断阈值, 越小越容易判定啸叫频点, 但可能误检导致音质音质变差
19     nhparam.SampleRate        = sr;
20     return notch_howling_phy(obuf, &nhparam, ppsound);
21 }
22
23
24 apps/app/bsp/common/sound_effect_list/notch_howling_api.c
```

自适应陷波啸叫抑制参数配置

图 1 自适应陷波啸叫抑制参数配置

29.2 相关函数

29.2.1 函数 `void *link_notch_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)`

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、sr：当前输入音频的采样率；
- 5、返回值：启动之后最新的音频通道句柄；

29.2.2 函数 `void *notch_howling_api(void *obuf, u32 sr, void **ppsound)`

该函数用于初始化自适应陷波抑制啸叫（notch_howling）的相关参数，由函数link_notch_howling_sound() 调用即可，需要关注的其内部参数的初始化：

- 1、gain：陷波器压制程度，该值越大防啸叫效果越好，但发声啸叫频点误检时音质会更差；
- 2、Q：陷波器带宽，该值越小防啸叫效果越好，但发声啸叫频点误检时音质会更差；
- 3、fade_time：陷波器启动与释放时间，该值越小启动与释放越快，但可能导致杂音出现和音质变差；
- 4、threshold：频点啸叫判定阈值，该值越小防啸叫效果越好，但越容易误检；
- 5、SampleRate：输入音频的采样率；

变音音效 (vo_pitch)

AD16N 和 AW30N SDK 均提供变音音效 (vo_pitch) 音效，该功能由 app_modules_h.c 里的 VO_PITCH_EN 宏控制开关。

本节主要介绍变音音效，其包含以下几种音效：

- ① 两种机器人声；
- ② 两种变速变调；
- ③ 两种 rap；
- ④ 卡通音效；
- ⑤ 混响音效；

30.1 变音音效参数配置

```
34 typedef struct _VOICE_PITCH_PARA_STRUCT_ {
35     unsigned short do_flag;           //选择变声效果
36     unsigned short samplerate;        //输入音频采样率，支持16K/24K
37     unsigned short noise_dc;         //2048 //能量阈值
38     unsigned short pitchrate;        //音调高低，128代表原音调，<128音调升高，>128音调降低
39     unsigned short hamorrate;        //64-256, //音色调整。<128音色变尖，>128音色变沉
40     const unsigned char *midi_file;  //RAP模式的音源输入
41     unsigned short midifile_len;     //RAP模式的音源大小
42     void (*callback)(void *priv, int pos); //RAP模式的状态回调
43     void *priv;
44 } VOICE_PITCH_PARA_STRUCT;
apps/include_lib/liba/ARCH/pi32v2_lto_r1/header/vo_pitch_api.h
```

变音音效参数配置

图 1 变音音效参数配置

变音音效的参数配置如图 1 变音音效参数配置

但是在选择不同的变声效果时，在某些效果下有些参数无效，如图 2 所示

其他音效模式具体参照 SDK 中的文件 apps/app/bsp/common/sound_effect/vo_pitch_api.c 中各个音效参数配置函数的注释说明。

```

269 /***** 变调模式2 *****/
270 void vp_cmd_pitchshift2(VOICE_PITCH_PARA_STRUCT *p_vc_parm)
271 {
272     log_info("voice pitch pitchshift2\n");
273     p_vc_parm->do_flag = HARM0_PITCHSHIFT2;
274     p_vc_parm->samplerate = read_audio_adc_sr();
275     p_vc_parm->noise_dc = 2048;
276     p_vc_parm->pitchrate = 60;    //50到250
277     p_vc_parm->hamorrate = 90;    //50到250
278     //无效参数
279     p_vc_parm->midi_file = midifile_file0_tab;
280     p_vc_parm->midifile_len = sizeof(midifile_file0_tab);
281     p_vc_parm->callback = rap_callback;
282     p_vc_parm->priv = NULL;
283 }
284

```

apps/app/bsp/common/sound_effect_list/vo_pitch_api.c

图 2 变调模式参数配置

30.2 变音音效算法链路链接函数

SDK 提供下面的函数，传递不同的 CMD 参数可以实现不同的音效。

30.2.1 函数 `sound_out_obj *link_voice_pitch_sound(sound_out_obj *p_curr_sound, void *p_dac_cbuf, void **pp_effect, u32 cmd)`

此函数实现主要实现变音算法的启动，将变音算法链接到音频链路中；

- ① 返回值：启动之后最新的音频通道句柄；
- ② void *p_sound_out: 当前算法前级的音频通道句柄；
- ③ void *p_dac_cbuf: 输出到 DAC 的 OBUF；
- ④ void **pp_effect: 存放当前音效算法的控制句柄指针的指针；
- ⑤ u32 cmd: 音效控制指令；

```

typedef enum {
    VP_CMD_PITCHSHIFT = 0,    //变调音效
    VP_CMD_PITCHSHIFT2,      //变调音效2
    VP_CMD_ROBOT,             //机器人音效
    VP_CMD_ROBOT2,            //机器人音效2
    VP_CMD_RAP,               //RAP音效
    VP_CMD_RAP_REALTIME,      //RAP音效2
    VP_CMD_CARTOON,           //卡通音效
    VP_CMD_ECHO,              //混响音效
} VP_CMD;

```

图 4 音效控制指令

如果需要在解码应用中接入该音效，需要做以下几个步骤：

- 1、修改 ld 文件，防止音效算法与解码格式资源复用。
- 2、添加音效串联接口，可仿照 decoder_api.c 中，decoder_list 函数串联 song_speed 和 eq 的方法做音效串联。
- 3、其中 pp_effect 参数可根据需要选择传 NULL 或者传自定义的 effect 句柄(可用于动态调整参数)
- 4、如果有动态调参需求，可以在对应的音效 api 文件中查找动态调参接口（函数名一般为 update_ 音效名 _parm），将修改后的音效参数传入调参接口即可实现动态调参。

30.2.2 函数 void *voice_pitch_api(void *obuf, u32 cmd, void **ppsound)

该函数用于初始化变音音效相关参数，由函数 link_voice_pitch_sound() 调用即可；

该函数内部会调用函数 vp_cmd_case()，用于配置所选的变音音效以及其参数。

30.2.3 函数 void rap_callback(void *priv, int pos)

该函数为 RAP 模式的回调函数，由解码器调用。其中的参数 pos 表示 RAP 模式的状态；

RAP 模式初始化后，pos 会处于 RAP_PREPARE 状态等待输入音源；

当输入音源大于所设置的 noise_dc 能量阈值后，会触发 RAP 模式，此时 pos 会在 PAR_START 状态；

当 RAP 播放结束后，pos 会在 RAP_END 状态循环。若需要重置 RAP 状态，需调用函数 rap_reopen()。

30.2.4 函数 void rap_reopen(void)

该函数用于 RAP 模式播放结束后，让 RAP 重回 RAP_PREPARE 状态。

30.3 变音音效算法参数函数

通过配置不同的参数，可以实现不同的音效，其中关键的函数有以下：

- 1、音效参数选择函数：

```
bool vp_cmd_case(VP_CMD cmd, VOICE_PITCH_PARA_STRUCT *p_vc_parm);
```

- 2、rap 音效参数配置函数：

```
void vp_cmd_rap(p_vc_parm);
```

- 3、实时 rap 音效参数配置函数：

```
void vp_cmd_rap_realtime(VOICE_PITCH_PARA_STRUCT *p_vc_parm);
```

- 4、平调机器人音效参数配置函数：

```
void vp_cmd_robot(VOICE_PITCH_PARA_STRUCT *p_vc_parm);
```

- 5、变调机器人音效参数配置函数：

```
void vp_cmd_robot2(VOICE_PITCH_PARA_STRUCT *p_vc_parm);
```

6、变速变调音效参数配置函数；

```
void vp_cmd_pitchshift(VOICE_PITCH_PARA_STRUCT *p_vc_parm);
```

7、变速变调 2 音效参数配置函数；

```
void vp_cmd_pitchshift2(VOICE_PITCH_PARA_STRUCT *p_vc_parm);
```

8、卡通音效参数配置函数；

```
void vp_cmd_echo(VOICE_PITCH_PARA_STRUCT *p_vc_parm);
```

9、混响音效参数配置函数；

```
void vp_cmd_echo(VOICE_PITCH_PARA_STRUCT *p_vc_parm);
```

30.4 两种 RAP 音效的区别

RAP 与实时 RAP 的音效是一样的，都是在输入音频数据流有能量时进行一次变音；但是他们在使用控制时有较大的区别；

1、RAP 在调用启动后，会在输入的音频信号有能量时进行一次变音，再之后就会停止；下次启动变音需要调用回调函数，回调函数中调用 rap_reopen 可以再次启动 RAP；

2、实时 RAP 在启动后，变音会一直工作；

30.5 RAP 和 RAP_REALTIME 模式文件替换说明

用户可以通过替换该音效依赖的 midi 文件表来进行播出不同的音调，midi 文件表位于头文件 midi_file0_h.h。

```

1 #ifndef _MIDIFILE_FILE0_H_
2 #define _MIDIFILE_FILE0_H_
3
4
5 static const unsigned char midifile_file0_tab[] = {
6     3, 76, 74, 116, 80, 81, 74, 117, 26, 27, 76, 117, 52, 106, 74, 117, 53, 107, 79, 117, 54,
7     8, 78, 117, 106, 213, 74, 117, 54,
8     55, 74, 117, 53, 53, 76, 117, 53, 107, 74, 117, 53, 107, 81, 117, 53, 107, 79, 117, 54, 21
9     74, 117, 54,
10    55, 74, 117, 52, 53, 86, 117, 53, 107, 83, 117, 53, 107, 79, 117, 53, 107, 78, 117, 53, 10
11    76, 117, 54,
12    107, 84, 117, 54, 55, 84, 117, 52, 53, 83, 117, 53, 106, 79, 117, 54, 108, 83, 117, 53, 10
13    81, 117, 107,
14    214, 74, 117, 54
15 };
16
17 static const unsigned char midifile_file1_tab[] = {
18     6, 69, 76, 127, 32, 35, 75, 127, 33, 36, 76, 127, 34, 37, 75, 127, 34, 35, 76, 127, 34, 37
19     71, 127, 34, 37, 74, 127, 33,
20     38, 72, 127, 33, 35, 69, 127, 106, 2, 57, 112, 73, 0, 60, 127, 36, 36, 64, 112, 216, 38, 6
21     112, 146,
22     36, 60, 127, 33, 35, 64, 127, 33, 36, 69, 127, 34, 39, 71, 127, 102, 1, 52, 112, 72, 35, 6
23     112, 184,
24     39, 68, 112, 145, 35, 64, 127, 33, 37, 68, 127, 33, 36, 71, 127, 34, 37, 72, 127, 102, 1,
25     , 112, 72,
26     36, 64, 112, 183, 38, 69, 112, 214, 35, 64, 127, 34, 35, 76, 127, 34, 37, 75, 127, 33, 38,
27     6, 127, 34,
28     37, 75, 127, 33, 35, 76, 127, 34, 38, 71, 127, 34, 35, 74, 127, 34, 39, 72, 127, 33, 36, 6
29     127, 110,
30     0, 57, 112, 74, 37, 64, 112, 203, 38, 69, 112, 145, 34, 60, 127, 33, 37, 64, 127, 33, 36,
31     , 127, 34,
32 };
33
34 static const unsigned char midifile_file2_tab[] = {
35     8, 67, 79, 110, 128, 0, 67, 70, 42, 42, 62, 70, 43, 43, 67, 70, 43, 43, 79, 110, 171, 0, 7
36     70, 43, 43, 67, 70, 43,
37     43, 62, 70, 42, 43, 55, 70, 255, 42, 78, 110, 43, 43, 79, 110, 43, 43, 81, 110, 43, 43, 79
38     110, 43,
39     43, 76, 110, 43, 43, 79, 110, 128, 0, 71, 70, 42, 43, 59, 70, 42, 42, 64, 70, 43, 43, 79,
40     0, 171,
41     0, 67, 70, 43, 43, 64, 70, 43, 43, 59, 70, 43, 43, 55, 70, 255, 43, 78, 110, 42, 43, 79, 1
42     , 42,
43     42, 81, 110, 43, 43, 79, 110, 43, 43, 74, 110, 43, 43, 76, 110, 128, 0, 67, 70, 43, 43, 64
44     70, 42,
45     43, 67, 70, 85, 42, 76, 110, 172, 43, 67, 70, 43, 43, 64, 70, 43, 43, 60, 70, 255, 43, 72,
46     10, 43,
47     43, 74, 110, 42, 43, 76, 110, 42, 42, 74, 110, 43, 43, 72, 110, 43, 43, 71, 110, 43, 0, 59
48     70, 255,
49     43, 66, 110, 43, 43, 71, 110, 42, 43, 74, 110, 42, 43, 71, 110, 42, 42, 66, 110, 43, 43, 5
50     70, 43,
51     43, 59, 70, 43, 43, 62, 70, 43, 43, 71, 70, 42, 43, 59, 70, 42, 42, 74, 70, 43, 43, 74, 11
52     255,
53     0, 55, 70, 43, 43, 57, 70, 43, 43, 60, 70, 43, 43, 69, 70, 43, 43, 64, 70, 42, 43, 67, 70,
54     2,
55     42, 72, 110, 86, 0, 69, 70, 129, 86, 74, 110, 43, 43, 76, 110, 85, 0, 60, 70, 128, 86, 78,
56     10, 42,

```

NORMAL | OpenSource_AD1... include_lib/vo_pitch/midi_file0.h.cpp 5% 16/283 :10

用户可以通过 get_miditabs 工具生成 midi 文件表，首先将需要使用的 midi 文件表放入工具目录，然后点击 t.bat 生成头文件

事先放入的midi文件 工具转换生成的头文件，内含midi文件表

名称	修改日期	类型	大小
cut_midlib.exe	2021/6/10 14:10	应用程序	142 KB
daoxiang.mid	2022/9/9 15:39	MIDI 序列	10 KB
midi_file0.h	2023/8/24 9:43	Header file	15 KB
midilist.txt	2023/8/24 9:43	文本文档	1 KB
msvcr100d.dll	2019/4/22 14:47	应用程序扩展	1,464 KB
t.bat	2021/6/9 16:09	Windows 批处理...	1 KB
vcruntime140.dll	2019/4/22 14:47	应用程序扩展	84 KB
说明.txt	2021/6/9 16:51	文本文档	1 KB

打开头文件可以看到生成的 midi 文件表

```

1  #ifndef _MIDIFILE_FILE0_H_
2  #define _MIDIFILE_FILE0_H_
3  static const unsigned char midifile_file3_tab[]={
4      8, 61, 45, 79, 38, 42, 52, 64, 39, 43, 64, 69, 77, 0, 59, 72, 19, 22, 61, 64, 19, 21, 57, 65, 39, 43, 40, 71, 38,
5      43, 52, 61, 38, 43, 64, 69, 77, 0, 56, 67, 77, 85, 42, 76, 39, 43, 52, 62, 39, 43, 64, 70, 77,
6      0, 59, 73, 39, 43, 57, 62, 39, 43, 49, 71, 38, 43, 59, 62, 38, 43, 64, 71, 77, 0, 61, 66, 77,
7      85, 47, 72, 39, 43, 54, 61, 39, 43, 64, 67, 77, 0, 57, 70, 77, 86, 40, 74, 38, 43, 52, 62, 38,
8      43, 64, 69, 77, 0, 57, 71, 77, 85, 45, 71, 39, 43, 52, 58, 39, 43, 64, 69, 154, 0, 59, 71, 39,
9      43, 57, 58, 19, 21, 59, 59, 39, 43, 57, 67, 20, 22, 52, 63, 38, 43, 59, 69, 38, 42, 64, 68, 43,
10     0, 52, 63, 39, 43, 45, 81, 39, 1, 69, 76, 42, 42, 52, 62, 39, 0, 73, 80, 43, 43, 64, 60, 20,
11     0, 76, 87, 129, 0, 59, 60, 19, 22, 61, 63, 19, 21, 57, 66, 39, 43, 40, 74, 38, 43, 52, 60, 38,
12     0, 71, 81, 43, 43, 64, 59, 38, 0, 68, 74, 43, 0, 56, 59, 38, 43, 52, 60, 38, 0, 64, 71, 42,
13     42, 42, 73, 39, 1, 69, 70, 42, 42, 49, 60, 39, 0, 73, 79, 43, 43, 59, 58, 39, 0, 76, 86, 129,
14     0, 64, 58, 39, 43, 57, 62, 39, 43, 49, 70, 38, 43, 56, 62, 38, 0, 71, 83, 43, 43, 61, 58, 38,
15     0, 68, 76, 43, 0, 64, 58, 38, 43, 59, 61, 38, 0, 64, 72, 42, 42, 47, 71, 39, 1, 69, 78, 115,
16     42, 54, 66, 39, 43, 62, 59, 39, 0, 57, 59, 39, 43, 64, 67, 43, 0, 54, 65, 39, 43, 40, 70, 38,
17     0, 69, 75, 116, 43, 52, 63, 38, 43, 64, 59, 38, 0, 57, 59, 38, 43, 64, 58, 42, 0, 52, 61, 38,
18     42, 45, 70, 39, 1, 61, 64, 231, 10, 64, 64, 114, 11, 69, 64, 255, 21, 52, 63, 39, 43, 64, 60, 39,
19     0, 59, 60, 39, 43, 57, 66, 19, 21, 59, 64, 39, 43, 57, 68, 20, 22, 52, 63, 38, 43, 59, 64, 38,
20     21, 69, 81, 22, 22, 57, 58, 38, 0, 69, 78, 21, 21, 66, 73, 21, 21, 45, 79, 39, 1, 69, 77, 42,
21     42, 52, 61, 39, 0, 66, 69, 22, 22, 69, 71, 43, 21, 57, 66, 39, 22, 66, 72, 21, 21, 52, 61, 39,
22     0, 69, 76, 21, 22, 66, 72, 21, 21, 44, 79, 39, 0, 71, 75, 21, 22, 71, 74, 21, 21, 52, 58, 38,
23     0, 71, 70, 21, 21, 71, 66, 22, 22, 56, 66, 38, 0, 73, 74, 43, 43, 52, 58, 38, 0, 69, 73, 38,
24     42, 42, 76, 39, 1, 69, 75, 42, 42, 49, 61, 39, 0, 66, 69, 22, 22, 69, 66, 43, 21, 54, 67, 39,
25     22, 66, 67, 21, 21, 49, 59, 39, 0, 69, 71, 21, 22, 66, 73, 21, 21, 40, 74, 39, 0, 71, 73, 21,
26     22, 71, 71, 21, 21, 49, 58, 38, 0, 71, 69, 21, 21, 71, 67, 22, 22, 52, 65, 38, 0, 73, 72, 43,
27     43, 49, 58, 38, 0, 69, 72, 38, 42, 47, 74, 39, 1, 69, 73, 42, 42, 54, 56, 39, 0, 66, 71, 22

```

最后将生成的文件表复制到 SDK 的头文件 midi_file0.h，然后在函数初始化设置配置文件表，图中以 rap_realtime 模式为例

```

182 void vp_cmd_rap_realtime(VOICE_PITCH_PARA_STRUCT *p_vc_parm)
183 {
184     p_vc_parm->do_flag = HARMO_RAP_REALTIME ;
185     p_vc_parm->noise_dc = 2048;
186     p_vc_parm->samplerate = read_audio_adc_sr();
187     p_vc_parm->hamorrage = 128; //改变音色, 50到250
188     p_vc_parm->pitchrate = 128; //改变音高, 50到250
189     p_vc_parm->midi_file = midifile_file3_tab; //改变旋律
190     p_vc_parm->midifile_len = sizeof(midifile_file3_tab);
191     //无效参数:
192     p_vc_parm->callback = rap_callback;
193     p_vc_parm->priv = NULL;
194 }
195

```

填入使用的文件表

实时变速变调 (SPEED, 用于扩音)

AD16N 和 AW30N SDK 均提供实时变速变调 (SPEED, 用于扩音) 音效, 该功能由 `app_modules.h.c` 里的 `HAS_SPEED_EN` 宏控制开关。

实时 `speed` 算法与 `song_speed` 算法功能基本一致, 但侧重点与运算方法不一致, `Speed` 运算量较小, 实时性好, 适用于 `mic` 直通喇叭等扩音应用此场景。在扩音场景下, `Speed` 音效可实时修改变调音效。

31.1 配置参数

- **Insample**: 输入音频采样率, 与 `audio_adc` 采样率一致即可;
- **Pitchrate**: 变调比例, 用于调节变调效果, 需要与 `const` 变量搭配使用;

当 `const` 变量 `SPITCH_PARM_PR_HResolution = 0` 时, `pitchrate = 128` 为不变调; 小于 128 时音调变高; 大于 128 时音调变高;

当 `const` 变量 `SPITCH_PARM_PR_HResolution = 1` 时, `pitchrate = 32768` 为不变调; 小于 32768 时音调变高; 大于 32768 时音调变高;

- **Speedin / Speedout**: 变速比例, 扩音应用下默认为 1;
- **Quality**: 变调运算质量, 与运算速度成正比, 配置范围 3~8;

```

16 SPEED_PITCH_PARA_STRUCT sp_parm AT(.sp_data);
17
18 const int SPITCH_PARM_PR_HResolution = 0; //pitchrate精度调节
19
20 void *speed_api(void *obuf, u32 insample, void **ppsound)
21 {
22     log_info("speed_api\n");
23
24     /* 输入音频采样率 */
25     sp_parm.insample = insample;
26
27     if (SPITCH_PARM_PR_HResolution) {
28         /* 变调比例, 32768:为不变调, <32768:音调变高, >32768:音调变低, 声音更沉 */
29         sp_parm.pitchrate = 32768;
30     } else {
31         /* 变调比例, 128:为不变调, <128:音调变高, >128:音调变低, 声音更沉 */
32         sp_parm.pitchrate = 100;
33     }
34
35     /* speedout/speedin 的结果大于0.6小于1.8,变快变慢最好都不要超过2倍 */
36     /* sp_parm.speedin: sp_parm.speedout为变速比例, 例如speedin=1;speedout=2;则为变慢1倍 */
37     sp_parm.speedin = 1;
38     sp_parm.speedout = 1;
39
40     /* 变调运算的运算质量, 跟速度成正比, 配置范围3-8 */
41     sp_parm.quality = 4;
42
43     /* 如果pitchflag=1,就是输入输出速度只由speedin: speedout决定,调节音调的时候, 速度比例保持不变。 */
44     /* 例如设置speedout: speedin=1:2, 这时候, 去调节pitchrate, 都是只变调, 速度一直是保持放慢1倍。 */
45     /* 如果pitchflag=0, 就是变速变调比例分开控制, 这样实际变速的比例是 (speedout/speedin)*pitchrate了。 */
46     sp_parm.pitchflag = 1;
47
48     return speed_phy(obuf, &sp_parm, pp_sound);
49 }

```

实时Speed参数初始化

apps/app/bsp/common/sound_effect_list/speed_api.c

图 1 实时变速变调参数初始化

31.2 相关函数

31.2.1 函数 void *link_speed_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 in_sr)

该函数为串联音频链路的标准接口, 实现响应算法的启动, 并添加到音频链路上, 其中参数::

- ① p_sound_out: 当前算法的前级音频通道句柄;
- ② p_dac_cbuf: 输出的音频buff;
- ③ pp_effect: 存放当前音效算法控制句柄指针的指针;
- ④ sr: 当前输入音频的采样率;
- ⑤ 返回值: 启动之后最新的音频通道句柄;

歌曲变速变调 (SONG_SPEED, 用于解码)

AD16N 和 AW30N SDK 均提供混响 (echo) 音效, 该功能由 app_modules.h.c 里的 HAS_SONG_SPEED_EN 宏控制开关。

song_speed 一般用于歌曲变速变调, 支持单双声道。在 96M 时钟下使用 song_speed 解码 48K mp3 时, 需要使用快速计算模式。计算模式由变量 PSFAST_FLAG 进行配置, 使用快速计算模式时, BUF 需要增大 256 个字节。

32.1 配置参数

```

13 #define SONG_SPEED_FAST      1
14 #if SONG_SPEED_FAST
15 const int PSFAST_FLAG = 1;    //0:普通模式      1:快速模式(需要增加buf大小)
16 const int PSADD_BUF = 1;     //0:不增加buf大小  1:增加256个byte, 用于快速模式
17 u8 buflen[0x147c] AT(.song_sp_data);
18 #else
19 const int PSFAST_FLAG = 0;
20 const int PSADD_BUF = 0;
21 u8 buflen[0x137c] AT(.song_sp_data);
22 #endif
23
24 PS69_CONTEXT_CONF ps_parm AT(.song_sp_data);
25
26 void *song_speed_api(void *obuf, u16 ch, u32 insample, void **ppsound)
27 {
28     ps_parm.chn    = ch;        //输入音源的声道信息
29     ps_parm.sr     = insample;  //输入音源的采样率信息
30     ps_parm.speedV = 120;       //>80是变快, <80 是变慢, 建议范围: 40到130
31     ps_parm.pitchV = 45000;     //>32768是音调变高, <32768音调变低, 建议范围20000到50000
32     return song_speed_phy(obuf, &ps_parm, ppsound);
33 }

```

Song_Speed参数配置和初始化

apps/app/bsp/common/sound_effect_list/song_speed_api.c

图 1 歌曲变速变调参数配置和初始化

32.2 相关函数

32.2.1 函数 `void *link_song_speed_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 in_sr, u8 ch)`

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

- | ① 返回值：启动之后最新的音频通道句柄；
- | ② `void *p_sound_out`：当前算法前级的音频通道句柄；
- | ③ `void *p_dac_cbuf`：输出到DAC的OBUF；
- | ④ `void **pp_effect`：存放当前音效算法的控制句柄指针的指针；
- | ⑤ `u32 sr`：当前输入音频的采样率；
- | ⑥ `u8 ch`：当前使用的声道数；

32.2.2 函数 `void *song_speed_phy(void *obuf, PS69_CONTEXT_CONF *psp_parm, void **ppsound)`

- ① 返回值：当前算法的音频通道句柄的指针；
- ② `void *obuf`：输出到DAC的OBUF；
- ③ `PS69_CONTEXT_CONF *psp_parm`：音效参数；
- ④ `void **ppsound`：当前算法的音频通道句柄；

混合变音音效 (voicechanger)

SDK 中提供 voicechanger 变音音效，可自行调整参数做到不同的效果，发出不同的人声或者鸟叫声。

33.1 可选音效

- EFFECT_VC_AV_PITCHSHIFT: 人声变音 1
- EFFECT_VC_F0_TD: 人声变音 2
- EFFECT_VC_AV_F0_TD_AUTOFIT: 人声变音 3
- EFFECT_VC_AV_PITCHSHIFT_AUTOFIT: 人声变音 4
- EFFECT_VC_AV_BIRD: 鸟叫声
- EFFECT_VC_AV_BIRD1: 鸟叫声 1
- EFFECT_VC_AV_BIRD2: 鸟叫声 2
- EFFECT_VC_AV_BIRD3: 鸟叫声 3
- EFFECT_VC_AV_BIRD4: 鸟叫声 4
- EFFECT_VC_AV_BIRD5: 鸟叫声 5
- EFFECT_VC_AV_BIRD6: 鸟叫声 6

其中 EFFECT_VC_AV_PITCHSHIFT 跟 EFFECT_VC_AV_F0_TD 是 2 种变声算法

EFFECT_VC_AV_PITCHSHIFT 中 formant_shift 参数无效；EFFECT_VC_AV_F0_TD 中可以通过组合不同的 shiftv 跟 formant_shift 调出不同的效果。

例如变女声可以设置成 formant_shift=80;shiftv=60;

EFFECT_VC_AV_F0_TD_AUTOFIT 跟 EFFECT_VC_AV_PITCHSHIFT_AUTOFIT 是会检测声音。

例如如果是童音即使 pitch 设高，也不要调高那么多。

EFFECT_VC_AV_BIRD 到 EFFECT_VC_AV_BIRD6 是把人声转换成鸟声，例如人声唱一段旋律，然后转换出来对应的鸟叫声旋律。其中 pitch 会改变鸟声的整体音高，鸟叫建议用 EFFECT_VC_AV_BIRD5 或 EFFECT_VC_AV_BIRD6

33.2 配置参数说明

- const int VC_NG_THRES: 能量阈值，低于这个能量的声音会被 mute 掉，建议默认设成 0，根据实际底噪水平设置。

voicechanger 变音音效的参数由两个结构体管理

结构体 VOICECHANGER_AV_PARM

- shiftv 调小，音调变高，变声模式建议范围 50 到 200，鸟叫声模式建议范围 30 到 100
- formant 在 EFFECT_VC_AV_F0_TD,EFFECT_VC_AV_F0_TD_AUTOFIT 模式下，调小音色变亮，调小音色变沉。范围 50 到 200
- speedv 变速参数，范围 50 到 200. 如果是实时变声，ad 直接变声到 da 的，因为进出数据量需要一致。这时候 speedv 必须设成 100，不然会有杂音。

下面参数是 EFFECT_VC_AV_BIRD5 跟 EFFECT_VC_AV_BIRD6 模式下是调节颤音用的

结构体 VOICESYN_AV_PARM

- vibrate_rate_u 范围 0-100，非 0 代表开启颤音，0 代表关闭
- vibrate_rate_d 范围 0-100，影响颤音最大幅度，越大颤音幅度越大（BIRD6 无效）
- vibrate_lenCtrl 范围 0-100，颤音衰减速度，影响颤音持续时长，100 代表最短，0 代表最长
- randpercent 有效范围 50-100 【或者 0 到 50，镜像对称】，微调音色的（BIRD6 无效）

33.3 相关函数接口

33.3.1 函数 void *link_voice_changer_sound(void *p_sound_out, void p_dac_cbuf, void **pp_effect, u32 in_sr)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

- 1、p_sound_out：当前算法的前级音频通道句柄；
- 2、p_dac_cbuf：输出的音频buff；
- 3、pp_effect：存放当前音效算法控制句柄指针的指针；
- 4、in_sr：采样率；
- 5、返回值：启动之后最新的音频通道句柄；

如果需要在解码应用中接入该音效，需要做以下几个步骤：

- 1、修改 ld 文件，防止音效算法与解码格式资源复用。
- 2、添加音效串联接口，可仿照 decoder_api.c 中，decoder_list 函数串联 song_speed 和 eq 的方法做音效串联。
- 3、其中 pp_effect 参数可根据需要选择传 NULL 或者传自定义的 effect 句柄 (可用于动态调整参数)
- 4、如果有

动态调参需求，可以在对应的音效 api 文件中查找动态调参接口（函数名一般为 update_ 音效名 _parm），将修改后的音效参数传入调参接口即可实现动态调参。

33.3.2 函数 void *voice_changer_api(void *obuf, u32 sr, void **ppsound)

该函数用于初始化变音音效相关参数，由函数 link_voice_changer_sound() 调用即可；

33.3.3 函 数 void update_voice_changer_parm(VOICECHANGER_AV_PARM *new_vc_parm, VOICESYN_AV_PARM *new_vsyn_ctrol)

该函数用于运行 voicechanger 变音音效时，动态调整参数，使用时注意采样率要和之前一致，其中参数：

- 1、new_vc_parm：修改配置后新的参数结构体1
- 2、new_vsyn_ctrol：修改配置后新的参数结构体2

音频降噪算法 (ANS)

AW30N 从 V1.4.0 版本开始支持音频降噪算法 (缩写: ANS)。

SDK 默认关闭音频降噪算法 (缩写: ANS), 用户需在 `app_modules.h` 打开。

使用音频降噪算法 (缩写: ANS) 需要注意以下配置:

1. 输入给降噪算法数据的采样率 (最好为 16k 采样率)
2. 运行降噪算法时的系统时钟需要在 107M 以上
3. 在链接文件里 (`app_ld.c`) 降噪算法使用的 `ans_data` 段与其他功能的复用, 确认在使用降噪算法时, 该空间不会被其他功能干扰。



```
104 #define ENCODER_JLA_LW_EN 1
105
106 // MIO功能使能
107 #define HAS_MIO_EN 0
108
109 // ANS功能使能
110 #define ANS_EN 0
111
112 // 实时SPEED功能(用于扩音)使能
113 #define HAS_SPEED_EN 1
114
115 // 歌曲SPEED功能(用于解码)使能
116 #define HAS_SONG_SPEED_EN 1
117
118 // 陷波/移频啸叫抑制使能(二选一)
```

apps/app/src/mbox_flash/bd49/app_modules.h

音频降噪宏

音频降噪算法 (缩写: ANS) 的使用可以参考下方 [gitee](#) 链接:

FAQ: 音频降噪算法的使用

34.1 涉及文件

源文件:

ans_api.c

头文件:

ans_api.h

NoiseSuppressLib.h

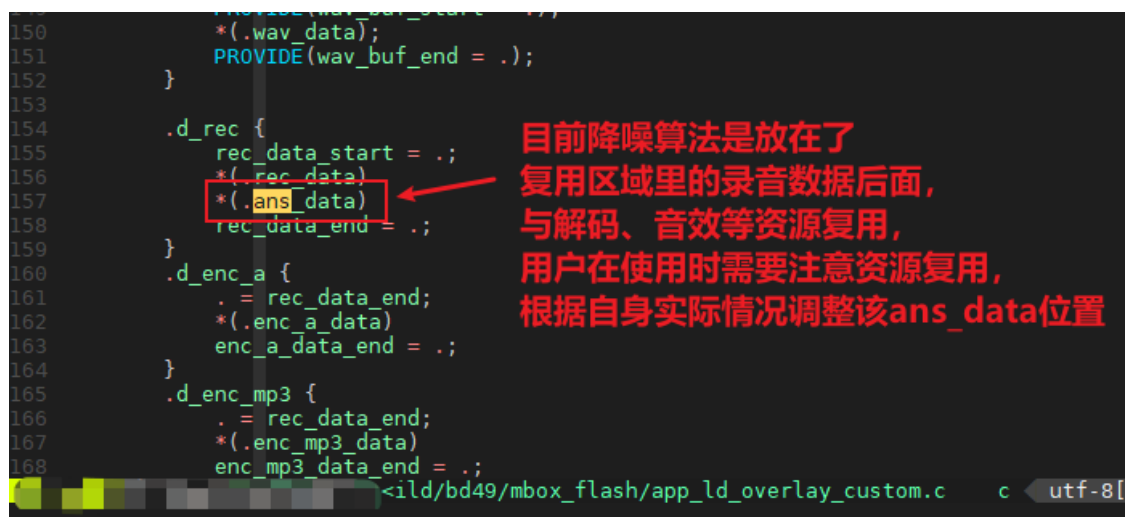
库文件:

libNoiseSuppress_pi32_OnChip.a

lib_SW_FFT_pi32_OnChip.a

链接文件:

app_ld.C 或 app_ld_overlay_custom.c 文件



降噪算法在链接文件复用位置

SDK 在录音模式已加入音频降噪算法。用户可作参考。

```

1 encoder_api.c
20 #include "log.h"
21
22 #if (defined(ANS_EN) && (ANS_EN))
23 #include "ans_api.h"
24 cbuffer_t cbuf_ans AT(.ans_data);
25 u32 ans_buff[512 / 4] AT(.ans_data);
26 #endif
27
28 cbuffer_t cbuf_adc AT(.rec_data);
29 static u8 adc_buff[7680] AT(.rec_data); //norfs擦写时间最长不超过80ms,最小缓存不小于sr*2*80ms
30
31 /* #if (0 == FPGA) */
32 #ifndef FPGA
33 sound_out_obj rec_sound;
34 sound_out_obj *enc_in_sound;
35
36 #define START_ADC_RUN rec_sound.enable |= (B_DEC_RUN_EN | B_REC_RUN)
37 #define STOP_ADC_RUN rec_sound.enable &= ~(B_DEC_RUN_EN | B_REC_RUN)
38
39 _WEAK_
40 void rec_cbuf_init(void *cbuf_t)
41 {
42     cbuf_init(cbuf_t, &adc_buff[0], sizeof(adc_buff));
43 }
44
45 void rec_phy_init(void)
46 {
47     memset(&rec_sound, 0, sizeof(rec_sound));
48     stream_frame_init(IRQ_SPEAKER_IP); // 1. 数据流中断初始化
49
50     cbuf_init(&cbuf_adc, &adc_buff[0], sizeof(adc_buff));
51     rec_sound.p_obuf = &cbuf_adc; //cbuf_adc为audio_adc的数据
52
53     sound_out_obj *p_curr_sound = &rec_sound; // 2. 音频降噪算法连接到数据流
54
55     #if (defined(ANS_EN) && (ANS_EN))
56     cbuf_init(&cbuf_ans, &ans_buff[0], sizeof(ans_buff)); //cbuf_ans 为link
57     p_curr_sound = link_ans_sound(p_curr_sound, &cbuf_ans, read_audio_adc_sr());
58     #endif
59
60     if (p_curr_sound != &rec_sound) {
61         void *kick = NULL;
62         kick = regist_stream_channel(&rec_sound, kick_encode_api);
63         regist_audio_adc_channel(&rec_sound, NULL, (void *)kick);
64     } else {
65         regist_audio_adc_channel(&rec_sound, NULL, (void *)kick_encode_api);
66     }
67
68     enc_in_sound = p_curr_sound;
69     rec_sound.enable |= B_DEC_RUN_EN | B_DEC_FIRST;
70
71 }
72 void rec_phy_suspend(void)

```

录音模式串联音频降噪

34.2 相关参数

1. 采样率:

降噪算法只支持 8k 和 16k 采样率。
建议使用 16k 采样率。

```

1 encoder_api.c 2 ans_api.c
1 #include "circular_buf.h"
2 #include "errno-base.h"
3 #include "NoiseSuppressLib.h"
4 #include "app_modules.h"
5 #include "sound_effect_api.h"
6 #include "remain_output.h"
7
8 #define LOG_TAG_CONST      NORM
9 /* #define LOG_TAG_CONST    OFF */
10 #define LOG_TAG            "[ans]"
11 #include "log.h"
12
13 #if defined(ANS_EN) && (ANS_EN)
14
15 /*
16  * ANS STATE BITMAP
17  */
18
19 #define NS_MODE            0
20 #define NS_IS_WIDEBAND     1//0:8K 1:16K
21
22 #define READSIZE           128    //每次run处理样点数
23 #define ANS_RUN_BUFSIZE   5400
24 #define ANS_TMP_BUFSIZE   3604
25 #define ANS_NEAR_SIZE     (READSIZE * 2)
26
27 static int ans_runbuf[ANS_RUN_BUFSIZE / 4] AT(.ans_data);
28 static int ans_tmpbuf[ANS_TMP_BUFSIZE / 4] AT(.ans_data);
29
30 static short ans_output_buff[READSIZE] AT(.ans_data);
31
32 static remain_ops ans_remain_ops AT(.ans_data);
33 static EFFECT_OBJ ans_effect_obj AT(.ans_data);
34 /* ----- */
35
36

```

音频降噪采样率配置

2. 系统时钟:

降噪算法需要较高的系统时钟。

AW30N 经测试最低需要 107MHz 时钟。

34.3 相关函数

34.3.1 void *link_ans_sound(void *p_sound_out, void *p_ans_obuf)

该函数为 LINK 音频降噪算法，实现算法的启动，并添加到音频链路上，其中参数：::

1. p_sound_out: 前级 sound
2. p_ans_obuf: 数据输出 cbuf

34.3.2 int ans_run(void *hld, short *inbuf, int len)

该函数为降噪算法数据处理流程。

该降噪算法固定输入 128 个样点，经（NoiseSuppress_Process）处理后输出。

频谱分析算法

AD16N 从 V1.4.0 版本开始支持频谱分析算法

该频谱分析算法使用 `link` 接口，可在解码、扩音等应用中串联该功能。该功能算法仅对输出的音频样点进行运算获取指定频点幅度，不会修改原本输出的音频样点，不会改变音频输出效果。

35.1 涉及文件

源文件：

`apps\app\bsp\common\sound_effect_list\spectral_anal.c`

头文件：

`apps\include_lib\audio_eq\hw_eq.h`

`apps\include_lib\sound_effect_list\follow.h`

`apps\include_lib\sound_effect_list\spectral_anal.h`

库文件：

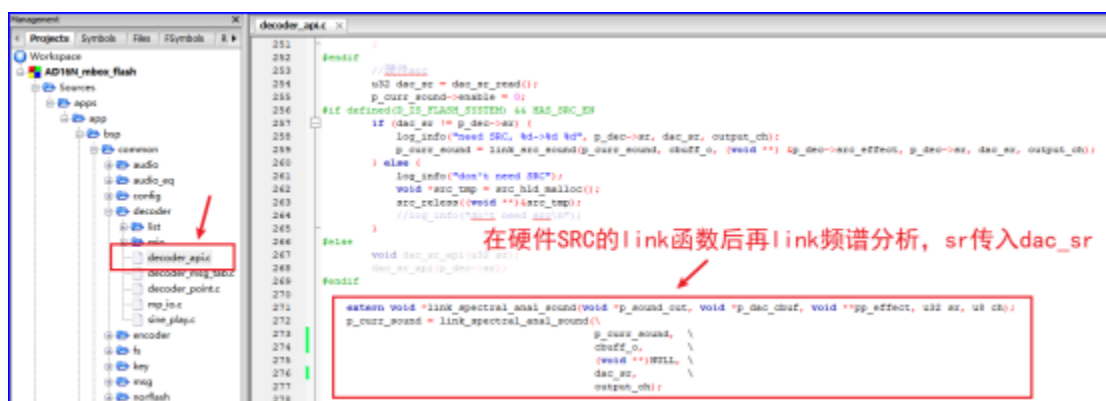
`apps\include_lib\liba\uc03\mbox_flash\audio_eq_lib.a`

35.2 使用流程

该频谱分析算法使用 link 接口，可在解码、扩音等应用中串联该功能。该功能算法仅对输出的音频样点进行运算获取指定频点幅度，不会修改原本输出的音频样点，不会改变音频输出效果。

35.2.1 在解码链路中使用频谱分析算法

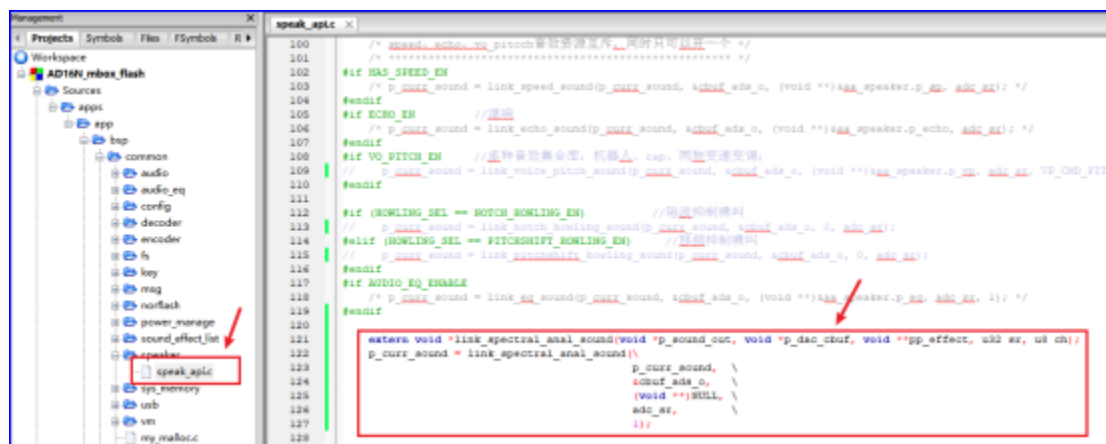
频谱分析 link 函数在硬件 SRC 的 link 后调用，示例如下：



将频谱分析算法链接到解码链路

35.2.2 在扩音链路中使用频谱分析算法

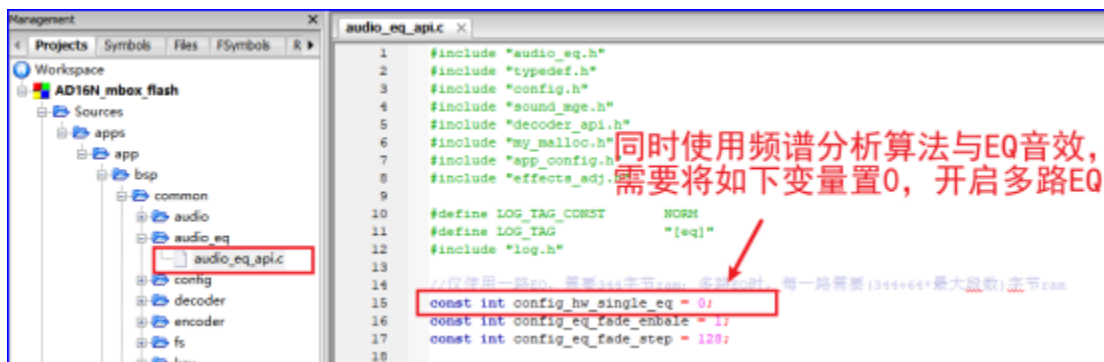
频谱分析 link 函数在硬件 SRC 的 link 后调用，示例如下：



将频谱分析算法链接到扩音链路

35.2.3 硬件 EQ 复用说明

频谱分析算法用到了硬件 EQ 模块，与 EQ 音效同时使用时，需要开启多路 EQ；



硬件 EQ 配置

35.2.4 获取频点幅度

频谱分析算法每次 run 完后，会将数据存放在运算 buff 中，SDK 可通过函数 getFollowdB_api() 定期去获取运算出来的结果



获取频点幅度接口说明

35.3 相关函数

35.3.1 void *link_spectral_anal_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u8 ch)

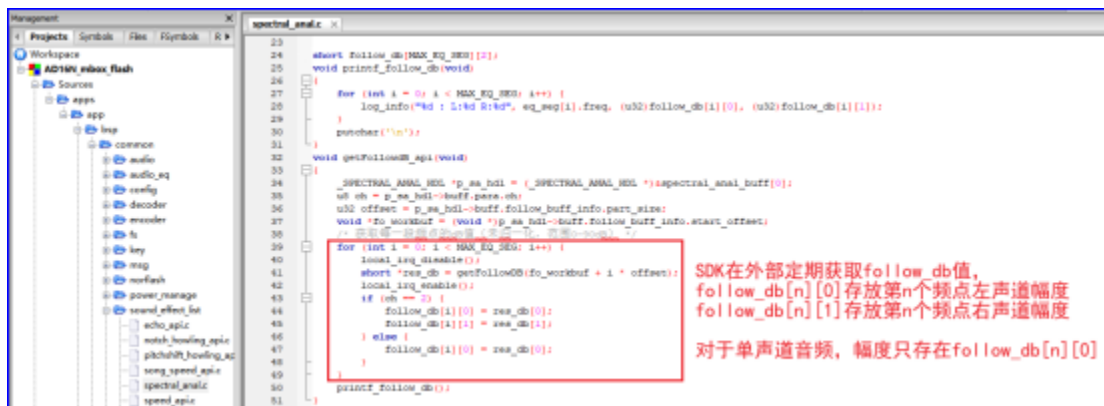
该函数为 LINK 频谱分析算法，实现算法的启动，并添加到音频链路上，其中参数：::

- void *p_sound_out: 当前算法前级的音频通道句柄；
- void *p_dac_cbuf: 输出到 DAC 的 OBUF；
- void **pp_effect: 存放当前音效算法的控制句柄指针的指针；

35.3.2 void getFollowdB_api(void)

该函数为获取频点幅度接口。

频谱分析算法每次 run 完后，会将数据存放在运算 buff 中，SDK 可通过函数 getFollowdB_api() 定期去获取运算出来的结果



获取频点幅度接口说明

AD16N SDK 提供了系统记忆管理接口，用于保存断电之后还需要保存的信息；

目前系统记忆用于管理新 VM 和旧 VM 掉电记忆；

系统记忆存储使用的步骤分为三步：

- 第一步：系统初始化时，调用函数 `sysmem_init_api()` 初始化所有设备。
- 第二步：`sys_memory.h` 头文件中定义数据记忆的 `SYSMEM_INDEX`。
- 第三步：使用函数 `sysmem_write_api(...)` 写入需要的记忆数据。
- 第四步：使用函数 `sysmem_read_api(...)` 读出需要的记忆数据。

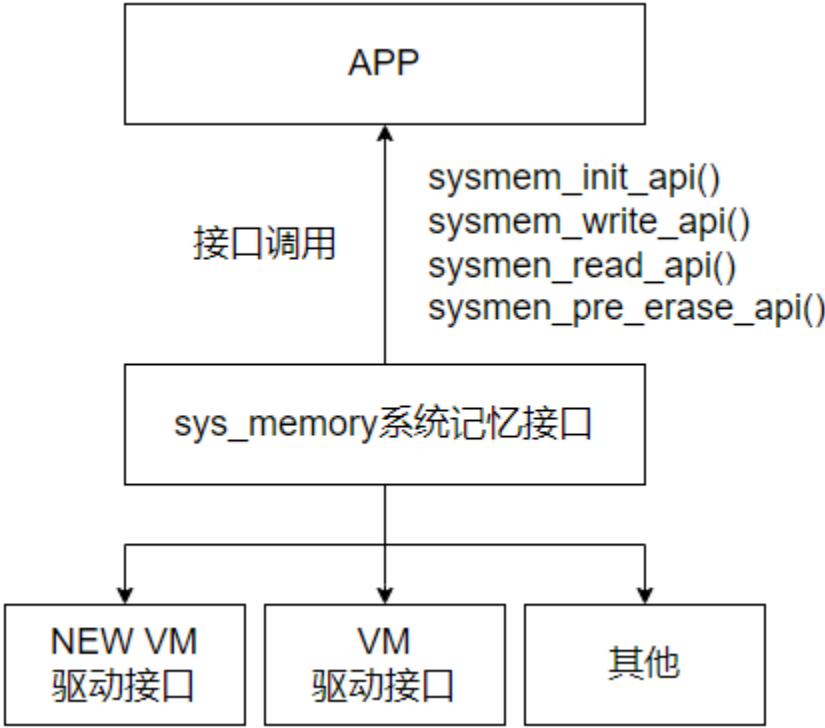


图 1.1 系统记忆结构

36.1 系统记忆存储接口

36.1.1 函数 int sysmem_init_api(u32 mem_addr, u32 mem_size)

此函数初始化设备记忆信息，其中参数：

- 1、mem_addr：存储掉电数据的EEPROM区域的起始地址
- 2、mem_size：存储掉电数据的EEPROM区域的长度

36.1.2 函数 int sysmem_read_api(u32 id, u8 *data_buf, u16 len)

此函数用于读取记忆数据，其中参数：

- 1、id：存储掉电数据的SYSTEM_INDEX
- 2、data_buf：读取记忆数据的buffer
- 3、len：读取记忆数据的长度
- 4、返回值：
 - 成功，返回读取长度len
 - 失败，错误值请查看errno-base.h

36.1.3 函数 `int sysmem_write_api(u32 id, u8 *data_buf, u16 len)`

此函数用于写入记忆数据，其中参数：

- 1、id：存储掉电数据的SYSMEM_INDEX
- 2、data_buf：写入记忆数据的buffer
- 3、len：写入记忆数据的长度
- 4、返回值：
 - 成功，返回写入长度len
 - 失败，错误值请查看errno-base.h

36.1.4 函数 `void sysmem_pre_erase_api(void)`

此函数用于预擦除设备。

设备记忆 (OTP 工程)

AD16N SDK 提供了设备记忆存储接口，用于保存断电之后还需要保存的信息。设备记忆存储使用的步骤分为三步：

- 第一步：系统初始化时，调用函数 `devices_init_api()` 初始化所有设备。
- 第二步：`device_memory.h` 头文件中定义需要存储的信息 `OFFSET` 和长度 `LEN`。
- 第三步：使用函数 `device_memory_control(..., 'w')` 把需要保存的数据写入指定设备。
- 第四步：使用函数 `device_memory_control(..., 'r')` 把读出指定设备的数据。

设备记忆项需要手动指定，并进行偏移，其定义的结构图如下：



图设备记忆项的定义

37.1 设备记忆存储接口

37.1.1 函数 u32 device_memory_control(u8 devmm_index, u8 *buf, u32 offset, u32 len, char set)

此函数用来读写设备存储的内容，数据会保存在设备保留区中，其中参数：

- 1、devmm_index：需要读写的设备号
 - ① SD0_INDEX：SD卡设备号；
 - ② UDISK_INDEX：U盘设备号；
- 2、buf：需要存储的信息 / 需要读出信息的buff
- 3、offset：偏移地址
- 4、len：读取数据长度
- 5、set：读出 / 写入选择
 - ① set = 'w'：记录存储信息；
 - ② set = 'r'：读出存储信息；

VM 掉电存储详细说明

下面分为 4 个部分来介绍 VM 的基本原理与特点：

1. “*VM* 记忆的基本原理”
2. “*VM* 记忆的系统局限性”
3. “*NEW VM* 的预擦除”
4. “*NEW VM* 常见 *BUG*”

备注：VM 使用原则：

VM 读写功能建议不要在中断操作!!!!

VM 读写功能建议不要在中断操作!!!!

VM 读写功能建议不要在中断操作!!!!

//一方面在 **中断处**进行 VM 写操作可能需要进行擦除动作导致中断执行过久；

//一方面裸机系统在中断处读写 VM，会有重入风险，极端情况下会导致数据记忆错误或丢失。

38.1 VM 记忆的基本原理

SDK 提供了 **vm 掉电记忆存储功能**，该功能用于在 **内置 flash** 里保存断电之后还需要保存的信息。

- 1、VM 会将自己所分得的内部 FLASH 区域划为 A,B 两块区域，
- 2、例如 A 区域负责记录 VM 数据，B 区域此时处于空闲状态，当 A 区域数据趋于饱和时，会把 A 区域的数据整合到 B 区域（即格式整理），后续 B 区域记录 VM 数据，A 区域处于空闲状态，以此往复。
- 3、若此时格式整理后，A 区域原有数据没有被擦除（未调用擦除函数）
- 4、用户去读取数据，会读取到原有区域的数据（即旧数据），导致数据错误或者读不到数据。

38.2 VM 记忆的系统局限性

VM 的数据记忆主要记录到 **系统 flash** 中，**系统 flash** 中存放着 **整个系统的代码**，大量的代码需要在系统 flash 中执行；

VM 的数据记忆过程中有可能需要对 **系统 flash** 进行擦除操作；

系统 flash 在被擦除时不可以被访问，且不同的 flash 擦除时间不一致；

基于以上几点 **当 VM 触发到系统 flash 擦除时，这时系统 flash 代码是不能执行的**，典型的现象就是 **解码卡音**。

为了优化上述的问题，编写新的 NEW VM 驱动，在一定程度上改善此问题。

38.3 NEW VM 的预擦除

为了解决解码时，擦除系统 flash 可能导致的卡音，new vm 尽量不在系统繁忙时对系统 flash 进行擦除；

new vm 提供了预擦除函数 `nvm_pre_erase_next`，可以在系统空闲时对系统 flash 当前与空闲区进行 **提前预擦除**、或 **滞后擦除**；

new vm 的写函数在 AB 区格式整理时，也不会对老区域进行擦除，而依赖预擦除在空闲时擦除；（未擦除再次上电会有老数据残留风险）

38.4 NEW VM 常见 BUG

部分客户在使用 NEW VM，自己编写功能中，遗忘或者未能合理调用预擦除功能导致，数据不能有效更新；

此问题有两种解决方式：

- 1、在写入数据后，系统掉电或关机前，调用预擦除；（推荐）
- 2、打开 `config_vm_erase_after_format_en` 变量，打开此变量后，new vm 驱动会在需要擦除时实时擦除系统 flash，这样一来 NEW VM 驱动的行为将和老的 VM 一直，不再具备对卡音问题的优化；

下面分为 6 个部分来介绍 VM 功能的使用说明：

1. “VM 掉电存储使用说明”
2. “VM 掉电存储版本简单说明”
3. “新旧 VM 文件组成及版本选择”
4. “VM 相关参数介绍”
5. “VM 掉电记忆存储接口”
6. “VM 常见问题介绍”

38.5 VM 掉电存储使用说明

VM 记忆存储使用的步骤分为三步：

- 第一步：系统初始化时，调用函数 `vm_init_api()` 初始化 VM 功能。
- 第二步：使用 `sysmem_write_api()` 接口把需要保存的数据写入 FLASH。
- 第三步：使用 `sysmem_read_api()` 接口读取写入的数据。

可参考” VM 简单使用 *demo*： ”

38.6 VM 掉电存储版本简单说明

目前 SDK 中 VM 有新旧两个版本可以选择：

备注：旧版 VM：

1. 可存储 id 号数量：128 个；
2. 单次写入最大数据长度：128 个 byte；
3. 没有 cache 管理；
4. 写数据时，会直接擦除 flash；

备注：新版 VM

1. 可存储 id 号数量：512 个；
2. 单次写入最大数据长度：4096 个 byte；（若 EEPROM 分配给 VM 区域只有 8K，则单次写入最大数据长度为：4092 个 byte）；
3. 增加 cache 管理，提升读取速度；
4. 写数据时，需要擦除 flash 时，可选择擦除，之后在空闲时手动调用预擦除功能实现擦除；（解决解码卡音的问题）
5. 增加预擦除功能。

备注：新旧版本 VM 功能对比：

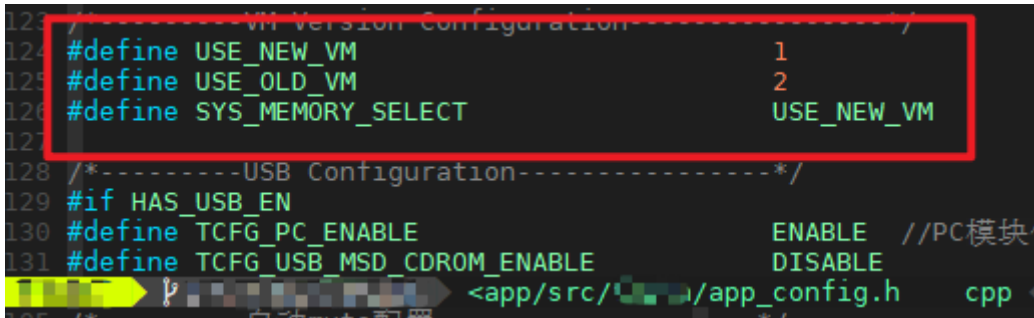
1. 新版 VM id 号数量增多；

- 2. 新版 VM 单次写入最大数据长度增多;
- 3. 新版 VM 增加 cache 管理, 提升 VM 读取速度;
- 4. 新版 VM 增加预擦除功能, 可以在系统空闲时进行预擦除操作, 可尽可能避免 VM 整理时关闭 flash 时间过长导致的卡音问题;
- 5. 新版 ID 号与旧版 index 句柄含义相同;

38.7 新旧 VM 文件组成及版本选择

vm 模块由 vm_api.c 和 vm_api.h 文件构成 API 层, 包含 ovm_api.c 和 nvmm_api.c 文件;

备注: 用户可在 app_config.h 文件里选择使用新 / 旧版 VM.



新旧 VM 功能选择

38.8 VM 相关参数介绍

对于用户, 我们需要关心的参数分别为:

- 1. “ID 号 (vm_api.h) ”
- 2. “读/写入数据长度”
- 3. “vm 存储区域大小”

对于新版 VM

- 4. “新版 VM 格式整理时缓存空间设置 new_vm_buff (new_vm.h) ”
- 5. “新版 vm 格式整理后是否立即擦除旧半区区域功能”
- 6. “新版 vm 多段读功能开关”

下面逐一介绍:

38.8.1 ID 号 (vm_api.h)

```
int systemem_write_api(u32 id, u8 *data_buf, u16 len)
{
```

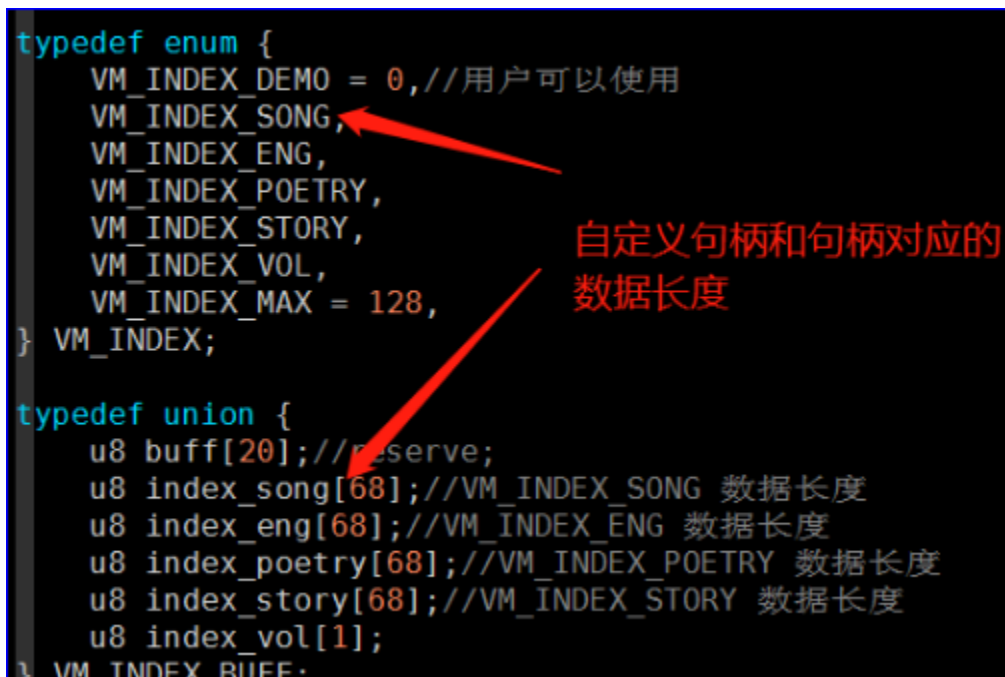
systemem_write_api 参数

重要：旧版 VM ID 号最多 128 个，新版 VM ID 号最多 512 个；（旧版 VM index 号在新版 vm 仍可通用）

新版 VM ID 号最多支持 512 个，且不再需要自定义对应 ID 的数据长度。

用户可在 sys_memory.h 的 VM_INDEX 里添加自己的 id 号。

注意：1~32 号 ID 为系统使用，用户不可修改其顺序



```
typedef enum {
    VM_INDEX_DEMO = 0, // 用户可以使用
    VM_INDEX_SONG,
    VM_INDEX_ENG,
    VM_INDEX_POETRY,
    VM_INDEX_STORY,
    VM_INDEX_VOL,
    VM_INDEX_MAX = 128,
} VM_INDEX;

typedef union {
    u8 buff[20]; // reserve;
    u8 index_song[68]; // VM_INDEX_SONG 数据长度
    u8 index_eng[68]; // VM_INDEX_ENG 数据长度
    u8 index_poetry[68]; // VM_INDEX_POETRY 数据长度
    u8 index_story[68]; // VM_INDEX_STORY 数据长度
    u8 index_vol[1];
} VM_INDEX_BUFF;
```

自定义句柄和句柄对应的数据长度

旧版 VM 掉电存储记忆项的定义 (vm_api.h)

38.8.2 读/写入数据长度

```
int systemem_write_api(u32 id, u8 *data_buf, u16 len)
{
```

systemem_write_api 参数 length

重要：旧版 VM 读/写长度最多支持 128 个 byte；

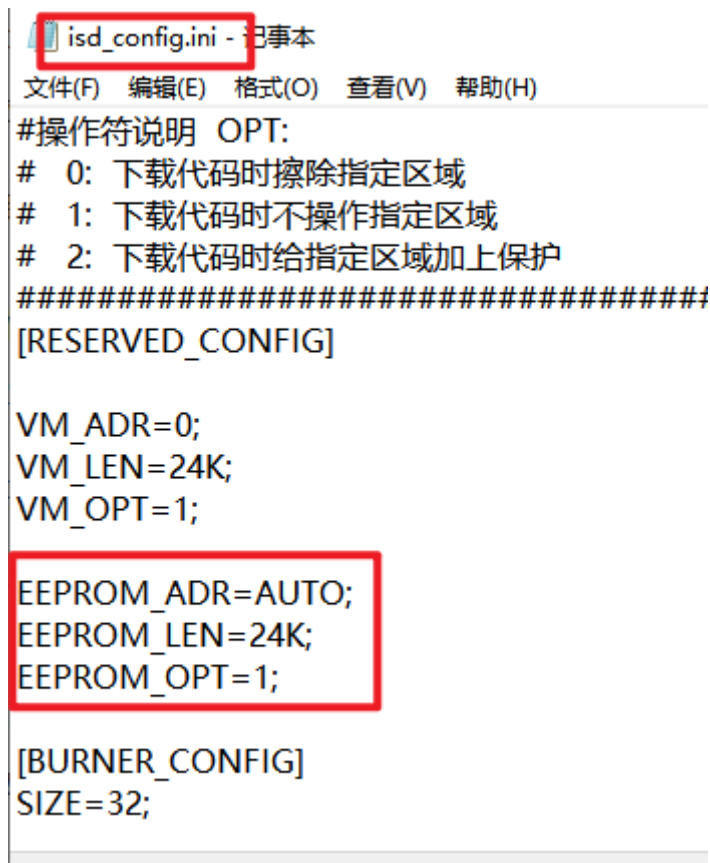
新版 VM 读/写长度最多支持 4096 个 byte；

38.8.3 vm 存储区域大小

重要： VM 记忆的数据是 EEPROM 区划分出一定空间大小给 VM 使用，该区域大小可由下载目录里的 isd_config.ini 文件配置。（该配置参数详细说明可参考” isd_config.ini 文件介绍”）

注意：

VM 存储区域大小分配的 EEPROM 空间不得低于 8K 字节，且需为 4K 字节的倍速，即 8K，12K，16K 等等。



```

isd_config.ini - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
#操作符说明 OPT:
# 0: 下载代码时擦除指定区域
# 1: 下载代码时不操作指定区域
# 2: 下载代码时给指定区域加上保护
#####
[RESERVED_CONFIG]

VM_ADR=0;
VM_LEN=24K;
VM_OPT=1;

EEPROM_ADR=AUTO;
EEPROM_LEN=24K;
EEPROM_OPT=1;

[BURNER_CONFIG]
SIZE=32;
  
```

VM 区域大小设置”

38.8.4 新版 VM 格式整理时缓存空间设置 new_vm_buff (new_vm.h)

重要： new_vm_buff，该 buff 用于 vm 格式整理时数据的缓存。

该 buff 由 用户自定义最大 id 数量 (BIT_MAP_SIZE) + 单个 id 在整理时数据缓存的最大长度 (BIT_MAX_LEN) 两部分组成。

BIT_MAP_SIZE：即 BIT_MAP：vm 最多可以存储的 id 数量，固定为 512 个。

BIT_MAX_LEN：即 VM 格式整理时数据缓存的大小，该值越大，则整理速度越快，该值越小，则整理速度越慢。（注意：该 buf 与读/写数据最大长度为两个概念）

```

14 #include "log.h"
15 NEW_VM_OBJ g_nvm_obj;
16
17 u32 new_vm_buff[NVM_BUFF_SIZE / 4]; //需要32bit对齐
18 /**
19 **/
20 void *nvm_buf_for_lib(NEW_VM_OBJ *p_nvm, u32 *p_len)
21 {
22     *p_len = sizeof(new_vm_buff);
23 }
24
25 #define BIT_MAP (32 * 16)
26 #define BIT_MAP_SIZE (BIT_MAP / 8)
27
28 #define NVM_MAX_LEN 128
29 #define NVM_BUFF_SIZE (NVM_MAX_LEN * BIT_MAP_SIZE)
30
31 typedef struct __new_vm {
32     void *device;
33 }

```

VM 掉电存储记忆项的定义 (nvm_api.c 和 new_vm.h)

38.8.5 新版 vm 格式整理后是否立即擦除旧半区区域功能

```

27
28 #define NVM_CACHE_ENABLE 1
29 #define NVM_CACHE_NUMBER 6
30 const bool config_vm_multiple_read_en = 0;
31 const bool config_vm_erasure_after_format_en = 0;
32
33 #if NVM_CACHE_ENABLE
34
35 NVM_ENTRY g_nvm_entry[NVM_CACHE_NUMBER];
36
37 NORMAL

```

config_vm_erasure_after_format_en

config_vm_erasure_after_format_en 用于控制新版 vm 格式整理后是否立即擦除旧半区区域;

0: 不立即擦除 (默认值)

1: 立即擦除

重要: 该功能直观影响为: 打开该功能后, vm 进行格式整理 会调用到 flash 擦除, 如果系统此时正在解码, 那么解码可能会出现卡音的现象。

而默认关闭该功能后, vm 进行格式整理 不会调用到 flash 擦除, 而是等到 系统空闲时程序员调用预擦除函数 进行擦除动作, 这样就不会影响到系统此时正进行的解码。

关闭该功能后, 未在系统空闲时进行擦除动作, 那么可能导致下次上电, VM 读取到老旧的数据。

38.8.6 新版 vm 多段读功能开关

```

27
28 #define NVM_CACHE_ENABLE 1
29 #define NVM_CACHE_NUMBER 6
30 const bool config_vm_multiple_read_en = 0;
31 const bool config_vm_erasure_after_format_en = 0;
32
33 #if NVM_CACHE_ENABLE
34
35 NVM_ENTRY g_nvm_entry[NVM_CACHE_NUMBER];
36
37 NORMAL

```

config_vm_multiple_read_en

config_vm_multiple_read_en 用于控制新版 vm 是否支持多段读功能;

0: 不支持多段读 (默认值)

1: 支持多段读

38.9 VM 掉电记忆存储接口

函数接口在 sys_memory.c

38.9.1 函数 u32 syscfg_vm_init(u32 mem_addr, u32 mem_size)

此函数用来初始化 vm 虚拟存储系统, 其中参数:

1. eeprom_saddr: vm区域的起始地址。
2. eeprom_size: vm区域的空间长度。
3. 返回值:
 - 成功: 0;
 - 失败: 错误号, 可参考errno-base.h;

38.9.2 函数 u32 sysmem_write_api(u32 id,u8 *data_buf, u32 len)

此函数用来把对应 hdl 的数据存储到 vm 虚拟存储系统, 其中参数:

1. hdl: 访问vm的id。
2. buf: vm读数据buffer。
3. len: 数据长度。
4. 返回值:
 - 成功: 成功写入VM数据长度;
 - 失败: 错误号, 可参考errno-base.h;

38.9.3 函数 u32 sysmem_read_api(u32 id ,u8 *data_buf, u32 len)

此函数用来把对应 hdl 的数据从 vm 虚拟存储系统读出来, 其中参数:

1. id: 访问vm的id。
2. buf: vm读数据buffer。
3. len: 数据长度。
4. 返回值:
 - 成功: 成功获取到的数据长度;
 - 失败: 错误号, 可参考errno-base.h;

38.9.4 函数 u32 sysmem_pre_erase_api(void)

此函数用于预擦除设备，在系统空闲时主动调用提前擦除 flash，可尽量避免 VM 做整理时关 flash 时间过长导致卡音问题；

该函数只有在新版 VM 处生效，旧版 VM 没有该功能。

示例：音乐模式下，500MS 消息来时若系统处于空闲状态，则会进行预擦除动作：

```

248         goto __out_music_mode;
249     case MSG_500MS:
250         /* decoder_time(pctl[0].p_dec_obj); */
251         UI_menu(MENU_MAIN);
252         UI_menu(MENU_HALF_SEC_REFRESH);
253         if (MUSIC_PLAY != get_decoder_status(pctl[0].p_dec_obj)) {
254             sysmem_pre_erase_api();
255             app_powerdown_deal(0);
256         } else {
257             app_powerdown_deal(1);
258         }
259     default:

```

系统空闲时进行擦除动作

图 2.1 系统空闲预擦除动作

备注：该函数在擦除前会判断要擦除的目标区域是否已经擦除过，如果已经擦除过，则不会继续执行擦除动作，所以该函数可以重复调用，不需要担心每次调用都会对 flash 进行擦除。

38.9.5 函数 void nvm_pre_eraser_next(NEW_VM_OBJ *p_nvm, u16 using_next, u16 idle_next)

此函数为 NEW VM 中独有接口，由 2.4 函数 u32 vm_pre_erase() 调用，用于系统空闲时选择擦除当前 VM 已存储的数据往后的 flash 扇区，以及空闲 VM 需要往后擦除的 flash 扇区数，用户可自行修改需要擦除的 flash 扇区数，其中参数：

- 1、p_nvm：NEW VM 句柄指针；
- 2、using_next：当前 VM 存储块需要往后擦除的 flash 扇区数；
- 3、idle_next：空闲 VM 存储块需要往后擦除的 flash 扇区数；
- 4、返回值：无

38.10 VM 常见问题介绍

38.10.1 VM 简单使用 demo：

用户可在 app/bsp/start/flash_init.c 里的 vm_init_api() 函数找到 demo 示例。

```

95 }
96
97 static struct vfs_attr eeeprom_attr;
98 void vm_init_api(void)
99 {
100     u32 err;
101     void *pvfs = 0;
102     void *pvfile = 0;
103     void *device = 0;
104     u32 capacity = 0;
105
106
107     err = vfs_mount(&pvfs, (void *)NULL, (void *)NULL);
108     ASSERT(!err, "fii vfs mount : 0x%x\n", err);
109     err = vfs_openbypath(pvfs, &pvfile, "/app_area_head/EEPROM");
110     ASSERT(!err, "fii vfs openbypath : 0x%x\n", err);
111     err = vfs_ioctl(pvfile, FS_IOCTL_FILE_ATTR, (int)&eeeprom_attr);
112     ASSERT(!err, "fii vfs ioctl : 0x%x\n", err);
113     log_info("EEPROM size : 0x%x\nEEPROM sclust : 0x%x\n", eeeprom_attr.fsize, eeeprom_attr.sclust);
114
115     ;
116     vfs_file_close(&pvfile);
117     vfs_fs_close(&pvfs);
118
119     system_init_api(eeeprom_attr.sclust, eeeprom_attr.fsize);
120     /* extern void nvm_demo(u32 start, u32 size); */
121     /* nvm_demo(eeeprom_attr.sclust, eeeprom_attr.fsize); */
122     /*
123     *demo
124     u8 data_buf[10];
125     log_info("memset\n");
126     memset(data_buf, 0xaa, 10);
127
128     log_info("vm write\n");
129     system_write_api(SYSTEMMEM_INDEX_DEMO, data_buf, 10);
130     memset(data_buf, 0x00, 10);
131     log_info("vm read\n");
132     system_read_api(SYSTEMMEM_INDEX_DEMO, data_buf, 10);
133     log_info_hexdump(data_buf, 10);
134     */
135 }
136 struct vfs_attr *get_eeeprom_attr_p(void)

```

vm_demo

38.10.2 调整 VM 区域大小设置:

节省 VM 占用系统空间可以从两个地方入手:

1. 减少 VM 区域大小。

具体参考上面” *vm* 存储区域大小” 章节的 *vm* 存储区域大小部分

2. 新版 VM 可以减少 VM 格式整理时的缓存 *new_vm_buff*

可以适当减少 *BIT_MAX_LEN* 的长度。

具体参考上面” 新版 VM 格式整理时缓存空间设置 *new_vm_buff* (*new_vm.h*) ” 章节的新版 VM 格式整理时缓存空间设置部分。

38.10.3 新版 VM 在关机或者掉电后可能存在数据丢失 BUG:

该问题原因可能有:

1. AD14v1.7.0、v1.8.0 voice_toy 工程 app_ld.c 文件缺少 (*.data) 段 (优先排查);

解决办法: FAQ: AD14v1.7.0、v1.8.0 voice_toy 工程 new_vm 存在掉电后读取数据错误问题?

2. 系统在关机前格式整理后没有做预擦除动作。

解决办法: FAQ: AD14/15/17/AC104 new_vm 功能在关机或者掉电后可能存在数据丢失 BUG

38.10.4 VM 格式整理原理:

VM 格式整理原理:

VM 功能将 EEPROM 区域划分为 A、B 两块, A 块记录 VM 数据, B 块处于空闲状态, 当 A 块数据接近饱和时, 会擦除 B 块并将 A 块的数据整合到 B 块, 后续 B 块记录 VM 数据, A 块处于空闲状态, 以此往复。

杰理玩具系列 SDK 支持的消息见下图 1：

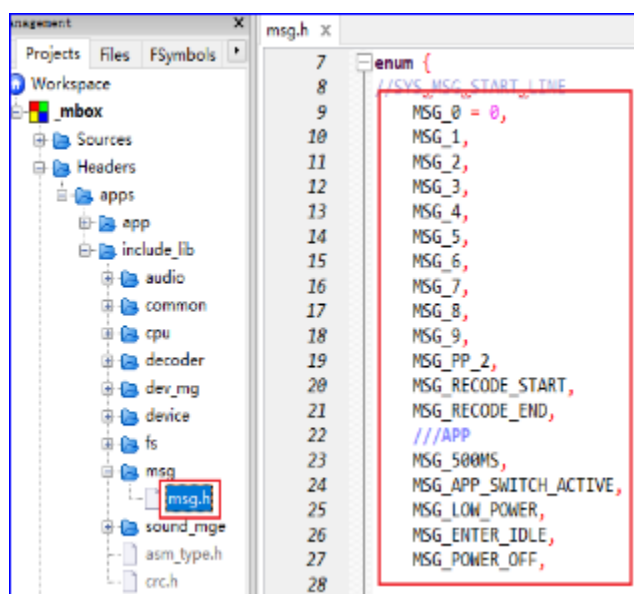


图 1 消息列表图

杰理玩具系列 SDK 目前支持的事件见下图 2，事件最多支持 32 种，系统已经定义的事件如图 2。如果用户要增加其它事件需同时在 msg.h 和 msg.c 的 event2msg[] 同样顺序下加入事件。

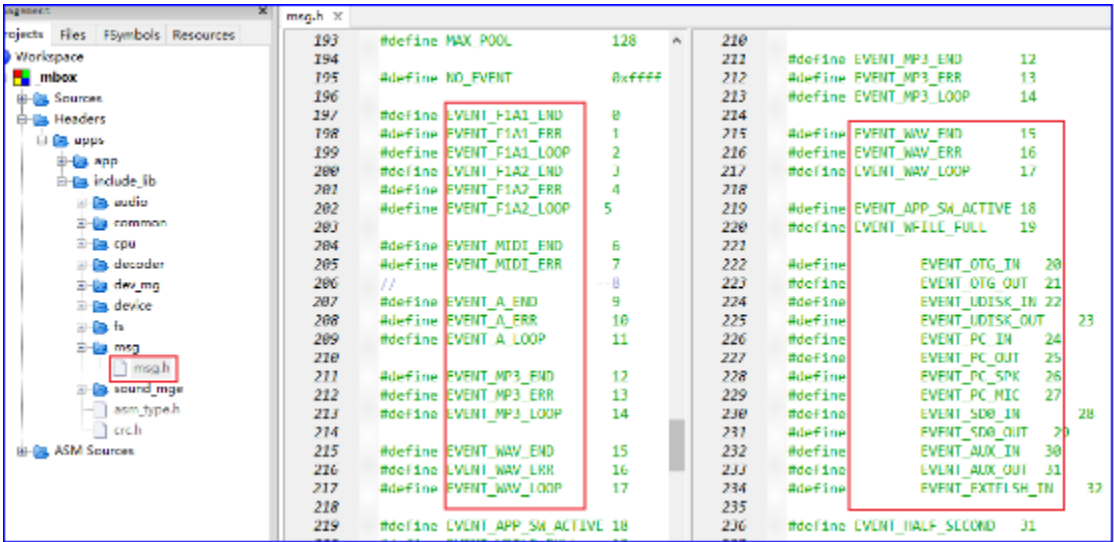


图 2 已有事件列表图

消息与事件的目录位于：apps/include_lib/msg/msg.h。

其中主要的函数有：

```
void event_init(void);
void message_init();
static u32 event2msg_api(u32 event);
bool get_event_status(u32 event);
void clear_one_event(u32 event);
int get_msg(int len , int *msg);
int post_event(int event);
int post_msg(int argc , ...);
void clear_all_message(void);
```

39.1 消息与事件使用一般流程

在开机时先调用 event_init()、message_init() 对消息系统初始化，主要初始化一些消息列表相关变量及事件状态变量。初始化后有用到消息的地方就可以设置消息或获取消息。

- a. 开机调用 event_init() 及 message_init() 完成对事件和消息的初始化；
- b. 调用 post_event(int event) 标记事件，调用 post_msg(int argc, ...) 标记消息；
- c. 调用 get_msg(int len, int *msg) 获取事件消息或其他消息；

39.2 消息 (msg) 函数

这是一种基于 FIFO 结构循环池的消息机制。当消息池累计太多消息，导致消息池满后，会写不进消息。

39.2.1 函数 void message_init()

此函数会初始化消息信息，将 msg_cbuf 结构体消息状态清空，这一步一般在开机时进行。

39.2.2 函数 int post_msg(int argc, ...)

当某个消息发生时，本函数实现将消息放入消息列表：

- 1、argc: 后面可变参数 (...) 的数量；
- 2、. . . : 可变参数，参数个数可变，可带入一个或多个文件；
可变参数 (...) 有参考下表；
- 3、返回值：MSG_NO_ERROR 无实际作用；

消息	详情
MSG_500MS	ticktime 时钟到 500ms 消息
MSG_PRIV_FILE	播放前一个文件消息
MSG_NEXT_FILE	播放下一个文件消息
MSG_NEXT_DIR	播放下一个文件夹消息
MSG_PLAY_FILE1,	播放文件 1 消息
MSG_PLAY_FILE2,	播放文件 2 消息
MSG_PP,	播放文件 1 暂停消息
MSG_PP_2,	播放文件 2 暂停消息
MSG_VOL_UP	播放声音增大消息
MSG_VOL_DOWN	播放声音减小消息
MSG_A_PLAY	播放文件 A 消息
MSG_REC_MODE_SWITCH	开始录音与开始播放录音切换消息
MSG_REC_SPEED_EN	录音速率模式使能消息
MSG_MIDI_MODE_SWITCH	midi 模式切换消息
MSG_MIDI_OKON_GOON	midi one key one note mode 下一音节消息
MSG_NEXT_MODE	app 模式切换消息
MSG_RECODE_START	重新编码开始消息
MSG_RECODE_END	重新编码结束消息
MSG_LOW_POWER	低电消息
MSG_ENTER_IDLE	cpu 进入空闲消息
MSG_POWER_OFF	关机消息

39.2.3 函数 int get_msg(int len, int *msg)

本函数实现获取事件对应的消息信息以及消息列表中消息的功能当获取到事件消息时，不在获取消息列表中消息，当没有事件消息时，查询消息列表，如果有消息，这保存消息信息到形参 msg。形参 msg 需要自定义。

1、len：形参msg的长度
长度由实际要获取的消息长度决定，msg长度小于要获取的消息长度时返回MSG_BUF_NOT_ENOUGH错误信息；
2、msg：保存获取到的消息信息（可获取到的消息如图1 消息列表图）；
3、返回值：
MSG_NO_ERROR：获取消息无误
MSG_BUF_NOT_ENOUGH：形参len长度小于实际消息长度

可获取到的消息如图 1 消息列表图

39.2.4 函数 void clear_all_message(void)

本函数实现清除所有消息的功能，清除后消息列表为空，没有消息可获取。

39.3 事件 (EVENT) 函数

这是一种基于位图的事件机制。

- 每个事件占一个比特，当写入某个事件时，对应比特位会被置一；
- 当读取某个事件，对应比特位如果为一，那么返回时间并清除对应比特位；否则返回空事件；
- 由于上述两点的原因，所以某个事件在读取之前被多次写入后，也仅会响应一次。

以下代码段是所有事件的定义：

```
#define NO_EVENT          0xffff

#define EVENT_F1A1_END    0
#define EVENT_F1A1_ERR    1
#define EVENT_F1A1_LOOP   2
#define EVENT_F1A2_END    3
#define EVENT_F1A2_ERR    4
#define EVENT_F1A2_LOOP   5

#define EVENT_MIDI_END     6
#define EVENT_MIDI_ERR     7
//                        --8
#define EVENT_A_END        9
#define EVENT_A_ERR       10
#define EVENT_A_LOOP       11

#define EVENT_MP3_END      12
#define EVENT_MP3_ERR      13
#define EVENT_MP3_LOOP     14

#define EVENT_WAV_END      15
```

(续下页)

(接上页)

```

#define EVENT_WAV_ERR      16
#define EVENT_WAV_LOOP     17

#define EVENT_APP_SW_ACTIVE 18
#define EVENT_WFILE_FULL   19

#define EVENT_OTG_IN       20
#define EVENT_OTG_OUT      21
#define EVENT_UDISK_IN     22
#define EVENT_UDISK_OUT    23
#define EVENT_PC_IN        24
#define EVENT_PC_OUT       25
#define EVENT_PC_SPK       26
#define EVENT_PC_MIC       27
#define EVENT_SD0_IN       28
#define EVENT_SD0_OUT      29
#define EVENT_AUX_IN       30
#define EVENT_AUX_OUT      31
#define EVENT_EXTFLSH_IN   32

#define EVENT_HALF_SECOND  31

```

代码段 1 事件定义列表

下表为事件列表:

事件列表	含义
EVENT_F1A1_END	F1A1 文件播放结束事件
EVENT_F1A1_ERR	F1A1 文件播放错误事件
EVENT_F1A1_LOOP	F1A1 文件播放循环事件
EVENT_F1A2_END	F1A2 文件播放结束事件
EVENT_F1A2_ERR	F1A2 文件播放错误事件
EVENT_F1A2_LOOP	F1A2 文件播放循环事件
EVENT_MIDI_END	MIDI 文件播放结束事件
EVENT_MIDI_ERR	MIDI 文件播放错误事件
EVENT_A_END	A 文件播放结束事件
EVENT_A_ERR	A 文件播放错误事件
EVENT_A_LOOP	A 文件播放循环事件
EVENT_MP3_END	MP3 文件播放结束事件
EVENT_MP3_ERR	MP3 文件播放错误事件
EVENT_MP3_LOOP	MP3 文件播放循环事件
EVENT_WAV_END	WAV 文件播放结束事件
EVENT_WAV_ERR	WAV 文件播放错误事件
EVENT_WAV_LOOP	WAV 文件播放循环事件
EVENT_APP_SW_ACTIVE	APP 切换活跃事件
EVENT_WFILE_FULL	空间已满事件
EVENT_OTG_IN	OTG 插入事件
EVENT_OTG_OUT	OTG 拔出事件
EVENT_UDISK_IN	U 盘插入事件
EVENT_UDISK_OUT	U 盘拔出事件
EVENT_PC_IN	PC 插入事件
EVENT_PC_OUT	PC 拔出事件
EVENT_PC_SPK	音箱插入事件

续下页

表 1 – 接上页

事件列表	含义
EVENT_PC_MIC	麦克风插入事件
EVENT_SD0_IN	SD 卡插入事件
EVENT_SD0_OUT	SD 卡拔出事件
EVENT_AUX_IN	音频接口接入事件
EVENT_AUX_OUT	音频接口拔出事件
EVENT_EXTFLSH_IN	外部 FLASH 插入事件
NO_EVENT	无事件发生

表 3.1 事件列表

39.3.1 函数 void event_init(void)

此函数会初始化事件状态信息，将 event_buf 事件状态清空（清楚整个事件表），这一步一般在开机时进行。

39.3.2 函数 u32 event2msg_api(u32 event)

此函数实现根据事件 event 信息获取对应的消息信息的功能。

- 1、event：事件编号，定义在“代码段1 事件定义列表”；
2、返回值：与事件对应的消息信息；

事件与消息翻译表如下代码段：

```
static const u16 event2msg[] = {
    MSG_F1A1_FILE_END,      /* 0 */
    MSG_F1A1_FILE_ERR,      /* 1 */
    MSG_F1A1_LOOP,
    MSG_F1A2_FILE_END,
    MSG_F1A2_FILE_ERR,
    MSG_F1A2_LOOP,          /* 5 */
    MSG_MIDI_FILE_END,
    MSG_MIDI_FILE_ERR,
    NO_MSG,
    MSG_A_FILE_END,
    MSG_A_FILE_ERR,         /* 10 */
    MSG_A_LOOP,
    MSG_MP3_FILE_END,
    MSG_MP3_FILE_ERR,
    MSG_MP3_LOOP,
    MSG_WAV_FILE_END,       /* 15 */
    MSG_WAV_FILE_ERR,
    MSG_WAV_LOOP,
    MSG_APP_SWITCH_ACTIVE,
    MSG_WFILE_FULL,

    MSG_OTG_IN,             /* 20 */
    MSG_OTG_OUT,
    MSG_USB_DISK_IN,
    MSG_USB_DISK_OUT,
    MSG_PC_IN,
```

(续下页)

(接上页)

```

MSG_PC_OUT,    /* 25 */
MSG_PC_SPK,    /* 26 */
MSG_PC_MIC,    /* 27 */
MSG_SDMMCA_IN,
MSG_SDMMCA_OUT,
MSG_AUX_IN,
MSG_AUX_OUT,
MSG_EXTFLSH_IN,

NO_MSG,
};

```

代码段 2 事件翻译到消息的列表

39.3.3 函数 int post_event(int event)

当某个事件 event 发生时，本函数实现将事件 event 对应的事件状态标号（event_buf 里相应的位）设置为 1。只有事件 event 状态位被置 1 后，消息获取函数 get_msg 才能获取到与事件对应的消息。

事件最多支持 32 种，系统已经定义的事件如图 2。事件发生与不发生的状态 32 位的变量 event_buf 表示；

其中变量的每个位表示一个事件的状态（1：事件发生；0：事件未发生）。

当某个事件 event 发生时对应的位将被置 1，其它位状态不变。

如果用户需求事件量高于 32，其事件可类似定义变量和事件宏，用变量位表示其状态，并修改文件中本函数 post_event、get_event 函数及事件消息对照数组 event2msg[EVENT_TOTAL] 等使其支持高于 32 种事件的事件。

- 1、event：事件列表中的某个事件，位置见上“表 3.1 事件列表”，可选事件：
- 2、返回值：返回无误信息 MSG_NO_ERROR，返回值无作用；

39.3.4 函数 bool get_event_status(u32 event)

本函数实现获取事件 event 状态的功能。

- 1、event：事件序号，位置见上“表 3.1 事件列表”为可选事件：
- 2、返回值：
 - 1：事件进行中
 - 0：事件未执行

按键系统 Key

SDK 提供了系统 key 来进行人机交互。现在 SDK 包含的按键类型有：

- IO 按键；
- AD 按键；
- 红外按键；
- 矩阵按键。

可以选择单独开其中 1 类按键，也可以选择同时开启多类按键。按键可以调整按键门阈值。

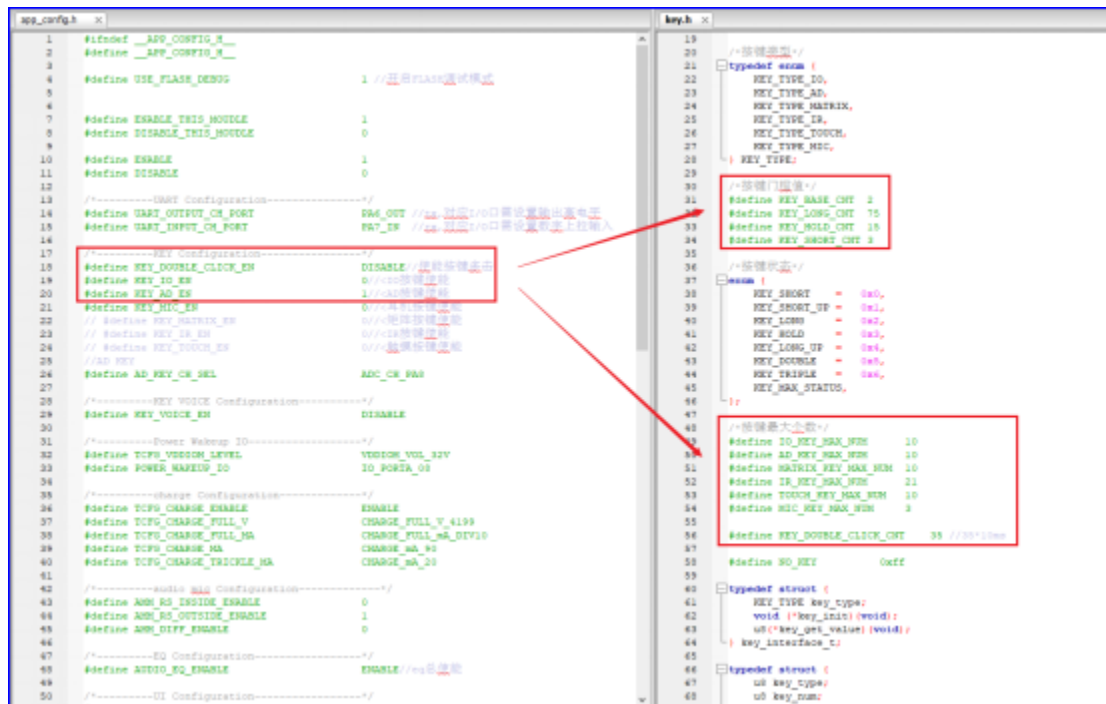


图 1 按键基本配置 (app_config.h / key.h)

按键接口：

```
void key_init(void)
void key_table_sel(void *msg_filter)
void key_scan(void)
```

40.1 按键类型介绍

本节介绍各种不同类型的按键。

40.1.1 IO 按键介绍

IO 按键的基本配置在 “key_drv_io.h” 文件中。IO 按键需要配置选择的 IO 口和 IO 是否需要内部上下拉。

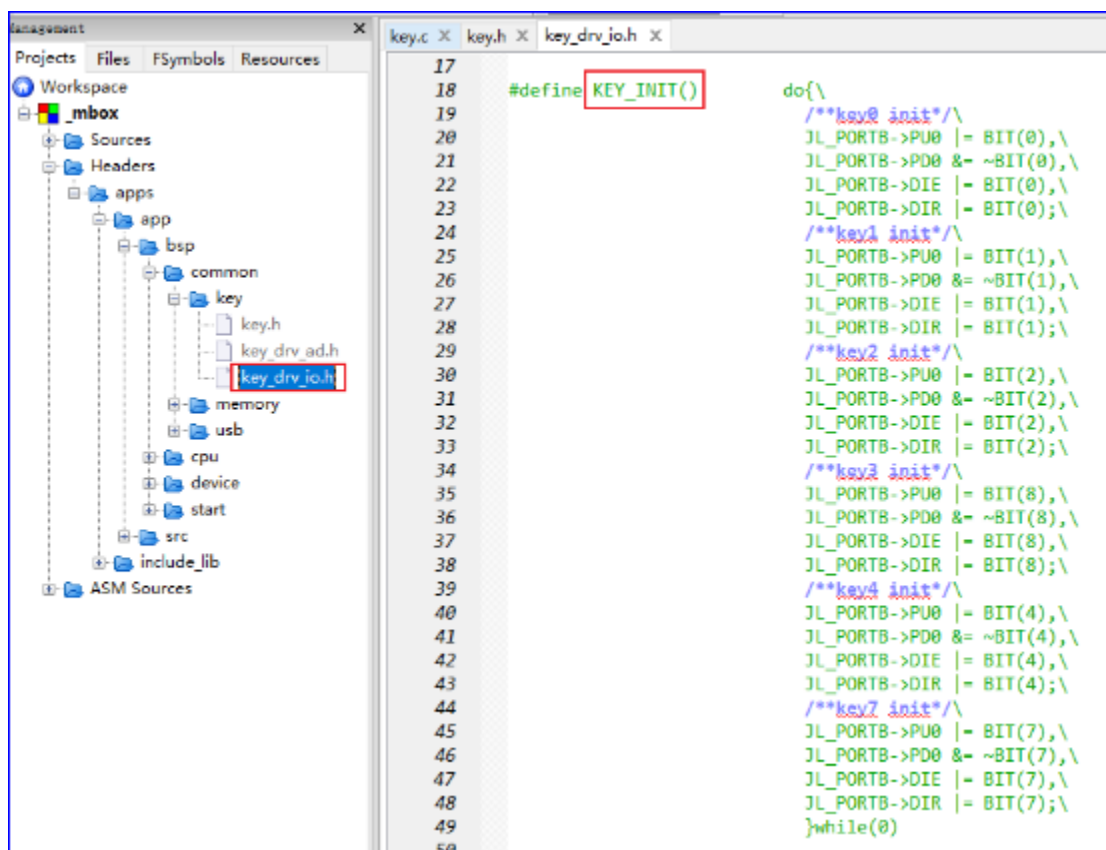


图 1.1 IO 按键配置 (key_drv_io.h)

40.1.2 AD 按键介绍

AD 按键的基本配置在“key_drv_ad.c”中。AD 按键需要配置选择的 ADC 通道和外上拉电阻的阻值，如果外上拉电阻选择阻值为 0，即使选择内上拉电阻。

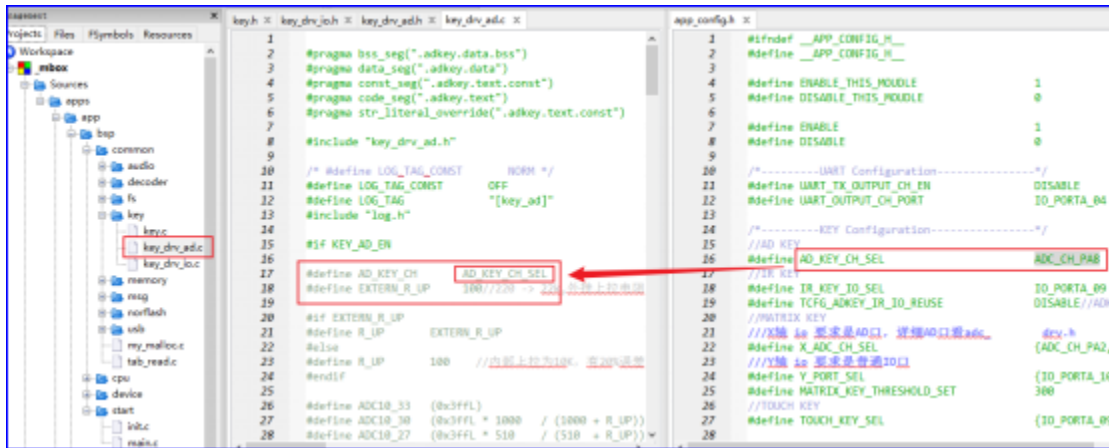


图 1.2 AD 按键配置 (key_drv_ad.c)

40.1.3 红外按键介绍

红外按键的基本配置在“key_drv_ir.c”中。红外按键需要配置选择的红外信号采集通道和采集需要的定时器，AD16 默认使用 PA5 和 TIMER2，AW30N 默认使用 PA6 和 TIMER0。

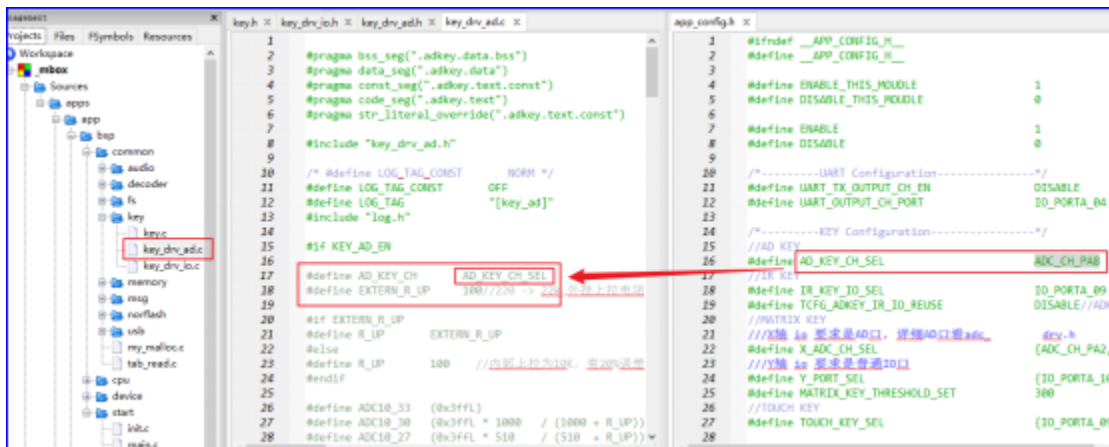


图 1.3 红外按键配置 (key_drv_ir.c)

40.1.4 触摸按键介绍

触摸按键的基本配置在“key_touch.h”中，触摸按键需要配置触摸按键 IO 口和灵敏度阈值。

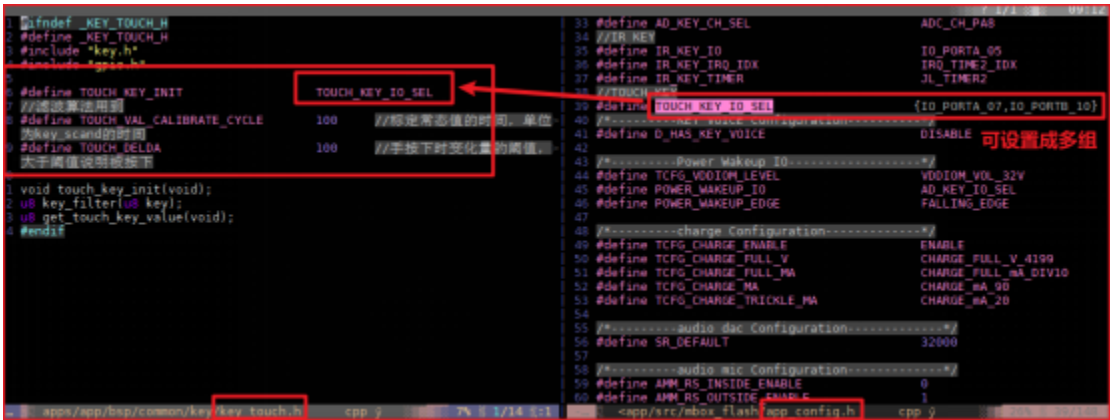


图 1.4 触摸按键配置 (key_touch.h)

图 1.4 触摸按键配置 (key_touch.h)

40.2 按键接口介绍

40.2.1 函数 void key_init(void)

此函数根据 key 的使能类型来初始化不同的 key。

40.2.2 函数 void key_table_sel(void *msg_filter)

此函数用来注册一个 msg_filter 函数，来获取 key 事件对应的 msg。

1、msg_filter: key 事件转换成消息的函数接口。

40.2.3 函数 void key_scan(void)

此函数在 tick_timer 中断中调用，用来扫描处理 key 事件。

40.3 按键消息配置

按下按键后，系统通过消息机制来响应、处理响应的按键事件。不用应用场景下，按键的消息可能不同。各应用模式下，会有自己的按键消息响应表，其中：

Music 模式按键响应表：

```
apps\app\src\music\music_key_table.c
```

USB 从机模式按键响应表：

```
apps\app\src\music\usb_slave_mode_key.c
```


41.1 低功耗相关参数配置

AD16N 支持 Soft off 和 Power down 低功耗流程，同时支持多种唤醒方式；Soft off 模式功耗为 1.8uA+，Soft off 唤醒后芯片需要从 MASKROM 开始重新跑流程，约等同于复位；Power down 模式功耗为 38uA+，Power down 唤醒后芯片会从进入 powerdown 的函数后开始跑。

低功耗方式	RAM 保持	功耗	唤醒后状态	唤醒时间
Power down	保持	38uA+	原地唤醒	2.15ms+
Soft off	不保持	1.8uA+	复位	7.72ms+

在 apps/app/bsp/cpu/uc03/power_api.c 文件中，配置 power_param 结构体以下成员可实现相关功能：

- 1、vddiom_lev：系统正常运行时的 VDDIO 电压；
- 2、vddiow_lev：系统低功耗时的 VDDIO 电压，该电压值需要比正常运行时的 VDDIO 电压低至少 200mV；
- 3、flash_pg_keep：FLASH 电源配置，A0 芯片使用 VDDIO 给外接 FLASH 供电时需要配置为 1，以降低关机功耗；使用 PD 口供电不用配置；

本章着重介绍 Soft off、Power down 以及唤醒。

Soft off 及 Power down 相关函数：

```
void sys_softoff();  
void sys_power_down(u32 usec);
```

唤醒相关函数：

```
void wakeup_callback(u8 index);  
u8 is_wakeup_source(enum WAKEUP_REASON index);
```

41.2 唤醒相关参数配置

Soft off 与 Power down 共用唤醒模块，AD16N 最多可以设置 12 个 IO 唤醒源。可以在 power_api.c 的 wkup_param 中增加 port_wakeup 类型成员来增加唤醒源。

在 apps/app/bsp/cpu/uc03/power_api.c 文件中，配置以下 const 变量可实现相关功能：

- 1、config_record_wkup：使能唤醒源记录
- 2、config_record_reset：使能复位源记录
- 3、config_max_wakeup_port：设置数字 I/O 唤醒数量

唤醒前需要先配置唤醒参数结构体 wakeup_param，内容包含 PORT_FLT 和 port_wakeup 结构体：

PORT_FLT：边沿唤醒选择消抖的脉冲时长，仅模拟端口有效

- ① PORT_FLT_NULL;
- ② PORT_FLT_256us;
- ③ PORT_FLT_512us;
- ④ PORT_FLT_1ms;
- ⑤ PORT_FLT_2ms;
- ⑥ PORT_FLT_4ms;
- ⑦ PORT_FLT_8ms;
- ⑧ PORT_FLT_16ms;

port_wakeup 结构体：唤醒端口的参数配置

- ① pullup_down_enable：上/下拉使能；
- ② edge：边沿触发方式；
- ③ filter：滤波方式；

iomap：I/O 口选择；

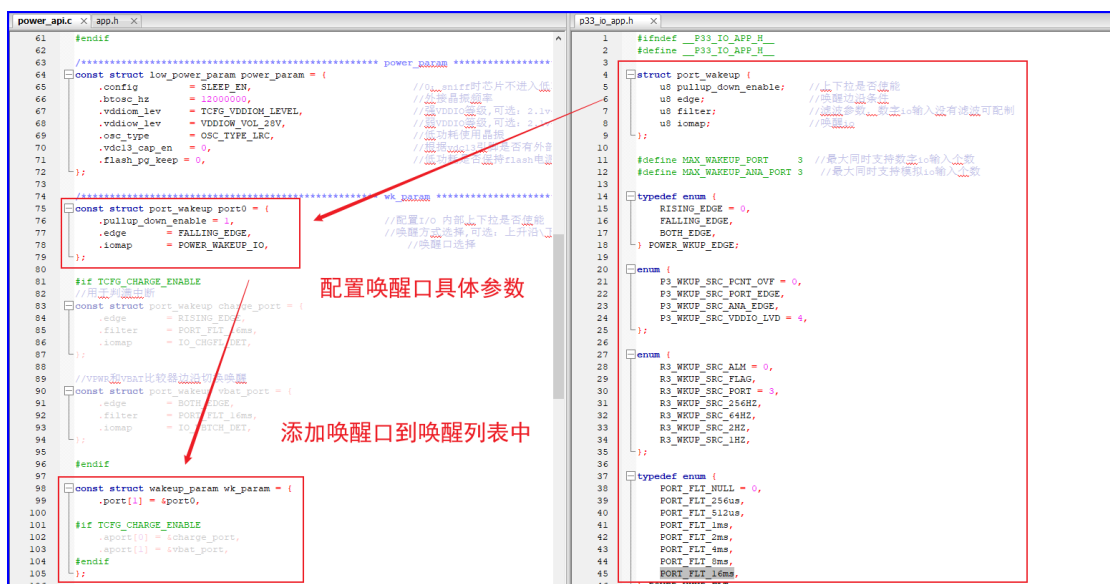


图 13-1 唤醒初始化配置参数

41.3 低功耗相关函数

41.3.1 函数 void music_idle_deal(void) (OTP 工程)

应用层函数，此函数实现进入 powerdown 的处理以及退出 powerdown 的恢复。

41.3.2 函数 void app_powerdown_deal(u8 is_busy) (FLASH 工程)

应用层函数，此函数实现在系统空闲时，指定超时时间进入 powerdown 的处理以及退出 powerdown 的恢复，其中参数：

1、is_busy：系统繁忙标志位

- ① 0：系统空闲；
- ② 1：系统繁忙中；

41.3.3 函数 void sys_softoff(void)

此函数实现使系统进入软关机。

41.3.4 函数 void board_set_soft_poweroff(void)

此函数实现在进入软关机之前将 IO 口设置成高阻，并将唤醒引脚设为对应的待唤醒状态，若需要保留某些 I/O 口原来状态，可调用函数 void soff_gpio_protect(u32 gpio)。

41.3.5 函数 void sys_power_down(u32 usec)

此函数实现进入系统 power down，并通过延时或者触发唤醒。其中参数：

- 1、usec：唤醒时间（微秒），该时间不可超过看门狗复位时间的一半；
- 2、返回值：无；

41.4 唤醒相关函数

41.4.1 函数 u8 is_wakeup_source(enum WAKEUP_REASON index)

此函数实现查询传入的唤醒源参数，是否与系统 soft off 的唤醒源一致。该接口需要在系统初始化 sys_power_init() 之前调用，否则获取的唤醒源不准确，其中参数：

1、index：唤醒端口；

- ① PWR_WK_REASON_EDGE_INDEX0：唤醒源 0~11；
- ② 具体可传入的唤醒源请查看 power_wakeup.h 中的 WAKEUP_REASON 枚举；

2、返回值：

- 1 传入的唤醒源为本次系统的唤醒源；
- 0 传入的唤醒源非本次系统的唤醒源；

41.4.2 函数 void wakeup_callback(u8 index)

此函数为 powerdown 唤醒的回调函数，通过其传入的 index 值即可知道唤醒源，其中参数：

1、index：唤醒端口；

返回值：无

42.1 简介

本章主要介绍 AD16 的 LCDC 驱动模块的使用说明，其单元设计最多支持 6COM*31SEG 或者 5COM*32SEG，支持强弱驱灵活调整，可使用 RTC32K 晶振时钟或内部 LRC 时钟作为模块时钟源。AD16 SDK 提供以 4COM*8SEG LCD 为示例的驱动作为使用参考，详细见 SDK 中文件 apps/app/bsp/cpu/uc03/lcd_seg4x8_driver.c 与其同名头文件。

42.2 使用流程

LCDC 的使用场景基本为以下两种：

- LCD IO 不复用；
- LCD IO 复用；

LCDC 模块在使用时，需要注意 COM 脚不能复用且不能随意选择，比如 4COM，则 IO 必须占用 COM0/1/2/3，而不能 COM1/2/3/4，一般建议按顺序接线；SEG 脚在使用中可以根据实际情况选择 SEG0~SEG31 接线，如 LCD_SEG0->CHIP_SEG7，LCD_SEG1->CHIP_SEG8 等。详情可参考 sdk 中的 lcd 驱动测试程序；

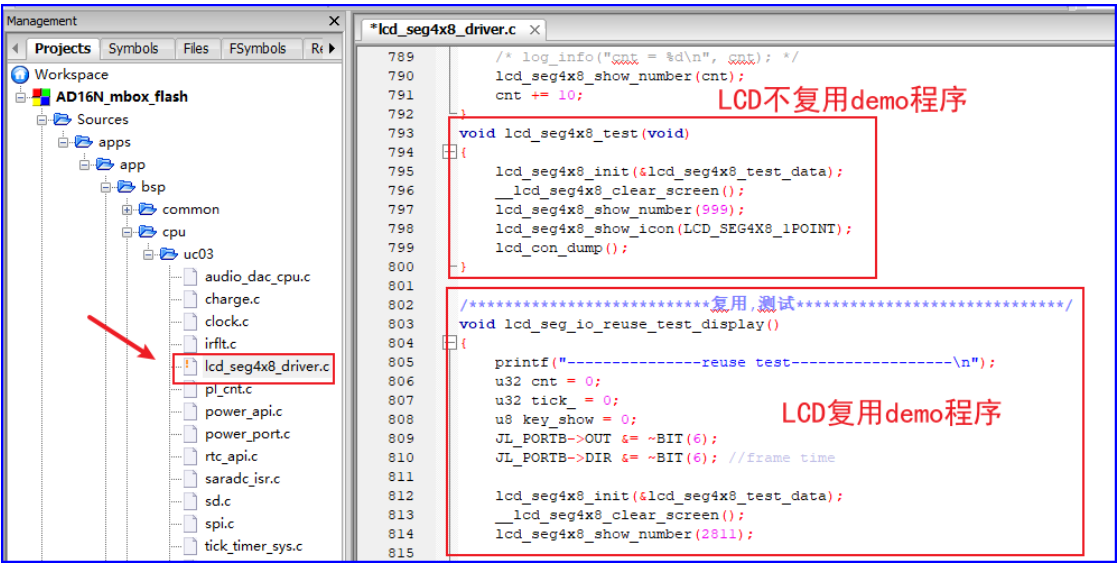


图 1 LCD 驱动示例程序

42.3 与低功耗的配合使用和注意事项

AD16 V1.3.0 的 SDK 新增了 powerdown 保持 LCD 显示的功能，该功能可以使在连续推屏状态下的 LCD 屏在低功耗状态下保持显示，可配合 RTC 的定时唤醒功能进行使用，相关示例流程已经加入了 RTC 模式。

在使用该功能时，需要保证低功耗保持 LCD 显示的功能开关打开，并且需要保持 IOVDD 的电压，需要注意的是，低功耗保持 LCD 显示会造成一定的功耗（powerdown 功耗约 200uA 以内）。



图 2 低功耗保持 LCD 功能开关

43.1 RTC 功能介绍

1、读写时钟时间

读取/修改时钟时间

2、读写闹钟时间

读取/修改闹钟设置时间

3、闹钟功能

当设定的闹钟时间与 RTC 的时钟时间相同时，会触发中断并执行相应的回调函数（可配置）

4、定时唤醒功能

开启该功能后，每隔一段时间会触发唤醒中断并且执行相应的回调函数（可配置）

注 1: AD16N_V1.4.0 开始, RTC 的接口出现变化 (例如初始化的接口从 `rtc_init` 变成 `rtc_dev_init`), 用户如果需要查看 V1.2.0 和 V1.3.0 版本的接口, 请在 gitee 的 SDK 包中查看 SDK 文档。

注 2: AD16N 有且只有硬件 RTC。

注 3: 在打开 RTC 配置的情况下, 需要先初始化 RTC 之后, 才能进入低功耗休眠。

43.2 RTC 配置参数

```
*rtc_mode.c x
52 const static struct rtc_config_init rtc_config = {
53     .default_sys_time = (struct sys_time *) &clock_time,
54     .default_alarm = (struct sys_time *) &clock_alarm,
55     .cbfun = alarm_callback, //闹钟中断的回调函数,此回调在中断运行,不可执行时间过长
56     #if RTC_LCD
57     .timefun = lcd_seg4x8_wakeupshow, //时基唤醒中断的回调函数,此回调在中断运行,不可执行时间过长
58     #else
59     /* .timefun = NULL, //时基唤醒中断回调,软件rtc没有 */
60     #endif
61     .rtc_clk = CLK_SEL_LRC, //时钟源选择
62     .rtc_sel = VIR_RTC, //软件rtc选择,对AD16无效/
63 };
```

图 1 RTC 配置和初始化

注意事项:

- 1、RTC 时钟源的设置可选内部 LRC 或者外部 32K 晶振
 - (1) 设置 LRC 为时钟源时, RTC 时钟会有偏差;
 - (2) 设置外部晶振为时钟源时, RTC 时钟准确;
- 注意: AD16N 开发板上使用外部 32K 晶振需要短接顶板部分电路:

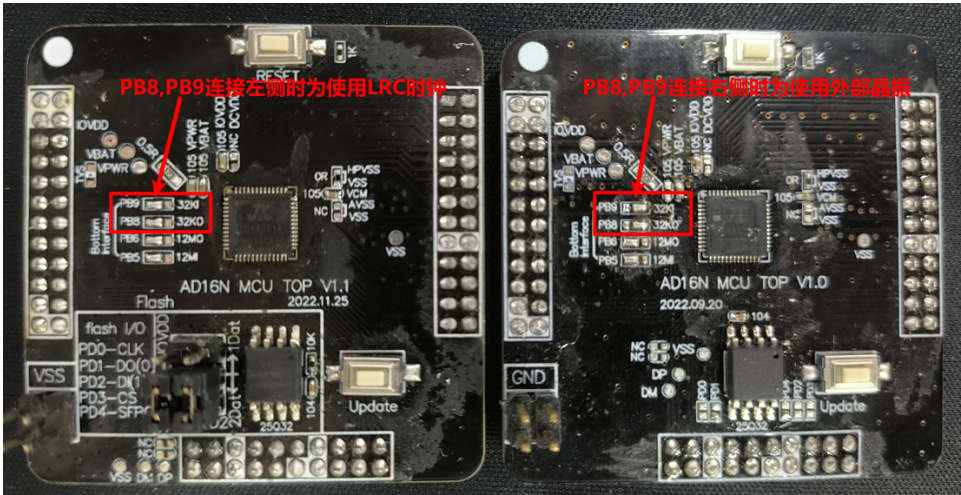


图 1 RTC 时钟源硬件配置

- 2、定时唤醒和闹钟唤醒的回调函数是在中断做处理的, 所以执行时间不可过长。
- 3、当开启 RTC 功能后, powerdown 和 soft off 的功耗会增加 5uA+ 的功耗。

43.3 RTC 接口函数

43.3.1 void rtc_dev_init(const struct rtc_config_init *arg);

该函数是 rtc 初始化的接口。

arg: rtc 配置的结构体指针

43.3.2 void read_current_time();

该函数是获取当前时间并打印显示出来，直接使用即可。

43.3.3 void write_clock_time(u16 year , u8 month , u8 day , u8 hour , u8 min , u8 sec);

该函数是修改时钟时间；

- ①u16 year : 年
- ②u8 month : 月
- ③u8 day : 日
- ④u8 hour : 时
- ⑤u8 min : 分
- ⑥u8 sec : 秒

43.3.4 void rea_alarm_time();

该函数是获取设置的闹钟时间并打印显示出来，直接使用即可。

43.3.5 void write_alarm_time(u16 year , u8 month , u8 day , u8 hour , u8 min , u8 sec);

该函数是修改闹钟时间；

- ①u16 year : 年
- ②u8 month : 月
- ③u8 day : 日
- ④u8 hour : 时
- ⑤u8 min : 分
- ⑥u8 sec : 秒

43.3.6 void time_wakeup_set(u8 index , u8 en);

该函数是设置 RTC 定时唤醒开启/关闭；①index：RTC 定时唤醒时间；参数如下：

```
* RTC_WKUP_SRC_256HZ
* RTC_WKUP_SRC_64HZ
* RTC_WKUP_SRC_2HZ      // 每隔 0.5s 唤醒一次
* RTC_WKUP_SRC_1HZ      // 每隔 1s 唤醒一次
```

②en：定时唤醒使能；参数如下：

0：关闭
1：开启

43.3.7 void rtc_alarm_en(u32 en);

该函数是开启/关闭闹钟功能；参数如下：en：0 关闭，1 开启

43.4 RTC 简单使用示例

RTC 模块有两个示范程序：rtc_app(); 和 rtc_timed_wakeup_app();

可以选择在 app.c 里选择 rtc 模式使用哪个模式

1、rtc_app();

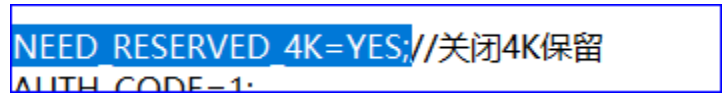
该模式主要展示 RTC 的基本功能和简单使用方法，如读写时钟，读写闹钟，开关闹钟，开关定时唤醒；

2、rtc_timed_wakeup_app();

该模式主要展示芯片在进入 powerdown 后能够定时唤醒执行相应动作（如读取当前时间）；

Reserved_Area 区域说明

可以选择 FLASH 最后的 4K 作为一个特殊的保留区，这个保留区只有使用烧写器烧写程序才能进行擦写动作。一些希望烧写完程序后就不经常给擦写的信息可以选择放到这个区域里面。使用这个保留区需要在 isd_config.ini 文件中把 NEED_RESERVED_4K 置成 YES。
第三方数据 (认证码 / 三元组 / 序列号)



```
NEED_RESERVED_4K=YES; //关闭4K保留
AUTH_CODE=1;
```

图 1 isd_config 中配置保留区

44.1 认证码相关信息

认证码是一个烧写在保留区 (Reserved_Area 区域) 中的一个信息，用来给每一个产品烧写一个特殊的 ID 使用。

认证码功能需要搭配烧写器支持，而支持该功能的一拖二烧写器最低版本号为：2.27.9

44.2 认证码功能 isd_config_ini.c 配置

认证码使用的时候需要在 isd_config.ini 文件添加相关信息（如果不需要认证码功能，第 2、3 点需要注释掉）

- 1: 把 NEED_RESERVED_4K=YES;
- 2: 在 [SYS_CFG_PARAM] 段中的如图位置，增加一个 AUTH_CODE=1; 配置项。

3: 在 [BURNER_PASSTHROUGH_CFG] 段中的如图位置, 增加一个 AUTH_CODE_IN_FLASH=1; 配置项。

```

isd_config_ini.c
49
50
51 [CHIP_VERSION]
52 SUPPORTED_LIST=A
53
54 ##### UBOOT配置项
55 [SYS_CFG_PARAM]
56 // #clk [0-255]
57 // #data_width[0 1 2 3 4] 3的时候uboot自动识别2或者4线
58 // #mode:
59 // #      0 RD_OUTPUT,      1 cmd      1 addr
60 // #      1 RD_I/O,        1 cmd      x addr
61 // #      2 RD_I/O_CONTINUE] no_send_cmd x add
62 // #SPI = data_width,clk,mode;
63 AUTH_CODE=1;
64 SPI=2_3_0_0; #data_width,clk,mode;
65 // #OSC=btosc;
66 // #OSC_FREQ=24MHz; #[24MHz 12MHz]
67 // #SYS_CLK=48MHz; #[48MHz 24MHz]
68 UTTX=PA06; //uboot串口tx
69 UTBD=1000000; //uboot串口波特率
70 // #UTRX=DP; //串口升级 [PB00 PB05 PA05], 默认PB05
71 // #RESET=PB01_08_0; //port口_长按时间_有效电平 (长按时间有00、04、0
72 // #CACHE_WAY=1; // 范围1~4
73 // #WAIT_TIME=10; // * 100ms
74 #if TFG_EXT_FLASH_EN

```

图 2 isd_config 文件 [SYS_CFG_PARAM] 段增加 AUTH_CODE 配置项

```

isd_config_ini.c
85
86 #####
87 [TOOL_CONFIG]
88 1TO2_MIN_VER=2.27.1
89 1TO8_MIN_VER=3.1.17
90 //BTBOX_MIN_VER=1.2.4.f
91 #####
92
93 #####
94 //烧写器配置项
95 //OTP_BOOT 0,FLASH模式 1,OTP模式
96 #####
97 [BURNER_PASSTHROUGH_CFG]
98 AUTH CODE IN FLASH=1;
99 LIGHTING2TYPE=0 //uboot_start
100 OTP_BOOT=0
101 // #FLASH_WRITE_PROTECT=NO;
102
103 #####
104 //烧写器配置项
105 //2.078v 2.188v 2.297v 2.406v 2.515v 2.624v 2.733v 2.840v

```

图 3 isd_config 文件 [BURNER_PASSTHROUGH_CFG] 段增加 AUTH_CODE_IN_FLASH 配置项

44.3 认证码信息接口

44.3.1 void rsv_auth_analysis(void)

该接口用于打印出烧写在芯片的认证码信息。

里面的 auth->data_p 即为烧录进去的认证码 (三元组 / 序列号)

```

24  *start addr:reserved area last 256 byte
25  data start:start addr + 80 byte
26  */
27
28 void rsv_auth_analysis(void)
29 {
30     auth_header_t *auth = (auth_header_t *)flash_addr2cpu_addr(rsv_flash_addr_get() + RESERVED_AREA_SIZE - 256 + 80);
31     u8 *crc_data = (u8 *)(&auth->data_len);
32     u32 crc_len = auth->data_len + sizeof(auth->data_len);
33     u16 crc16 = CRC16(crc_data, crc_len);
34
35     if (crc16 != auth->crc16) {
36         log_error("no auth data !!!\n ");
37         return;
38     }
39
40     log_info("auth data:");
41     log_info_hexdump((u8 *)auth->data_p, auth->data_len);
42 }
43

```

对应烧写在芯片的认证码 / 三元组 / 序列号

NORMAL /apps/app/bsp/common/reserved_area/reserved_area.c

图 4 认证码信息

44.4 认证码文件说明

认证码文件的格式是：认证码格式头 + 数据。

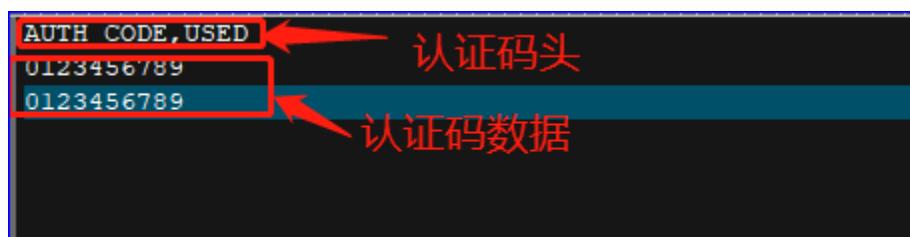


图 5 认证码文件格式

认证码的数据烧写到固件的时候会有 2 种格式

当文件的后缀名为 “*.csv” 时候，数据格式按照 ASCII 方式读取烧写。
当文件的后缀名为 “*.hex” 时候，数据格式按照 HEX 方式读取烧写。

异常

芯片在运行代码时，由于软件 / 硬件状态出错，当该错误状态未在硬件 / 软件程序设计覆盖的容错范围时，就会引起系统处于未知状态的异常。

在触发异常后，会有两种后果：

- 一、系统复位；
- 二、进入系统异常中断函数并死循环住；

备注：用户可以通过配置 `config_asser` 来选择触发异常后是系统复位还是进入系统异常中断函数卡死。

```

37 const int IRQ_AES_IP          = 3;    //aes
38
39 //内存管理malloc内部断言
40 const char MM_ASSERT          = TRUE;
41 //内核异常打印
42 const u8 config_asser = 1;
43
44 const u8 config_spi_code_user_cache = 1; //sfc放code区
45
46 const u8 config_audio_adc_enable = 1;
47
48 #if (DECODER_MIDI_EN || DECODER_MIDI_KEYBOARD_EN)
49 //midi主轨选择方式
50 const int MAINTRACK_USE_CHN    = 0;    //0:用track号来区分 1:用channel号来区分。
51 const int MAX_DEC_PLAYER_CNT   = 8;    //midi乐谱解码最大同时发声的key数,范围[1,31]
52 const int MAX_CTR_PLAYER_CNT   = 8;    //midi琴最大同时发声的key数,范围[1,31]
53 const int NOTE_OFF_TRIGGER     = 0;    //midi琴note off回调 1: time传0时, 不会回调 0:
54 const int MIDI_MAX_MARK_CNT    = 0;    //midi解码最大支持的mark数
55 #endif
56
57 #if RTC_EN

```

0: 系统复位

1: 进入异常中断函数卡死

玩具系列芯片统一

`app_config.c`

系统异常触发后果选择

45.1 异常分析

用户在触发异常后，可以观察异常信息里的 `reti` 和 `rets` 的地址，在下载目录里的 `sdk.lst` 文件查看对应地址的代码是出现在那里。

下面例子为 SDK 操作了一个指向空白地址的指针给他赋值导致的 `cpu` 写空白地址异常。（其他异常分析均可类似分析）

1. 分析异常信息

```

[09:03:56.673] [Info] [update]update module init ok
[09:03:56.673] [Info] [PMU]trim_flag:0, adp_trim_flag:0, vm_rd_err:0,
trim_err:0, vm_wr_err:0
[Info] [PMU]wvdd_level: 3, wvdd_level_now: 3
[Info] [PMU]except_cnt: 0
E
[09:03:56.673] X
[09:03:56.673] C
[09:03:56.673] <Error> [EMU]
[09:03:56.673] -----Err: function:exception_analyze() @line:625
[09:03:56.673] <Error> [EMU]_EMU_CON : 0xc000001f
<Error> [EMU]_EMU_MSG : 0x80000000
<Error> [EMU]CPU0 4023892 --> 4024258 --> 4012df4 --> 402136a
[09:03:56.673] <Error> [EMU]usp : 0x3f005b0
<Error> [EMU]ssp : 0x3f01f40
<Error> [EMU]sp : 0x3f02a58
<Error> [EMU]Stack Top: 0x3f02ae0
<Error> [EMU]reti : 0x4021380
<Error> [EMU]rets : 0x4012df4
<Error> [EMU]rete : 0x0
<Error> [EMU]psr : 0x0
<Error> [EMU]icfg : 0x7010a80
<Error> [EMU]--reti---
7F BC FA E1 2F 41 F8 E1 EF C7 00 EA 04 70 80 EA
[09:03:56.673] 30 00 91 80 01 78 01 EA 03 70 81 EA 30 00 A2 81
[09:03:56.673] 0A 78 02 EA 02 70 82 EA 30 00 B3 82 13 78 03 EA
[09:03:56.673] 01 70 83 EA 30 00 C6 83 1E 78 09 58 12 58 41 F6
[09:03:56.673] <Error> [EMU]--rets---
49 57 82 EA 00 04 28 C6 31 C6 08 E1 30 A2 7B 5C
[09:03:56.686] 02 EA 6C 57 82 EA 00 04 28 C6 31 C6 08 E1 A0 A1
[09:03:56.686] 10 CD 19 C0 60 80 FF E1 CF 7A 98 20 00 81 FF E1
[09:03:56.686] 8F 7A 58 20 08 81 FF E1 4F 7A 00 98 5F 10 00 EA
[09:03:56.686] <Error> [EMU]-- sp ---
04 70 30 00 03 70 30 00 34 00 00 00 01 00 00 00
[09:03:56.686] 57 05 00 04 5E 05 00 04 C8 2A F0 03 CC 47 05 04
[09:03:56.686] 14 42 ED 03 0C 02 EE 03 04 05 EE 03 1C 04 EE 03
[09:03:56.686] 14 04 EE 03 40 33 F0 03 B0 22 F0 03 80 13 02 04
[09:03:56.686] 00 00 00 00 F4 2D 01 04 00 00 00 00 40 1F F0 03
[09:03:56.686] 80 0A 01 07 B0 05 F0 03 DF 05 00 04 26 1C F0 03
[09:03:56.686] 1E 2E 04 04 00 00 00 00 00 00 00 00 00 00 00
[09:03:56.686] DF FF 00 00 FF FF 00 00 F0 FF 00 00 03 00 00 00
[09:03:56.686] 00 B0 71 0B 49 A2 2C 08
[09:03:56.686]
[09:03:56.686] <Error> [EMU] [0-CPU] emu err msg : sys exception
<Error> [EMU]JL_CEMU->MSG0 : 0x20000000
<Error> [EMU]JL_CEMU->MSG1 : 0x0
<Error> [EMU]JL_CEMU->MSG2 : 0x0
<Error> [EMU] [1-HCORE] hmem err msg : cpu write hmem exception
[09:03:56.695] <Error> [EMU]x0 = 0x30000000

```

1.cpu运行地址

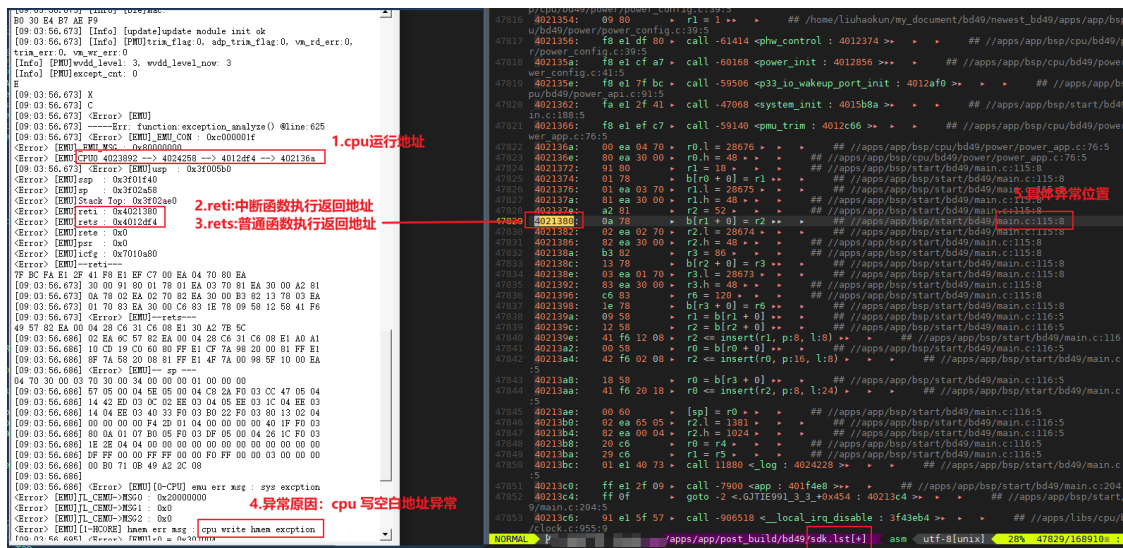
2.reti:中断函数执行返回地址

3.rets:普通函数执行返回地址

4.异常原因: cpu 写空白地址异常

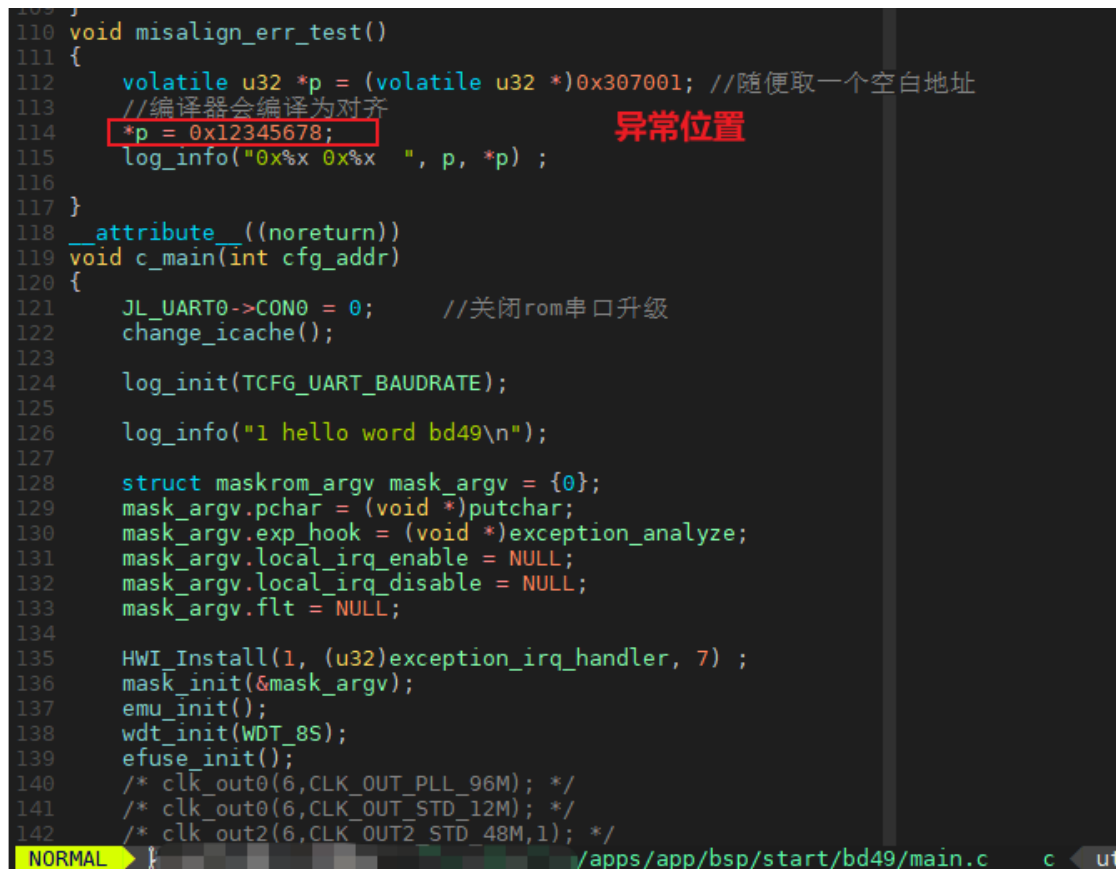
异常信息

2. 在下载目录里的 `sdk.lst` 文件查找 `rets` 和 `reti` 的位置（若分析该两处地址分析不出具体异常原因，可查看异常信息里的 `cpu` 的运行地址来反推异常位置）



异常信息在 sdk.lst 位置

3. 追踪到 SDK 具体异常位置。SDK 操作了一个指向空白地址的指针给他赋值导致的 cpu 写空白地址异常。



异常位置

备注：有一些异常信息往往不在具体异常代码那一行，而是在附近

45.2 常见异常

常见异常信息有：

1. pc_limit：程序跑飞，指 CPU 的 PC 指针跑到了不允许操作的地址；
2. misalign_err：非对齐访问，指 CPU 访问内存数据时，寻址指针要求地址 4 字节对齐访问；
3. cpu read / write hmem exception：cpu 读 / 写空白地址访问异常，指 CPU 读/写了内存里的空白地址；
4. watchdog time out：看门狗异常，指系统未在看门狗规定范围内进行清狗操作。

更多异常信息和 demo 例子可查看下方链接的《系统异常信息》部分：

[系统异常分析](#)

双备份升级

双备份升级存储结构有两个 app code 区域，当芯片正在运行 app code1，升级时将新固件写入 app code 2 区域，反之亦然。升级时对 app code N 区域数据校验成功后，复位即可运行 app code N 代码。当升级中断或者新写入固件校验不成功，不会影响原来 app code 的运行

杰理双备份升级有两种方式实现

1. 设备主动升级（由设备端主动向远端获取升级数据，通信协议需要能支持文件内容获取能力）
2. 设备被动升级（由远端主动推送升级数据，设备端被动写入的方式，远端可控制升级流程）

46.1 支持双备份升级的芯片以及对应 SDK 版本

支持双备份升级的芯片：AD14/15/16/18/、AC104 和 AD24

AD14/15/AC104 的 v1.7.0 SDK 和 AD16 v1.3.0SDK 需要添加补丁支持双备份设备被动升级。后续版本 SDK 会直接支持。

46.2 SDK 相关配置

首先需要在 isd_config.ini 加入以下配置：

```
[EXTRA_CFG_PARAM]
BR22_TWS_DB=YES;           //dual bank flash framework enable (only valid for_
↪dual_bank)
FLASH_SIZE=1M;             //flash_size cfg (only valid for dual_bank)
BR22_TWS_VERSION=0;        //default fw version(only valid for dual_bank)
DB_UPDATE_DATA=YES;        //generate db_update_data.bin(only valid for dual_bank)
SPECIAL_OPT=0;             //special cfg : single bin generating(only valid for_
```

(续下页)

(接上页)

```

↪dual_bank)
FORCE_4K_ALIGN=YES; // force align with 4k bytes (only valid for dual_bank)
#NEW_FLASH_FS=YES; // enable single bank flash framework (only valid for_
↪single_bank)

```

另外, AD14/15/16/18 以及 AC104 需要关闭所有写保护, 其中软件写保护位于

```

85
86 AT_SPI_CODE /*该函数放置段不可更改*/
87 u32 flash_code_protect_callback(u32 offset, u32 len)
88 {
89     return 0; // 返回0即可
90     u32 limit_addr = vm_attr.sclust;
91     /* log_info("0x%x 0x%x", limit_addr, offset); */
92     if ((offset < limit_addr) || ((offset + len) > boot_info.flash_size)) {
93         /* 超过正常擦写区域, 不进行擦写操作 */
94         return 1;
95     } else {
96         /* 进行擦写操作 */
97         return 0;
98     }
99 }
100

```

图 1 软件写保护

如果升级使用 simple_fat 读取升级文件, 需要去掉框内的限制

```

4
5 u32 smpl_fat_read_api(void *pfile, void *buff, u32 len)
6 {
7     u32 rlen = f_read(&pfi, buff, len);
8     /* if (rlen != len) { */
9     /* return 0; */
10    /* } */
11    return rlen;
12 }
13
14 u32 smpl_fat_seek_api(void *pfile, u32 offset, u32 fromwhere)
15 {
16     return f_seek(&pfi, offset);
17 }
18
19 u32 smpl_fat_file_close_api(void **ppfile)
20 {
21     return (u32) * ppfile;
22 }

```

图 2 simple_fat 读函数

设备被动升级采用 db_update_data.bin 作为升级文件, 升级文件会在编译下载的时候生成, 在 SDK 示例中由于 fat 文件系统有 8+3 的长名限制, 名字被缩短为 db_data.bin

46.3 相关接口说明

46.3.1 函数 u32 jlfs_get_idle_bank_info(u32 *bank_addr, u32 *bank_size)

此函数获取 idle_bank 的信息, 其中参数:

1. bank_addr: 空闲的code区地址, 接口内部会更新到指针;
2. bank_size: 空闲的code区大小, 接口内部会更新到指针
3. 返回值:

(续下页)

(接上页)

成功 返回0;
失败 其他值;

46.3.2 函数 u32 jlfs_updata_dual_bank_info(u32 bank_addr, u16 data_crc)

此函数更新 idle_bank 的信息其中参数:

1. bank_addr: 空闲的code区地址;
2. data_crc: 升级文件校验值;
3. 返回值:
成功 返回0;
失败 其他值;

46.3.3 函数 u32 jlfs_check_dual_bank_info(u32 update_bank_addr)

此函数校验写入升级信息后的 idle_bank 信息, 其中参数:

1. update_bank_addr: 升级的code区地址;
2. 返回值:
成功 返回0;
失败 其他值;

46.4 常见客户问题

1、AD15 双备份升级后，使用强制升级工具升级失败？

AD15 的 download 工具在下载的时候不会去主动擦除 code2 区的代码，所以需要在下载目录的 download.bat.c 加入 -format all 全擦除 flash 指令擦除 code2 区域代码，加入指令后需要编译一次，看到图中的全擦除模式生效即为配置成功。

```
diff_00 0 [FAVG_IS_NULL]
打开设备 (COM3) 成功
SPI nor flash online.
Online flash id: ef4016
Online flash size: 4M
Online external flash id: ef4017
Online external flash size: 8M
Erase Flash Size is 4096
ISDdownload

-----FLASH INFO-----
| PID : SH54
| FLASH_BIN_SIZE : 0x16000
| FLASH_NEED_SIZE : 0x100000
| FLASH_REAL_SIZE : 0x400000
| VM_REAL_SIZE : 0x6000
| VM_START_ADDR : 0xf4000
| VM_END_ADDR : 0xfa000
| EEPROM_RESERVED_SIZE : 0x6000
| EEPROM_RESERVED_START : 0xfa000
| EEPROM_RESERVED_END : 0x100000
| CODE_BOUNDARY_LINE : 0x7a000
| LAVE_SIZE : 0x64000
| ENTRY_ADDR : 0x100100
| ERASE MODE : all
| UBOOT VERSION : 2023-06-25 17:46:17, UBOOT-$-@20230625-$1242b31
| LOADER VERSION : 2023-06-30 20:11:01, LOADER-v1.0.0-$-@20230630-$a806557
-----
正在擦除FLASH[0x0-0x400000].....
...
.
```

图 3 ad15 编译擦除 flash 示意图

2、双备份升级流程，获取到的版本号出现溢出？

双备份升级的获取到的版本号是使用 u16 变量保存，升级到版本号为 65535 之后，再升级就会出现版本号溢出。但是不影响后续的升级和使用流程。

46.5 双备份升级接口使用示例

双备份升级接口使用示例可以查看 gitee 文档中的示例代码部分 <https://gitee.com/Jieli-Tech/fw-AD15N/issues/I7MIDZ>

CHAPTER 47

Indices and tables

- `genindex`
- `modindex`
- `search`